

Matemáticas y Algoritmos en Ciencia de Datos Aplicada

Fundamentos matemáticos, algoritmos y práctica computacional

César Galindo

2026-07-13

Tabla de contenidos

Prefacio	16
Cómo leer este libro	18
Del caso a la formulación	18
Definiciones y derivaciones	18
Algoritmos y código	19
Diagnósticos y problemas	19
 Parte I: Datos, variación y aprendizaje a partir de muestras	 20
Observaciones, variables y preguntas	22
Unidades de observación y niveles de análisis	23
Variables, valores y mediciones	24
El diccionario de datos	26
Población, marco de observación y muestra	27
De una pregunta amplia a una pregunta precisa	28
Familias de tareas	29
De la tabla a la notación matemática	30
Modelo, algoritmo y resultado ajustado	32
Disponibilidad temporal y filtración de información	34
Calidad de datos como propiedad de la pregunta	35
Análisis completo del caso de Bogotá	35
Una ficha mínima para iniciar un análisis	36
Síntesis	37
Problemas integrados	37
Respuestas seleccionadas	39
 Variables aleatorias, distribuciones y muestras	 40
Antes y después de observar	40
La distribución	41
Esperanza y variación	42
Distribuciones conjuntas, independencia y covarianza	43
De la población a una muestra	44
Distribución empírica y estadísticos	44
Por qué se estabiliza un promedio	45
Síntesis	46
Problemas integrados	46
Respuestas seleccionadas	47

Condicionamiento, estimación y riesgo	49
Lo que cambia al conocer las entradas	49
De la cantidad poblacional al valor calculado	52
Pérdida, riesgo y aprendizaje a partir de una muestra	53
Regreso a las subzonas de Bogotá	56
Síntesis	56
Problemas integrados	56
Respuestas seleccionadas	57
 Parte II: Regresión, optimización y clasificación	 59
Regresión lineal y mínimos cuadrados	61
Una recta elegida por mínimos cuadrados	61
Del problema escalar a la matriz de diseño	62
La pérdida cuadrática en notación matricial	63
Gradiente y ecuaciones normales	64
Cómo calcular la solución	66
Proyección y diagnóstico de residuales	67
Rango, colinealidad e interpretación	68
Regreso a las subzonas de Bogotá	68
Síntesis	70
Problemas integrados	70
Respuestas seleccionadas	71
 Descenso del gradiente	 73
De la derivada a un algoritmo	73
La tasa decide la trayectoria	75
La Hessiana explica la estabilidad	76
Escala y condicionamiento	77
Verificar gradiente y solución	78
Costo y gradientes por subconjuntos	79
Regreso a los dos casos	79
Síntesis	80
Problemas integrados	80
Respuestas seleccionadas	81
 Validación y regularización	 82
Capacidad y comportamiento fuera de muestra	82
Tres funciones para tres conjuntos de datos	83
Transformaciones y filtración de información	84
Ridge: encoger direcciones inestables	85
Lasso: una penalización con esquina	87
Seleccionar sin consultar prueba	88
Regreso a las subzonas de Bogotá	88
Síntesis	89
Problemas integrados	90
Respuestas seleccionadas	91

Regresión logística	92
De una clase a una probabilidad condicional	92
La pérdida proviene del modelo Bernoulli	93
Gradiente, Hessiana y convexidad	94
La probabilidad todavía no es una decisión	96
Matriz de confusión y métricas	96
Un umbral fijado con validación	98
Calibración y alcance de las probabilidades	99
Síntesis	99
Problemas integrados	99
Respuestas seleccionadas	100
Problemas integradores de regresión y clasificación	101
Síntesis de la parte	101
Problemas integrados	101
Respuestas seleccionadas	105
Parte III: Redes neuronales	106
Perceptron, neurona logística y forward propagation	108
Pregunta aplicada	108
Objetivos del capítulo	108
Distribución predictiva y riesgo	108
Perceptron como unidad básica	109
Neurona logística	109
De una neurona a una capa	110
Red de una capa oculta	110
ReLU	111
Softmax	111
Entropía cruzada multiclase	112
Por qué hace falta una no linealidad	112
Costo de una pasada forward	112
Ejemplo resuelto: softmax y pérdida	113
Ejemplo resuelto: dimensiones	114
Caso longitudinal: dígitos escritos a mano	114
Punto de control	114
Verificaciones seleccionadas	115
Ejemplo resuelto: auditoría de formas en dos capas	115
Para explorar más	115
Revisión del capítulo	116
Términos clave	116
Ejercicios	116
Backpropagation	118
Pregunta aplicada	118
Objetivos del capítulo	118
Del riesgo esperado al gradiente calculable	118

El grafo de cómputo	119
Gradiente de softmax con entropía cruzada	120
Gradientes de la segunda capa	120
Propagar hacia la capa oculta	121
Backpropagation como algoritmo	121
Costo, memoria y alcance de la garantía	122
Verificación por diferencias finitas	123
Ejemplo resuelto: diferencia finita escalar	123
Ejemplo resuelto: formas de los gradientes	124
Errores frecuentes	124
Punto de control	124
Caso longitudinal: backpropagation en dígitos	125
Verificaciones seleccionadas	125
Para explorar más	125
Revisión del capítulo	125
Términos clave	126
Ejercicios	126
Activaciones, inicialización, regularización y diagnóstico	128
Pregunta aplicada	128
Objetivos del capítulo	128
Fuentes distintas de aleatoriedad	128
Activaciones	129
Inicialización	129
De la propagación de varianza a Xavier y He	130
Regularización L2 en redes	131
Ejemplo resuelto: L2 en pérdida y gradiente	131
Dropout como idea	132
Curvas de entrenamiento	132
Diagnósticos comunes	132
Ejemplo resuelto: leer una curva	133
Checkpoint técnico del proyecto	133
Caso longitudinal: curva de entrenamiento en dígitos	133
Verificaciones seleccionadas	134
Ejemplo resuelto: regla de parada	134
Lectura de curvas: tres patrones frecuentes	134
Punto de control	135
Para explorar más	135
Revisión del capítulo	135
Términos clave	136
Ejercicios	136
Optimizadores y PyTorch	138
Pregunta aplicada	138
Objetivos del capítulo	139
Gradiente estocástico y filtración	139
Mini-batch gradient descent	139
Momentum	140

Ejemplo resuelto: dos pasos con momentum	140
Derivación guiada: momentum como memoria exponencial	141
RMSProp y Adam	141
Ejemplo resuelto: Adam en una coordenada	142
Qué puede afirmarse sobre convergencia	143
Costo y memoria del optimizador	143
PyTorch: qué automatiza	143
Loop mínimo	144
Ejemplo resuelto: lectura de un loop	144
Comparación justa	145
Caso longitudinal: experimento reproducible en dígitos	145
Punto de control	145
Verificaciones seleccionadas	145
Ejemplo resuelto: presupuesto de entrenamiento	146
Convergencia no es validación	146
Para explorar más	146
Revisión del capítulo	147
Términos clave	147
Ejercicios	147
Ejercicios integradores: redes neuronales	149
Resultados de práctica	149
Mapa del banco	149
Cómo usar esta sección	149
Columna probabilística del módulo	150
Bloque A: dimensiones y forward	150
Bloque B: backpropagation	150
Bloque C: diagnóstico	151
Bloque D: PyTorch	151
Respuestas seleccionadas	151
Parte IV: Evaluación y estrategia de modelado	152
Métricas y particiones	154
Pregunta aplicada	154
Objetivos del capítulo	154
Una métrica es un estadístico	154
Matriz de confusión poblacional y empírica	156
El condicionamiento inverso y la prevalencia	156
Métrica principal	158
Accuracy no siempre basta	158
Ejemplo resuelto: accuracy engañosa	158
Ejemplo resuelto: costo de falsos negativos	159
Derivación guiada: F1 como equilibrio operativo	159
De costos a una regla de decisión	160
Baseline	160
Particiones	161

Test se toca una vez	161
Estratificación	161
Ejemplo resuelto: conteos en una partición estratificada	161
Leakage	162
Pipeline como defensa	162
Caso longitudinal: clasificación con slice reciente	162
Punto de control	163
Verificaciones seleccionadas	163
Para explorar más	163
Revisión del capítulo	163
Términos clave	164
Ejercicios	164
Validación cruzada y curvas de aprendizaje	166
Pregunta aplicada	166
Objetivos del capítulo	166
Qué aleatoriedad intenta representar la validación	166
Bootstrap y unidad de remuestreo	167
Cross-validation	167
Qué reportar	168
Ejemplo resuelto: elegir entre dos modelos	168
Derivación guiada: media y variabilidad de CV	168
Grid search	169
Selección y validación anidada	169
Costo computacional de validar	170
Learning curve	170
Ejemplo resuelto: leer una learning curve	171
Validation curve	171
Caso longitudinal: estabilidad del modelo de clasificación	172
Ejemplo resuelto: decisión con diferencia pequeña	172
Sesgo, varianza y ruido	172
Punto de control	173
Verificaciones seleccionadas	173
Ejemplo resuelto: cuándo no hacer grid más grande	173
Para explorar más	173
Criterio práctico de decisión	173
Revisión del capítulo	174
Términos clave	174
Ejercicios	174
Error analysis y data mismatch	176
Pregunta aplicada	176
Objetivos del capítulo	176
Riesgo condicional por slice	176
Cambio de distribución como cambio de medida	177
Error analysis	177
Slices	178
Incertidumbre y búsqueda entre slices	178

Data mismatch	179
Diagnóstico de mismatch	179
Ejemplo resuelto: mismatch por proporción de clase	180
Derivación guiada: qué métricas pueden promediarse por slices	180
Recomendación técnica	180
Ejemplo resuelto: dos modelos con el mismo F1	181
Ejemplo resuelto: brecha contra el promedio	181
Caso longitudinal: slice reciente	182
Punto de control	182
Verificaciones seleccionadas	182
Procedimiento de auditoría por slice	182
Para explorar más	183
Revisión del capítulo	183
Términos clave	183
Ejercicios	184
Ejercicios integradores: estrategia de machine learning	185
Resultados de práctica	185
Mapa del banco	185
Criterio de respuesta	185
Columna probabilística del módulo	186
Bloque A: métricas	186
Bloque B: particiones y leakage	186
Bloque C: validación y curvas	187
Bloque D: error analysis	187
Respuestas seleccionadas	187
Parte V: Aprendizaje no supervisado	188
SVD y PCA	190
Pregunta aplicada	190
Objetivos del capítulo	190
PCA poblacional y PCA empírico	191
SVD	192
PCA como SVD del dato centrado	192
Proyección	193
Varianza explicada	193
Ejemplo resuelto: error desde valores singulares	193
Derivación guiada: reconstrucción y pérdida de información	194
Costo y elección del método numérico	195
Ejemplo resuelto: elegir k	195
Reconstrucción	195
Ejemplo resuelto: proyección y reconstrucción	196
Caso longitudinal: PCA sintético	197
PCA no es interpretabilidad automática	197
Punto de control	197
Verificaciones seleccionadas	198

Ejemplo resuelto: PCA dentro de un pipeline	198
Para explorar más	198
Revisión del capítulo	198
Términos clave	199
Ejercicios	199
K-Means y clustering	201
Pregunta aplicada	201
Objetivos del capítulo	201
Distorsión poblacional y objetivo empírico	202
Función objetivo	202
Algoritmo	203
Por qué la inercia no aumenta	203
Inercia	204
Silhouette score	204
Ejemplo resuelto: por qué escalar cambia clusters	204
Ejemplo resuelto: asignación a centroides	205
Ejemplo resuelto: una iteración completa	205
Derivación guiada: por qué el centroide es el promedio	206
Complejidad y casos degenerados	206
Escalamiento	207
Limitaciones	207
Caso longitudinal: clusters sintéticos	207
Estabilidad por inicialización	207
Punto de control	208
Verificaciones seleccionadas	208
Ejemplo resuelto: elegir k con información incompleta	208
Para explorar más	209
Revisión del capítulo	209
Términos clave	209
Ejercicios	209
Recomendación básica y estructura latente	211
Pregunta aplicada	211
Objetivos del capítulo	211
El mecanismo de observación no es neutral	211
Riesgo offline y utilidad de uso	212
Matriz usuario-ítem	212
Similitud coseno	212
Ejemplo resuelto: similitud coseno	213
Derivación guiada: qué ignora la similitud coseno	213
Recomendación item-item	213
Ejemplo resuelto: recomendación item-item	214
Evaluación offline: Precision@k	214
Baseline de popularidad	215
SVD para factores latentes	215
Factorización sobre entradas observadas	216
Alternancia, costo e identificación	217

Caso longitudinal: interacciones de recomendación	218
Riesgos de recomendación	218
Punto de control	218
Verificaciones seleccionadas	218
Ejemplo resuelto: cobertura mínima	219
Para explorar más	219
Revisión del capítulo	219
Términos clave	219
Ejercicios	220
Ejercicios integradores: aprendizaje no supervisado	221
Resultados de práctica	221
Mapa del banco	221
Antes de responder	221
Columna probabilística del módulo	222
Bloque A: PCA	222
Bloque B: K-Means	222
Bloque C: recomendación	223
Respuestas seleccionadas	223
 Parte VI: Reproducibilidad y comunicación	 224
Síntesis conceptual: redes, estrategia y no supervisado	226
Pregunta aplicada	226
Alcance	226
Objetivos del capítulo	226
Mapa probabilístico de síntesis	226
Mapa de contenidos	227
Redes neuronales	227
Softmax y entropía cruzada	228
Diagnóstico de modelos	228
No supervisado	228
Forma del Parcial 2	228
Regla de preparación	229
Cómo leer una pregunta integradora	229
Ejemplo resuelto: softmax estable en síntesis	229
Ejemplo resuelto: estructura no supervisada dentro de una respuesta	230
Ejemplo resuelto: respuesta integradora	230
Caso longitudinal: respuesta integradora	231
Verificaciones seleccionadas	231
Ejemplo resuelto: matriz de decisión integradora	231
Mini-rúbrica para una respuesta integradora	232
Para explorar más	232
Revisión del capítulo	232
Términos clave	233
Ejercicios	233

Reproducibilidad y tracking	234
Pregunta aplicada	234
Qué significa reproducir	234
Objetivos del capítulo	235
Reproducibilidad computacional y replicación estadística	235
El paquete mínimo reproducible	235
Semillas	236
Manifest de experimento	236
Ejemplo resuelto: manifest mínimo	236
Ejemplo resuelto: resultado no reproducible	237
Ejemplo resuelto: auditar un manifest	237
Tracking de experimentos	238
Persistencia de modelos	238
Hash de artefactos	239
Checklist de reproducibilidad	239
Punto de control	239
Caso longitudinal: manifest del clasificador	239
Verificaciones seleccionadas	240
Ejemplo resuelto: manifest mínimo en JSON	240
Qué debe poder volver a ejecutarse	241
Para explorar más	241
Revisión del capítulo	241
Términos clave	241
Ejercicios	242
 Comunicación técnica	 243
Pregunta aplicada	243
Tesis de un reporte	243
Objetivos del capítulo	244
Comunicar una cantidad aleatoria	244
Estructura recomendada	244
Figuras útiles	245
Lenguaje de alcance	245
Ejemplo resuelto: de bitácora a argumento	245
Ejemplo resuelto: mejorar una conclusión	246
Ejemplo resuelto: mejora absoluta y relativa	246
Respuesta oral de 45 segundos	247
Caso longitudinal: conclusión del clasificador	247
Verificaciones seleccionadas	247
Ejemplo resuelto: tabla de soporte de afirmaciones	247
Plantilla de resultados	248
De resultado a recomendación	248
Para explorar más	249
Revisión del capítulo	249
Términos clave	249
Ejercicios	249

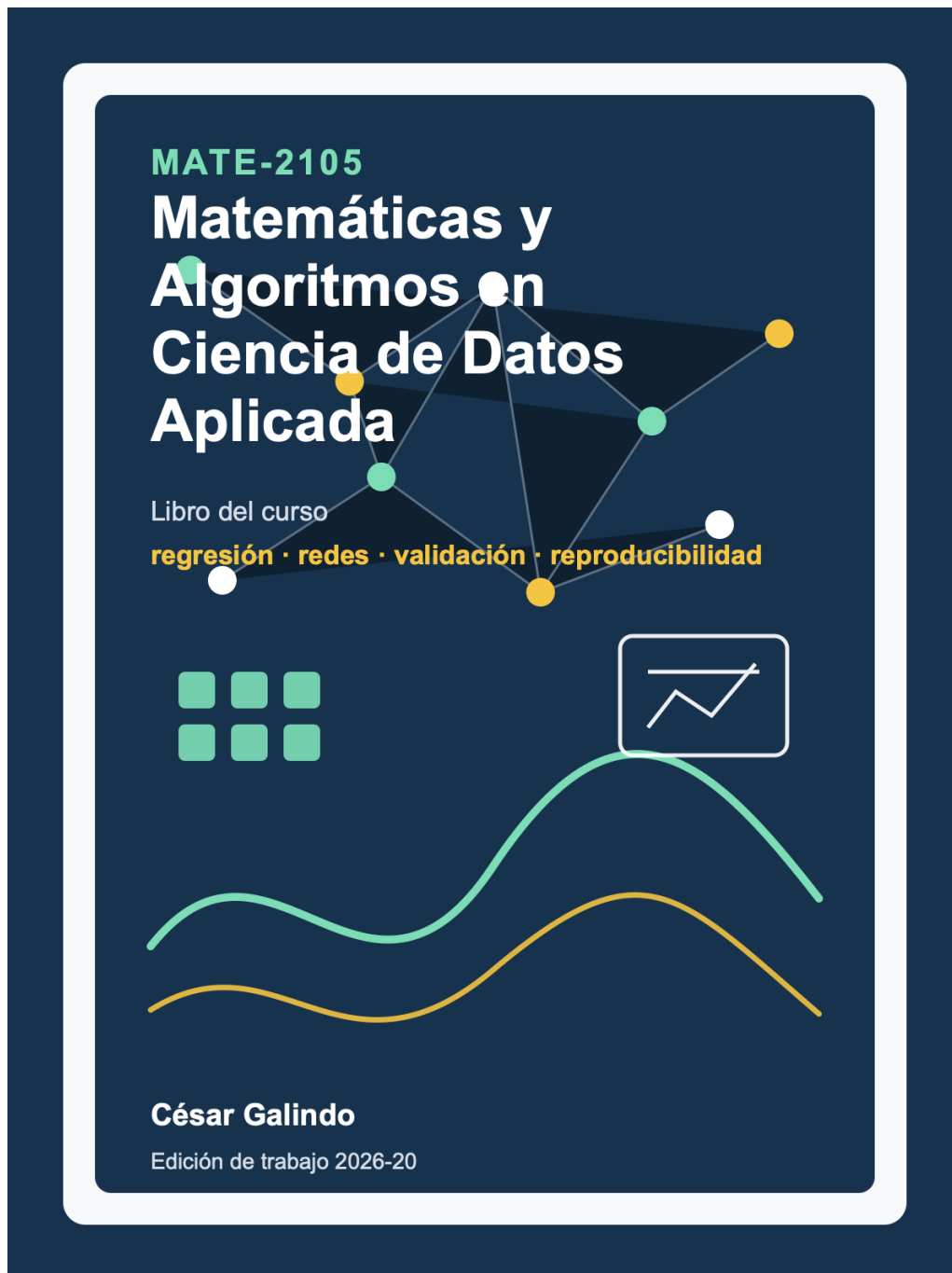
Entrega final y defensa	251
Pregunta aplicada	251
Paquete final	251
Objetivos del capítulo	251
Defensa del fundamento probabilístico	251
Estructura del repositorio de proyecto	252
Criterio de paquete ejecutable	252
Ejemplo resuelto: contribución individual	252
Peso del proyecto en el curso	253
Rúbrica del reporte y repositorio	253
Ejemplo resuelto: cálculo de nota ponderada	253
Defensa oral individual	254
Rúbrica de defensa 0/1/2	255
Caso longitudinal: defensa del clasificador	255
Verificaciones seleccionadas	256
Ejemplo resuelto: pregunta difícil	256
Checklist individual antes de entrar a defensa	256
Carga docente mínima	257
Contribución individual y trazabilidad	257
Para explorar más	257
Revisión del capítulo	257
Términos clave	258
Ejercicios	258
Ejercicios integradores: reproducibilidad y defensa	259
Resultados de práctica	259
Mapa del banco	259
Criterio de calidad	259
Columna probabilística del cierre	260
Ejercicio 1: Softmax estable	260
Ejercicio 2: Diagnóstico	260
Ejercicio 3: PCA y reconstrucción	261
Ejercicio 4: Reproducibilidad	261
Ejercicio 5: Defensa	261
Respuestas seleccionadas	261
Apéndices de consulta	263
Prerrequisitos matemáticos mínimos	264
Cómo usar este apéndice	264
Álgebra lineal mínima	264
Normas, distancias y pérdida cuadrática	265
Cálculo y gradientes mínimos	266
Notación probabilística de primera aparición	266
Notación NumPy y PyTorch	267
Checklist de formas	267
Ejemplo resuelto: gradiente de MSE en una línea	267

Respuestas rápidas de autoverificación	268
Glosario y notación	269
Términos clave	269
Notación frecuente	271
Convención de formas	272
Índice temático	273
Cómo usar este índice	273
Modelación y representación	273
Álgebra lineal y optimización	274
Regresión, regularización y clasificación	274
Validación, métricas y diagnóstico	275
Redes neuronales	275
Aprendizaje no supervisado y recomendación	276
Reproducibilidad y comunicación	276
Recursos de consulta rápida	276
Índice de algoritmos, datasets y funciones	278
Cómo usar este apéndice	278
Mapa por primer uso	278
Referencia de API por capítulo	279
Algoritmos principales	280
Datasets y generadores	281
Funciones NumPy frecuentes	282
Herramientas scikit-learn frecuentes	283
Métricas y diagnósticos	283
PyTorch mínimo del curso	284
Regla de lectura rápida	284
Formularios y respuestas seleccionadas	285
Cómo usar esta parte	285
Mapa de apéndices	285
Política de respuestas	285
Apéndice Módulo 1: formulario y respuestas seleccionadas	287
Cómo usar este apéndice	288
Notación mínima	288
Formulario mínimo	288
Patrones de código	291
Checklist antes de entregar	292
Respuestas seleccionadas	292
Guía de estudio para el Parcial 1	294
Apéndice Módulo 2: formulario y respuestas seleccionadas	295
Cómo usar este apéndice	296
Notación mínima	296

Fórmulas clave de forward	296
Fórmulas clave de backward	297
Diagnóstico de entrenamiento	298
Patrones de código	298
Checklist antes de entregar	298
Respuestas seleccionadas	299
Apéndice Módulo 3: formulario y respuestas seleccionadas	301
Cómo usar este apéndice	302
Notación mínima	302
Métricas de clasificación binaria	302
Validación cruzada	303
Reglas operativas	303
Checklist de evaluación	303
Patrones de decisión	303
Patrones de código	304
Respuestas seleccionadas	304
Apéndice Módulo 4: formulario y respuestas seleccionadas	306
Cómo usar este apéndice	307
Notación mínima	307
SVD	307
PCA	308
K-Means	308
Silhouette	308
Cosine similarity	308
Checklist de interpretación	309
Patrones de código	309
Respuestas seleccionadas	309
Apéndice Módulo 5: formulario y respuestas seleccionadas	312
Cómo usar este apéndice	313
Notación mínima	313
Fórmulas clave	313
Checklist de paquete reproducible	314
Estructura mínima de manifest	314
Lenguaje de defensa	315
Respuestas seleccionadas	315
Referencias y atribuciones	318
Referencias de exposición y edición	318
Referencias matemáticas y algorítmicas	318
Documentación técnica	319
Datasets y generadores	319
Figuras	319
Cómo citar este libro	320

Sobre esta edición	321
Autoría y cita	321
Accesibilidad	321
Datos, código y atribuciones	321
Licencia y materiales reservados	321
Errata y actualización	322

Prefacio



Este libro fue escrito para el curso **MATE-2105 Matemáticas y Algoritmos en Ciencia de Datos Aplicada**. Presenta algunos de los modelos fundamentales y de uso más frecuente en ciencia de datos. Para cada uno estudia la formulación matemática, los algoritmos utilizados para estimarlo o calcularlo a partir de datos y la interpretación de sus resultados, supuestos y limitaciones.

El texto supone familiaridad con cálculo diferencial y vectorial, álgebra lineal, estadística descriptiva y programación básica en Python. La primera parte desarrolla las nociones de probabilidad necesarias para estudiar modelos con datos: variables aleatorias, distribuciones, muestras, condicionamiento, estimación y riesgo. Esas ideas no quedan aisladas en un capítulo preliminar; reaparecen en regresión, clasificación, validación y reducción de dimensión. En todo el libro, definiciones, derivaciones, ejemplos resueltos y problemas conectan las ideas matemáticas con su implementación y con situaciones concretas.

La idea que orienta el libro es sencilla: las matemáticas, los algoritmos y el análisis de datos deben estudiarse juntos. Una fórmula adquiere sentido cuando se entiende qué objeto describe, bajo qué supuestos se obtiene y cómo conduce a un procedimiento que puede ejecutarse. De manera recíproca, ejecutar un algoritmo no basta para comprenderlo; también es necesario saber qué objetivo persigue, qué garantías ofrece y en qué situaciones puede fallar.

Por esta razón, los capítulos parten de problemas concretos y avanzan desde su formulación hasta la evaluación de los resultados. Las derivaciones se presentan con suficiente detalle para que el lector pueda reconstruir los argumentos. Los ejemplos resueltos, el código y los ejercicios permiten comprobar las ideas y examinar su comportamiento en casos particulares. Una implementación correcta, sin embargo, no compensa una pregunta mal formulada; tampoco una métrica bien calculada garantiza que las conclusiones sean válidas para la población en la que se quiere utilizar el modelo.

El recorrido comienza con el significado de los datos, la variación entre muestras y la formulación de cantidades poblacionales que pueden aproximarse a partir de observaciones. La regresión introduce después la primera relación entre una de esas cantidades, una pérdida empírica y un algoritmo de ajuste. A partir de allí se desarrollan optimización, regularización, clasificación y redes neuronales. Los capítulos posteriores se ocupan de validación, análisis de errores, reducción de dimensión, agrupamiento y recomendación. El libro concluye con reproducibilidad, comunicación técnica y evaluación crítica de los resultados; los apéndices permiten consultar prerequisites, notación, índices y respuestas seleccionadas.

El propósito no es ofrecer un catálogo exhaustivo de métodos de aprendizaje automático, sino estudiar un conjunto representativo con el detalle suficiente para comprender sus fundamentos, derivar sus algoritmos, implementarlos, analizar su comportamiento y reconocer sus limitaciones. Aunque el texto nació para acompañar un curso semestral, sus capítulos están organizados para que también puedan leerse de manera continua y consultarse de forma independiente.

César Galindo
Bogotá, 2026

Cómo leer este libro

Los capítulos están organizados para una lectura acumulativa. La primera parte establece qué representan los datos y cómo una muestra se relaciona con una población. Después aparecen los modelos supervisados, la optimización, las redes neuronales, la evaluación, el aprendizaje no supervisado y la reproducibilidad. Cada parte utiliza objetos introducidos antes, pero vuelve a explicarlos cuando adquieren una función nueva.

También es posible consultar un capítulo de manera independiente. En ese caso conviene reconstruir primero su pregunta principal y revisar las definiciones a las que remite. Los apéndices reúnen los prerrequisitos de álgebra lineal y cálculo, el glosario, los índices y respuestas seleccionadas. No reemplazan el desarrollo: permiten recuperar una identidad o una convención sin interrumpir innecesariamente la lectura.

Del caso a la formulación

Buena parte de la matemática del libro aparece porque un primer análisis resulta insuficiente. Una tabla no identifica por sí sola la población que representa; un promedio no describe toda una distribución; una recta no siempre explica la estructura de los residuales; una pérdida pequeña en entrenamiento no garantiza un buen resultado en observaciones nuevas. Esas dificultades introducen las definiciones y los algoritmos que siguen.

Por esta razón, un ejemplo no debe leerse como una aplicación posterior de una fórmula ya terminada. Con frecuencia es el lugar donde se descubre qué fórmula hace falta. Después del desarrollo matemático, el texto vuelve al caso para examinar qué cambió y qué limitaciones permanecen.

Tres conjuntos de datos cumplen un papel recurrente. Una población sintética de regresión permite conocer el mecanismo generador y repetir muestras; un conjunto de 32 subzonas de Bogotá obliga a trabajar con documentación incompleta y datos agregados; una población sintética de clasificación permite comparar una probabilidad conocida con la estimada por un modelo. Los casos sintéticos sirven para comprobar propiedades matemáticas. El caso real sirve para recordar que una propiedad demostrada bajo un modelo no corrige una limitación de medición o de cobertura.

Definiciones y derivaciones

Las definiciones fijan objetos que se utilizarán después. Al leer una, conviene identificar qué ambigüedad resuelve y construir un ejemplo que la satisfaga y otro que no. Una proposición debe leerse junto con sus supuestos: retirar uno de ellos puede cambiar por completo la conclusión.

Las derivaciones contienen los pasos que conectan una formulación con un algoritmo. El gradiente de mínimos cuadrados, la ecuación normal, *backpropagation* y la actualización de K-Means no son fórmulas aisladas. Cada una surge de un objetivo y de una representación particular. Reconstruir esos pasos con lápiz permite distinguir una identidad matemática de una elección de implementación.

En el texto matemático, X representa un predictor aleatorio y $\mathbf{X}$ una matriz de diseño observada. Las realizaciones individuales se escriben x_i , y el vector de respuestas observadas se denota por $\mathbf{y}$. En el código se conservan los nombres usuales $\mathbf{X}$ y $\mathbf{y}$; el contexto y las dimensiones indican qué arreglos representan.

Cuando una expresión no resulte clara, hay tres comprobaciones útiles:

1. escribir las dimensiones de cada objeto;
2. identificar qué cantidades son aleatorias antes de observar la muestra;
3. separar el objetivo poblacional de la función calculada con los datos.

Estas comprobaciones detectan muchos errores antes de ejecutar una sola línea de código.

Algoritmos y código

Para comprender un algoritmo hay que reconocer qué recibe, qué produce y qué objetivo modifica. También interesa saber de dónde proviene cada actualización, cuál es su costo y bajo qué condiciones puede fallar. Una implementación que produce un número no responde todavía ninguna de estas preguntas.

Los fragmentos de Python utilizan principalmente NumPy, pandas, scikit-learn y PyTorch. Antes de ejecutarlos conviene anticipar la forma de las entradas, la salida y una comprobación sencilla. Después de ejecutarlos, el resultado debe compararse con esa expectativa. Las bibliotecas se usan para verificar y ampliar una implementación, no para sustituir la formulación matemática.

Los notebooks contienen cálculos más extensos y visualizaciones interactivas. El libro conserva el argumento que permite interpretar esos resultados. Por ello, leer solamente el código o solamente las fórmulas deja incompleta la explicación.

Diagnósticos y problemas

Los diagnósticos aparecen dentro del desarrollo. Un modelo puede tener un coeficiente de determinación alto y dejar curvatura en los residuales; un método iterativo puede reducir la pérdida demasiado lentamente o divergir; un clasificador puede alcanzar accuracy alta ignorando una clase poco frecuente. Estos casos no son excepciones que deban memorizarse al final: muestran qué información no contiene una métrica aislada.

Los problemas de cierre combinan varias tareas sobre una misma situación. Pueden pedir formular una población, efectuar un cálculo, implementar un procedimiento, examinar una gráfica y escribir una conclusión. En los problemas abiertos, una respuesta se considera completa cuando declara sus supuestos y distingue lo que los resultados muestran de lo que todavía no permiten afirmar.

Parte I: Datos, variación y aprendizaje a partir de muestras

En un conjunto de datos, una fila puede representar una persona, una transacción, una región o el resultado de un experimento. Su significado depende del procedimiento con el que fue registrada, de las variables disponibles y de la población sobre la que se quiere formular una conclusión. Identificar estos elementos es el primer paso para construir un modelo.

La tabla, por sí sola, no determina la pregunta. Un mismo archivo puede utilizarse para describir lo observado, predecir una respuesta futura o investigar una relación causal, pero cada propósito exige supuestos diferentes. El primer capítulo parte de un conjunto de datos de vivienda nueva por subzonas de Bogotá. La unidad de una de sus columnas no está documentada y cada fila resume muchas viviendas. Estas limitaciones permiten precisar qué significan una observación, una variable, una población y una conclusión basada en datos agregados.

Una muestra observada tampoco agota las posibilidades de la población. Si el procedimiento de recolección se repitiera, cambiarían las filas, los promedios y los modelos ajustados. El segundo capítulo utiliza una población sintética cuyo mecanismo generador es conocido para describir esa variación mediante variables aleatorias, distribuciones, esperanza, covarianza y muestras. El caso de Bogotá regresa para mostrar que el supuesto de muestra aleatoria no se obtiene por el solo hecho de tener una tabla.

El tercer capítulo pregunta qué cantidad intentamos aproximar cuando hacemos una predicción. El condicionamiento permite describir cómo cambia una respuesta con las variables disponibles; la teoría de estimación distingue una cantidad poblacional del procedimiento y del valor que obtenemos con una muestra. Esta distinción conduce al riesgo esperado, al riesgo empírico y a los baselines que se utilizarán en los capítulos de modelos.

Al terminar esta parte, las matrices, pérdidas y algoritmos posteriores tendrán un referente preciso: observaciones producidas por un procedimiento, una distribución que representa la población de interés y una muestra finita de la que depende cualquier resultado calculado.

Observaciones, variables y preguntas

Cómo definir los objetos de un problema con datos

La Secretaría Distrital del Hábitat publica una caracterización de precios de vivienda nueva en Bogotá. Para los ejemplos de este libro se congeló un subconjunto de 32 subzonas y siete columnas. Cinco filas tienen la forma siguiente.

subzona	precio reportado	estrato prom.	área prom.
Rosales	16.365	5.922	177.316
Chicó	13.939	5.905	128.044
Bosque Medina	10.751	4.699	151.226
Multicentro	10.365	5.647	94.071
Centro	9.504	3.195	63.383

subzona	alcobas prom.	baños prom.	parqueaderos prom.
Rosales	2.673	3.776	2.981
Chicó	2.225	3.105	2.152
Bosque Medina	2.533	3.037	2.467
Multicentro	2.086	2.625	1.786
Centro	1.744	1.799	0.988

La tabla puede leerse de varias maneras equivocadas. Una fila no representa una vivienda, sino una subzona. Las columnas numéricas son promedios o indicadores de esa subzona, no mediciones individuales. Además, el archivo fuente llama **precios** a una de las columnas, pero no documenta su unidad en el recurso descargado. Por esta razón usamos el nombre **precio_reportado** y no afirmamos que la cifra esté expresada en pesos, millones de pesos o precio por metro cuadrado.



Figura 1: Las 32 filas representan subzonas de Bogotá y no viviendas individuales. El eje horizontal usa el campo de precio reportado, cuya unidad no está documentada en el archivo fuente.

La figura permite comparar el orden y la dispersión del indicador dentro de las 32 filas. No permite valorar una vivienda particular ni completar una unidad de medida ausente. Esta diferencia entre lo que muestran los datos y lo que quisiéramos saber será recurrente a lo largo del libro.

Unidades de observación y niveles de análisis

Unidad de observación. La unidad de observación es la entidad, evento o intervalo representado por un registro. Puede ser una persona, una transacción, una imagen, una subzona, un paciente en una visita o el resultado de un experimento.

La descripción de la unidad debe incluir más que un sustantivo. También interesa el periodo de registro, el lugar, los criterios de inclusión y el procedimiento que produjo la fila. “Paciente” y “visita de un paciente” definen unidades distintas. Si una misma persona aparece en cinco visitas, las cinco filas no constituyen cinco personas independientes.

La **unidad de análisis** es la entidad sobre la cual se formula la conclusión. Con frecuencia coincide con la unidad de observación, pero no siempre. Una tabla puede registrar transacciones y utilizarse para estudiar comercios; puede registrar estudiantes y utilizarse para comparar colegios; puede registrar subzonas y pretender describir viviendas. Cuando las unidades no coinciden, es necesario especificar cómo se agregan los registros y qué información se pierde.

Ejemplo 1: registros hospitalarios. Un archivo contiene una fila por consulta. La pregunta “¿qué consultas terminarán en hospitalización?” tiene la consulta como unidad de observación y de análisis. La pregunta “¿qué pacientes tendrán al menos una hospitalización durante el año?” exige

agrupar consultas por paciente y fijar un intervalo temporal. Usar cada consulta como si fuera un paciente alteraría el denominador y podría colocar consultas de una misma persona en conjuntos distintos.

Identificadores y repetición

Un identificador permite reconocer la unidad registrada. No siempre es una variable útil para predecir. Un número de documento puede ser indispensable para detectar duplicados y, al mismo tiempo, ser inadecuado como entrada de un modelo.

Conviene distinguir:

- **identificador de registro**, que diferencia filas;
- **identificador de unidad**, que permite reconocer observaciones repetidas de la misma entidad;
- **identificador de grupo**, que señala hospital, colegio, región o dispositivo;
- **marca temporal**, que sitúa la observación y permite reconstruir el orden.

Estas columnas determinan dependencias que no se ven en una matriz numérica. Más adelante condicionarán la manera de construir muestras de entrenamiento y evaluación.

Agregación y falacia ecológica

En la tabla de Bogotá, **area_promedio** es un resumen de subzona. Una relación entre **area_promedio** y **precio_reportado** describe asociaciones entre subzonas. No se deduce de ella la relación correspondiente entre viviendas dentro de cada subzona.

Inferir un comportamiento individual a partir de datos agregados se conoce como **falacia ecológica**. El problema no se resuelve aumentando el número de subzonas: la información individual fue reemplazada por resúmenes. Tampoco puede recuperarse la variación interna de cada grupo a partir de un promedio aislado.

Ejemplo 2: dos subzonas con el mismo promedio. En una subzona, las áreas de dos viviendas pueden ser 80 y 120 metros cuadrados; en otra, 99 y 101. Ambas tienen promedio 100, pero su variación interna es diferente. Una tabla que conserva solo el promedio no permite distinguirlas.

Variables, valores y mediciones

Variable. Una variable es una característica que puede tomar distintos valores en las unidades consideradas. Su definición incluye un dominio de valores posibles y una regla, explícita o implícita, para asignar un valor a cada unidad.

Una columna es la representación computacional de los valores registrados para una variable. Variable y columna no son sinónimos perfectos. Una variable puede requerir varias columnas, como una fecha separada en año, mes y día; una columna puede ser un identificador técnico sin interpretación sustantiva; y una variable no observada directamente puede construirse a partir de otras mediciones.

Dominio y tipo de variable

El dominio describe qué valores admite una variable. Algunas categorías útiles son:

Tipo conceptual	Ejemplo	Operaciones con sentido
binaria	hospitalización: sí/no	proporción, comparación de categorías
categorica nominal	localidad	igualdad, conteos, frecuencias
categorica ordinal	nivel bajo/medio/alto	orden y comparaciones ordinales
conteo	número de visitas	suma, diferencia, razón bajo condiciones
continua	área construida	diferencias, promedios, transformaciones
temporal	fecha de venta	orden, duración, estacionalidad
texto, imagen o señal	descripción, radiografía, audio	representación específica del objeto

El tipo almacenado por un programa no decide el tipo conceptual. Un código postal puede aparecer como entero y seguir siendo categórico; convertirlo a `float` no hace razonable promediarlo. De forma inversa, un número guardado como texto puede representar una medición continua que necesita limpieza.

Escala y unidad

La unidad determina la interpretación de diferencias y proporciones. Una longitud en metros y la misma longitud en centímetros contienen la misma información, pero producen coeficientes numéricos diferentes. Una temperatura en grados Celsius admite diferencias con sentido, aunque la razón “20 es el doble de 10” no tenga la interpretación física usual de una escala con cero absoluto.

Por ello, antes de calcular conviene registrar:

1. definición de la variable;
2. unidad y escala;
3. instante o periodo al que se refiere;
4. instrumento o fuente;
5. transformaciones aplicadas;
6. códigos especiales y valores faltantes.

Medición y construcción

No toda variable es observada de manera directa. “Ingreso mensual” puede proceder de una declaración, un registro tributario o una imputación. “Satisfacción” puede ser la respuesta a una escala. “Riesgo” puede ser un índice construido a partir de varias columnas. Cada procedimiento define un objeto diferente, aunque los nombres se parezcan.

Procedimiento de medición. Es la regla mediante la cual una propiedad de una unidad se transforma en un valor registrado. Incluye instrumento, protocolo, momento, resolución, redondeo y tratamiento de casos no observables.

El error de medición es la diferencia entre el valor registrado y la cantidad que se pretendía medir. No es lo mismo que un residual de un modelo, que se calcula después de comparar una observación con una predicción. La distinción se retomará al estudiar modelos probabilísticos.

Datos faltantes

Un valor faltante no es necesariamente cero y tampoco constituye una categoría ordinaria. Puede faltar porque la medición no aplica, porque no se realizó, porque el participante no respondió o porque el sistema perdió el registro. Esas causas pueden contener información.

Antes de imputar o eliminar filas deben responderse al menos tres preguntas:

- ¿qué significa la ausencia en esta variable?;
- ¿puede conocerse la causa con las demás columnas?;
- ¿cambia la frecuencia de ausencia entre grupos, periodos o valores del resultado?

La teoría estadística de los mecanismos de ausencia requiere supuestos adicionales. En esta etapa basta con no ocultar la ausencia bajo una transformación automática.

El diccionario de datos

Un diccionario de datos conserva el significado que se pierde al convertir una tabla en un arreglo. Para cada columna debería indicar, cuando corresponda:

Campo	Pregunta que debe responder
nombre	¿cómo se identifica en el archivo?
descripción	¿qué propiedad representa?
dominio	¿qué valores son admisibles?
unidad	¿en qué escala se expresa?
procedencia	¿qué sistema o instrumento la produjo?
disponibilidad	¿en qué momento se conoce?
transformación	¿es original, agregada o derivada?
faltantes	¿qué códigos y causas de ausencia existen?

Para el subconjunto de Bogotá puede escribirse un diccionario parcial:

variable	papel	interpretación disponible	limitación principal
subzona	identificador	nombre de la subzona	no identifica viviendas
precio_reportado	posible salida	indicador llamado precios en la fuente	unidad no documentada

variable	papel	interpretación disponible	limitación principal
estrato_promedio	entrada	promedio reportado por subzona	oculta variación interna
area_promedio	entrada	área promedio reportada	tipo de área no documentado aquí
alcobas_promedio	entrada	promedio de alcobas	no es conteo de una vivienda
banos_promedio	entrada	promedio de baños	no es conteo de una vivienda
parqueaderos_promedio	entrada	promedio de parqueaderos	no es conteo de una vivienda

Este diccionario no completa información ausente. Su valor consiste precisamente en separar lo conocido de lo supuesto.

Población, marco de observación y muestra

Una tabla contiene unidades observadas. Para interpretar una conclusión también debemos precisar el conjunto más amplio al que se refiere.

Población objetivo. Es el conjunto de unidades, lugares y periodos sobre los que se desea formular una conclusión.

Marco de observación. Es el conjunto de unidades que el procedimiento de recolección podía identificar o incluir. Puede ser más pequeño o diferente de la población objetivo.

Muestra observada. Es el conjunto de unidades del marco que finalmente aparece en los datos después de aplicar criterios de inclusión, respuesta, disponibilidad y limpieza.

Estas tres colecciones pueden diferir. Si la población objetivo son todos los hogares de una ciudad y el marco contiene solo hogares con conexión fija a internet, ningún tamaño de muestra dentro de ese marco incluye a quienes quedaron fuera. El problema es de cobertura, no de precisión numérica.

La relación puede resumirse así:

población objetivo → marco de observación → muestra registrada → tabla analizada.

Cada flecha corresponde a un mecanismo: cobertura, selección, respuesta, medición o procesamiento. Una conclusión sobre la población requiere que esos mecanismos sean compatibles con la comparación que se pretende realizar.

Un censo también tiene alcance limitado

Observar todas las unidades de un marco en un instante elimina el error asociado a seleccionar solo algunas de ellas, pero no resuelve otros problemas. El marco puede excluir unidades, las mediciones pueden contener errores y la población futura puede cambiar. Un “censo” de transacciones de 2025 sigue siendo una muestra de los periodos posibles si la pregunta se refiere a 2026.

¿Qué población corresponde al caso de Bogotá?

La respuesta depende de la pregunta. Podríamos interesarnos por las 32 subzonas en el periodo documentado, por todas las subzonas de Bogotá, por proyectos de vivienda nueva o por viviendas particulares. La tabla no convierte automáticamente una de estas opciones en población objetivo. Además, no conocemos aquí el mecanismo de selección de las 32 subzonas ni la unidad del indicador. Por ello, el caso sirve para estudiar asociaciones entre los resúmenes disponibles, no para valorar viviendas individuales ni para describir sin reservas toda la ciudad.

El capítulo siguiente representará muestras y poblaciones mediante distribuciones de probabilidad. Esa formalización no reemplaza la descripción del marco: primero debe quedar claro de qué proceso podría ser muestra la tabla.

De una pregunta amplia a una pregunta precisa

“Analizar vivienda en Bogotá” expresa un tema, pero no una pregunta matemática. Una pregunta precisa debe indicar qué unidad se estudia, qué cantidad se quiere obtener y a qué población se refiere.

Preguntas descriptivas

Una pregunta descriptiva resume lo observado:

¿Cómo se distribuye `precio_reportado` entre las 32 subzonas del archivo?

La respuesta puede incluir tablas, gráficos, medias o cuantiles. Su alcance inmediato son las filas observadas. Extenderla a otras subzonas o periodos exige un argumento adicional sobre la manera en que se obtuvo la muestra.

Preguntas predictivas

Una pregunta predictiva busca anticipar un resultado que no estará disponible en el momento de aplicar la regla. Conviene especificar cinco elementos:

1. **unidad:** ¿para qué tipo de caso se produce la predicción?;
2. **salida:** ¿qué valor o categoría se quiere anticipar?;
3. **momento:** ¿cuándo se realiza la predicción y para qué horizonte?;
4. **información disponible:** ¿qué variables se conocen en ese instante?;
5. **población de uso:** ¿sobre qué casos se aplicará el procedimiento?

Una formulación compatible con el ejemplo sería:

Para una subzona descrita por los promedios disponibles, estimar el indicador `precio_reportado` y evaluar el procedimiento en subzonas comparables que no participaron en el ajuste.

Esta formulación no promete valorar viviendas ni explicar por qué cambia el indicador.

Preguntas causales

Una pregunta causal se refiere al efecto de una intervención o exposición:

¿Cómo cambiaría el resultado si, manteniendo lo demás apropiadamente comparable, se modificara una condición determinada?

Una asociación útil para predecir no identifica por sí sola ese efecto. Si `estrato_promedio` y `precio_reportado` cambian juntos, un modelo puede aprovechar la asociación sin que una modificación administrativa del estrato produzca el cambio predicho. Para sostener una afirmación causal se necesita definir la intervención, establecer qué comparación la representa y justificar supuestos sobre asignación, confusión y medición.

Una misma tabla admite preguntas diferentes

El conjunto `load_wine` de `scikit-learn` contiene mediciones químicas de vinos y tres clases de cultivar. La misma tabla permite formular, entre otras, estas preguntas:

- describir la distribución de una concentración en la muestra;
- predecir la clase de cultivar a partir de las mediciones;
- estimar una concentración continua usando las demás variables;
- explorar una representación de baja dimensión sin escoger una salida.

La tabla no determina por sí sola la tarea. El papel de una columna depende de la pregunta. Una variable puede ser entrada en un análisis y salida en otro.

Familias de tareas

Una vez definida la pregunta, la forma de la salida orienta la familia de métodos.

Pregunta	Salida matemática	Familia habitual
estimar una cantidad	número real	regresión
asignar una categoría	elemento de un conjunto finito	clasificación
estimar una probabilidad	vector de probabilidades	clasificación probabilística
ordenar alternativas	puntaje u orden	ranking y recomendación
representar estructura sin salida	coordenadas, grupos o factores	aprendizaje no supervisado

Pregunta	Salida matemática	Familia habitual
estimar efecto de una intervención	contraste entre resultados potenciales	inferencia causal

Las fronteras no siempre son absolutas. Una variable ordinal puede tratarse como categoría o como valor ordenado; un conteo puede modelarse mediante regresión con una distribución específica; una probabilidad estimada puede convertirse en una decisión después de fijar costos y umbrales. La elección debe corresponder al significado de la salida, no únicamente al tipo almacenado en el archivo.

De la tabla a la notación matemática

Sea una tabla con n registros y q variables registradas. Denotamos la fila i por

$$z_i = (z_{i1}, \dots, z_{iq}),$$

donde cada coordenada representa una variable registrada. La colección ordenada de registros es

$$d_n = (z_1, \dots, z_n).$$

Usamos aquí letras minúsculas porque los valores ya fueron observados. En el capítulo siguiente introduciremos las variables aleatorias Z_i y reservaremos $D_n = (Z_1, \dots, Z_n)$ para la muestra antes de observarla.

Problemas supervisados

Si una variable cumple el papel de salida, escribimos

$$z_i = (x_i, y_i),$$

donde x_i reúne la información de entrada y y_i es el resultado observado. Una transformación $\phi : \mathcal{X}_{\text{raw}} \rightarrow \mathbb{R}^p$ produce la representación numérica que utilizará el modelo. En los capítulos de regresión, su primera coordenada será uno para incluir el intercepto. Las observaciones se organizan como

$$\mathbf{X} = \begin{bmatrix} \phi(x_1^{\text{raw}})^\top \\ \vdots \\ \phi(x_n^{\text{raw}})^\top \end{bmatrix} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

La fila i de $\mathbf{X}$ y la entrada i de $\mathbf{y}$ deben referirse a la misma unidad. La forma de los arreglos es parte del contrato matemático:

- n es el número de observaciones;

- p es el número de columnas de la matriz de diseño, incluido el intercepto;
- $\phi(x_i^{\text{raw}}) \in \mathbb{R}^p$ describe la representación de una observación;
- y_i pertenece al espacio de salidas, que puede ser $\mathbb{R}$, un conjunto de clases u otro dominio.

Para el caso de Bogotá podríamos seleccionar cinco entradas:

```
features = [
    "estrato_promedio",
    "area_promedio",
    "alcobas_promedio",
    "banos_promedio",
    "parqueaderos_promedio",
]

import numpy as np

X_features = tabla[features].to_numpy()
X = np.column_stack([np.ones(len(tabla)), X_features])
y = tabla["precio_reportado"].to_numpy()

assert X.shape == (32, 6)
assert y.shape == (32,)
```

Tabla original y matriz de diseño

No toda tabla puede introducirse directamente en $\mathbb{R}^{n \times p}$. Las categorías necesitan una codificación; las fechas pueden transformarse en duraciones o componentes estacionales; el texto y las imágenes requieren una representación; los valores faltantes necesitan un tratamiento explícito.

Podemos escribir una transformación de características como

$$\phi : \mathcal{X}_{\text{raw}} \rightarrow \mathbb{R}^p, \quad \phi(x_i^{\text{raw}}) = [1 \quad \phi_1(x_i^{\text{raw}}) \quad \cdots \quad \phi_{p-1}(x_i^{\text{raw}})]^\top.$$

La matriz formada por los vectores $\phi(x_i^{\text{raw}})$ es una **matriz de diseño**. La transformación ϕ puede estar completamente fijada de antemano o contener cantidades aprendidas de los datos, como medias para estandarizar o categorías observadas. En el segundo caso, su ajuste forma parte del procedimiento estadístico y debe realizarse sin consultar observaciones reservadas.



La matriz no conserva toda la información

Al convertir una tabla a **numpy**, los valores y las dimensiones permanecen, pero pueden desaparecer nombres, unidades, identificadores y fechas. La representación numérica debe acompañarse de un diccionario y de las reglas que produjeron cada columna.

Modelo, algoritmo y resultado ajustado

En este libro un **modelo predictivo** es una familia de funciones

$$\mathcal{F} = \{f_\theta : \theta \in \Theta\}, \quad f_\theta : \mathcal{X} \longrightarrow \mathcal{Y}.$$

El vector θ identifica una función dentro de la familia. Por ejemplo, una familia lineal utiliza

$$f_\theta(x) = \phi(x)^\top \theta.$$

La primera coordenada de θ es el intercepto y las restantes son los coeficientes de las características representadas.

Un **algoritmo de ajuste** recibe datos y produce parámetros. Si lo denotamos por A , entonces

$$A(d_{\text{train}}) = \hat{\theta}.$$

La función $f_{\hat{\theta}}$ es el **modelo ajustado**. Para una entrada nueva x produce

$$\hat{y} = f_{\hat{\theta}}(x).$$

Estos objetos no deben confundirse:

Objeto	Ejemplo	Qué lo determina
familia de modelos	funciones lineales	decisión de modelación
parámetro	coeficientes θ	algoritmo y datos de ajuste
hiperparámetro	intensidad de regularización	procedimiento de selección
modelo ajustado	$f_{\hat{\theta}}$	familia, algoritmo y muestra
predicción	$f_{\hat{\theta}}(x)$	modelo ajustado y entrada nueva

Un hiperparámetro controla la familia o el algoritmo, pero no se obtiene en el ajuste ordinario de parámetros. Su elección también utiliza datos y, por tanto, debe incluirse al describir el procedimiento completo.

Ajustar y aplicar son operaciones distintas

Durante el ajuste,

$$d_{\text{train}} \mapsto \hat{\theta}.$$

Durante la aplicación,

$$x_{\text{nuevo}} \mapsto f_{\hat{\theta}}(x_{\text{nuevo}}),$$

con $\hat{\theta}$ ya fijado. Esta distinción permite preguntar qué información estaba disponible en cada momento. Si una variable solo se conoce después del resultado, no puede formar parte de x_{nuevo} en un sistema que intenta anticiparlo.

Un primer experimento computacional

El conjunto `load_wine` de `scikit-learn` contiene 178 análisis químicos y una etiqueta con tres clases. No será uno de los casos conductores del libro; se usa aquí porque está disponible sin conexión y permite observar completo un primer ajuste. Antes de elegir un modelo, los conteos de clase ya definen una regla de comparación.

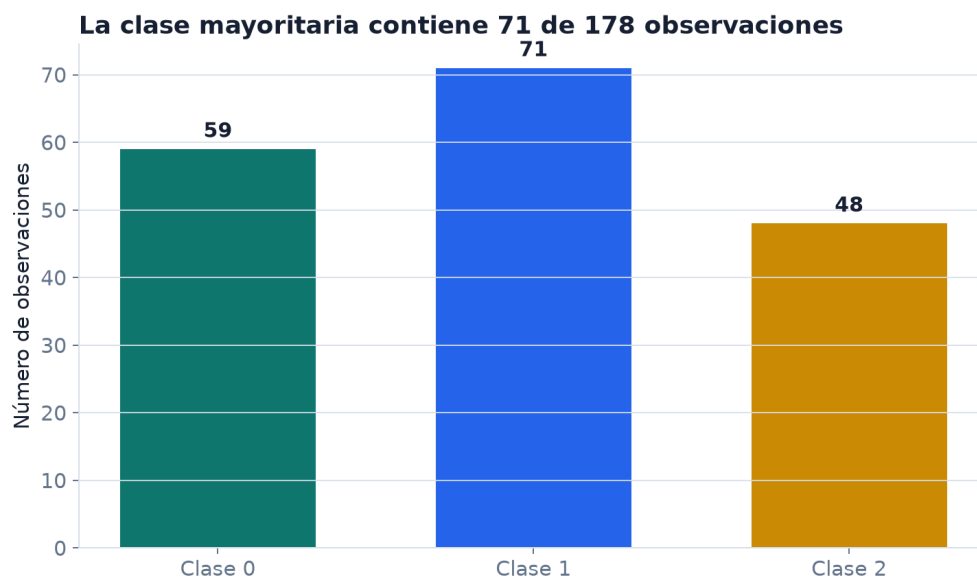


Figura 2: La clase mayoritaria contiene 71 de las 178 observaciones de Wine y define un baseline de clasificación antes de ajustar un modelo.

La regla que siempre anuncia la clase 1 alcanza $\text{accuracy } 71/178 \approx 0.399$ si se calcula sobre la tabla completa. En una evaluación correcta, la clase mayoritaria se determina solo con los datos de ajuste. Con una partición fijada de 133 observaciones para ajuste y 45 para prueba, un pipeline con

escalamiento y regresión logística acierta 44 casos. La matriz de confusión localiza el error que la cifra 44/45 no muestra.

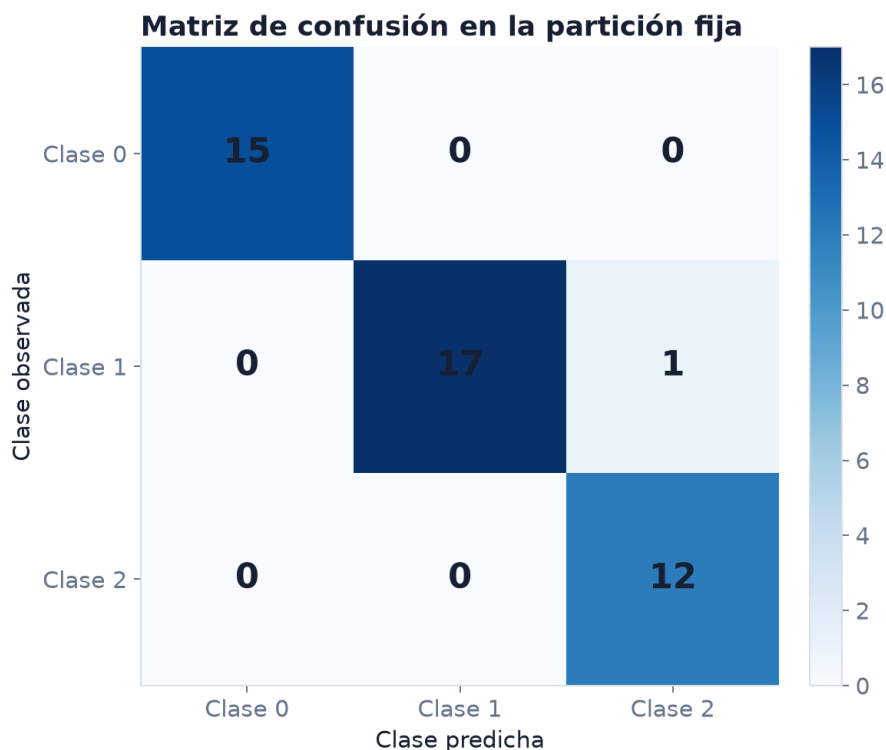


Figura 3: La matriz de confusión del primer modelo contiene 44 aciertos y un único error: una observación de clase 1 fue predicha como clase 2.

Este experimento separa cuatro objetos: la tabla observada, la regla baseline, el algoritmo que ajusta parámetros y el resultado sobre casos reservados. Todavía no explica por qué funciona la regresión logística ni cuánto variaría la métrica con otra muestra. Esas preguntas requieren la formulación probabilística de los capítulos siguientes.

Disponibilidad temporal y filtración de información

Para cada variable predictora conviene registrar un tiempo de disponibilidad. Si t_0 es el momento de predicción, una entrada válida debe estar disponible en t_0 bajo las condiciones reales de uso.

Filtración de información. Hay filtración, o *data leakage*, cuando el procedimiento utiliza para ajustar, seleccionar o representar un caso información que no estaría disponible en el momento de uso o que pertenece a observaciones reservadas para evaluación.

Ejemplos frecuentes son:

- predecir hospitalización con un código registrado al dar de alta;
- predecir fraude con el resultado de la investigación posterior;
- calcular una media de estandarización con toda la tabla antes de separar datos;

- imputar faltantes utilizando observaciones reservadas;
- permitir que registros de una misma persona aparezcan a ambos lados de una partición que pretende evaluar personas nuevas.

Una transformación determinista fijada de antemano, como elevar una medición al cuadrado, no consulta otras filas. En cambio, una transformación que estima medias, categorías, componentes principales o variables seleccionadas aprende de la muestra y debe tratarse como parte del algoritmo A.

Calidad de datos como propiedad de la pregunta

La calidad no es una propiedad absoluta del archivo. Una columna puede ser suficiente para una descripción agregada e insuficiente para una predicción individual. Por eso la revisión debe relacionarse con la pregunta.

Dimensión	Pregunta de revisión
cobertura	¿qué unidades de la población no podían aparecer?
selección	¿por qué unas unidades quedaron incluidas y otras no?
medición	¿qué se midió y con qué precisión?
completitud	¿qué valores faltan y por qué?
unicidad	¿hay duplicados o registros repetidos legítimos?
consistencia	¿las definiciones se mantienen entre fuentes y periodos?
temporalidad	¿cada variable estaba disponible al producir la salida?
granularidad	¿la unidad registrada coincide con la unidad de la pregunta?
trazabilidad	¿puede reconstruirse la transformación desde la fuente?

La revisión establece qué afirmaciones admite un conjunto de datos para la pregunta formulada y cuáles requerirían información diferente.

Análisis completo del caso de Bogotá

Podemos ahora describir el ejemplo sin atribuirle información que no contiene.

Unidad de observación. Una subzona de Bogotá resumida por la fuente.

Nivel de agregación. Cada fila contiene promedios o indicadores de la subzona; no hay registros de viviendas individuales.

Muestra disponible. Un subconjunto congelado de 32 subzonas. El archivo del libro conserva la fuente y las transformaciones en un manifiesto, pero no completa la unidad ausente de `precio_reportado`.

Pregunta descriptiva admisible. Comparar los valores registrados y las asociaciones entre columnas dentro de las 32 subzonas.

Pregunta predictiva posible. Estudiar si los promedios disponibles contienen información para anticipar `precio_reportado` en subzonas comparables, declarando que el tamaño es pequeño y que la población de uso debe justificarse.

Afirmaciones no sustentadas. Valorar una vivienda, interpretar el coeficiente como efecto causal, asignar una unidad al indicador o generalizar automáticamente a todos los proyectos de la ciudad.

El archivo puede inspeccionarse con:

```
import pandas as pd

ruta = "datos/vivienda_nueva_bogota_subzonas.csv"
tabla = pd.read_csv(ruta)

print(tabla.shape)
print(tabla.dtypes)
print(tabla.isna().sum())
print(tabla.head())
```

Estas operaciones verifican forma, tipos computacionales y faltantes. No verifican por sí solas la interpretación. Para ello siguen siendo necesarios la ficha de la fuente, el diccionario de datos y el análisis de la unidad de observación.

Una ficha mínima para iniciar un análisis

Antes de ajustar un modelo resulta útil escribir una ficha breve:

1. **Fuente y versión:** procedencia, fecha de descarga y licencia.
2. **Unidad de observación:** qué representa una fila y cómo se identifica.
3. **Población objetivo:** dónde y cuándo se quiere aplicar la conclusión.
4. **Marco y selección:** qué unidades podían aparecer y cómo fueron incluidas.
5. **Variables:** significado, dominio, unidad y tiempo de disponibilidad.
6. **Pregunta:** descriptiva, predictiva o causal; salida y uso previsto.
7. **Representación:** cómo se transformará la tabla en objetos matemáticos.
8. **Limitaciones iniciales:** información ausente, agregación, medición y posibles dependencias.

Esta ficha no reemplaza el análisis exploratorio. Evita que el análisis comience con objetos mal definidos y ofrece un punto de referencia cuando cambian las decisiones de modelación.

Síntesis

Una fila representa una unidad registrada mediante un procedimiento concreto. La unidad puede diferir de aquella sobre la que se desea concluir, especialmente en datos repetidos o agregados. Una columna representa valores observados de una variable, pero su nombre y su tipo computacional no sustituyen la definición, la unidad ni el mecanismo de medición.

La muestra disponible procede de un marco de observación y este puede diferir de la población objetivo. Cobertura, selección, medición y tiempo determinan el alcance de los datos. Una pregunta descriptiva resume lo observado; una pregunta predictiva anticipa una salida para casos de uso definidos; una pregunta causal compara intervenciones y requiere supuestos adicionales.

La traducción matemática organiza los registros como $d_n = (z_1, \dots, z_n)$ y, en un problema supervisado, como pares (x_i, y_i) . Una familia de modelos $\{f_\theta\}$, un algoritmo de ajuste A y un modelo ajustado $f_{\hat{\theta}}$ son objetos diferentes. Esta separación será esencial cuando la muestra misma se represente como aleatoria y se estudie la variación de los resultados obtenidos a partir de ella.

Problemas integrados

Registros, unidades y poblaciones

Los primeros ocho problemas utilizan distintos sistemas de registro. En cada caso debe identificarse la unidad antes de proponer cálculos o modelos.

1. Un archivo de transporte contiene una fila por validación de tarjeta. Proponga una pregunta cuya unidad de análisis sea la validación y otra cuya unidad sea la persona. ¿Qué identificadores necesitaría para la segunda?
2. Explique por qué 32 promedios de subzona no equivalen a 32 mediciones de viviendas. Construya dos conjuntos de valores individuales con el mismo promedio y distinta variación.
3. Dé un ejemplo en el que el tipo `int` de una columna no corresponda a una variable cuantitativa. Indique qué operaciones carecerían de sentido.
4. Distinga población objetivo, marco de observación y muestra en una encuesta enviada por correo institucional a estudiantes matriculados.
5. Explique por qué un censo de todas las ventas realizadas durante un mes puede seguir siendo insuficiente para una pregunta sobre ventas del año siguiente.
6. Formule sobre un mismo conjunto de datos una pregunta descriptiva, una predictiva y una causal. Identifique qué información adicional exige cada una.
7. ¿Por qué el papel de “variable objetivo” no es una propiedad permanente de una columna?
8. Distinga familia de modelos, algoritmo de ajuste, parámetro, hiperparámetro y predicción mediante un ejemplo de regresión.

De la tabla a una representación

En esta secuencia, además del resultado numérico, debe explicarse qué información conserva la representación y qué interpretación no puede recuperarse de ella.

9. Una tabla contiene 240 pacientes, 12 variables numéricas, 3 variables categóricas y una salida binaria. Si cada variable categórica se codifica con 4 columnas indicadoras, ¿cuál es la dimensión de la matriz de diseño sin contar intercepto? Declare cualquier supuesto necesario.
10. Escriba explícitamente la matriz de características sin intercepto y el vector de respuestas para las observaciones $(x_1, y_1) = ((1, 2), 5)$, $(x_2, y_2) = ((0, 3), 4)$ y $(x_3, y_3) = ((-1, 1), 0)$. Indique las dimensiones.
11. Una tabla registra 800 visitas de 300 pacientes. Explique por qué una partición de filas puede colocar información de un mismo paciente en ajuste y evaluación. Proponga una unidad apropiada para formar los grupos.
12. Para una variable de fecha t , proponga una transformación $\phi(t)$ que represente mes y día de la semana. Indique qué información se conserva y cuál se pierde.
13. Considere $f_\theta(x) = \theta_0 + x^\top \theta_{1:5}$ con $x \in \mathbb{R}^5$. ¿Cuántos parámetros escalares tiene el modelo? ¿Cambia ese número si hay 32 o 3200 observaciones?
14. Una variable se expresa primero en metros y luego en centímetros. Explique cómo cambia su columna y por qué la realidad medida no cambia. Anticipe qué ocurriría con el coeficiente de una regresión lineal que utiliza esa variable.

Auditoría de datos y código

Estas tareas deben resolverse como una auditoría breve: comprobación reproducible, hallazgo e implicación para el análisis.

15. Cargue el CSV de Bogotá. Verifique forma, nombres, tipos y faltantes. Produzca un diccionario de datos en una tabla separada; no complete unidades que la fuente no documenta.
16. Busque duplicados en `subzona`. Explique qué significaría encontrar uno y qué información necesitaría antes de eliminarlo.
17. Escriba una función que reciba una tabla, una lista de predictores y una salida, y verifique que las columnas existen, que la salida no está entre los predictores y que X y y tienen el mismo número de filas.
18. Con `load_wine`, identifique unidad de observación, salida, número de entradas y dominio de cada tipo de variable. Formule dos preguntas distintas que usen la misma tabla.
19. Construya un ejemplo pequeño donde una categoría nueva aparezca después de haber definido una codificación. Explique por qué la transformación debe especificar cómo tratar categorías desconocidas.

Formulación de un estudio

Cada respuesta debe terminar con una conclusión admisible y una afirmación que los datos descritos todavía no permitirían sostener.

20. Un hospital quiere predecir reingreso a 30 días. Para cada una de estas variables, determine si estaría disponible al momento del alta: diagnóstico inicial, medicación de alta, número de llamadas posteriores y código del reingreso. Justifique.
21. Una aplicación recoge datos solo de usuarios que aceptan compartir su ubicación. Describa la población objetivo, el marco observado y una posible diferencia sistemática entre usuarios incluidos y excluidos.
22. Redacte una pregunta predictiva completa para un problema de su interés. Debe indicar unidad, salida, momento, información disponible y población de uso.
23. Para el caso de Bogotá, escriba dos conclusiones admisibles y dos conclusiones que excedan la información disponible. Señale exactamente qué dato o supuesto falta en cada caso.
24. Prepare la ficha mínima de un conjunto de datos que considere para un proyecto. Identifique al menos una limitación de cobertura, una de medición y una de temporalidad.

Respuestas seleccionadas

2. Los conjuntos $(80, 120)$ y $(99, 101)$ tienen promedio 100, pero el primero presenta mucha mayor dispersión. El promedio no permite reconstruir la distribución interna de una subzona.

4. La población objetivo podría ser todo el estudiantado sobre el que se desea concluir. El marco contiene estudiantes matriculados con correo institucional activo. La muestra observada contiene quienes recibieron y respondieron la encuesta. Cada transición puede excluir unidades diferentes.

9. Bajo el supuesto de cuatro columnas por cada una de las tres variables categóricas, la dimensión es $240 \times (12 + 3 \cdot 4) = 240 \times 24$. Si se elimina una categoría de referencia por variable, la dimensión cambia a 240×21 .

10.

$$\mathbf{X}_{\text{features}} = \begin{bmatrix} 1 & 2 \\ 0 & 3 \\ -1 & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad y = \begin{bmatrix} 5 \\ 4 \\ 0 \end{bmatrix} \in \mathbb{R}^3.$$

13. El intercepto aporta un parámetro y $\beta \in \mathbb{R}^5$ aporta cinco; hay seis parámetros escalares. El número no depende del tamaño de muestra, aunque la información disponible para estimarlos sí cambia.

20. El diagnóstico inicial y la medicación de alta pueden estar disponibles al alta si el sistema los registra a tiempo. Las llamadas posteriores y el código de reingreso ocurren después y producirían filtración para una predicción realizada al alta.

Variables aleatorias, distribuciones y muestras

Cómo describir la variación antes y después de observar los datos

El archivo sintético de regresión del libro contiene 160 filas generadas con una semilla fija. En esa población, el promedio de `target` es 4.121. Si elegimos doce filas sin reemplazo, el promedio depende de cuáles hayan sido seleccionadas. Cuatro selecciones posibles producen:

selección	tamaño	promedio de <code>target</code>
1	12	2.916
2	12	4.600
3	12	4.467
4	12	4.065

No hay contradicción entre esos cuatro números. Cada uno resume una muestra distinta de la misma población. Antes de seleccionar las filas no sabemos cuál de esos promedios, ni cuál de muchos otros, obtendremos. Después de observar la muestra aparece un valor concreto.

Esta diferencia entre lo posible antes de observar y lo registrado después es el punto de entrada a la probabilidad. En los capítulos posteriores cambiarán los objetos calculados: en lugar de un promedio tendremos coeficientes, pérdidas o métricas. La fuente de variación será la misma: otra muestra puede producir otro resultado.

Antes y después de observar

Consideremos una unidad escogida de una población y la cantidad `target` que registraríamos. Antes de escogerla representamos esa cantidad mediante una variable aleatoria Y . Después de observar la unidad obtenemos una realización y .

Variable aleatoria y realización. Una variable aleatoria real es una función

$$Y : \Omega \longrightarrow \mathbb{R}$$

que asigna un valor numérico a cada resultado posible. Si ocurre el resultado ω_i , el valor observado es $y_i = Y(\omega_i)$.

La función Y no cambia al repetir el procedimiento; cambia el resultado ω_i al que se aplica. Por convención, las mayúsculas representan variables aleatorias y las minúsculas sus realizaciones. Una

columna $\mathbf{y} = (y_1, \dots, y_n)^\top$ contiene valores observados; Y representa la cantidad antes de observar una unidad nueva.

Si cada unidad tiene varias características, usamos un vector aleatorio

$$\mathbf{X} = (X_1, \dots, X_d) \in \mathbb{R}^d.$$

Una fila de un problema supervisado es una realización

$$\mathbf{z}_i = (x_i, y_i)$$

del par aleatorio $Z = (X, Y)$. La negrita distingue el predictor aleatorio de la matriz de diseño $\mathbf{X}$ que construiremos con varias filas.

Eventos y una formulación probabilística

Un modelo probabilístico se expresa mediante una terna

$$(\Omega, \mathcal{F}, \mathbb{P}).$$

El espacio Ω contiene los resultados posibles; $\mathcal{F}$ contiene los eventos a los que se asignará probabilidad; y $\mathbb{P}$ satisface no negatividad, $\mathbb{P}(\Omega) = 1$ y aditividad numerable sobre eventos disjuntos. Estas condiciones permiten calcular probabilidades de afirmaciones como $Y \leq 4$ o $X_1 > 0$.

No necesitaremos construir $\mathcal{F}$ en cada ejemplo. Su presencia aclara, sin embargo, que la probabilidad pertenece a resultados posibles antes de observarlos. El valor 4.242557 de una fila concreta ya no tiene una probabilidad por estimar: es una realización registrada.

La distribución

La distribución de Y describe con qué probabilidad aparece cada región de valores. Para un conjunto $B \subseteq \mathbb{R}$, escribimos

$$P_Y(B) = \mathbb{P}(Y \in B).$$

La función de distribución acumulada

$$F_Y(t) = \mathbb{P}(Y \leq t)$$

existe para toda variable aleatoria real y determina su distribución. Si Y es discreta, puede describirse mediante una función de masa

$$p_Y(y) = \mathbb{P}(Y = y), \quad \sum_y p_Y(y) = 1.$$

Si Y es continua y posee densidad f_Y , las probabilidades se obtienen integrando:

$$\mathbb{P}(a < Y \leq b) = \int_a^b f_Y(y) dy.$$

Una densidad puede tomar valores mayores que uno; lo que debe estar entre cero y uno es el área integrada sobre un conjunto. Para una variable continua usual, $\mathbb{P}(Y = y) = 0$ en cada punto aislado aunque los intervalos tengan probabilidad positiva.

Dos distribuciones que reaparecerán

Una variable Bernoulli con parámetro p toma valores 0 y 1:

$$\mathbb{P}(Y = 1) = p, \quad \mathbb{P}(Y = 0) = 1 - p.$$

Se utilizará para representar una clase binaria y también para representar si una predicción fue correcta. Una variable normal

$$Y \sim \mathcal{N}(\mu, \sigma^2)$$

es continua, simétrica alrededor de μ y está determinada por su media y su varianza. Será útil como modelo y como aproximación de ciertas distribuciones muestrales, pero no debe adoptarse por costumbre: valores extremos, asimetría o restricciones del dominio pueden contradecirla.

Esperanza y variación

La esperanza resume el promedio definido por una distribución. Para una variable discreta,

$$\mathbb{E}[Y] = \sum_y y p_Y(y),$$

y, para una variable continua con densidad,

$$\mathbb{E}[Y] = \int_{-\infty}^{\infty} y f_Y(y) dy,$$

si la integral existe. La esperanza no tiene que ser un valor que Y pueda tomar. Una variable Bernoulli solo toma 0 y 1, pero su esperanza es p .

La esperanza es lineal. Si las cantidades involucradas son integrables,

$$\mathbb{E}[aY + bW + c] = a\mathbb{E}[Y] + b\mathbb{E}[W] + c,$$

sin necesidad de independencia. Esta propiedad permitirá convertir un promedio de pérdidas observadas en un estimador de una pérdida esperada.

La varianza mide dispersión cuadrática alrededor de la media:

$$\text{Var}(Y) = \mathbb{E}[(Y - \mathbb{E}[Y])^2] = \mathbb{E}[Y^2] - \mathbb{E}[Y]^2.$$

Su raíz cuadrada es la desviación estándar. Dos distribuciones pueden tener la misma media y la misma varianza y ser muy diferentes; estos momentos son resúmenes, no descripciones completas.

Distribuciones conjuntas, independencia y covarianza

El par (X_1, X_2) posee una distribución conjunta. De ella se obtienen las distribuciones marginales de cada componente. Decir que X_1 y X_2 son independientes significa que, para conjuntos apropiados A y B ,

$$\mathbb{P}(X_1 \in A, X_2 \in B) = \mathbb{P}(X_1 \in A)\mathbb{P}(X_2 \in B).$$

La covarianza mide asociación lineal:

$$\text{Cov}(X_1, X_2) = \mathbb{E}[(X_1 - \mathbb{E}[X_1])(X_2 - \mathbb{E}[X_2])].$$

Si las variables tienen desviaciones estándar positivas, su correlación es

$$\text{Corr}(X_1, X_2) = \frac{\text{Cov}(X_1, X_2)}{\text{sd}(X_1)\text{sd}(X_2)}.$$

La independencia implica covarianza cero cuando existen segundos momentos. La conversas es falsa: una relación no lineal puede tener covarianza cero. Tampoco la covarianza distingue causalidad de asociación.

Para un vector aleatorio $X \in \mathbb{R}^d$, la matriz de covarianza

$$\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^\top], \quad \mu = \mathbb{E}[X],$$

reúne varianzas en la diagonal y covarianzas fuera de ella. Más adelante, PCA buscará direcciones determinadas por esta estructura.

En la población sintética de regresión, $\mathbf{x}_2$ fue construido a partir de $\mathbf{x}_1$ más ruido. Por ello ambas variables están asociadas. El hecho de conocer el mecanismo generador permite explicar esa asociación. En un dataset observacional, una covarianza por sí sola no revelaría el mecanismo.

De la población a una muestra

Representamos una muestra aleatoria de tamaño n mediante

$$D_n = (Z_1, \dots, Z_n).$$

Antes de observarla, D_n es aleatoria. Después obtenemos la realización

$$d_n = (z_1, \dots, z_n),$$

que puede almacenarse como una tabla. Escribir

$$Z_1, \dots, Z_n \stackrel{\text{iid}}{\sim} P$$

afirma dos cosas: las observaciones son independientes y todas tienen la misma distribución P . No es una propiedad del formato CSV; es un supuesto sobre el procedimiento que produjo las filas.

Mediciones repetidas de una persona, observaciones sucesivas de una serie y estudiantes dentro de un mismo colegio pueden depender entre sí. Una recolección que excluye sistemáticamente parte de la población puede producir observaciones con una distribución distinta de la población objetivo. En cualquiera de esos casos, la notación iid requiere modificación o una justificación limitada.

El caso de las subzonas de Bogotá

Las 32 subzonas del capítulo anterior no están documentadas como una muestra aleatoria de todas las subzonas, de todos los proyectos ni de las viviendas de Bogotá. Cada fila es además un resumen agregado. Podemos describir la distribución empírica de sus columnas, pero no convertirla automáticamente en una distribución poblacional de viviendas.

Esta distinción permanecería aunque el archivo tuviera 3200 subzonas. Un tamaño grande reduce algunas formas de variación muestral bajo un diseño apropiado; no repara una unidad equivocada ni un mecanismo de selección desconocido.

Distribución empírica y estadísticos

La distribución empírica asigna masa $1/n$ a cada observación:

$$\hat{P}_n = \frac{1}{n} \sum_{i=1}^n \delta_{z_i}.$$

Para una variable numérica, su función de distribución empírica es

$$\hat{F}_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i \leq t\}.$$

La media y la varianza empíricas son

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \quad s_y^2 = \frac{1}{n-1} \sum_{i=1}^n (y_i - \bar{y})^2.$$

La primera aproxima una esperanza bajo condiciones de muestreo. La segunda usa $n - 1$ para estimar la varianza poblacional sin sesgo bajo el modelo iid con segundo momento finito.

Estadístico. Un estadístico es una función de la muestra que no depende de cantidades poblacionales desconocidas:

$$T_n = T(D_n).$$

Como D_n es aleatoria, T_n también lo es. Su distribución bajo repetición del muestreo se llama distribución muestral. Después de observar d_n , el estadístico toma el valor $t_n = T(d_n)$. Los cuatro promedios que abrieron el capítulo son cuatro realizaciones del mismo estadístico.

Por qué se estabiliza un promedio

Supongamos que $Y_1, \dots, Y_n$ son iid, con media μ y varianza $\sigma^2 < \infty$. Entonces

$$\mathbb{E}[\bar{Y}_n] = \mu$$

y, por independencia,

$$\text{Var}(\bar{Y}_n) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n Y_i\right) = \frac{\sigma^2}{n}.$$

La desviación estándar del promedio decrece como $1/\sqrt{n}$. Para reducirla a la mitad se necesita, bajo estos supuestos, multiplicar el tamaño por cuatro.

Ley de los grandes números. Si $Y_1, Y_2, \dots$ son iid y tienen esperanza finita, entonces

$$\bar{Y}_n \xrightarrow{\mathbb{P}} \mathbb{E}[Y].$$

La ley justifica aproximar una esperanza mediante un promedio cuando el modelo de muestreo es apropiado. No corrige sesgo de selección, dependencia, errores de medición ni cambio de población.

El siguiente experimento repite el cálculo que abrió el capítulo:

```

import numpy as np
import pandas as pd

tabla = pd.read_csv("datos/regresion_lineal_sintetica.csv")

for n in [8, 20, 80]:
    medias = [
        tabla.sample(n=n, replace=True, random_state=2105 + b)["target"].mean()
        for b in range(1000)
    ]
    print(n, np.mean(medias), np.std(medias, ddof=1))

```

Al aumentar n , las medias simuladas se concentran alrededor del promedio de la población sintética. La simulación ilustra la proposición; no demuestra que las 32 subzonas de Bogotá sean iid.

Síntesis

Una variable aleatoria representa una cantidad antes de observarla y una realización es el valor registrado. Su distribución describe las posibilidades; la esperanza, la varianza y la covarianza resumen aspectos de esa distribución.

Una muestra aleatoria también es un objeto aleatorio. Por ello cambian sus promedios, covarianzas, coeficientes ajustados y métricas. La distribución empírica describe la tabla disponible; conectarla con una población exige un supuesto sobre muestreo o sobre el proceso que genera nuevas observaciones.

La ley de los grandes números explica por qué un promedio puede estabilizarse. Su conclusión depende de las condiciones del muestreo y no convierte un archivo grande en una muestra representativa.

Problemas integrados

1. Muestras de una población conocida

Use `regresion_lineal_sintetica.csv` como una población finita.

1. Calcule la media, varianza y cuartiles de `target` en las 160 filas.
2. Extraiga 500 muestras con reemplazo para cada tamaño $n \in \{8, 20, 80\}$ y calcule sus medias.
3. Compare la media y la desviación estándar de las tres distribuciones muestrales con $\sigma/\sqrt{n}$.
4. Repita el experimento con la mediana. Explique qué parte del argumento de la varianza del promedio ya no puede reutilizarse directamente.
5. Escriba una conclusión que distinga la población sintética, una muestra y el estadístico calculado.

2. Qué distribución describen las subzonas

Use `vivienda_nueva_bogota_subzonas.csv`.

1. Construya la distribución empírica de `area_promedio` y reporte media, desviación estándar y mediana.
2. Identifique la unidad de observación y explique por qué esos resúmenes no son la distribución de áreas de viviendas individuales.
3. Escriba una población respecto a la cual el promedio tendría sentido y la información de muestreo que faltaría para justificarla.
4. Describa una posible dependencia entre subzonas que haría dudosa la independencia.
5. Redacte una conclusión descriptiva admisible y una generalización que el archivo no permite.

3. Bernoulli, indicadores y promedios

Sea $Y_i \sim \text{Bernoulli}(0.3)$.

1. Demuestre que $\mathbb{E}[Y_i] = 0.3$ y $\text{Var}(Y_i) = 0.3(0.7)$.
2. Interprete $\bar{Y}_n$ como una proporción.
3. Calcule la desviación estándar teórica de esa proporción para $n = 25, 100, 400$.
4. Verifique los tres valores mediante 5000 simulaciones.
5. Explique por qué la concordancia con la teoría no prueba que una proporción observada en datos reales provenga de ensayos iid.

4. Covarianza no significa independencia

Sea X uniforme en $\{-1, 0, 1\}$ y $Y = X^2$.

1. Escriba la distribución conjunta de (X, Y) .
2. Calcule $\mathbb{E}[X]$, $\mathbb{E}[Y]$ y $\text{Cov}(X, Y)$.
3. Muestre que la covarianza es cero.
4. Pruebe que X y Y no son independientes.
5. Explique qué advertencia ofrece este ejemplo al interpretar una matriz de correlaciones antes de ajustar un modelo.

Respuestas seleccionadas

Problema 3. Como $Y_i^2 = Y_i$,

$$\mathbb{E}[Y_i] = 0.3, \quad \text{Var}(Y_i) = 0.3 - 0.3^2 = 0.21.$$

Por tanto,

$$\text{sd}(\bar{Y}_n) = \sqrt{0.21/n},$$

que vale aproximadamente 0.0917, 0.0458 y 0.0229 para los tres tamaños.

Problema 4. Se tiene $\mathbb{E}[X] = 0$, $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0$, luego la covarianza es cero. Sin embargo, Y queda determinado por X ; por ejemplo, $\mathbb{P}(Y = 0 \mid X = 0) = 1 \neq \mathbb{P}(Y = 0) = 1/3$. No son independientes.

Condicionamiento, estimación y riesgo

Qué aprende un modelo de una muestra y qué cantidad intenta aproximar

El conjunto sintético de regresión contiene 160 observaciones. Como conocemos el mecanismo que lo produjo, podemos tratar esas filas como una población finita y comparar dos reglas de predicción. La primera ignora las entradas y asigna a toda observación el promedio de `target`, que es 4.121. Su error cuadrático medio en la población es 5.277. La segunda utiliza la función con la que se generaron los datos,

$$m(x) = 4 + 2.5x_1 - 1.2x_2 + 0.7x_3.$$

Su error cuadrático medio es 0.289. La diferencia no proviene de que el segundo número haya sido calculado con más filas: ambas reglas se evaluaron sobre las mismas 160 observaciones. Proviene de usar información sobre $X = (X_1, X_2, X_3)$ para describir cómo cambia Y .

En datos reales no conocemos de antemano la función m . Tenemos una muestra y un procedimiento que produce una aproximación. Para entender qué significa ese ajuste debemos separar tres niveles: la distribución poblacional, el estimador que actúa sobre una muestra y el valor concreto obtenido con los datos observados. El condicionamiento y el riesgo permiten expresar esa separación.

Lo que cambia al conocer las entradas

Si A y B son eventos con $\mathbb{P}(B) > 0$, la probabilidad condicional de A dado B es

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

El denominador restringe el espacio de comparación a los resultados compatibles con B . Por ejemplo, una proporción calculada entre observaciones con $X_1 > 0$ aproxima una cantidad diferente de la proporción calculada en toda la población.

De esta definición se obtiene la regla del producto,

$$\mathbb{P}(A \cap B) = \mathbb{P}(A \mid B)\mathbb{P}(B),$$

y, si $B_1, \dots, B_K$ forman una partición del espacio, la ley de probabilidad total,

$$\mathbb{P}(A) = \sum_{k=1}^K \mathbb{P}(A | B_k) \mathbb{P}(B_k).$$

Estas identidades distinguen la frecuencia de un resultado dentro de cada grupo de la frecuencia con la que los grupos aparecen en la población.

Una prueba en una población de baja prevalencia. Supongamos que una condición está presente en el 1 % de una población. Una prueba resulta positiva en el 95 % de quienes tienen la condición y en el 5 % de quienes no la tienen. La fórmula de Bayes da

$$\mathbb{P}(C | +) = \frac{0.95(0.01)}{0.95(0.01) + 0.05(0.99)} \approx 0.161.$$

La sensibilidad de 0.95 no es la probabilidad de tener la condición después de un resultado positivo. La prevalencia también interviene.

Distribuciones y esperanzas condicionales

Para variables discretas X y Y , la distribución condicional puede escribirse como

$$p_{Y|X}(y | x) = \frac{p_{X,Y}(x, y)}{p_X(x)}, \quad p_X(x) > 0.$$

Si las variables poseen densidades, la razón correspondiente es

$$f_{Y|X}(y | x) = \frac{f_{X,Y}(x, y)}{f_X(x)}, \quad f_X(x) > 0.$$

Cuando X es continua, normalmente $\mathbb{P}(X = x) = 0$. La densidad condicional no se obtiene dividiendo por esa probabilidad puntual. En lo que sigue suponemos que existe una versión de la distribución condicional para los valores de interés.

Esperanza condicional. Si Y es integrable, la esperanza condicional de Y dado X se representa por

$$\mathbb{E}[Y | X].$$

En condiciones usuales es una función de X . Escribimos

$$m(x) = \mathbb{E}[Y | X = x]$$

para la función de regresión poblacional.

La palabra *regresión* no implica aquí que m sea lineal. La linealidad será una restricción de la familia de funciones con la que intentaremos aproximarla.

Dos identidades sitúan la media condicional dentro de la distribución conjunta. La ley de la esperanza total establece

$$\mathbb{E}[\mathbb{E}[Y \mid X]] = \mathbb{E}[Y],$$

y, si existe el segundo momento, la ley de la varianza total afirma

$$\text{Var}(Y) = \mathbb{E}[\text{Var}(Y \mid X)] + \text{Var}(\mathbb{E}[Y \mid X]).$$

El primer término mide la variación promedio que permanece al fijar las entradas. El segundo mide cuánto cambian las medias condicionales entre entradas. La identidad es probabilística; no convierte una asociación en una explicación causal.

Por qué aparece la media condicional en regresión

La comparación inicial entre el promedio y $m(x)$ puede formularse como un problema de optimización poblacional.

La media condicional minimiza la pérdida cuadrática. Si Y y $g(X)$ tienen segundo momento finito, entonces

$$\begin{aligned} \mathbb{E}[(Y - g(X))^2 \mid X] &= \text{Var}(Y \mid X) \\ &\quad + (\mathbb{E}[Y \mid X] - g(X))^2. \end{aligned}$$

Por tanto, $g^*(x) = m(x)$ minimiza el error cuadrático esperado entre todas las funciones admisibles.

Se suma y resta $m(X)$:

$$Y - g(X) = Y - m(X) + m(X) - g(X).$$

Al elevar al cuadrado y condicionar en X , el término cruzado tiene esperanza cero porque

$$\mathbb{E}[Y - m(X) \mid X] = 0.$$

Quedan la varianza condicional y un cuadrado no negativo. Este último se anula cuando $g(X) = m(X)$.

En la población sintética,

$$Y = m(X) + \varepsilon, \quad \mathbb{E}[\varepsilon \mid X] = 0.$$

El MSE 0.289 de la regla conocida no es un fallo del algoritmo: refleja la variación de ε que permanece incluso al conocer las entradas. El MSE 5.277 del promedio mezcla esa variación con las diferencias sistemáticas entre medias condicionales.

El análogo binario

Si $Y \in \{0, 1\}$, la media condicional coincide con una probabilidad:

$$\eta(x) = \mathbb{E}[Y \mid X = x] = \mathbb{P}(Y = 1 \mid X = x).$$

Una predicción probabilística $q \in (0, 1)$ puede evaluarse mediante la pérdida logarítmica

$$\ell(q, y) = -y \log q - (1 - y) \log(1 - q).$$

Condicionado en $X = x$, su esperanza es

$$L_x(q) = -\eta(x) \log q - (1 - \eta(x)) \log(1 - q).$$

Su derivada se anula únicamente en $q = \eta(x)$ y la segunda derivada es positiva. Así, la probabilidad condicional es también el objetivo poblacional natural de la log-loss. Una regla de clasificación requiere además un umbral y una descripción de los costos de los dos tipos de error; esa decisión se estudiará en el capítulo de regresión logística.

De la cantidad poblacional al valor calculado

El promedio de **target**, un coeficiente de regresión y el riesgo de una regla no son objetos del mismo tipo. Para distinguirlos conviene reservar tres palabras.

Estimando, estimador y estimación. Un *estimando* es una cantidad definida por la población, por ejemplo $\mu = \mathbb{E}[Y]$ o un parámetro θ^* que minimiza un riesgo. Un *estimador* es una función de la muestra, como

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n Y_i.$$

Una *estimación* es el valor que toma el estimador después de observar los datos, por ejemplo $\hat{\mu}_n = 4.600$ en una muestra particular.

Antes de observar $D_n = (Z_1, \dots, Z_n)$, el estimador $\hat{\mu}_n(D_n)$ es aleatorio. Repetir el muestreo produce una distribución muestral. Esa distribución describe la variación del procedimiento, no la distribución de los valores individuales de Y .

En regresión lineal, el estimando puede definirse sin suponer que la media condicional sea exactamente lineal:

$$\theta^* \in \arg \min_{\theta \in \mathbb{R}^p} \mathbb{E} \left[\frac{1}{2} (Y - \phi(X)^\top \theta)^2 \right],$$

donde $\phi(X)$ incluye el intercepto y las características utilizadas. El estimador de mínimos cuadrados reemplaza la esperanza por un promedio de muestra. El capítulo siguiente derivará su forma.

Sesgo, varianza y error cuadrático medio

Sea T un estimando escalar y $\hat{T}_n$ un estimador. Su sesgo es

$$\text{Bias}(\hat{T}_n; T) = \mathbb{E}[\hat{T}_n] - T.$$

El error cuadrático medio del estimador se descompone como

$$\mathbb{E}[(\hat{T}_n - T)^2] = \text{Var}(\hat{T}_n) + \text{Bias}(\hat{T}_n; T)^2.$$

Esta igualdad se obtiene escribiendo $\hat{T}_n - T = (\hat{T}_n - \mathbb{E}[\hat{T}_n]) + (\mathbb{E}[\hat{T}_n] - T)$ y observando que el término cruzado tiene esperanza cero.

Un estimador insesgado puede variar demasiado para ser útil, y un estimador con un sesgo pequeño puede tener menor error cuadrático medio. Esta tensión reaparece con Ridge y Lasso: se introduce sesgo para reducir la sensibilidad del ajuste a la muestra.

Estabilidad no equivale a pertinencia. Un estimador puede producir valores muy semejantes en muestras repetidas y, aun así, estimar una cantidad distinta de la pregunta sustantiva. Promediar `precio_reportado` en las 32 subzonas es un cálculo estable si se repite sobre las mismas filas; no estima por ello el precio medio de una vivienda en Bogotá. La unidad y el mecanismo de selección definen el estimando antes de que una fórmula pueda estimarlo.

Pérdida, riesgo y aprendizaje a partir de una muestra

Una pérdida compara una acción o predicción con el resultado observado. Para una regla f y una observación $Z = (X, Y)$ escribimos

$$\ell(f; Z) = \ell(f(X), Y).$$

Riesgo esperado. El riesgo de f bajo la distribución poblacional P es

$$R_P(f) = \mathbb{E}_P[\ell(f; Z)].$$

Es una propiedad conjunta de la regla, la pérdida y la población.

El subíndice P importa. Una misma regla puede tener riesgos distintos en dos poblaciones, aun cuando su código no cambie. Cuando la distribución sea clara escribiremos simplemente $R(f)$.

Como P suele ser desconocida, calculamos el riesgo empírico

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(f; z_i).$$

Para derivar mínimos cuadrados utilizaremos $\ell(f_\theta; z_i) = \frac{1}{2}(f_\theta(x_i) - y_i)^2$. Por tanto,

$$\widehat{R}_n(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2.$$

El MSE que se reporta como métrica es

$$\text{MSE}_n(\theta) = \frac{1}{n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 = 2\widehat{R}_n(\theta).$$

El factor $1/2$ no cambia el minimizador; simplifica el gradiente. Mantener la distinción evita comparar accidentalmente pérdidas con escalas diferentes.

El riesgo empírico de una regla fija estima su riesgo esperado. Si $Z_1, \dots, Z_n$ son iid y f se elige sin utilizar esa muestra, entonces

$$\mathbb{E}[\widehat{R}_n(f)] = R(f),$$

si la pérdida es integrable.

Por linealidad de la esperanza y porque cada Z_i tiene distribución P ,

$$\mathbb{E}[\widehat{R}_n(f)] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[\ell(f; Z_i)] = R(f).$$

La condición de que f sea fija es decisiva. Si el mismo conjunto de datos se usa para escoger f , la regla ajustada depende de cada Z_i . La demostración anterior ya no se aplica directamente y el error de entrenamiento puede ser optimista. Las particiones de entrenamiento, validación y prueba se introducirán cuando la selección de complejidad haga visible este conflicto.

Minimización empírica y baseline

Una familia de modelos $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$ convierte el aprendizaje en el problema

$$\widehat{\theta} \in \arg \min_{\theta \in \Theta} \widehat{R}_n(\theta).$$

El algoritmo busca un minimizador o una aproximación; los datos determinan cuál miembro de la familia resulta elegido. Cambiar la muestra puede cambiar $\widehat{\theta}$ aunque el algoritmo sea idéntico.

Un baseline es una regla sencilla que fija una comparación mínima. Bajo pérdida cuadrática, la mejor predicción constante en una muestra es $\bar{y}$. Demostrarlo requiere minimizar

$$\frac{1}{n} \sum_{i=1}^n (c - y_i)^2$$

respecto de c , cuya derivada se anula en $c = \bar{y}$. Un modelo que no mejora esta regla no ha utilizado de manera efectiva las entradas.

Cuatro muestras, cuatro ajustes

Tomemos cuatro muestras de 40 filas de la población sintética y ajustemos por mínimos cuadrados un intercepto y tres coeficientes. Los resultados son:

semilla	intercepto	coeficiente 1	coeficiente 2	coeficiente 3	MSE muestra	MSE población
11	4.069	2.487	-1.220	0.790	0.324	0.297
29	4.203	2.468	-1.025	0.717	0.334	0.344
47	4.094	2.569	-1.313	0.640	0.292	0.308
83	3.941	2.411	-1.135	0.633	0.237	0.311

La cuarta muestra obtiene el menor MSE de entrenamiento, pero no el menor MSE en la población. Los coeficientes varían porque cada ajuste es una realización del estimador. En este ejemplo podemos calcular el MSE poblacional porque conservamos las 160 filas; en una aplicación ordinaria esa cantidad permanece desconocida.

El cálculo se reproduce con:

```
import numpy as np
import pandas as pd

tabla = pd.read_csv("datos/regresion_lineal_sintetica.csv")
X_poblacion = np.column_stack(
    [np.ones(len(tabla)), tabla[["x1", "x2", "x3"]].to_numpy()]
)
y_poblacion = tabla["target"].to_numpy()

for semilla in [11, 29, 47, 83]:
    muestra = tabla.sample(n=40, replace=False, random_state=semilla)
    X = np.column_stack(
        [np.ones(len(muestra)), muestra[["x1", "x2", "x3"]].to_numpy()]
    )
    y = muestra["target"].to_numpy()
    theta = np.linalg.lstsq(X, y, rcond=None)[0]
    mse_muestra = np.mean((X @ theta - y) ** 2)
    mse_poblacion = np.mean((X_poblacion @ theta - y_poblacion) ** 2)
    print(semilla, theta, mse_muestra, mse_poblacion)
```

El ejemplo separa dos preguntas. La optimización determina qué parámetro reduce la pérdida en la muestra. La evaluación pregunta cómo se comporta la regla fuera de las observaciones utilizadas para ajustarla. El primer problema conduce a mínimos cuadrados y descenso del gradiente; el segundo, a validación y análisis de incertidumbre.

Regreso a las subzonas de Bogotá

Podríamos usar `area_promedio`, `estrato_promedio` y otras columnas para predecir `precio_reportado`, calcular residuales y comparar el resultado con el baseline de la media. El procedimiento sería matemáticamente válido para las 32 filas. Su interpretación seguiría limitada por dos hechos anteriores al ajuste:

1. una fila representa una subzona y no una vivienda;
2. la unidad de `precio_reportado` no está documentada en el recurso congelado.

Reducir el MSE describiría una asociación entre indicadores de las subzonas incluidas. No estimaría automáticamente el precio de una vivienda ni justificaría una conclusión para todas las subzonas futuras. El riesgo siempre se refiere a una distribución y a una pérdida; si la población de uso no está definida, un número de evaluación queda incompleto.

Síntesis

El condicionamiento describe cómo cambia la distribución de la respuesta cuando se conocen las entradas. Bajo pérdida cuadrática, la media condicional es la predicción poblacional óptima; para una respuesta binaria, esa misma media es la probabilidad condicional de la clase positiva.

Un estimando pertenece a la población, un estimador actúa sobre una muestra y una estimación es el valor observado. La variación entre muestras se transmite a los coeficientes y a las predicciones. El riesgo esperado expresa el objetivo poblacional y el riesgo empírico proporciona una cantidad calculable. Ambos solo coinciden de manera controlada bajo supuestos sobre el muestreo y sobre cómo se eligió la regla.

El capítulo siguiente comenzará con los residuales de una regresión de una sola variable. Allí el riesgo empírico cuadrático se convertirá en una función explícita de θ y podremos derivar el algoritmo de mínimos cuadrados.

Problemas integrados

1. De un baseline a una función condicional

Use `regresion_lineal_sintetica.csv` como población finita.

1. Calcule el promedio de `target` y el MSE del predictor constante.
2. Calcule las predicciones de $m(x) = 4 + 2.5x_1 - 1.2x_2 + 0.7x_3$ y su MSE.
3. Use la descomposición de varianza total para explicar conceptualmente la diferencia entre ambos errores.
4. Extraiga 200 muestras de tamaño 40, ajuste por `np.linalg.lstsq` y conserve los cuatro coeficientes de cada ajuste.
5. Grafique sus distribuciones muestrales y compare sus promedios con $(4, 2.5, -1.2, 0.7)$.
6. Para cada ajuste, compare el MSE de la muestra con el MSE en las 160 filas. Explique por qué el menor error de entrenamiento no identifica necesariamente el mejor ajuste poblacional.

2. Una media marginal puede ocultar grupos

Una población contiene dos grupos. Se tiene $\mathbb{P}(X = 0) = 0.7$, $\mathbb{P}(X = 1) = 0.3$, $\mathbb{E}[Y | X = 0] = 2$ y $\mathbb{E}[Y | X = 1] = 8$.

1. Calcule $\mathbb{E}[Y]$.
2. Compare el riesgo cuadrático del predictor constante $g_0(x) = \mathbb{E}[Y]$ con el de $g_1(x) = \mathbb{E}[Y | X = x]$.
3. Expresé la diferencia mediante $\text{Var}(\mathbb{E}[Y | X])$.
4. Suponga ahora que los grupos aparecen en proporciones 0.4 y 0.6 en la población de uso. Identifique qué cantidades cambian aunque las medias condicionales permanezcan iguales.
5. Explique qué información adicional necesitaría para calcular los dos riesgos numéricamente.

3. Probabilidad, log-loss y umbral

Para una entrada fija se sabe que $\eta(x) = \mathbb{P}(Y = 1 | X = x) = 0.8$.

1. Calcule la log-loss condicional esperada para $q = 0.8$, $q = 0.6$ y $q = 0.95$.
2. Verifique mediante derivación que el mínimo ocurre en $q = 0.8$.
3. Compare las clases producidas por umbrales 0.5 y 0.9.
4. Construya un ejemplo de costos en el que sea razonable usar cada umbral.
5. Explique por qué estimar una probabilidad y escoger una acción son problemas relacionados, pero distintos.

4. Qué se estaría estimando en Bogotá

Use `vivienda_nueva_bogota_subzonas.csv`.

1. Defina con precisión el estimando correspondiente al promedio de `precio_reportado` si las 32 filas se consideran una población finita.
2. Proponga un estimando diferente referido a todas las viviendas nuevas de Bogotá y enumere la información que falta para conectarlo con el archivo.
3. Ajuste una regresión de `precio_reportado` sobre `area_promedio` y compare su MSE con el baseline constante.
4. Examine el gráfico de residuales e identifique una observación que merezca revisión documental.
5. Redacte una conclusión admisible sobre las 32 subzonas y otra conclusión que los datos no permiten sostener.

Respuestas seleccionadas

Problema 2. La esperanza marginal es

$$\mathbb{E}[Y] = 0.7(2) + 0.3(8) = 3.8.$$

La diferencia entre el riesgo del predictor constante y el de la media condicional es

$$\text{Var}(\mathbb{E}[Y \mid X]) = 0.7(2 - 3.8)^2 + 0.3(8 - 3.8)^2 = 7.56.$$

Para obtener cada riesgo por separado hace falta conocer además $\mathbb{E}[\text{Var}(Y \mid X)]$.

Problema 3. El riesgo condicional es

$$L(q) = -0.8 \log q - 0.2 \log(1 - q).$$

Sus valores aproximados son 0.500 para $q = 0.8$, 0.592 para $q = 0.6$ y 0.641 para $q = 0.95$. El umbral 0.5 asigna la clase positiva y el umbral 0.9 la clase negativa; ninguna de esas acciones modifica la probabilidad estimada.

Parte II: Regresión, optimización y clasificación

Predecir una cantidad numérica obliga a distinguir dos problemas. Con los datos observados podemos minimizar una suma de errores; fuera de ellos queremos que la regla conserve un desempeño adecuado en nuevas observaciones. La regresión lineal permite estudiar ambos problemas con una combinación especialmente clara de probabilidad, álgebra lineal y cálculo.

El primer capítulo presenta la media condicional como objetivo poblacional bajo pérdida cuadrática y los mínimos cuadrados como su aproximación dentro de una familia lineal. El segundo convierte el gradiente de esa pérdida en un algoritmo iterativo y estudia cómo el escalamiento y el condicionamiento afectan su convergencia. Cuando la familia de modelos se vuelve más flexible, la variación entre muestras conduce de manera natural a regularización y validación.

El último modelo de esta parte cambia una respuesta numérica por una etiqueta. La regresión logística introduce entonces una probabilidad condicional, un modelo Bernoulli y la log-loss. El modelo produce una probabilidad estimada; la decisión final requiere además una métrica, un umbral y una descripción de los errores que resultan relevantes en el problema.

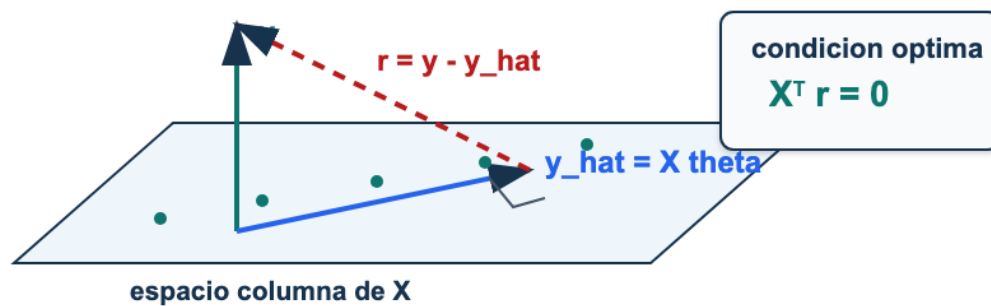


Figura 4: Geometría de mínimos cuadrados: las predicciones forman una proyección de y sobre el espacio columna de X ; el residual queda perpendicular a ese espacio.

La figura muestra la geometría del ajuste lineal. La predicción se encuentra en el espacio generado por las columnas de la matriz de diseño y el residual queda perpendicular a ese espacio. Esta interpretación geométrica reaparecerá al estudiar proyecciones, regularización y reducción de dimensión.

Regresión lineal y mínimos cuadrados

De una recta y sus residuales a la formulación matricial

El conjunto sintético de regresión fue generado con tres variables y una respuesta numérica. Las primeras filas son:

id	x1	x2	x3	target
reg-001	1.352	2.206	1.344	5.723
reg-002	-0.342	-0.375	1.180	4.243
reg-003	-1.743	-1.013	0.331	1.630
reg-004	0.074	-0.793	0.397	6.130
reg-005	0.536	0.705	1.965	6.755
reg-006	0.547	1.383	-0.070	3.768

Empecemos ignorando **x2** y **x3**. Queremos aproximar **target** mediante una recta

$$\hat{y}_i = \theta_0 + \theta_1 x_{i1}.$$

La recta no pasará por todos los puntos. La diferencia

$$r_i = y_i - \hat{y}_i$$

es el residual de la observación i . El problema no consiste en borrar los residuales, algo imposible en general, sino en fijar una regla para compararlos y encontrar los parámetros que optimizan esa regla.

Una recta elegida por mínimos cuadrados

Para n observaciones definimos

$$\hat{R}_n(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (\theta_0 + \theta_1 x_{i1} - y_i)^2.$$

El factor $1/2$ simplifica las derivadas y no cambia el minimizador. Cuando se reporte el error cuadrático medio usaremos

$$\text{MSE}_n = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = 2\hat{R}_n.$$

Derivar respecto del intercepto y de la pendiente produce

$$\frac{\partial \hat{R}_n}{\partial \theta_0} = \frac{1}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x_{i1} - y_i),$$

$$\frac{\partial \hat{R}_n}{\partial \theta_1} = \frac{1}{n} \sum_{i=1}^n x_{i1} (\theta_0 + \theta_1 x_{i1} - y_i).$$

En un mínimo, ambas derivadas se anulan. La primera condición dice que los residuales suman cero; la segunda, que su suma ponderada por x_{i1} también es cero. Si los valores de x_{i1} no son todos iguales, al resolver las dos ecuaciones se obtiene

$$\hat{\theta}_1 = \frac{\sum_{i=1}^n (x_{i1} - \bar{x}_1)(y_i - \bar{y})}{\sum_{i=1}^n (x_{i1} - \bar{x}_1)^2}, \quad \hat{\theta}_0 = \bar{y} - \hat{\theta}_1 \bar{x}_1.$$

Esta fórmula muestra por qué la pendiente depende de la covariación entre la entrada y la respuesta, medida en relación con la variación de la entrada.

Al usar las 160 filas se obtiene

$$\hat{y} = 4.055 + 1.652x_1,$$

con MSE 2.004. El baseline que predice siempre $\bar{y} = 4.121$ tiene MSE 5.277. La variable **x1** reduce el error dentro de esta población sintética, pero la recta aún deja variación sustancial.

Una pendiente no recupera por sí sola el mecanismo generador. El coeficiente 1.652 no coincide con el valor 2.5 utilizado para generar **target**. La variable **x2** está correlacionada con **x1** y también interviene en la respuesta. Al omitirla, la pendiente de la regresión simple resume una asociación marginal diferente del coeficiente parcial del mecanismo con tres variables.

Del problema escalar a la matriz de diseño

Una regresión con varias características conserva la misma idea. Para cada observación construimos

$$\phi(x_i) = [1 \quad x_{i1} \quad \cdots \quad x_{i,p-1}]^\top \in \mathbb{R}^p.$$

La primera coordenada representa el intercepto. Al apilar las filas obtenemos

$$\mathbf{X} = \begin{bmatrix} \phi(x_1)^\top \\ \vdots \\ \phi(x_n)^\top \end{bmatrix} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

El vector de parámetros y las predicciones son

$$\theta = [\theta_0 \quad \theta_1 \quad \cdots \quad \theta_{p-1}]^\top, \quad \hat{\mathbf{y}} = \mathbf{X}\theta.$$

La matriz de diseño no tiene que coincidir con las columnas originales de la tabla. Puede contener transformaciones, indicadores e interacciones. Lo que sí debe permanecer es la correspondencia entre la fila i de $\mathbf{X}$ y el valor y_i .

objeto	dimensión	interpretación
$\mathbf{X}$	$n \times p$	diseño, incluido el intercepto
θ	p	parámetros
$\mathbf{X}\theta$	n	una predicción por observación
$\mathbf{y}$	n	respuestas observadas
$\mathbf{X}^\top \mathbf{X}$	$p \times p$	productos entre columnas del diseño
$\mathbf{X}^\top \mathbf{y}$	p	productos entre columnas y respuesta

En código, la construcción explícita es:

```
import numpy as np

def add_intercept(X):
    X = np.asarray(X, dtype=float)
    if X.ndim != 2:
        raise ValueError("X debe ser una matriz bidimensional")
    return np.column_stack([np.ones(X.shape[0]), X])

X_design = add_intercept(X_raw)
y_hat = X_design @ theta
```

La comprobación de dimensiones antecede a cualquier interpretación estadística: si `X_design @ theta` no produce una predicción por fila, el objeto matemático no ha sido construido correctamente.

La pérdida cuadrática en notación matricial

El vector de residuales se define con la convención

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\theta.$$

La función que minimizaremos es

$$\widehat{R}_n(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2.$$

Para el ejemplo pequeño

$$x = (0, 1, 2, 3), \quad y = (1.1, 2.9, 5.2, 6.8),$$

cada par (θ_0, θ_1) determina una recta y un valor de la pérdida.

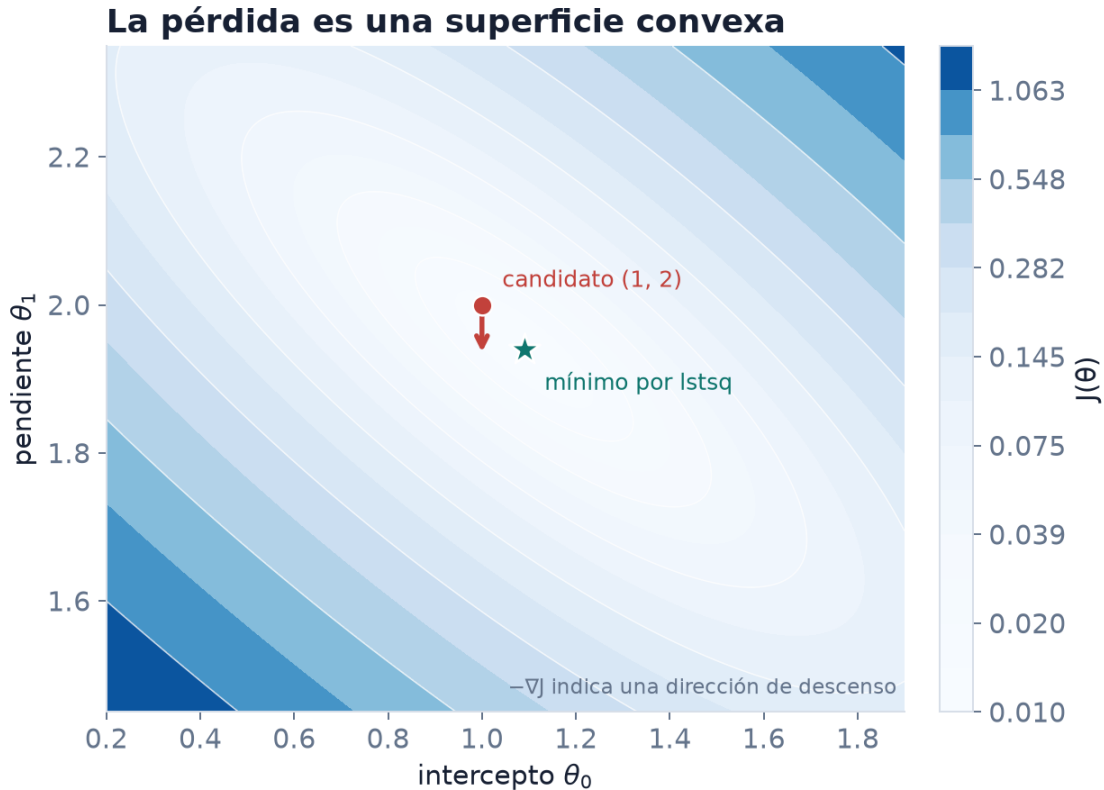


Figura 5: La pérdida cuadrática del ejemplo es convexa. El punto candidato queda cerca del mínimo calculado con `lstsq`, y el gradiente local señala cómo reducir la pérdida.

Los contornos agrupan parámetros con la misma pérdida. La convexidad permitirá caracterizar el mínimo mediante el gradiente, pero no determina cuál método numérico será más estable para calcularlo.

Gradiente y ecuaciones normales

Expandimos la forma cuadrática:

$$\widehat{R}_n(\theta) = \frac{1}{2n} (\theta^\top \mathbf{X}^\top \mathbf{X} \theta - 2\mathbf{y}^\top \mathbf{X} \theta + \mathbf{y}^\top \mathbf{y}).$$

Como $\mathbf{X}^\top \mathbf{X}$ es simétrica,

$$\nabla \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}).$$

Una implementación que conserva exactamente esa normalización es:

```
def half_mse_loss(X, y, theta):  
    error = X @ theta - y  
    return 0.5 * np.mean(error ** 2)  
  
def mse_gradient(X, y, theta):  
    n = X.shape[0]  
    return X.T @ (X @ theta - y) / n
```

Condición de mínimos cuadrados. Todo minimizador $\hat{\theta}$ de la pérdida cuadrática satisface

$$\mathbf{X}^\top \mathbf{X} \hat{\theta} = \mathbf{X}^\top \mathbf{y}.$$

Si $\mathbf{X}$ tiene rango columna completo, el minimizador es único.

La Hessiana es

$$\nabla^2 \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top \mathbf{X},$$

que es semidefinida positiva. Por tanto, cualquier punto donde el gradiente se anula es un mínimo global. Igualar el gradiente a cero produce las ecuaciones normales. Si $\mathbf{X}$ tiene rango columna completo, la Hessiana es definida positiva y el mínimo es único.

Hasta aquí $\mathbf{X}$ y $\mathbf{y}$ pueden verse como matrices ya observadas. Si las filas provienen de una muestra aleatoria, entonces $\hat{\theta}$ también cambia de una muestra a otra: es un estimador aleatorio. El objeto poblacional que motiva el ajuste es la media condicional $\mathbb{E}[Y \mid X = x]$; una función lineal la representa exactamente solo bajo supuestos adicionales. Las ecuaciones normales caracterizan el óptimo empírico, no demuestran por sí solas que la relación poblacional sea lineal ni que el ajuste se generalice.

Cuando $\mathbf{X}^\top \mathbf{X}$ es invertible, la identidad algebraica

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

describe la solución. No es la receta numérica recomendada.

Cómo calcular la solución

Formar una inversa explícita añade trabajo y error de redondeo. Además, en norma 2 y bajo rango columna completo,

$$\kappa_2(\mathbf{X}^\top \mathbf{X}) = \kappa_2(\mathbf{X})^2.$$

Las ecuaciones normales elevan al cuadrado el número de condición. Una factorización QR o una SVD trabaja directamente con $\mathbf{X}$. En NumPy, `lstsq` ofrece una interfaz robusta y también devuelve el rango y los valores singulares:

```
def fit_lstsq(X, y):
    theta, residual_sum, rank, singular_values = np.linalg.lstsq(
        X, y, rcond=None
    )
    return theta, rank, singular_values
```

método	objeto principal	costo denso si $n \geq p$	observación
ecuaciones normales	$\mathbf{X}^\top \mathbf{X}$	$O(np^2 + p^3)$	empeora el condicionamiento
QR	$\mathbf{X} = \mathbf{QR}$	$O(np^2)$	estable con rango completo
SVD / <code>lstsq</code>	$\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top$	$O(np^2)$	revela rango y solución de norma mínima

La fórmula cerrada explica el estimador; `lstsq` calcula la solución sin formar $\mathbf{X}^\top \mathbf{X}$. Esta distinción entre identidad matemática y método numérico reaparecerá en otros modelos.

Un cálculo completo con cuatro puntos

Para los cuatro valores anteriores,

```
x = np.array([0, 1, 2, 3], dtype=float)
y = np.array([1.1, 2.9, 5.2, 6.8], dtype=float)
X = add_intercept(x.reshape(-1, 1))

theta, rank, singular_values = fit_lstsq(X, y)
y_hat = X @ theta
residual = y - y_hat
mse = np.mean(residual ** 2)
orthogonality = X.T @ residual
```

El resultado es

$$\hat{\theta} = \begin{bmatrix} 1.09 \\ 1.94 \end{bmatrix}, \quad \mathbf{r} = [0.01 \quad -0.13 \quad 0.23 \quad -0.11]^\top,$$

con MSE 0.0205. Salvo error de redondeo, $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$.

Proyección y diagnóstico de residuales

La predicción $\hat{\mathbf{y}} = \mathbf{X}\hat{\theta}$ pertenece al espacio generado por las columnas de $\mathbf{X}$. Las ecuaciones normales equivalen a

$$\mathbf{X}^\top (\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}.$$

El residual queda perpendicular al espacio columna.

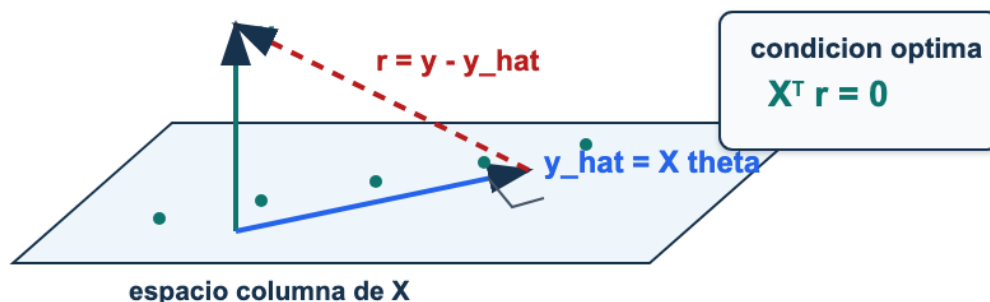


Figura 6: Geometría de mínimos cuadrados: las predicciones forman una proyección de $\mathbf{y}$ sobre el espacio columna de $\mathbf{X}$; el residual queda perpendicular a ese espacio.

La ortogonalidad es una propiedad algebraica de los datos utilizados en el ajuste. No implica que los residuales sean independientes, que tengan varianza constante ni que carezcan de estructura no lineal. Un gráfico de residual contra predicción puede revelar curvatura, dispersión creciente u observaciones influyentes incluso cuando $\mathbf{X}^\top \mathbf{r}$ es numéricamente cero.

Con las tres variables sintéticas, `lstsq` produce

$$\hat{\theta} = [4.043 \quad 2.491 \quad -1.201 \quad 0.715]^\top$$

y MSE 0.287. Los valores están cerca del mecanismo generador $(4, 2.5, -1.2, 0.7)$ y mejoran tanto la recta con $\mathbf{x}_1$ como el baseline. Esto es posible porque el caso sintético conserva las variables utilizadas para generar la respuesta; no debe esperarse la misma coincidencia en datos observacionales.

Rango, colinealidad e interpretación

Si una columna de $\mathbf{X}$ es combinación lineal de otras, el rango es menor que p . Las predicciones de mínimos cuadrados siguen estando bien definidas como proyección, pero puede haber varios vectores de parámetros que producen la misma predicción.

```
x1 = np.array([1, 2, 3, 4], dtype=float)
x2 = 2 * x1
X = np.column_stack([np.ones_like(x1), x1, x2])

assert np.linalg.matrix_rank(X) < X.shape[1]
```

Con colinealidad aproximada, la solución puede ser única y, sin embargo, muy sensible a pequeñas variaciones de los datos. Coeficientes grandes de signos opuestos, cambios bruscos entre muestras y un número de condición alto son señales para revisar.

En el modelo

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_{p-1} x_{p-1},$$

θ_j mide el cambio de la predicción cuando x_j aumenta una unidad y las otras columnas del diseño permanecen fijas. La afirmación depende de la escala y de las transformaciones. No es una afirmación causal: mantener fijas columnas correlacionadas puede describir una comparación rara o incluso imposible en la población.

Regreso a las subzonas de Bogotá

En las 32 subzonas, `estrato_promedio` y `precio_reportado` presentan una asociación positiva.

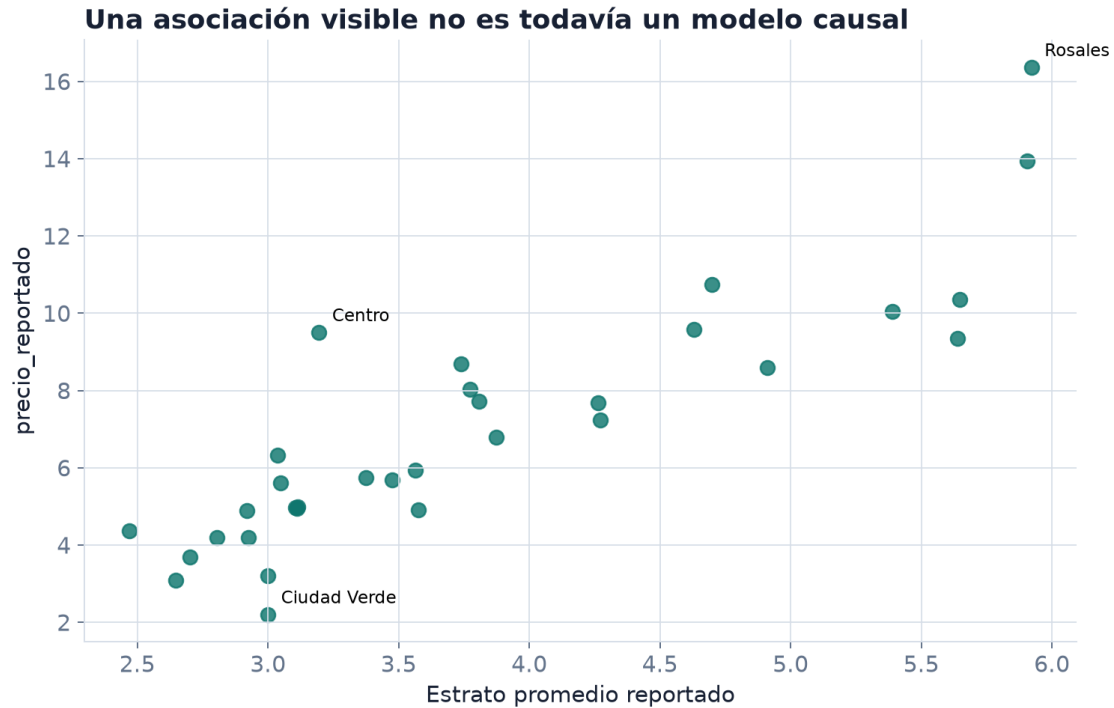
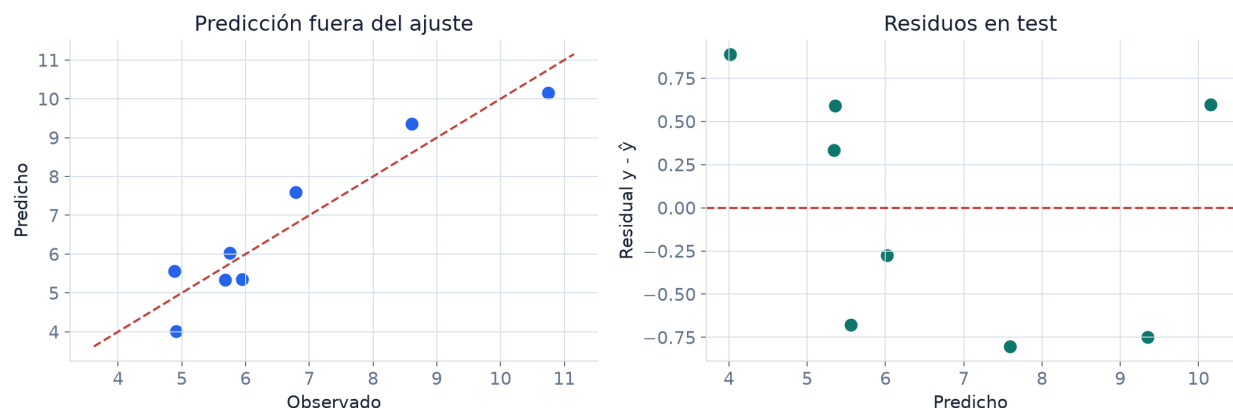


Figura 7: Estrato promedio y precio reportado presentan una asociación positiva en las 32 subzonas, pero la gráfica no identifica una relación causal.

Un ajuste con intercepto y las cinco columnas numéricas obtiene MSE de entrenamiento 1.077, frente a 9.720 para el baseline constante. También tiene un número de condición cercano a 1237, influido por escalas muy diferentes y por la relación entre promedios de área, alcobas, baños y parqueaderos.

Los MSE anteriores describen el ajuste de las 32 filas. Para obtener una primera comprobación fuera del ajuste, reservamos ocho subzonas con una semilla fijada. En esa partición, el MAE del baseline es 1.723 y el del modelo lineal es 0.615. La gráfica muestra cada predicción y el residual correspondiente.

Partición fija: MAE baseline 1.723; modelo 0.615



Ejemplo docente con 32 filas agregadas; no constituye validación operativa.

Figura 8: En una partición fija de ocho subzonas, el modelo supera el baseline; las predicciones y los residuos muestran dónde se concentra el error.

Una partición no basta para establecer que la mejora sea estable. El capítulo de validación repetirá la comparación sin escoger la semilla por su resultado. Aun entonces, una fila seguirá representando una subzona y la unidad de `precio_reportado` seguirá sin estar documentada. El álgebra no completa la información que el archivo no contiene.

Síntesis

Mínimos cuadrados comienza con residuales y una regla para agregarlos. En una regresión de una variable, las condiciones de optimalidad producen una pendiente determinada por covariación y un intercepto que hace cero el residual medio. La matriz de diseño extiende el mismo cálculo a varias características.

La pérdida $\|\mathbf{X}\theta - \mathbf{y}\|^2 / (2n)$ tiene gradiente $\mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}) / n$. Sus puntos críticos satisfacen las ecuaciones normales, y las predicciones son la proyección de $\mathbf{y}$ sobre el espacio columna de $\mathbf{X}$. Para calcular el ajuste se prefiere `lstsq` a formar una inversa o a construir $\mathbf{X}^\top \mathbf{X}$ sin necesidad.

La ortogonalidad residual certifica que el problema algebraico fue resuelto; no certifica que la forma lineal sea adecuada ni que el modelo generalice. El baseline, los gráficos de residuales, el rango y la documentación de las variables responden preguntas distintas y complementarias.

Problemas integrados

1. Una recta antes de una matriz

Considere los puntos $(0, 2)$, $(1, 2)$ y $(2, 5)$.

1. Calcule $\bar{x}$, $\bar{y}$, $\hat{\theta}_0$ y $\hat{\theta}_1$ mediante las fórmulas escalares.

2. Obtenga las predicciones, los residuales y el MSE.
3. Verifique las dos condiciones de optimalidad para los residuales.
4. Construya $\mathbf{X}$ con intercepto y reproduzca la solución con `lstsq`.
5. Compare el MSE con el baseline constante y escriba qué aporta la entrada.

2. Recuperar un mecanismo conocido

Use `regresion_lineal_sintetica.csv`.

1. Ajuste primero `target ~ x1` y luego `target ~ x1 + x2 + x3`.
2. Compare coeficientes, MSE y ortogonalidad residual.
3. Explique por qué la pendiente de `x1` cambia entre los dos diseños.
4. Añada la columna `x4 = 2*x1`, vuelva a ajustar y examine rango y valores singulares.
5. Construya dos vectores de parámetros distintos que produzcan las mismas predicciones en el diseño de rango deficiente.

3. Fórmula algebraica y algoritmo numérico

Construya matrices sintéticas con números de condición aproximadamente 10^1 , 10^3 y 10^6 .

1. Compruebe numéricamente la relación entre $\kappa_2(\mathbf{X})$ y $\kappa_2(\mathbf{X}^\top \mathbf{X})$.
2. Compare `np.linalg.inv`, `np.linalg.solve` y `np.linalg.lstsq`.
3. Mida el residual de las ecuaciones normales y el error en los coeficientes respecto de una solución conocida.
4. Explique por qué una identidad exacta puede dar lugar a implementaciones con comportamientos numéricos diferentes.

4. Un ajuste agregado en Bogotá

Use `vivienda_nueva_bogota_subzonas.csv`.

1. Defina la matriz de diseño con intercepto y las cinco columnas numéricas.
2. Reporte rango, valores singulares, número de condición, coeficientes y MSE de entrenamiento.
3. Compare con el baseline de la media y grafique residuales contra predicciones.
4. Examine cómo cambian los coeficientes después de estandarizar las cinco características sin modificar el intercepto.
5. Redacte una conclusión sobre las 32 subzonas que distinga ajuste, interpretación de coeficientes y alcance de los datos.

Respuestas seleccionadas

Problema 1. Se tiene $\bar{x} = 1$ y $\bar{y} = 3$. La pendiente es

$$\hat{\theta}_1 = \frac{(-1)(-1) + 0(-1) + 1(2)}{(-1)^2 + 0^2 + 1^2} = \frac{3}{2},$$

y el intercepto es $\hat{\theta}_0 = 1.5$. Las predicciones son $(1.5, 3, 4.5)$, los residuales son $(0.5, -1, 0.5)$ y el MSE es 0.5. Tanto la suma de residuales como $\sum_i x_i r_i$ son cero.

Problema 2. Con una sola variable se obtiene aproximadamente $(4.055, 1.652)$ y MSE 2.004. Con las tres variables se obtiene $(4.043, 2.491, -1.201, 0.715)$ y MSE 0.287. La diferencia no muestra que el primer algoritmo haya fallado: muestra que los dos modelos responden a diseños distintos.

Descenso del gradiente

Iteraciones, estabilidad y geometría de la optimización

En el capítulo anterior, `np.linalg.lstsq` calculó directamente el minimizador de

$$\widehat{R}_n(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2.$$

Supongamos ahora que solo podemos evaluar su gradiente. Para tres observaciones con $x = (-1, 0, 1)$, $y = (1, 3, 5)$ e intercepto, comenzamos en $\theta^{(0)} = (0, 0)$. El gradiente inicial es

$$\nabla \widehat{R}_n(\theta^{(0)}) = \frac{1}{3} \mathbf{X}^\top (\mathbf{X}\theta^{(0)} - \mathbf{y}) = \begin{bmatrix} -3 \\ -4/3 \end{bmatrix}.$$

Moverse una distancia $\alpha = 0.2$ en la dirección opuesta produce

$$\theta^{(1)} = \theta^{(0)} - 0.2 \nabla \widehat{R}_n(\theta^{(0)}) = \begin{bmatrix} 0.6 \\ 0.267 \end{bmatrix}.$$

La pérdida baja de 5.833 a 3.881. No hemos resuelto un sistema ni formado una inversa: usamos información local para construir una secuencia de parámetros.

De la derivada a un algoritmo

Para una función diferenciable $J : \mathbb{R}^p \rightarrow \mathbb{R}$, el gradiente $\nabla J(\theta)$ señala la dirección de mayor crecimiento local. La aproximación de Taylor

$$J(\theta + h) \approx J(\theta) + \nabla J(\theta)^\top h$$

muestra que, entre desplazamientos de norma fija y suficientemente pequeña, la elección h opuesta al gradiente produce la mayor disminución de primer orden. El descenso del gradiente repite

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}),$$

donde $\alpha > 0$ es la tasa de aprendizaje.

Para mínimos cuadrados,

$$\nabla \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}).$$

La siguiente implementación guarda la pérdida inicial y la obtenida después de cada actualización. Las firmas coinciden con las utilizadas en los notebooks del módulo.

```
import numpy as np

def half_mse_loss(X, y, theta):
    error = X @ theta - y
    return 0.5 * np.mean(error ** 2)

def gradient_descent(X, y, alpha=0.1, max_iter=1000, tol=1e-8):
    n, p = X.shape
    theta = np.zeros(p, dtype=float)
    history = [half_mse_loss(X, y, theta)]

    for _ in range(max_iter):
        gradient = X.T @ (X @ theta - y) / n
        if not np.all(np.isfinite(gradient)):
            raise FloatingPointError("gradiente no finito")
        if np.linalg.norm(gradient) <= tol:
            break

        theta = theta - alpha * gradient
        loss = half_mse_loss(X, y, theta)
        if not np.isfinite(loss):
            raise FloatingPointError("pérdida no finita")
        history.append(loss)

    return theta, np.asarray(history)
```

En el ejemplo inicial, las primeras iteraciones con $\alpha = 0.2$ son:

iteración	intercepto	pendiente	pérdida	norma del gradiente
0	0.000	0.000	5.833	3.283
1	0.600	0.267	3.881	2.664
2	1.080	0.498	2.595	2.165
3	1.464	0.698	1.745	1.764
4	1.771	0.872	1.179	1.441
5	2.017	1.022	0.802	1.180

El minimizador es (3, 2). Cada columna progresa a una velocidad distinta; la secuencia todavía no ha convergido después de cinco pasos, pero la pérdida y la norma del gradiente disminuyen de manera coherente.

La tasa decide la trayectoria

La dirección del gradiente no fija la longitud del paso. En una superficie convexa, un paso pequeño puede avanzar lentamente y uno grande puede atravesar el valle y aumentar la pérdida.

Descenso del gradiente: trayectoria, escala

Cada actualización usa el gradiente local para reducir la pérdida, pero el tamaño

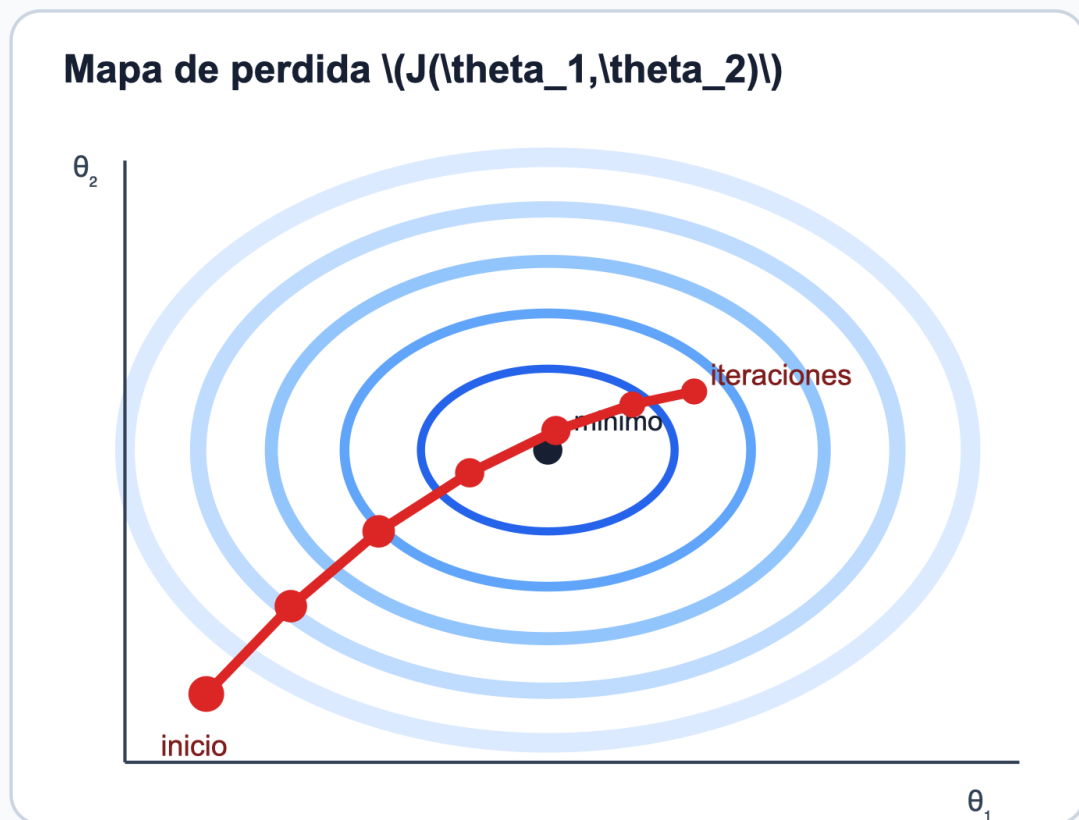


Figura 9: Descenso del gradiente: la trayectoria avanza sobre contornos de pérdida y la tasa de aprendizaje controla si el avance es lento, estable u oscilatorio.

Con el mismo ejemplo, $\alpha = 1.8$ hace oscilar el intercepto alrededor de 3, aunque la pérdida decrece. Con $\alpha = 2.2$ la pérdida comienza en 5.833 y luego toma los valores 6.770, 9.394, 13.451 y 19.352: el algoritmo diverge. Ni la convexidad de la pérdida ni la corrección del gradiente hacen estable cualquier tasa.

Una curva de pérdida debe leerse junto con la norma del gradiente y el estado de los parámetros:

comportamiento	explicación compatible	comprobación siguiente
disminución suave	paso estable	comparar con una solución de referencia
disminución muy lenta	paso pequeño o geometría alargada	revisar escala y espectro
oscilación decreciente	paso cercano al límite de estabilidad	reducir α
crecimiento sostenido nan o inf	paso fuera del intervalo estable desbordamiento tras divergencia	detener y reducir α revisar primero la historia finita
pérdida estable pero alta	familia limitada o parada prematura	separar optimización de modelado

Que la pérdida de entrenamiento converja solo responde si el algoritmo resolvió el problema empírico. No demuestra que la familia sea adecuada ni que el modelo funcione en observaciones nuevas.

La Hessiana explica la estabilidad

En mínimos cuadrados la Hessiana es constante:

$$\mathbf{H} = \nabla^2 \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top \mathbf{X}.$$

Supongamos primero que $\mathbf{X}$ tiene rango columna completo. Si θ^* es el minimizador y $e_t = \theta^{(t)} - \theta^*$, entonces

$$\begin{aligned} e_{t+1} &= e_t - \alpha \mathbf{H} e_t \\ &= (\mathbf{I} - \alpha \mathbf{H}) e_t. \end{aligned}$$

Sea $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$ y denote sus valores propios por $0 < \lambda_1 \leq \dots \leq \lambda_p$. La componente del error en la dirección propia j se multiplica por $1 - \alpha \lambda_j$ en cada iteración. Todas las componentes se contraen exactamente cuando

$$|1 - \alpha \lambda_j| < 1 \quad \text{para todo } j,$$

o, de manera equivalente,

$$0 < \alpha < \frac{2}{\lambda_{\max}(\mathbf{H})}.$$

Convergencia en la pérdida cuadrática. Si $\mathbf{X}$ tiene rango columna completo y $0 < \alpha < 2/\lambda_{\max}(\mathbf{H})$, el descenso del gradiente con paso constante converge linealmente al único minimizador.

Si $\mathbf{H}$ es singular, las componentes iniciales en su espacio nulo no cambian y el vector de parámetros límite puede no ser único.

La tasa que equilibra las dos direcciones extremas de una cuadrática definida positiva es

$$\alpha_{\text{opt}} = \frac{2}{\lambda_{\min}(\mathbf{H}) + \lambda_{\max}(\mathbf{H})},$$

con factor de contracción máximo

$$\rho = \frac{\kappa_2(\mathbf{H}) - 1}{\kappa_2(\mathbf{H}) + 1}.$$

La fórmula explica el fenómeno, pero no es una receta general. Calcular el espectro completo puede costar tanto como resolver el problema y, fuera de una cuadrática, la Hessiana cambia con los parámetros.

Escala y condicionamiento

Una misma variable expresada en unidades distintas contiene la misma información, pero produce una geometría numérica diferente. Multipliquemos $\mathbf{x}_2$ del conjunto sintético por 1000. El diseño resultante tiene aproximadamente

$$\kappa_2(\mathbf{X}) = 1311, \quad \lambda_{\max}(\mathbf{H}) = 1.189 \times 10^6.$$

La condición teórica de estabilidad exige entonces

$$\alpha < 1.68 \times 10^{-6}.$$

Después de centrar cada característica y dividirla por su desviación estándar, sin transformar la columna de intercepto, se obtiene

$$\kappa_2(\mathbf{X}_{\text{esc}}) = 2.16, \quad \lambda_{\max}(\mathbf{H}_{\text{esc}}) = 1.657,$$

y el límite aumenta a aproximadamente 1.207. No apareció información nueva; se modificaron las coordenadas en las que el algoritmo recorre la pérdida.

La estandarización de una característica j utiliza

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

```

mu = X_features.mean(axis=0)
scale = X_features.std(axis=0)
scale[scale == 0] = 1.0

Z = (X_features - mu) / scale
X_scaled = add_intercept(Z)

```

Cuando se evalúe fuera de la muestra de ajuste, μ_j y s_j deberán calcularse solo con entrenamiento y reutilizarse sin volverlos a estimar. El capítulo siguiente formalizará esta restricción al introducir particiones y filtración de información.

Escalar no corrige colinealidad. La estandarización puede equilibrar unidades, pero dos columnas casi redundantes continúan casi redundantes. Si un valor singular permanece cercano a cero después de escalar, el problema requiere revisar el diseño o introducir regularización.

Verificar gradiente y solución

Una implementación del gradiente debe comprobarse antes de interpretar su curva. La diferencia central en la coordenada j es

$$g_j^{\text{num}} = \frac{J(\theta + he_j) - J(\theta - he_j)}{2h}.$$

Para un h moderadamente pequeño, el vector numérico debe aproximar el gradiente analítico:

```

def finite_difference_gradient(loss, theta, h=1e-6):
    gradient = np.empty_like(theta, dtype=float)
    for j in range(theta.size):
        direction = np.zeros_like(theta, dtype=float)
        direction[j] = h
        gradient[j] = (
            loss(theta + direction) - loss(theta - direction)
        ) / (2 * h)
    return gradient

theta0 = np.zeros(X.shape[1])
analytic = X.T @ (X @ theta0 - y) / X.shape[0]
numeric = finite_difference_gradient(
    lambda theta: half_mse_loss(X, y, theta), theta0
)
assert np.allclose(analytic, numeric, rtol=1e-5, atol=1e-7)

```

Después se compara el límite del algoritmo con `lstsq` usando exactamente la misma matriz:

```

theta_ref = np.linalg.lstsq(X_scaled, y, rcond=None)[0]
theta_gd, history = gradient_descent(
    X_scaled, y, alpha=0.5, max_iter=10_000
)

parameter_gap = np.linalg.norm(theta_gd - theta_ref)
gradient_norm = np.linalg.norm(
    X_scaled.T @ (X_scaled @ theta_gd - y) / X_scaled.shape[0]
)

```

Una diferencia grande puede deberse a una tasa inestable, pocas iteraciones, un criterio de parada mal definido o un gradiente incorrecto. Comparar solo las pérdidas puede ocultar diferencias entre parámetros cuando el diseño tiene rango deficiente.

Costo y gradientes por subconjuntos

Calcular $\mathbf{X}\theta$ y después multiplicar el error por $\mathbf{X}^\top$ cuesta $O(np)$ por iteración batch. Tras T iteraciones, el costo es $O(Tnp)$, además de almacenar el diseño. Una factorización directa paga aproximadamente $O(np^2)$ una vez. El método conveniente depende de n , p , la estructura dispersa y el número de iteraciones necesario.

Para un subconjunto B de observaciones definimos

$$g_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta).$$

Si las posiciones se seleccionan uniformemente, condicionado en el conjunto de datos,

$$\mathbb{E}_B[g_B(\theta)] = \nabla \widehat{R}_n(\theta).$$

Por tanto, el gradiente de mini-batch es un estimador condicionalmente insesgado del gradiente empírico completo. Su aleatoriedad proviene de escoger el subconjunto B , una vez fijados los datos; no debe confundirse con la variación que produciría observar otra muestra de la población. Un mini-batch reduce el costo de cada paso e introduce variación entre pasos. Esta observación prepara el uso de optimizadores estocásticos en redes neuronales; en este módulo, el algoritmo de referencia seguirá utilizando todo el conjunto.

Regreso a los dos casos

En la población sintética, el diseño con intercepto y tres características tiene número de condición 2.17. El descenso con datos sin reescalar ya es manejable y debe converger al mismo ajuste que `lstsq`. Multiplicar `x2` por 1000 no cambia las predicciones que puede representar el modelo, pero obliga a reducir drásticamente la tasa si no se estandariza.

En las subzonas de Bogotá, el diseño sin escalar tiene número de condición cercano a 1237. Parte del valor se debe a que `area_promedio` vive en una escala muy distinta de los demás promedios. Estandarizar facilita la optimización y la comparación numérica de columnas, pero no corrige la agregación por subzona, la unidad desconocida de `precio_reportado` ni la dependencia entre variables. Un algoritmo puede converger con precisión a un modelo cuya interpretación siga siendo limitada.

Síntesis

El descenso del gradiente convierte una derivada en una sucesión de parámetros. La pérdida, la norma del gradiente y la comparación con una solución de referencia permiten distinguir convergencia, oscilación, divergencia y errores de implementación.

En una pérdida cuadrática, los valores propios de $\mathbf{X}^\top \mathbf{X}/n$ determinan el intervalo estable de tasas y la velocidad relativa de las direcciones. El escalamiento puede transformar una geometría muy alargada en otra más equilibrada sin cambiar la información del problema. No elimina colinealidad ni sustituye la evaluación estadística.

El próximo capítulo mostrará un conflicto diferente: una familia más flexible puede reducir la pérdida de entrenamiento y empeorar el comportamiento fuera de muestra. Allí la optimización será una parte del procedimiento, no el criterio para escoger por sí sola el modelo final.

Problemas integrados

1. Seis pasos que pueden calcularse a mano

Use $x = (-1, 0, 1)$, $y = (1, 3, 5)$ y $\theta^{(0)} = (0, 0)$.

1. Derive el gradiente y la Hessiana.
2. Calcule seis iteraciones con $\alpha = 0.2$.
3. Repita con $\alpha = 1.8$ y $\alpha = 2.2$; compare las pérdidas.
4. Obtenga los valores propios de la Hessiana y determine el intervalo teórico de convergencia.
5. Explique cada trayectoria mediante los factores $1 - \alpha\lambda_j$.

2. El mismo dato en otra unidad

Use `regresion_lineal_sintetica.csv` y multiplique `x2` por 1000.

1. Calcule número de condición, espectro de la Hessiana y tasa máxima teórica antes de escalar.
2. Ejecute gradiente con tasas 10^{-7} , 10^{-5} y 0.1.
3. Estandarice las características, mantenga intacto el intercepto y repita.
4. Convierta los coeficientes del modelo escalado a las unidades originales y compruebe que las predicciones coinciden.
5. Distinga qué cambió en la optimización, en el modelo representable y en la interpretación de los coeficientes.

3. Auditoría de una implementación

Introduzca deliberadamente uno de estos errores: omitir $1/n$, cambiar el signo del gradiente o devolver una pérdida anterior al último parámetro.

1. Diseñe una prueba por diferencias finitas que detecte el error.
2. Compare el resultado con `lstsq`.
3. Registre pérdida, norma del gradiente y distancia entre parámetros.
4. Corrija la función sin cambiar su firma pública.
5. Escriba qué prueba habría fallado aun si la pérdida final pareciera pequeña.

4. Optimización en las subzonas

Use las cinco columnas numéricas del caso de Bogotá.

1. Compare el espectro y el número de condición antes y después de estandarizar.
2. Encuentre una tasa estable para cada representación sin seleccionar una semilla por su resultado.
3. Compare las predicciones de gradiente y `lstsq`.
4. Grafique la historia de pérdida con ejes y escala claramente identificados.
5. Redacte un diagnóstico que separe convergencia numérica, ajuste en las 32 filas y alcance sustantivo del dataset.

Respuestas seleccionadas

Problema 1. La Hessiana es

$$\mathbf{H} = \begin{bmatrix} 1 & 0 \\ 0 & 2/3 \end{bmatrix},$$

por lo que el intervalo estable es $0 < \alpha < 2$. Con $\alpha = 2.2$, la componente asociada a $\lambda = 1$ se multiplica por -1.2 y crece en magnitud; la divergencia no se debe a un mínimo local.

Problema 2. Antes de escalar, $\kappa_2(\mathbf{X}) \approx 1311$ y $\alpha_{\text{máx}} \approx 1.68 \times 10^{-6}$. Después de escalar, $\kappa_2(\mathbf{X}) \approx 2.16$ y $\alpha_{\text{máx}} \approx 1.207$. Las dos parametrizaciones pueden producir las mismas predicciones lineales; sus recorridos numéricos no son iguales.

Validación y regularización

Cómo escoger complejidad sin confundir ajuste con generalización

La variable `x1` del conjunto sintético produjo una recta con MSE cercano a 2. Podemos construir una familia mucho más flexible sin añadir nuevas mediciones:

$$\phi_d(x) = [x \ x^2 \ \dots \ x^d]^\top.$$

Dividamos las 160 filas, con semillas fijadas, en 80 para entrenamiento, 40 para validación y 40 para prueba. Ajustar polinomios por mínimos cuadrados produce:

grado	MSE entrenamiento	MSE validación
1	1.993	2.223
3	1.965	2.172
8	1.749	2.379
15	1.542	3.254

El grado 15 resuelve mejor el problema que se le pidió durante el ajuste: tiene el menor error de entrenamiento. También produce el peor resultado en las 40 observaciones que no utilizó. La optimización no falló. El criterio de selección era incompleto.

Capacidad y comportamiento fuera de muestra

Una regresión polinomial sigue siendo lineal en sus parámetros:

$$f_\theta(x) = \theta_0 + \theta_1 x + \dots + \theta_d x^d.$$

Al aumentar d , la familia contiene a las anteriores y el mínimo de la pérdida de entrenamiento no puede aumentar, salvo diferencias numéricas. Esa propiedad no se extiende al riesgo poblacional. Los parámetros de orden alto pueden adaptarse a fluctuaciones particulares de la muestra.

```
def polynomial_features_1d(x, degree):  
    x = np.asarray(x, dtype=float).reshape(-1, 1)  
    columns = [np.ones((x.shape[0], 1))]  
    columns.extend(x ** power for power in range(1, degree + 1))  
    return np.hstack(columns)
```

La transformación $x \mapsto (x, x^2, \dots, x^d)$ es determinista por fila. No consulta otras observaciones. El grado, en cambio, define una familia y debe elegirse sin utilizar la evaluación final.

Si D_n representa la muestra de ajuste y $\hat{f}_{D_n}$ la función obtenida, repetir el muestreo produce otra función. Para pérdida cuadrática y una entrada fija x , bajo condiciones usuales,

$$\begin{aligned}\mathbb{E}_{D_n, Y|x}[(Y - \hat{f}_{D_n}(x))^2] &= \text{Var}(Y | X = x) \\ &+ \text{Bias}(x)^2 \\ &+ \text{Var}_{D_n}(\hat{f}_{D_n}(x)),\end{aligned}$$

donde

$$\text{Bias}(x) = \mathbb{E}_{D_n}[\hat{f}_{D_n}(x)] - \mathbb{E}[Y | X = x].$$

Una familia demasiado rígida puede tener sesgo alto. Una familia muy flexible puede responder con intensidad a las particularidades de D_n . La regularización modifica este balance; la validación proporciona datos para compararlo.

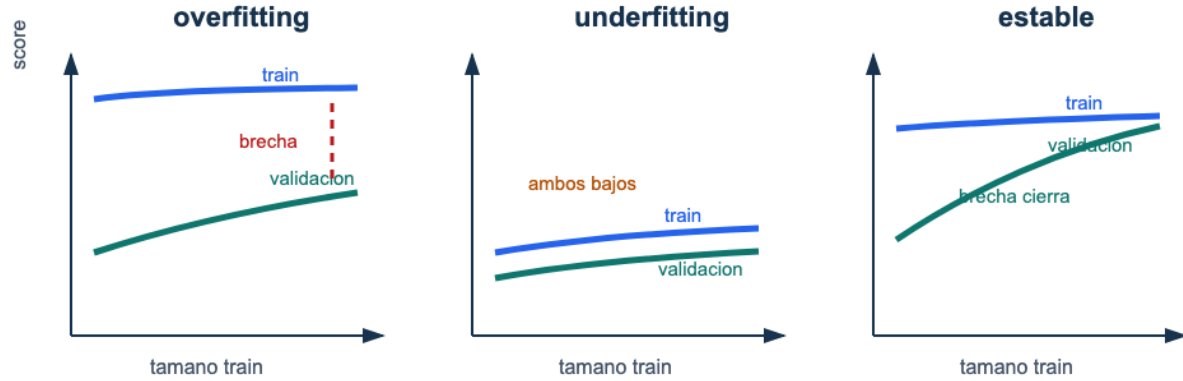


Figura 10: Curvas de aprendizaje: el sobreajuste mantiene una brecha entre entrenamiento y validación, el subajuste mantiene ambos errores altos y un caso estable reduce la brecha al aumentar los datos.

Una curva de aprendizaje es un diagnóstico, no una prueba automática de una causa. Una brecha puede sugerir sensibilidad a la muestra; errores altos en ambos conjuntos pueden sugerir una familia inadecuada, una representación pobre o una optimización incompleta.

Tres funciones para tres conjuntos de datos

Entrenamiento, validación y prueba no son tres nombres para muestras intercambiables.

conjunto	decisiones que permite
entrenamiento	ajustar parámetros y transformaciones aprendidas
validación	escoger grado, penalización y otras configuraciones
prueba	evaluar una vez el procedimiento ya fijado

Validación de una regla fijada. Sea f una función que no utilizó el conjunto de validación. Si $Z_1^{\text{val}}, \dots, Z_v^{\text{val}}$ son iid de la población P , entonces

$$\mathbb{E}[\widehat{R}_{\text{val}}(f) \mid f] = R_P(f),$$

si la pérdida es integrable.

Condicionado en f , cada pérdida $\ell(f; Z_i^{\text{val}})$ tiene esperanza $R_P(f)$. La linealidad de la esperanza aplicada al promedio da la igualdad.

Después de comparar muchos modelos y escoger el menor valor observado en esa misma validación, la función elegida ya depende de los datos de validación. El mínimo incorpora optimismo por selección. El conjunto de prueba se reserva para el procedimiento completo: transformaciones, familia, hiperparámetros, umbral y regla de entrenamiento.

Cuando hay pocas observaciones, la validación cruzada reutiliza de manera estructurada distintas partes de los datos para comparar configuraciones. No elimina la incertidumbre ni autoriza usar prueba durante el diseño. El capítulo de estrategia estudiará sus estimaciones con más detalle.

Transformaciones y filtración de información

Una transformación puede ser fija o aprender cantidades de los datos.

- $x \mapsto x^2$ se aplica a una fila sin estimar nada de las demás.
- La estandarización necesita medias y desviaciones.
- La imputación necesita reglas o valores de reemplazo.
- Una codificación puede aprender qué categorías existen.
- La selección de variables utiliza una medida calculada con respuestas.

Las cantidades aprendidas se ajustan solo con entrenamiento y se aplican sin reestimarlas a validación y prueba. De lo contrario, información reservada cambia la representación usada para entrenar.

Dos filtraciones diferentes. Calcular la media de estandarización con toda la tabla filtra información entre observaciones. Incluir una variable registrada después del resultado filtra información temporal. Una tubería de software ayuda con la primera; la segunda exige comprender cómo se produce cada columna.

Una implementación con `scikit-learn` mantiene juntos transformación y ajuste:

```

from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

model = make_pipeline(
    PolynomialFeatures(degree=3, include_bias=False),
    StandardScaler(),
    Ridge(alpha=1.0, fit_intercept=True),
)
model.fit(X_train, y_train)
validation_prediction = model.predict(X_validation)

```

El método `fit` del pipeline aprende polinomio, escalas y regresión únicamente a partir de entrenamiento. `predict` reutiliza esos objetos.

Ridge: encoger direcciones inestables

Ridge añade una penalización cuadrática a la pérdida. Con el intercepto en la primera columna de $\mathbf{X}$, definimos

$$\mathbf{D} = \text{diag}(0, 1, \dots, 1)$$

y

$$J_{\text{Ridge}}(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \theta^\top \mathbf{D} \theta.$$

El cero inicial evita penalizar el intercepto. El gradiente es

$$\nabla J_{\text{Ridge}}(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}) + \lambda \mathbf{D} \theta.$$

Igualarlo a cero produce

$$(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{D}) \hat{\theta}_\lambda = \mathbf{X}^\top \mathbf{y}.$$

```

def ridge_closed_form(X, y, lam):
    n, p = X.shape
    penalty = np.eye(p)
    penalty[0, 0] = 0.0
    system = X.T @ X + n * lam * penalty
    return np.linalg.solve(system, X.T @ y)

```

Esta función implementa la convención del libro. `sklearn.linear_model.Ridge` minimiza

$$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_2^2.$$

Por tanto, para el mismo diseño y sin penalizar el intercepto, $\alpha = n\lambda$. Usar el mismo número para α y λ sin considerar la normalización no reproduce el mismo problema.

Qué ocurre en las direcciones singulares

Para estudiar los coeficientes no intercepto, supongamos que las características están centradas y escribamos $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Ridge actúa como

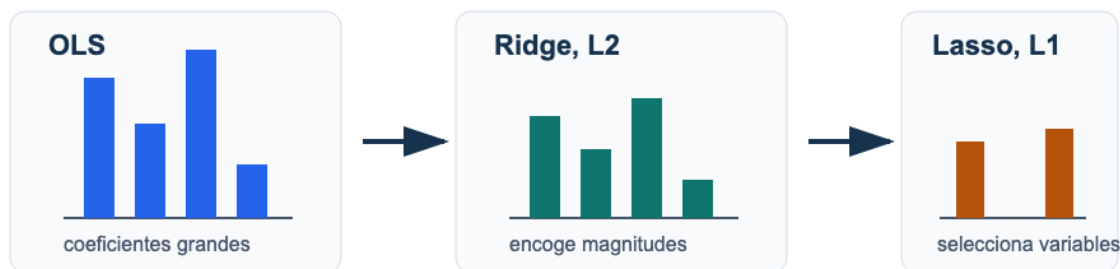
$$\hat{\theta}_\lambda = \mathbf{V}(\mathbf{\Sigma}^\top\mathbf{\Sigma} + n\lambda\mathbf{I})^{-1}\mathbf{\Sigma}^\top\mathbf{U}^\top\mathbf{y}.$$

En la dirección derecha v_j , el factor

$$\frac{\sigma_j}{\sigma_j^2 + n\lambda}$$

reemplaza el factor $1/\sigma_j$ de mínimos cuadrados. Las direcciones con σ_j pequeño, que son las más sensibles, se encogen con mayor intensidad. La estabilidad se obtiene a cambio de sesgo.

Regularización: controlar complejidad sin cambiar la pregunta



Idea matemática: minimizar pérdida + penalización; validar lambda antes de decidir.

Figura 11: Regularización: mínimos cuadrados puede producir coeficientes grandes, Ridge reduce magnitudes y Lasso puede llevar coeficientes exactamente a cero.

Un λ positivo no garantiza menor riesgo. Si es demasiado grande, las predicciones se acercan a una regla casi constante y pueden perder estructura real. Su valor se selecciona con validación.

Lasso: una penalización con esquina

Lasso utiliza

$$J_{\text{Lasso}}(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 + \lambda \sum_{j=1}^{p-1} |\theta_j|.$$

De nuevo, el intercepto no se penaliza. La función $|t|$ no es diferenciable en cero; su subdiferencial es

$$\partial|t| = \begin{cases} \{1\}, & t > 0, \\ [-1, 1], & t = 0, \\ \{-1\}, & t < 0. \end{cases}$$

Sea $\mathbf{x}_j$ la columna j del diseño. Una solución satisface

$$0 \in \frac{1}{n} \mathbf{x}_j^\top (\mathbf{X}\hat{\theta} - \mathbf{y}) + \lambda \partial|\hat{\theta}_j|.$$

La coordenada puede quedar exactamente en cero cuando

$$\left| \frac{1}{n} \mathbf{x}_j^\top (\mathbf{y} - \mathbf{X}\hat{\theta}) \right| \leq \lambda.$$

La esquina de la penalización permite que un intervalo de gradientes residuales sea compatible con cero. El operador de umbral suave

$$S(a, \lambda) = \text{sign}(a) \max\{|a| - \lambda, 0\}$$

aparece al resolver una coordenada a la vez. Si $\mathbf{r}_{-j} = \mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \theta_k$, la actualización es

$$\theta_j \leftarrow \frac{S(\mathbf{x}_j^\top \mathbf{r}_{-j}/n, \lambda)}{\|\mathbf{x}_j\|_2^2/n}.$$

Con variables muy correlacionadas, Lasso puede escoger una y dejar otra en cero de manera sensible a la muestra. Un cero describe la solución penalizada para ese diseño y ese λ ; no demuestra irrelevancia causal ni ausencia de asociación poblacional.

aspecto	Ridge	Lasso
penalización	suma de cuadrados	suma de valores absolutos
efecto habitual	encogimiento continuo	algunos ceros exactos
columnas correlacionadas	tiende a repartir peso	puede escoger una
cálculo	sistema lineal	algoritmo iterativo, por ejemplo coordenadas
propósito frecuente	estabilidad predictiva	esparsidad y predicción

Como ambas penalizaciones actúan sobre coeficientes, las características deben tener escalas comparables. El intercepto se mantiene separado de la estandarización y de la penalización.

Seleccionar sin consultar prueba

Volvamos al polinomio de grado 15. Después de ajustar `PolynomialFeatures` y `StandardScaler` con entrenamiento, obtenemos:

alpha de Ridge	MSE entrenamiento	MSE validación
0	1.542	3.254
0.0001	1.655	2.552
0.01	1.756	2.317
0.1	1.790	2.274
1	1.848	2.199
10	1.953	2.326
100	2.647	3.396

Ridge reduce la brecha del polinomio de grado 15. Sin embargo, el polinomio de grado 3 obtiene MSE de validación 2.172 y resulta ligeramente mejor entre las configuraciones comparadas. Se fija entonces el procedimiento de grado 3 y se consulta prueba por primera vez: su MSE es 2.103. El baseline, cuya media se calculó solo con entrenamiento, obtiene 5.533.

Esta evaluación no prueba que el grado 3 sea óptimo para toda muestra futura. Sí documenta una comparación reproducible en la que la prueba no participó en la selección.

Regreso a las subzonas de Bogotá

En el caso de Bogotá solo hay 32 filas. Una partición de 24 subzonas para ajuste y 8 para evaluación produce una métrica muy sensible a cuáles subzonas quedaron en cada grupo.

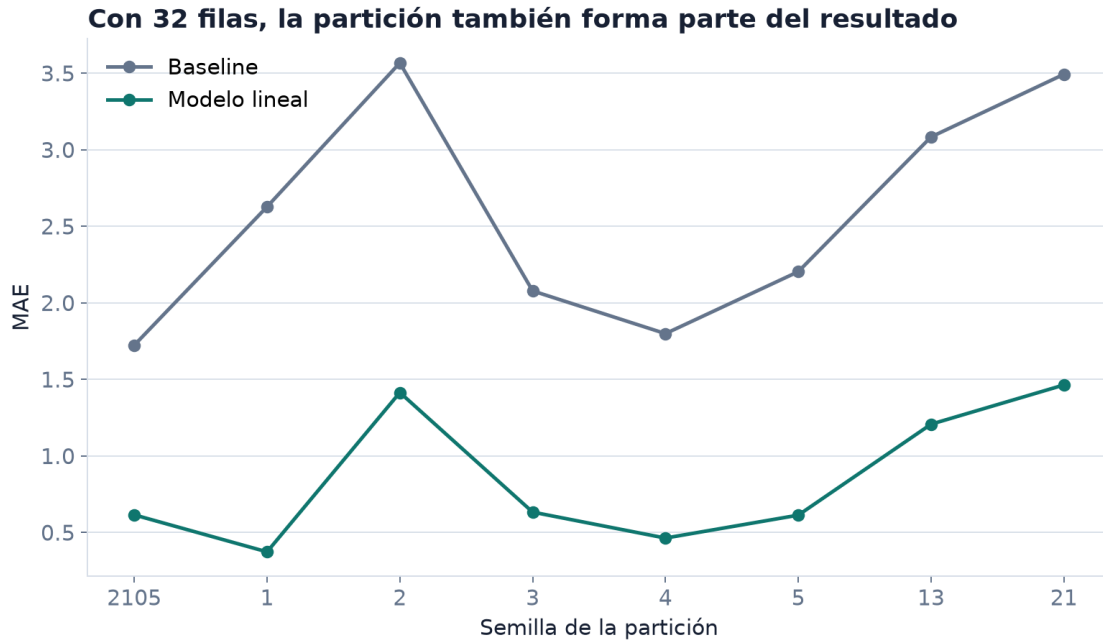


Figura 12: El MAE cambia entre particiones de las 32 subzonas. La semilla registra una fuente de variación; no debe escogerse por producir el menor error.

Escoger la semilla con menor MAE sería otra forma de usar la evaluación para diseñar el resultado. Una comparación responsable fija el mecanismo de partición antes de ver las métricas y muestra su variación. Con tan pocas unidades, la validación cruzada puede aprovechar mejor las filas, pero las unidades siguen siendo agregadas y no están documentadas como una muestra aleatoria de una población más amplia.

Ridge puede estabilizar los coeficientes correlacionados de área, alcobas, baños y parqueaderos. Esa estabilidad resuelve una dificultad del ajuste; no recupera la variación dentro de cada subzona ni la unidad ausente de `precio_reportado`.

Síntesis

Una familia más flexible siempre puede mejorar o conservar el mejor ajuste de entrenamiento. El ejemplo polinomial muestra por qué ese orden no determina el comportamiento fuera de muestra. Entrenamiento ajusta parámetros y transformaciones; validación compara configuraciones; prueba evalúa el procedimiento fijado.

Ridge añade una penalización cuadrática que encoge especialmente las direcciones mal determinadas. Lasso utiliza valores absolutos y puede producir ceros por su condición de subgradiente. En ambos casos, el intercepto no se penaliza y las características se estandarizan con estadísticas aprendidas solo en entrenamiento.

Regularizar, validar y evitar filtración son partes del mismo procedimiento. Una buena métrica de prueba no borra, sin embargo, las restricciones de unidad, muestreo y medición establecidas antes del ajuste.

Problemas integrados

1. Complejidad que mejora el objetivo equivocado

Use `x1` y `target` de `regresion_lineal_sintetica.csv` con las particiones descritas al inicio.

1. Reproduzca la tabla para grados 1, 3, 8 y 15.
2. Grafique cada polinomio dentro y fuera del rango central de entrenamiento.
3. Compare normas de coeficientes y números de condición.
4. Explique por qué el error de entrenamiento debe ser no creciente si los espacios polinomiales están anidados y la solución es exacta.
5. Seleccione un grado con validación y consulte prueba una sola vez.

2. Una tubería con y sin filtración

Construya una variable con faltantes y una categoría poco frecuente.

1. Divida los datos antes de imputar, codificar o estandarizar.
2. Implemente un pipeline que aprenda todas esas transformaciones con entrenamiento.
3. Implemente deliberadamente una versión que las aprenda con toda la tabla.
4. Compare los objetos aprendidos, no solo la métrica final.
5. Identifique cuál información reservada ingresó en cada transformación y por qué una diferencia pequeña en la métrica no vuelve correcto el procedimiento.

3. Ridge en una dirección casi redundante

Use el conjunto sintético y añada $\mathbf{x}_4 = \mathbf{x}_1 + 0.001 \cdot \text{ruido}$, con semilla fija.

1. Estandarice las características usando solo entrenamiento.
2. Compare valores singulares y coeficientes de OLS con los de Ridge para una rejilla de λ .
3. Verifique la fórmula $\sigma_j / (\sigma_j^2 + n\lambda)$ en las direcciones singulares.
4. Compare pérdida de entrenamiento, validación y norma de coeficientes.
5. Explique qué estabilidad se ganó y qué sesgo se introdujo.

4. Lasso y columnas correlacionadas

Genere dos predictores altamente correlacionados y una respuesta que dependa de ambos.

1. Ajuste Lasso para una secuencia de penalizaciones.
2. Verifique la condición de subgradiente en una coordenada igual a cero.
3. Repita el experimento con diez semillas y registre cuál variable queda activa.
4. Compare predicciones y selección de columnas entre repeticiones.
5. Redacte una conclusión que distinga esparsidad computada, estabilidad y relevancia sustantiva.

5. Validación con 32 subzonas

Use el caso de Bogotá.

1. Defina una tubería con estandarización y Ridge.
2. Compare una rejilla de penalizaciones mediante particiones fijadas o validación cruzada.
3. Incluya el baseline en cada división y conserve resultados por subzona.
4. Muestre la distribución de métricas y la estabilidad de coeficientes.
5. Escriba qué conclusión permiten esas 32 filas y qué población adicional no puede afirmarse sin un diseño de muestreo.

Respuestas seleccionadas

Problema 1. Los MSE de validación para grados 1, 3, 8 y 15 son aproximadamente 2.223, 2.172, 2.379 y 3.254. El mínimo de entrenamiento del grado 15 no justifica escogerlo: la configuración se elige con observaciones que no intervinieron en sus coeficientes.

Problema 3. Si una dirección tiene σ_j pequeño, OLS multiplica la componente correspondiente de $\mathbf{U}^\top \mathbf{y}$ por $1/\sigma_j$. Ridge la multiplica por $\sigma_j/(\sigma_j^2 + n\lambda)$, que permanece acotado y tiende a cero cuando λ crece.

Regresión logística

Probabilidades, pérdida y decisiones en clasificación binaria

El conjunto `clasificacion_binaria_sintetica.csv` contiene 240 observaciones. La respuesta `target` vale 1 o 0 y las entradas disponibles para el modelo son `x1` y `x2`. La columna `slice` identifica dos periodos de generación. En las 180 observaciones del periodo `base`, la proporción de unos es 0.428.

Una regla que siempre predice la clase mayoritaria alcanza accuracy 0.572 sin utilizar ninguna entrada y nunca detecta un positivo. Ese número fija un baseline, pero no responde la pregunta más importante: ¿podemos estimar cómo cambia la probabilidad de $Y = 1$ con $X = (X_1, X_2)$?

El archivo también publica `score_latente` y `probabilidad` para que el mecanismo sintético pueda auditarse. Esas columnas revelan directamente cantidades usadas para generar la respuesta. Incluirlas como predictores convertiría el ejercicio en filtración de información. El modelo se ajustará solo con `x1` y `x2`.

De una clase a una probabilidad condicional

Para una respuesta binaria,

$$Y \mid X = x \sim \text{Bernoulli}(\eta(x)),$$

donde

$$\eta(x) = \mathbb{P}(Y = 1 \mid X = x) = \mathbb{E}[Y \mid X = x].$$

η es una propiedad de la distribución poblacional. La regresión logística no la conoce; construye una aproximación dentro de la familia

$$p_{\theta}(x) = \sigma(\phi(x)^{\top} \theta),$$

con

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

La sigmoide es creciente, toma valores estrictamente entre 0 y 1 y satisface $\sigma(0) = 1/2$. Su implementación debe evitar desbordamientos para logits de gran magnitud:

```
import numpy as np

def sigmoid(z):
    z = np.asarray(z, dtype=float)
    output = np.empty_like(z)
    nonnegative = z >= 0
    output[nonnegative] = 1 / (1 + np.exp(-z[nonnegative]))
    exp_z = np.exp(z[~nonnegative])
    output[~nonnegative] = exp_z / (1 + exp_z)
    return output
```

Para un diseño $\mathbf{X} \in \mathbb{R}^{n \times p}$,

$$z = \mathbf{X}\theta, \quad \hat{p} = \sigma(z).$$

La igualdad $\eta(x) = p_\theta(x)$ es un supuesto de especificación, no una consecuencia de aplicar una sigmoide. Por eso diremos *probabilidad estimada* al hablar de la salida ajustada.

Una interpretación mediante odds

La inversa de la sigmoide da

$$\log \left(\frac{p_\theta(x)}{1 - p_\theta(x)} \right) = \phi(x)^\top \theta.$$

Si las demás columnas se mantienen fijas, aumentar una unidad la característica j suma θ_j a los log-odds y multiplica los odds por e^{θ_j} . El cambio en probabilidad no es constante: depende del punto de la sigmoide en el que se evalúa.

La pérdida proviene del modelo Bernoulli

Si una observación tiene probabilidad estimada p_i , su masa Bernoulli es

$$p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Bajo independencia condicional, la verosimilitud de la muestra es el producto de esas masas. Maximizar su logaritmo equivale a minimizar la log-loss promedio

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

Con $z_i = \phi(x_i)^\top \theta$, la misma cantidad puede escribirse de forma numéricamente estable:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n [\log(1 + e^{z_i}) - y_i z_i].$$

```
def log_loss_from_logits(y, z):
    y = np.asarray(y, dtype=float)
    z = np.asarray(z, dtype=float)
    return np.mean(np.logaddexp(0.0, z) - y * z)

def log_loss_binary(y, p, eps=1e-15):
    y = np.asarray(y, dtype=float)
    p = np.clip(np.asarray(p, dtype=float), eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

Recortar probabilidades evita evaluar $\log(0)$ cuando solo se dispone de probabilidades. Trabajar con logits mediante `logaddexp` es preferible durante el ajuste.

Por qué la log-loss busca la probabilidad condicional

Fijemos x y sea $\eta = \mathbb{P}(Y = 1 \mid X = x)$. El riesgo condicional de reportar $q \in (0, 1)$ es

$$L_x(q) = -\eta \log q - (1 - \eta) \log(1 - q).$$

La diferencia respecto del mínimo es

$$L_x(q) - L_x(\eta) = \text{KL}(\text{Bern}(\eta) \parallel \text{Bern}(q)) \geq 0.$$

La igualdad ocurre en $q = \eta$. La log-loss no solo revisa de qué lado de un umbral queda una observación: compara una distribución Bernoulli completa. Una probabilidad muy segura y equivocada recibe una penalización grande.

Gradiente, Hessiana y convexidad

Para una observación,

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

y la regla de la cadena reduce la derivada de la log-loss respecto del logit a

$$\frac{\partial \ell_i}{\partial z_i} = \hat{p}_i - y_i.$$

Como $\nabla_{\theta} z_i = \phi(x_i)$, al sumar las observaciones se obtiene

$$\nabla J(\theta) = \frac{1}{n} \mathbf{X}^{\top} (\hat{\mathbf{p}} - \mathbf{y}).$$

```
def logistic_gradient(X, y, theta):  
    probabilities = sigmoid(X @ theta)  
    return X.T @ (probabilities - y) / X.shape[0]
```

Sea $\mathbf{W}(\theta)$ la matriz diagonal con entradas

$$W_{ii} = \hat{p}_i(1 - \hat{p}_i).$$

La Hessiana es

$$\nabla^2 J(\theta) = \frac{1}{n} \mathbf{X}^{\top} \mathbf{W}(\theta) \mathbf{X}.$$

Para todo $v \in \mathbb{R}^p$,

$$v^{\top} \nabla^2 J(\theta) v = \frac{1}{n} \sum_{i=1}^n \hat{p}_i(1 - \hat{p}_i) (\phi(x_i)^{\top} v)^2 \geq 0.$$

Convexidad de la log-loss logística. La log-loss empírica de la regresión logística es convexa. Si no hay direcciones no identificadas y los pesos son positivos, es estrictamente convexa en la región correspondiente.

La convexidad no garantiza que el mínimo se alcance en un parámetro finito. Si una recta separa perfectamente las clases, multiplicar sus coeficientes puede llevar las probabilidades hacia 0 y 1 mientras la pérdida se aproxima a cero y $\|\theta\|$ crece sin límite.

Pérdida decreciente con coeficientes crecientes. En datos separables, esta trayectoria no indica que falten iteraciones. Indica que el problema sin penalización no tiene un minimizador finito. Una penalización L2, sin incluir el intercepto, controla esa dirección y produce una solución finita.

Un paso batch de gradiente cuesta $O(np)$. Los métodos de Newton utilizan la Hessiana, cuyo ensamblaje denso cuesta $O(np^2)$ y cuya solución cuesta $O(p^3)$. Las bibliotecas eligen entre métodos de Newton, quasi-Newton, gradiente y coordenadas según el tamaño y la penalización. El objetivo estadístico y el solver son decisiones diferentes.

La probabilidad todavía no es una decisión

Una regla binaria introduce un umbral τ :

$$\hat{y}_i(\tau) = \begin{cases} 1, & \hat{p}_i \geq \tau, \\ 0, & \hat{p}_i < \tau. \end{cases}$$

```
def predict_class(p, threshold=0.5):  
    return (np.asarray(p) >= threshold).astype(int)
```

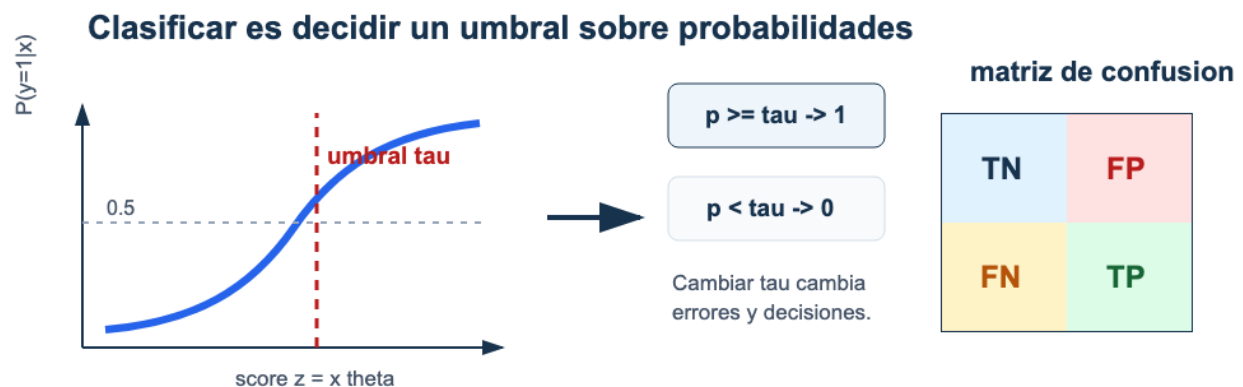


Figura 13: La sigmoide produce probabilidades. El umbral convierte esas probabilidades en clases y modifica la matriz de confusión.

El valor 0.5 no es una ley del modelo. Si un falso negativo cuesta más que un falso positivo, puede ser razonable predecir la clase positiva con una probabilidad menor. Si las probabilidades fueran conocidas y calibradas, con costos C_{FP} y C_{FN} la regla de costo esperado elegiría la clase positiva cuando

$$p \geq \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

En la práctica, p es estimada, los costos pueden ser incompletos y la calibración puede fallar. El umbral se fija con validación y su consecuencia se reporta mediante conteos, no solo mediante una métrica agregada.

Matriz de confusión y métricas

Adoptaremos la orientación habitual:

clase observada	predicho 0	predicho 1
real 0	TN	FP
real 1	FN	TP

```
def confusion_counts(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    return tp, fp, tn, fn
```

Los cuatro conteos producen

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \frac{\text{precision recall}}{\text{precision} + \text{recall}}.$$

Cuando un denominador es cero, la métrica necesita una convención explícita; no debe producirse un número silenciosamente.

```
def classification_metrics(y_true, y_pred):
    tp, fp, tn, fn = confusion_counts(y_true, y_pred)
    total = tp + fp + tn + fn
    accuracy = (tp + tn) / total if total else np.nan
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = (
        2 * precision * recall / (precision + recall)
        if precision + recall else 0.0
    )
    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

Accuracy pondera por igual las observaciones y puede ocultar que la clase positiva nunca se detecta. Precision condiona en las predicciones positivas; recall condiona en los positivos reales. F_1 combina ambas, pero tampoco incorpora por sí solo un costo, una capacidad de revisión ni la prevalencia de la población de uso.

Un umbral fijado con validación

Usamos únicamente las 180 observaciones del periodo **base**: 108 para entrenamiento, 36 para validación y 36 para prueba, con estratificación y semillas fijas. Una regresión logística ajustada con **x1** y **x2** obtiene log-loss de validación 0.533. Las clases de validación cambian así:

umbral	TN	FP	FN	TP	accuracy	precision	recall	F1
0.3	6	15	1	14	0.556	0.483	0.933	0.636
0.4	11	10	2	13	0.667	0.565	0.867	0.684
0.5	18	3	3	12	0.833	0.800	0.800	0.800
0.6	20	1	7	8	0.778	0.889	0.533	0.667
0.7	20	1	12	3	0.639	0.750	0.200	0.316

Supongamos que el requisito fijado antes de mirar la tabla es alcanzar recall al menos 0.85 y, entre los umbrales que lo cumplen, usar el mayor para limitar falsos positivos. En esta rejilla se escoge $\tau = 0.4$.

```
def choose_threshold_for_recall(y_true, p, min_recall=0.85):
    candidates = np.sort(np.unique(p))
    feasible = []
    for threshold in candidates:
        metrics = classification_metrics(
            y_true, predict_class(p, threshold)
        )
        if metrics["recall"] >= min_recall:
            feasible.append(threshold)
    return max(feasible) if feasible else None
```

Una vez fijado el modelo y el umbral, prueba se consulta una sola vez. Allí se obtienen TN 9, FP 11, FN 0 y TP 16: accuracy 0.694, precision 0.593, recall 1.0 y $F_1 = 0.744$. El umbral 0.5 habría dado mayor accuracy en prueba, pero escogerlo después de ver ese resultado violaría el procedimiento declarado.

La regla mayoritaria alcanza accuracy 0.556 en esta prueba y recall cero. El baseline muestra por qué la métrica primaria debe definirse antes de comparar modelos.

Calibración y alcance de las probabilidades

Un modelo está calibrado si, entre unidades con probabilidad estimada cercana a q , la frecuencia observada de positivos es cercana a q . La calibración es distinta del umbral y también de la capacidad para ordenar casos. Una sigmoide no garantiza calibración fuera de muestra.

En el dataset sintético podemos comparar las probabilidades estimadas con la columna **probabilidad**, porque conocemos el mecanismo generador. Esa comparación es un privilegio del experimento controlado. En una aplicación real, la calibración se estima con observaciones reservadas y conserva incertidumbre.

Las 60 filas del periodo **reciente** no se usarán para reajustar el ejemplo ni para escoger el umbral. Su distribución fue modificada deliberadamente y se retomará en la parte de estrategia para estudiar qué ocurre cuando cambian la prevalencia o las entradas. Mezclarlas aquí ocultaría precisamente el cambio que queremos diagnosticar después.

Síntesis

La regresión logística aproxima la probabilidad condicional de una respuesta Bernoulli mediante una sigmoide aplicada a un predictor lineal. La log-loss proviene de la verosimilitud Bernoulli y es una pérdida propia: su objetivo poblacional es la probabilidad condicional, no solo una etiqueta.

El gradiente conserva la forma matricial $\mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y})/n$ y la Hessiana es semidefinida positiva. La convexidad evita mínimos locales espurios, pero la separabilidad puede llevar los coeficientes al infinito si no hay regularización.

El modelo termina en una probabilidad estimada. El umbral añade una decisión y determina la matriz de confusión. Precision, recall, F_1 y accuracy resumen aspectos distintos; ninguna reemplaza la declaración de la clase positiva, los costos y el procedimiento de validación.

Problemas integrados

1. Del mecanismo Bernoulli al gradiente

Use las filas **base** de `clasificacion_binaria_sintetica.csv` y solo **x1**, **x2** como entradas.

1. Calcule la prevalencia y el baseline mayoritario.
2. Derive la log-loss desde la masa Bernoulli.
3. Implemente sigmoide, pérdida desde logits y gradiente.
4. Verifique el gradiente por diferencias finitas en cinco vectores de parámetros.
5. Compare su ajuste con `LogisticRegression` usando la misma penalización o una penalización suficientemente pequeña y documentada.

2. Una probabilidad y dos decisiones

Considere etiquetas $\mathbf{y} = (1, 0, 1, 0, 1, 0)$ y probabilidades $\hat{p} = (0.92, 0.61, 0.55, 0.40, 0.20, 0.10)$.

1. Calcule log-loss sin aplicar umbral.
2. Construya matrices de confusión para $\tau = 0.5$ y $\tau = 0.7$.
3. Calcule accuracy, precision, recall y F_1 .
4. Proponga costos que hagan razonable cada umbral.
5. Explique qué cambió en la decisión y qué permaneció igual en el modelo.

3. Selección de umbral reproducible

Reproduzca las particiones 108/36/36 del periodo **base**.

1. Ajuste estandarización y regresión solo con entrenamiento.
2. Construya la tabla de validación para una rejilla de umbrales.
3. Declare antes de consultar prueba una restricción de recall o una función de costo.
4. Fije el umbral, evalúe prueba una sola vez e incluya los cuatro conteos.
5. Compare con la clase mayoritaria y explique por qué no cambiaría el umbral después de ver prueba.

4. Separabilidad y regularización

Construya dos grupos linealmente separables en $\mathbb{R}^2$.

1. Ejecute gradiente sin penalización y registre pérdida y norma de parámetros.
2. Muestre que las clases permanecen correctas mientras la norma crece.
3. Añada una penalización L2 que no incluya el intercepto.
4. Compare convergencia, probabilidades y frontera para varias intensidades.
5. Explique por qué un minimizador finito es una propiedad distinta de tener accuracy de entrenamiento igual a uno.

Respuestas seleccionadas

Problema 2. Con $\tau = 0.5$ se obtiene $(TP, FP, TN, FN) = (2, 1, 2, 1)$ y las cuatro métricas son aproximadamente 0.667. Con $\tau = 0.7$ se obtiene $(1, 0, 3, 2)$: precision es 1, recall es $1/3$, accuracy es $2/3$ y $F_1 = 0.5$. Las probabilidades y su log-loss no cambian al mover el umbral.

Problema 4. En datos separables, multiplicar por una constante positiva un vector que separa aumenta los márgenes y hace que la log-loss se acerque a cero. La penalización L2 crece cuadráticamente con esa constante y evita que la norma pueda aumentar sin costo.

Problemas integradores de regresión y clasificación

De la muestra y la pérdida a una conclusión evaluada

Los problemas de este capítulo reúnen las ideas de la Parte II. No están separados en preguntas conceptuales, cálculos y programación: cada secuencia exige conectar esos niveles. Los datasets se encuentran en el directorio `datos/` del libro y fueron congelados para que los resultados sean reproducibles.

Una solución completa debe declarar la unidad de observación, las dimensiones de los arreglos, la función que se optimiza y los datos usados para cada decisión. Los resultados numéricos deben acompañarse de un diagnóstico y de una conclusión cuyo alcance coincida con la población descrita.

Síntesis de la parte

Regresión lineal, Ridge y regresión logística pueden escribirse mediante una matriz de diseño $\mathbf{X}$, un vector de parámetros θ y una pérdida empírica. Esa escritura común no vuelve idénticos sus objetivos: mínimos cuadrados aproxima una media condicional, log-loss aproxima una probabilidad condicional y las penalizaciones modifican el estimador para controlar su sensibilidad.

La optimización responde cómo calcular parámetros para un problema fijado. La validación responde cómo escoger entre procedimientos que aprendieron de los datos. Los residuales, las curvas de pérdida, el rango, las métricas y la matriz de confusión examinan fallos diferentes. Ninguno corrige una unidad de observación equivocada, una variable filtrada o una población de uso mal definida.

Problemas integrados

1. Una población conocida y muchas muestras

Considere las 160 filas de `regresion_lineal_sintetica.csv` como una población finita. El mecanismo generador conocido es

$$Y = 4 + 2.5X_1 - 1.2X_2 + 0.7X_3 + \varepsilon, \quad \mathbb{E}[\varepsilon \mid X] = 0.$$

1. Calcule el promedio de `target`, el riesgo cuadrático del baseline constante y el riesgo de la función generadora sobre las 160 filas.

2. Extraiga 500 muestras de tamaño 40. En cada una ajuste un intercepto y las tres variables mediante `np.linalg.lstsq`.
3. Grafique las distribuciones muestrales de los cuatro coeficientes y compare sus centros con $(4, 2.5, -1.2, 0.7)$.
4. Compare, para cada muestra, MSE de ajuste y MSE sobre la población finita.
5. Identifique una muestra cuyo error de ajuste sea menor que el de otra, pero cuyo error poblacional sea mayor.
6. Explique la diferencia entre parámetro poblacional, estimador, estimación y riesgo empírico en este experimento.
7. Indique qué parte del análisis dejaría de estar disponible si las 160 filas fueran una muestra y no una población conocida.

2. Mínimos cuadrados desde tres representaciones

Use

$$x = (0, 1, 2, 3), \quad y = (1.1, 2.9, 5.2, 6.8).$$

1. Obtenga intercepto y pendiente mediante las fórmulas centradas de regresión simple.
2. Construya $\mathbf{X}$ con intercepto y derive las ecuaciones normales.
3. Calcule la solución con `lstsq`; reporte rango, valores singulares, predicciones, residuales y MSE.
4. Verifique $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$ y explique su interpretación geométrica.
5. Añada una tercera columna igual a dos veces la segunda. Encuentre dos vectores de parámetros diferentes que generen las mismas predicciones.
6. Compare la solución de norma mínima de `lstsq` con una solución construida por usted.
7. Explique cuáles afirmaciones dependen del rango completo y cuáles permanecen válidas cuando el diseño es deficiente.

3. Una unidad que cambia la optimización

Use las tres características del conjunto sintético. Expresé $\mathbf{x}_2$ primero en su escala original y luego multiplicada por 1000.

1. Calcule $\kappa_2(\mathbf{X})$, el espectro de $\mathbf{H} = \mathbf{X}^\top \mathbf{X}/n$ y el intervalo estable de tasas en las dos representaciones.
2. Ejecute descenso del gradiente con una misma tasa y documente las dos curvas de pérdida.
3. Estandarice las características, sin transformar el intercepto, y repita el cálculo espectral.
4. Verifique el gradiente por diferencias finitas antes de interpretar la convergencia.
5. Compare el límite con `lstsq` y transforme los coeficientes a las unidades originales.
6. Demuestre que las predicciones coinciden después de deshacer la estandarización.
7. Distinga el efecto de la unidad sobre la realidad medida, los coeficientes, la geometría del algoritmo y la función ajustada.

4. Complejidad, selección y prueba

Use únicamente `x1` para predecir `target`. Construya una partición entrenamiento/validación/prueba con semillas fijadas antes de calcular métricas.

1. Compare grados 1, 3, 8 y 15 mediante un pipeline que incluya `PolynomialFeatures` y `StandardScaler`.
2. Grafique MSE de entrenamiento y validación en función del grado.
3. Para los grados 8 y 15, compare Ridge en una rejilla logarítmica de penalizaciones.
4. Registre MSE, norma de coeficientes y número efectivo de características.
5. Fije una regla de selección que use solo validación y aplíquela sin consultar prueba.
6. Evalúe la configuración seleccionada y el baseline una sola vez en prueba.
7. Repita la partición completa con cinco semillas para estudiar la estabilidad de la selección, sin reemplazar la evaluación final por la semilla más favorable.
8. Redacte una conclusión que separe optimización, selección e incertidumbre por partición.

5. Ridge y Lasso ante predictores correlacionados

Genere, con semilla fija,

$$X_2 = X_1 + 0.02U, \quad Y = 2X_1 + 2X_2 + \varepsilon,$$

donde U y ε son ruidos independientes centrados.

1. Divida los datos y ajuste la estandarización solo con entrenamiento.
2. Compare OLS, Ridge y Lasso sobre rejillas de penalización.
3. Muestre trayectorias de coeficientes y métricas de validación.
4. Para Ridge, relacione el encogimiento con los valores singulares del diseño.
5. Para una coordenada que Lasso deja en cero, verifique la condición de subgradiente.
6. Repita con veinte semillas y registre la frecuencia con la que cada variable queda activa.
7. Compare estabilidad de coeficientes y estabilidad de predicciones.
8. Explique por qué un coeficiente cero no demuestra que la variable carezca de relación con la respuesta.

6. De Bernoulli a regresión logística

Use únicamente las 180 filas `base` de `clasificacion_binaria_sintetica.csv`. No utilice `score_latente`, `probabilidad` ni `slice` como predictores.

1. Identifique la clase positiva, calcule su prevalencia y defina dos baselines.
2. A partir de la masa Bernoulli, derive log-loss, gradiente y Hessiana.
3. Implemente `sigmoid`, `log_loss_from_logits` y `logistic_gradient`.
4. Compruebe el gradiente por diferencias finitas en parámetros aleatorios con semilla fija.
5. Ajuste el modelo por gradiente y compare pérdida, parámetros y predicciones con una implementación de biblioteca cuya penalización esté documentada.
6. Verifique numéricamente que la Hessiana sea semidefinida positiva en varios puntos.

7. Compare las probabilidades estimadas con la columna `probabilidad` solo como auditoría posterior del mecanismo sintético.
8. Explique por qué esa comparación no estaría disponible en una aplicación ordinaria.

7. Un modelo, varios umbrales

Conserve una partición independiente de validación y otra de prueba para el problema anterior.

1. Calcule log-loss de validación sin convertir probabilidades en clases.
2. Para umbrales entre 0.1 y 0.9, construya los cuatro conteos y las métricas.
3. Grafique precision, recall y F_1 en función del umbral.
4. Defina dos escenarios de costos y derive el umbral teórico bajo probabilidades calibradas.
5. Fije con validación un umbral para cada escenario y explique cualquier diferencia respecto del umbral teórico.
6. Evalúe prueba una sola vez para el escenario elegido.
7. Compare contra la clase mayoritaria e incluya siempre su recall.
8. Escriba qué afirmaría sobre la decisión y qué no afirmaría sobre la probabilidad verdadera.

8. Cuando las clases son separables

Construya en $\mathbb{R}^2$ dos grupos que una recta separe perfectamente.

1. Ajuste regresión logística sin penalización durante un número creciente de iteraciones.
2. Registre log-loss, accuracy, margen mínimo y norma de parámetros.
3. Explique por qué accuracy puede estabilizarse en uno mientras la log-loss y los parámetros siguen cambiando.
4. Demuestre que multiplicar una dirección separadora reduce la pérdida sin alcanzar cero con parámetros finitos.
5. Añada Ridge sin penalizar el intercepto y compare las soluciones.
6. Estudie cómo cambia el margen y la calibración aparente al variar la penalización.
7. Distinga existencia de un minimizador, convergencia del solver y desempeño fuera de muestra.

9. Un análisis completo de las subzonas de Bogotá

Use `vivienda_nueva_bogota_subzonas.csv`.

1. Reconstruya el diccionario de las siete columnas y documente la unidad de observación. No invente la unidad de `precio_reportado`.
2. Formule una pregunta descriptiva y una predictiva que tengan a la subzona como unidad.
3. Compare baseline, OLS y Ridge mediante un procedimiento de validación fijado antes de ver resultados.
4. Mantenga estandarización, ajuste y selección dentro de un pipeline.
5. Reporte métricas por partición, residuales, rango, valores singulares y estabilidad de coeficientes.
6. Identifique una subzona cuyo residual merezca revisar la documentación, sin eliminarla automáticamente.

7. Explique por qué una mejora predictiva entre subzonas no describe viviendas individuales ni establece causalidad.
8. Escriba una conclusión de no más de 180 palabras con resultado, baseline, variación observada y dos limitaciones concretas.

10. Diseñar una investigación reproducible

Elija un conjunto tabular diferente de los tres casos conductores.

1. Defina unidad, población, salida, momento de predicción y uso previsto.
2. Justifique si el problema es regresión o clasificación y cuál objeto condicional intenta aproximar.
3. Declare un baseline, una métrica y una regla de partición antes del ajuste.
4. Implemente una función central desde cero y compruébela contra una biblioteca.
5. Compare al menos una decisión de complejidad o regularización con validación.
6. Incluya un diagnóstico de optimización y otro de comportamiento fuera de muestra.
7. Conserve semillas, versiones, nombres de columnas y resultados necesarios para reproducir el análisis.
8. Redacte una conclusión que incluya una afirmación respaldada, una limitación y la próxima comprobación necesaria.

Respuestas seleccionadas

Problema 1. En las 160 filas, el promedio de `target` es aproximadamente 4.121. El MSE del baseline constante es 5.277 y el de la función generadora es 0.289. El segundo valor no es cero porque la respuesta incluye ruido incluso condicionado en las entradas.

Problema 2. La solución es $\hat{\theta} = (1.09, 1.94)^\top$, con predicciones (1.09, 3.03, 4.97, 6.91), residuales (0.01, -0.13, 0.23, -0.11) y MSE 0.0205. La suma de residuales y su producto con la columna de x son cero salvo redondeo.

Problema 3. Al multiplicar `x2` por 1000, el número de condición del diseño es aproximadamente 1311 y el límite de paso es 1.68×10^{-6} . Después de estandarizar, esos valores son aproximadamente 2.16 y 1.207. La unidad altera los parámetros y la trayectoria, pero las predicciones pueden transformarse de manera equivalente.

Problema 6. La Hessiana tiene la forma

$$\frac{1}{n} \mathbf{X}^\top \mathbf{W} \mathbf{X}, \quad W_{ii} = \hat{p}_i(1 - \hat{p}_i) \geq 0.$$

Por ello $v^\top \mathbf{X}^\top \mathbf{W} \mathbf{X} v / n = \|\mathbf{W}^{1/2} \mathbf{X} v\|_2^2 / n \geq 0$ para todo v .

Problema 9. Una conclusión admisible se refiere a asociaciones y predicción entre las 32 subzonas incluidas. No puede convertirse en una conclusión sobre viviendas individuales: las filas son agregadas y el archivo no conserva la distribución interna de cada subzona.

Parte III: Redes neuronales

Una regresión logística transforma una combinación lineal de las variables en una probabilidad. Una red neuronal conserva esa idea y la repite en varias capas: cada capa aplica una transformación afín y una función no lineal, y la composición completa representa una familia mucho más flexible de funciones.

El estudio comienza con la propagación hacia adelante y con las dimensiones de los tensores que intervienen. La entropía cruzada extiende la log-loss a varias clases y permite interpretar la salida *softmax* como una distribución condicional estimada. Después, la regla de la cadena conduce a *backpropagation*, no como una fórmula para memorizar, sino como un procedimiento que recorre en sentido inverso el grafo de cálculo.

Las capas adicionales también introducen nuevos problemas. La inicialización es aleatoria, los gradientes pueden calcularse con minibatches y distintas corridas pueden producir modelos diferentes. Los capítulos finales de esta parte separan esas fuentes de variación, estudian regularización y optimizadores, y muestran qué automatiza PyTorch y qué debe seguir verificando quien implementa el modelo.

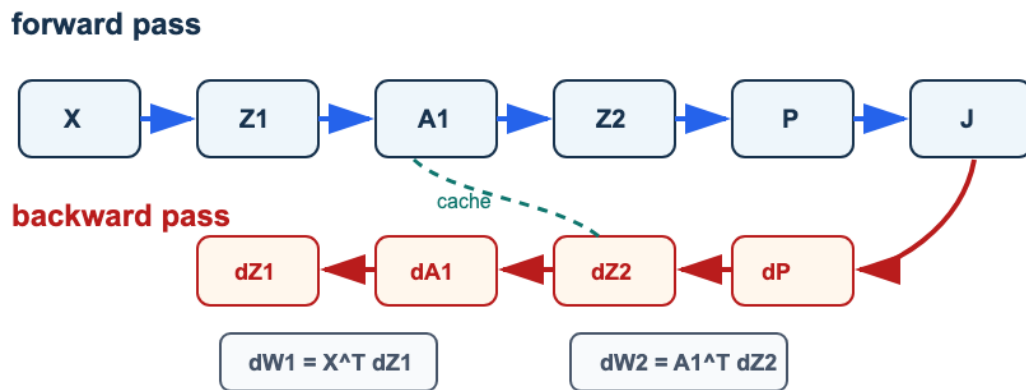


Figura 14: Backpropagation como grafo: el forward calcula activaciones y pérdida; el backward propaga gradientes usando la cache.

La figura resume las dos direcciones del cálculo. La pasada *forward* conserva las cantidades que serán necesarias después; la pasada *backward* combina derivadas locales para obtener los gradientes de los parámetros. Seguir las formas de esos objetos es una de las comprobaciones más eficaces de toda implementación.

Perceptron, neurona logística y forward propagation

Pregunta aplicada

Suponga que tenemos imágenes pequeñas de dígitos escritos a mano. Una regresión logística puede separar dos clases simples, pero ¿qué ocurre cuando la frontera entre clases requiere interacciones no lineales entre píxeles?

Una red neuronal densa permite aprender representaciones intermedias. La idea central puede escribirse sin metáforas:

Una **red neuronal feedforward** es una composición de transformaciones afines y funciones no lineales.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. reconocer una neurona logística como una capa lineal seguida de una activación;
2. escribir las dimensiones correctas de una capa densa;
3. explicar por qué una activación no lineal cambia la capacidad del modelo;
4. implementar softmax de forma estable;
5. interpretar entropía cruzada como pérdida para clasificación multiclase.
6. contar parámetros y estimar el costo de una pasada forward;
7. distinguir capacidad expresiva, estabilidad numérica y calidad estadística.

Distribución predictiva y riesgo

Para K clases, representamos

$$Y \mid X = x \sim \text{Categorical}(\pi_1(x), \dots, \pi_K(x)), \quad \sum_{k=1}^K \pi_k(x) = 1.$$

Una red produce logits $z(x; \theta)$ y probabilidades $q_\theta(x) = \text{softmax}(z(x; \theta))$. La entropía cruzada empírica aproxima el riesgo poblacional

$$R_{\text{CE}}(\theta) = \mathbb{E}[-\log q_{\theta,Y}(X)].$$

La distribución verdadera minimiza entropía cruzada. Condicionado en $X = x$, para cualquier vector de probabilidades q ,

$$-\sum_{k=1}^K \pi_k(x) \log q_k = H(\pi(x)) + \text{KL}(\pi(x) \| q) \geq H(\pi(x)).$$

El mínimo se alcanza en $q = \pi(x)$. La red aproxima esa distribución dentro de su familia paramétrica; softmax por sí solo no garantiza calibración.

Si al inicio todas las clases reciben probabilidad $1/K$, la pérdida esperada por observación es $\log K$. Este valor es una comprobación probabilística del forward, no una meta de entrenamiento.

Perceptron como unidad básica

Dada una observación $x \in \mathbb{R}^p$, un perceptron calcula:

$$z = w^\top x + b.$$

Si se aplica una regla de decisión:

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0, \end{cases}$$

obtenemos un clasificador lineal.

Advertencia

El perceptron clásico produce una decisión dura. Para entrenar con gradientes resulta más conveniente usar funciones diferenciables y pérdidas suaves.

Neurona logística

La regresión logística ya puede verse como una neurona:

$$\hat{p} = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

La salida $\hat{p}$ es la probabilidad **estimada por el modelo** para la clase positiva. No es una probabilidad poblacional conocida: depende de los datos, la especificación y el ajuste. Esta construcción conecta el Módulo 1 con redes neuronales:

`regresión logística = una capa lineal + sigmoide`

De una neurona a una capa

Una capa densa con h neuronas calcula, para m observaciones y d variables de entrada:

$$Z^{[1]} = XW^{[1]} + b^{[1]},$$

donde:

- $X \in \mathbb{R}^{m \times d}$;
- $W^{[1]} \in \mathbb{R}^{d \times h}$;
- $b^{[1]} \in \mathbb{R}^h$;
- $Z^{[1]} \in \mathbb{R}^{m \times h}$.

Luego aplicamos una activación elemento a elemento:

$$A^{[1]} = g(Z^{[1]}).$$

Red de una capa oculta

Para clasificación multiclase con k clases:

$$Z^{[1]} = XW^{[1]} + b^{[1]},$$

$$A^{[1]} = \text{ReLU}(Z^{[1]}),$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]},$$

$$\hat{P} = \text{softmax}(Z^{[2]}).$$

Esta escritura define una función parametrizada

$$f_{\theta} : \mathbb{R}^d \longrightarrow \Delta^{k-1},$$

donde Δ^{k-1} es el simplex de probabilidades sobre k clases y

$$\theta = (W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}).$$

Con h unidades ocultas, el número de parámetros escalares es

$$dh + h + hk + k.$$

El ancho h modifica simultáneamente la capacidad del modelo, el costo de cómputo y el número de cantidades estimadas a partir de la muestra. Aumentarlo no es una mejora gratuita.

ReLU

$$\text{ReLU}(z) = \max(0, z).$$

ReLU introduce no linealidad y evita parte de la saturación que aparece con sigmoides en redes profundas.

Su derivada por partes es:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

En $z = 0$ la derivada clásica no existe. En este curso, y en el laboratorio, usamos el valor computacional 0.

Softmax

Para logits $z_1, \dots, z_k$:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{\ell=1}^k e^{z_\ell}}.$$

En implementación usamos una versión estable:

```
shifted = Z - np.max(Z, axis=1, keepdims=True)
exp_scores = np.exp(shifted)
probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)
```

Restar el máximo no cambia las probabilidades, pero evita overflow.

La invariancia se verifica directamente. Para cualquier constante c ,

$$\text{softmax}(z + c\mathbf{1})_j = \frac{e^{z_j+c}}{\sum_{\ell} e^{z_\ell+c}} = \frac{e^c e^{z_j}}{e^c \sum_{\ell} e^{z_\ell}} = \text{softmax}(z)_j.$$

Elegir $c = -\max_j z_j$ hace que el mayor exponente sea $e^0 = 1$ y que los demás no excedan 1. Esta transformación resuelve overflow; no evita por sí sola que probabilidades muy pequeñas se redondeen a cero. Para calcular la pérdida es preferible usar una operación `logsumexp` estable sobre logits.

Entropía cruzada multiclase

Si Y es one-hot y $\hat{P}$ contiene probabilidades estimadas:

$$J = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^k Y_{ij} \log(\hat{P}_{ij}).$$

La pérdida premia asignar alta probabilidad a la clase correcta.

Por qué hace falta una no linealidad

Antes del cálculo numérico, conviene entender por qué una activación no lineal importa. Supón dos capas lineales sin activación:

$$H = XW^{[1]} + b^{[1]}, \quad Z = HW^{[2]} + b^{[2]}.$$

Sustituyendo H :

$$Z = (XW^{[1]} + b^{[1]})W^{[2]} + b^{[2]} = X(W^{[1]}W^{[2]}) + b^{[1]}W^{[2]} + b^{[2]}.$$

Si definimos $W^* = W^{[1]}W^{[2]}$ y $b^* = b^{[1]}W^{[2]} + b^{[2]}$, entonces:

$$Z = XW^* + b^*.$$

Dos capas afines sin activación equivalen a una sola transformación afín. La activación no lineal es lo que evita ese colapso.

La conclusión es estructural, no una afirmación de que cualquier red con ReLU aprenderá una buena frontera. ReLU hace que la función sea afín por regiones; la cantidad y disposición de esas regiones dependen de pesos, profundidad y ancho. La optimización todavía debe encontrar parámetros útiles y la evaluación debe determinar si la representación generaliza.

Costo de una pasada forward

Para un lote de m observaciones, la multiplicación $XW^{[1]}$ cuesta $O(mdh)$ y $A^{[1]}W^{[2]}$ cuesta $O(mhk)$. ReLU y softmax añaden $O(mh + mk)$. El costo dominante es, por tanto,

$$O(mh(d + k)).$$

Durante entrenamiento se guardan activaciones y preactivaciones para calcular gradientes. En esta red pequeña, la cache ocupa $O(mh + mk)$ números, además de los parámetros. En inferencia no hace falta conservar todo el grafo y la memoria puede reducirse.

Esta cuenta permite detectar un error conceptual frecuente: la notación de una capa puede caber en una línea, pero su costo crece con el tamaño del lote, la entrada, el ancho y la salida. Una comparación de arquitecturas debe reportar al menos parámetros, presupuesto de actualizaciones y tamaño de lote.

Ejemplo resuelto: softmax y pérdida

Supón que una observación produce tres logits:

$$z = (2, 1, 0).$$

Para calcular softmax de forma estable restamos el máximo, que es 2:

$$z - \max(z) = (0, -1, -2).$$

Entonces:

$$e^{z - \max(z)} = (1, e^{-1}, e^{-2}) \approx (1, 0.368, 0.135).$$

La suma es aproximadamente 1.503, así que:

$$\text{softmax}(z) \approx (0.665, 0.245, 0.090).$$

Si la clase correcta es la primera, la entropía cruzada para esta observación es:

$$-\log(0.665) \approx 0.408.$$

Si la clase correcta fuera la tercera, la pérdida sería:

$$-\log(0.090) \approx 2.408.$$

La misma predicción puede ser buena o mala según la etiqueta verdadera. Por eso no basta mirar logits grandes: hay que mirar la probabilidad asignada a la clase correcta.

Ejemplo resuelto: dimensiones

Suponga:

- $m = 100$ observaciones;
- $d = 64$ píxeles por imagen;
- $h = 32$ neuronas ocultas;
- $k = 10$ clases.

Entonces:

Objeto	Forma
X	100×64
$W^{[1]}$	64×32
$b^{[1]}$	32
$A^{[1]}$	100×32
$W^{[2]}$	32×10
$\hat{P}$	100×10

Caso longitudinal: dígitos escritos a mano

En los capítulos de redes usaremos como referencia conceptual el dataset `load_digits`: imágenes pequeñas de 8×8 píxeles, convertidas en vectores de 64 entradas. Este dataset no representa visión por computador moderna; su valor pedagógico es que permite ver todas las formas de una red pequeña sin depender de internet ni de archivos pesados.

Objeto	Forma típica	Interpretación
X	$m \times 64$	intensidad de píxeles aplanados
$W^{[1]}$	$64 \times h$	pesos de entrada a capa oculta
$A^{[1]}$	$m \times h$	representación intermedia
$W^{[2]}$	$h \times 10$	pesos hacia diez clases
$\hat{P}$	$m \times 10$	probabilidades estimadas por dígito

La primera comparación no debe ser contra una red grande. Debe ser contra un baseline simple: clase mayoritaria, regresión logística o una red pequeña sin tuning agresivo. Si la red mejora el baseline, todavía falta revisar validación, errores por clase y reproducibilidad del entrenamiento.

Punto de control

Antes de pasar a backpropagation, verifica que puedes responder sin mirar la tabla:

1. Si una capa recibe 64 variables y produce 32 activaciones, ¿qué forma tiene su matriz de pesos?
2. Si una red produce 10 logits por observación, ¿por qué softmax se aplica por fila?

3. Si se quita ReLU, ¿qué tipo de frontera queda después de componer capas lineales?

Respuesta corta. La matriz tiene forma 64×32 . Softmax se aplica por fila porque cada fila contiene las probabilidades de una observación sobre las 10 clases. Sin activaciones no lineales, la composición de capas sigue siendo una transformación lineal.

Verificaciones seleccionadas

1. Si X tiene forma 50×64 y $W^{[1]}$ tiene forma 64×16 , entonces $Z^{[1]}$ tiene forma 50×16 .
2. Si una red multiclase produce logits 50×10 , la suma de softmax debe ser 1 en cada fila, no en cada columna.
3. Si se eliminan todas las activaciones no lineales, la red completa equivale a una transformación lineal; por eso no aumenta capacidad geométrica real.

Ejemplo resuelto: auditoría de formas en dos capas

Supón un lote $X \in \mathbb{R}^{32 \times 8}$, una capa oculta con 5 neuronas y una salida de 3 clases. Una implementación consistente debe declarar

$$W^{[1]} \in \mathbb{R}^{8 \times 5}, \quad b^{[1]} \in \mathbb{R}^5, \quad W^{[2]} \in \mathbb{R}^{5 \times 3}, \quad b^{[2]} \in \mathbb{R}^3.$$

Entonces $Z^{[1]} = XW^{[1]} + b^{[1]}$ tiene forma 32×5 , la activación $A^{[1]}$ conserva esa forma y los logits finales tienen forma 32×3 . Softmax se aplica sobre las 3 columnas de cada fila, porque cada fila representa una observación. Si softmax se aplicara por columna, se estarían normalizando observaciones distintas entre sí, lo cual no tiene interpretación como distribución de clases para un caso individual.

Esta auditoría debe hacerse antes de entrenar. El notebook verifica formas en puntos intermedios, porque un error de broadcasting puede producir un arreglo numérico sin representar el modelo matemático esperado.

Para explorar más

- **Extensión matemática:** calcula manualmente una pasada forward con dos observaciones, dos entradas y dos neuronas ocultas.
- **Extensión computacional:** implementa la misma red con `numpy` y luego con `PyTorch`; compara formas y valores intermedios.
- **Conexión con proyecto:** identifica si tu problema necesita una arquitectura no lineal o si un modelo lineal regularizado sigue siendo un baseline suficiente.

Revisión del capítulo

Este capítulo construye una red neuronal desde sus operaciones de forward pass. Una neurona combina una transformación lineal con una activación; una red densa apila esas operaciones para producir logits y probabilidades.

La no linealidad es esencial. Sin activaciones no lineales, varias capas lineales se pueden reescribir como una sola transformación lineal. ReLU introduce capacidad adicional y softmax convierte logits multiclase en una distribución de probabilidad por observación.

El criterio de dominio de este capítulo es la trazabilidad de formas. Antes de entrenar, debes poder decir la forma de W , b , Z , A , logits, probabilidades y etiquetas. Sin esa auditoría, los errores de broadcasting pueden parecer resultados válidos.

Términos clave

- **Perceptron:** unidad que combina entradas con pesos y sesgo para producir un puntaje real.
- **Capa densa:** transformación afín $XW + b$ aplicada a un lote de observaciones.
- **Activación:** función no lineal aplicada a los scores de una capa.
- **ReLU:** activación $\max(0, z)$ usada para introducir no linealidad.
- **Logit:** score real antes de convertirlo en probabilidad.
- **Softmax:** transformación que convierte logits multiclase en probabilidades por fila.
- **One-hot:** codificación de clases como vectores indicadores.
- **Entropía cruzada:** pérdida que penaliza baja probabilidad asignada a la clase correcta.

Ejercicios

Preguntas conceptuales

1. Explique por qué una red sin activaciones no lineales se reduce a una transformación lineal.
2. ¿Qué diferencia hay entre logits y probabilidades?

Problemas cuantitativos

3. Si X tiene forma $(250, 20)$ y una capa oculta tiene 12 neuronas, ¿qué forma deben tener $W1$, $b1$ y $Z1$?
4. Para una salida con 5 clases y batch de 32 observaciones, indique la forma de los logits, las probabilidades softmax y la matriz one-hot Y .

Práctica computacional

5. Implemente una función `softmax_stable(Z)` y verifique que cada fila suma 1.
6. Implemente un `forward_pass(X, params)` para una red con una capa oculta ReLU y salida softmax.

Interpretación y pensamiento crítico

7. Una red obtiene probabilidades casi uniformes para todas las clases. Proponga dos hipótesis técnicas.

Profundización matemática y algorítmica

8. Demuestre que $\text{softmax}(\mathbf{z} + \mathbf{c})$ coincide con $\text{softmax}(\mathbf{z})$ para cualquier constante c y explique por qué se elige $c = -\max_j z_j$.
9. Para $d = 64$, $h = 32$ y $k = 10$, calcule el número de parámetros de la red, incluidos sesgos.
10. Estime el costo dominante de un forward con lote $m = 128$ y compare qué ocurre al duplicar h .
11. Explique por qué mayor número de regiones lineales no garantiza menor riesgo de validación.

Proyecto de cierre

12. Use un dataset multiclase generado con semilla fija o una muestra pequeña de imágenes. Defina dimensiones, implemente forward propagation y entregue una tabla con formas de cada objeto intermedio.

Backpropagation

Pregunta aplicada

Forward propagation nos dice cómo una red produce predicciones. Pero entrenar exige responder:

¿Cómo cambia la pérdida si modificamos cada peso de la red?

Backpropagation es una forma eficiente de aplicar la regla de la cadena a una composición de funciones.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. leer una red como grafo de cómputo;
2. derivar las formas de $dW^{[2]}$, $db^{[2]}$, $dW^{[1]}$ y $db^{[1]}$;
3. explicar por qué softmax con entropía cruzada produce $dZ^{[2]} = (\hat{P} - Y)/m$;
4. usar una cache de forward para calcular backward;
5. diseñar una prueba básica de gradientes.
6. derivar el Jacobiano de softmax y la regla de una capa afín;
7. explicar por qué backward tiene costo comparable con forward y por qué necesita una cache.

Del riesgo esperado al gradiente calculable

El objetivo poblacional de una red es

$$R_P(\theta) = \mathbb{E}_{Z \sim P}[\ell(\theta; Z)],$$

pero backpropagation se aplica a la pérdida empírica

$$\hat{R}_m(\theta) = \frac{1}{m} \sum_{i=1}^m \ell(\theta; Z_i).$$

Bajo condiciones que permiten intercambiar derivada y esperanza,

$$\nabla R_P(\theta) = \mathbb{E}[\nabla_{\theta} \ell(\theta; Z)].$$

Esta identidad explica por qué gradientes de observaciones o mini-batches pueden usarse como estimadores del gradiente poblacional. En el curso, la igualdad que podemos verificar exactamente es la correspondiente al riesgo empírico.

Una prueba de gradientes comprueba derivadas para una muestra y parámetros fijos. No comprueba representatividad de los datos, calibración ni bajo riesgo de uso.

El grafo de cómputo

Para una red de una capa oculta:

- entrada X ;
- preactivación Z_1 ;
- activación A_1 ;
- logits Z_2 ;
- probabilidades P ;
- pérdida J .

Cada flecha representa una operación diferenciable o casi diferenciable. Backpropagation recorre el grafo en sentido inverso:

- primero J ;
- luego P , Z_2 , A_1 y Z_1 ;
- finalmente los parámetros.

forward pass

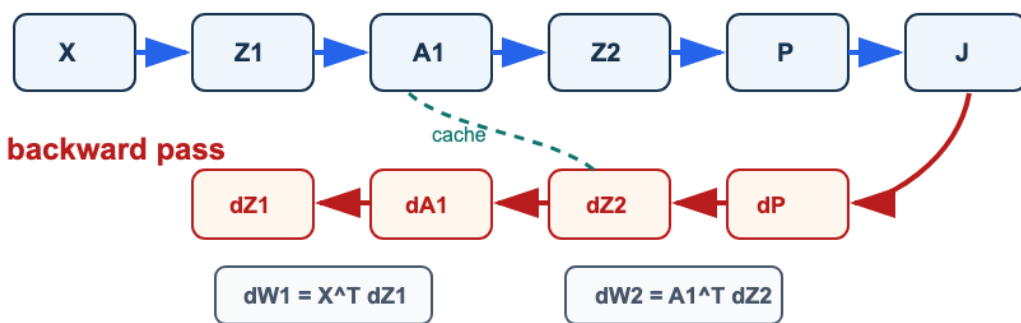


Figura 15: Backpropagation como grafo: el forward calcula X , Z_1 , A_1 , Z_2 , P y J ; el backward propaga dJ hacia dZ_2 , dW_2 , dA_1 , dZ_1 y dW_1 usando la cache.

Lectura de la figura. Forward produce valores intermedios; backward reutiliza esa cache para construir gradientes. Si una forma no coincide con el parámetro correspondiente, el error está en el recorrido del grafo o en una transposición.

Gradiente de softmax con entropía cruzada

La simplificación no se toma como una regla memorizada. Para una observación, escribamos

$$p_a = \frac{e^{z_a}}{\sum_{r=1}^k e^{z_r}}.$$

Al derivar p_a respecto a z_b aparecen dos casos, que se reúnen con la delta de Kronecker δ_{ab} :

$$\frac{\partial p_a}{\partial z_b} = p_a(\delta_{ab} - p_b).$$

Si $L = -\sum_{a=1}^k y_a \log p_a$, la regla de la cadena da

$$\begin{aligned} \frac{\partial L}{\partial z_b} &= -\sum_{a=1}^k \frac{y_a}{p_a} \frac{\partial p_a}{\partial z_b} \\ &= -\sum_{a=1}^k y_a (\delta_{ab} - p_b) \\ &= -y_b + p_b \sum_{a=1}^k y_a \\ &= p_b - y_b, \end{aligned}$$

porque una etiqueta one-hot satisface $\sum_a y_a = 1$. Al promediar m observaciones:

$$\frac{\partial J}{\partial Z^{[2]}} = \frac{1}{m}(\hat{P} - Y).$$

La cancelación entre el Jacobiano de softmax y la derivada del logaritmo es una de las razones por las que esta combinación es estándar. Si se cambia la pérdida o la transformación de salida, esta identidad debe derivarse de nuevo.

Gradientes de la segunda capa

Con:

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]},$$

definimos:

$$dZ^{[2]} = \frac{1}{m}(\hat{P} - Y).$$

Entonces:

$$dW^{[2]} = (A^{[1]})^\top dZ^{[2]},$$

$$db^{[2]} = \sum_i dZ_i^{[2]}.$$

Estas tres fórmulas provienen de una sola regla de capa. Para $Z = AW + \mathbf{1}_m b^\top$ y una perturbación diferencial,

$$dZ = dA W + A dW + \mathbf{1}_m (db)^\top.$$

Al emparejar cada término con el gradiente que llega desde la pérdida se obtiene

$$\nabla_W J = A^\top \nabla_Z J, \quad \nabla_b J = \mathbf{1}^\top \nabla_Z J, \quad \nabla_A J = \nabla_Z J W^\top.$$

La regla no depende de que la capa sea la primera o la última. Lo que cambia entre capas es la derivada de la activación y el gradiente que llega desde arriba.

Propagar hacia la capa oculta

Primero pasamos el gradiente hacia las activaciones:

$$dA^{[1]} = dZ^{[2]}(W^{[2]})^\top.$$

Luego atravesamos ReLU:

$$dZ^{[1]} = dA^{[1]} \odot \mathbf{1}\{Z^{[1]} > 0\}.$$

Finalmente:

$$dW^{[1]} = X^\top dZ^{[1]},$$

$$db^{[1]} = \sum_i dZ_i^{[1]}.$$

Backpropagation como algoritmo

Las identidades anteriores ya contienen la deducción. El algoritmo debe conservar los valores del forward correspondiente, iniciar con $dZ^{[2]} = (\hat{P} - Y)/m$ y aplicar la regla de capa en orden inverso. El código para dos capas es:

```
def backward_pass(X, Y, cache, params):
    P = cache["P"]
    A1 = cache["A1"]
    Z1 = cache["Z1"]
    W2 = params["W2"]

    dZ2 = (P - Y) / len(X)
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0)

    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
```

Para una red con L capas, el mismo argumento se recorre desde $\ell = L$ hasta 1:

```
calcular el gradiente de la pérdida respecto a la salida
para ell = L, ..., 1:
    atravesar la activación de la capa ell
    calcular dW[ell] y db[ell]
    propagar el gradiente hacia A[ell-1]
```

Backpropagation no es una fórmula particular de dos capas. Es evaluación en modo reverso de la regla de la cadena sobre un grafo acíclico de operaciones.

Costo, memoria y alcance de la garantía

Cada producto matricial del forward reaparece con dimensiones transpuestas en el backward. Por ello, el costo de backpropagation es del mismo orden que el forward: para la red de una capa oculta,

$$O(mh(d + k)).$$

El ahorro frente a diferencias finitas es decisivo. Si hay P parámetros, verificar todas las coordenadas por diferencias centrales requiere aproximadamente $2P$ evaluaciones forward; backpropagation obtiene los P gradientes en un solo recorrido inverso de costo comparable a unas pocas pasadas.

El precio es memoria. El backward necesita los valores de X , $Z^{[1]}$, $A^{[1]}$ y las probabilidades producidas por el mismo forward. Recalcularlos puede ahorrar memoria a cambio de más cómputo, estrategia conocida como *checkpointing*.

La eficiencia no es una garantía de encontrar el mínimo global. Backpropagation calcula, salvo errores numéricos y puntos no diferenciables elegidos por convención, el gradiente de la pérdida

empírica para los parámetros actuales. El optimizador decide cómo usarlo; la no convexidad de una red permanece.

Verificación por diferencias finitas

Una forma de detectar errores en backpropagation es comparar contra una aproximación numérica:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\theta_j + \varepsilon) - J(\theta_j - \varepsilon)}{2\varepsilon}.$$

Esta técnica no se usa para entrenar porque es costosa, pero es excelente para depurar.

Ejemplo resuelto: diferencia finita escalar

Considera la función simple:

$$J(w) = (w - 3)^2.$$

Su derivada exacta es:

$$J'(w) = 2(w - 3).$$

En $w = 2$, la derivada exacta es:

$$J'(2) = 2(2 - 3) = -2.$$

Ahora usa diferencia finita central con $\varepsilon = 0.01$:

$$\frac{J(2 + \varepsilon) - J(2 - \varepsilon)}{2\varepsilon} = \frac{J(2.01) - J(1.99)}{0.02}.$$

Calculamos:

$$J(2.01) = (-0.99)^2 = 0.9801, \quad J(1.99) = (-1.01)^2 = 1.0201.$$

Por tanto:

$$\frac{0.9801 - 1.0201}{0.02} = -2.$$

La aproximación coincide con la derivada exacta. En una red real comparamos el error relativo, no una igualdad exacta. Un ε demasiado grande introduce error de aproximación y uno demasiado pequeño puede sufrir cancelación numérica. Si la diferencia sigue siendo grande, revisamos signos, escalas, transpuestas y la división por m .

Ejemplo resuelto: formas de los gradientes

Supón que:

- $X \in \mathbb{R}^{100 \times 64}$;
- $W^{[1]} \in \mathbb{R}^{64 \times 32}$;
- $A^{[1]} \in \mathbb{R}^{100 \times 32}$;
- $W^{[2]} \in \mathbb{R}^{32 \times 10}$;
- $\hat{P}, Y \in \mathbb{R}^{100 \times 10}$.

Entonces:

Gradiente	Cálculo	Forma
$dZ^{[2]}$	$(\hat{P} - Y)/100$	100×10
$dW^{[2]}$	$(A^{[1]})^\top dZ^{[2]}$	32×10
$db^{[2]}$	suma por filas de $dZ^{[2]}$	10
$dA^{[1]}$	$dZ^{[2]}(W^{[2]})^\top$	100×32
$dW^{[1]}$	$X^\top dZ^{[1]}$	64×32

El primer chequeo es simple: cada gradiente debe tener la misma forma que el parámetro que actualiza. Pasar este chequeo no demuestra que sus valores sean correctos.

Errores frecuentes

1. Olvidar dividir por m .
2. Confundir $Y - P$ con $P - Y$.
3. Sumar db sin `axis=0`.
4. Usar las formas transpuestas incorrectas.
5. Usar valores de un forward distinto al que produjo la pérdida.
6. No estabilizar softmax.

Punto de control

Si el autograder dice que `dW2` tiene forma incorrecta, no empieces cambiando signos. Primero revisa:

1. ¿`A1.T @ dZ2` produce la misma forma que `W2`?
2. ¿`db2` suma sobre observaciones y no sobre clases?
3. ¿dividiste por m exactamente una vez?

Caso longitudinal: backpropagation en dígitos

En el caso `load_digits`, el forward produce probabilidades para diez clases. El backward debe producir gradientes con las mismas formas que los parámetros:

Parámetro	Forma si $h = 32$	Gradiente esperado
$W^{[1]}$	64×32	$dW^{[1]}$, misma forma
$b^{[1]}$	32	$db^{[1]}$, una entrada por neurona
$W^{[2]}$	32×10	$dW^{[2]}$, misma forma
$b^{[2]}$	10	$db^{[2]}$, una entrada por clase

La prueba de gradiente no se hace sobre todo el dataset. Se hace sobre un caso pequeño, con pocas observaciones y parámetros controlados, porque ahí se puede comparar contra diferencias finitas. Una red que “entrena” sin pasar una prueba de gradiente puede estar reduciendo la pérdida por accidente o por un error de implementación no detectado.

Verificaciones seleccionadas

1. Si $A^{[1]}$ tiene forma $m \times h$ y $dZ^{[2]}$ tiene forma $m \times k$, entonces $dW^{[2]} = (A^{[1]})^\top dZ^{[2]}$ tiene forma $h \times k$.
2. Una diferencia finita central usa $(J(\theta + \epsilon) - J(\theta - \epsilon))/(2\epsilon)$; suele ser más estable que usar solo $J(\theta + \epsilon) - J(\theta)$.
3. Un error de forma que NumPy resuelve con broadcasting puede producir un valor numérico sin representar el gradiente correcto.

Para explorar más

- **Extensión matemática:** verifica por diferencias finitas un solo peso de la primera capa y un solo peso de la segunda capa.
- **Extensión computacional:** agrega chequeos automáticos de forma para cada gradiente antes de actualizar parámetros.
- **Conexión con proyecto:** decide qué prueba mínima de gradientes o sanity check usarías antes de confiar en un entrenamiento propio.

Revisión del capítulo

Backpropagation aplica la regla de la cadena de forma organizada sobre un grafo de cómputo. El forward produce activaciones, logits, probabilidades y pérdida; el backward usa esos valores guardados para calcular gradientes.

La simplificación $dZ^{[2]} = (\hat{P} - Y)/m$ para softmax con entropía cruzada permite propagar gradientes hacia la segunda capa y luego hacia la capa oculta. Cada gradiente debe tener exactamente la misma forma que el parámetro que actualiza.

La verificación por diferencias finitas no reemplaza la derivación, pero sí detecta errores de signo, escala o forma. Una red que no mejora aunque sus formas sean correctas exige revisar inicialización, activaciones, tasa de aprendizaje y datos.

Términos clave

- **Grafo de cómputo:** representación de operaciones intermedias usadas para calcular una salida.
- **Cache:** valores guardados durante forward propagation para reutilizarlos en backward propagation.
- **Backpropagation:** aplicación eficiente de la regla de la cadena desde la pérdida hacia los parámetros.
- **Gradiente:** derivada de la pérdida respecto a un parámetro o activación.
- **Diferencias finitas:** aproximación numérica de derivadas para verificar gradientes.
- **Forma del gradiente:** dimensión que debe coincidir con el parámetro correspondiente.

Ejercicios

Preguntas conceptuales

1. Explique por qué backpropagation necesita valores guardados del forward pass.
2. Explique por qué $dZ1 = dA1 * (Z1 > 0)$ para ReLU.

Problemas cuantitativos

3. Derive la forma de $dW2$ a partir de $Z2 = A1 @ W2 + b2$.
4. Si $A^{[1]} \in \mathbb{R}^{80 \times 16}$ y $dZ^{[2]} \in \mathbb{R}^{80 \times 4}$, calcule la forma de $dW^{[2]}$ y $db^{[2]}$.

Práctica computacional

5. Diseñe una prueba pequeña para verificar que `backward_pass` devuelve gradientes con la misma forma que los parámetros.
6. Implemente una verificación por diferencias finitas para un parámetro escalar.

Durante esta verificación desactive dropout y cualquier otra operación aleatoria; la pérdida evaluada en $\theta + \varepsilon$ y $\theta - \varepsilon$ debe usar exactamente los mismos datos y el mismo grafo.

Interpretación y pensamiento crítico

7. Un gradiente tiene la forma correcta pero el entrenamiento no mejora. Proponga tres causas posibles.

Profundización matemática y algorítmica

8. Derive $\partial p_a / \partial z_b = p_a (\delta_{ab} - p_b)$ y úsela para obtener $\nabla_z L = p - y$.
9. A partir del diferencial de $Z = AW + \mathbf{1}b^\top$, obtenga los gradientes respecto a A , W y b .
10. Compare el costo de verificar P parámetros por diferencias centrales con el costo de una pasada backward.
11. Explique el intercambio memoria-cómputo de guardar activaciones frente a recalcularlas durante backward.

Proyecto de cierre

12. Tome una red pequeña de dos capas y documente una auditoría de gradientes: formas esperadas, formas obtenidas, norma de cada gradiente y resultado de una prueba numérica para un parámetro.

Activaciones, inicialización, regularización y diagnóstico

Pregunta aplicada

Una red puede fallar aunque el código no tenga errores. El entrenamiento puede divergir, saturarse, memorizar datos o quedarse sin aprender.

La implementación de backpropagation no basta:

hay que diagnosticar el entrenamiento.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. identificar fallas de saturación, overfitting y learning rate;
2. explicar por qué la inicialización afecta el entrenamiento;
3. incorporar regularización L2 en la pérdida y en los gradientes;
4. leer curvas de entrenamiento y validación;
5. convertir una curva en una hipótesis y un siguiente experimento.
6. deducir la escala de una inicialización a partir de propagación de segundos momentos;
7. explicar qué preserva dropout invertido en esperanza y qué variación añade.

Fuentes distintas de aleatoriedad

En una red aparecen al menos cuatro fuentes que conviene separar:

Fuente	Variable aleatoria	Qué cambia
muestreo	$D_m \sim P^m$	observaciones disponibles
inicialización	$W_0^{[\ell]}$	punto inicial del algoritmo
mini-batches	B_t	gradiente usado en cada paso
dropout	máscaras M_t	subred activa durante entrenamiento

Una semilla fija condiciona estas realizaciones en una ejecución; no elimina la incertidumbre del procedimiento ni demuestra estabilidad frente a otras muestras.

Para justificar reglas de inicialización usamos aproximaciones probabilísticas. Si w_j y x_j son independientes, centrados y con segundos momentos finitos,

$$\text{Var}\left(\sum_{j=1}^d w_j x_j\right) = d \text{Var}(w_j) \text{Var}(x_j).$$

Si las entradas están correlacionadas o no están centradas, aparecen términos de covarianza y la aproximación debe revisarse.

En dropout invertido, una máscara $M_j \sim \text{Bernoulli}(1 - p)$ produce

$$\tilde{A}_j = \frac{M_j}{1 - p} A_j, \quad \mathbb{E}[\tilde{A}_j \mid A_j] = A_j.$$

La igualdad conserva la activación en esperanza durante entrenamiento, pero aumenta su varianza.

Activaciones

Activación	Fórmula	Riesgo principal
Sigmoide	$\sigma(z)$	saturación y gradientes pequeños
Tanh	$\tanh(z)$	saturación en valores extremos
ReLU	$\max(0, z)$	neuronas muertas si muchos $z \leq 0$

ReLU suele ser una buena opción inicial en redes densas pequeñas por simplicidad y estabilidad.

Inicialización

La inicialización controla la escala inicial de las señales. Si una capa calcula:

$$z = \sum_{j=1}^d w_j x_j,$$

la varianza de z crece con d si los pesos se inicializan sin controlar escala. Bajo independencia aproximada y media cero:

$$\text{Var}(z) \approx d \text{Var}(w) \text{Var}(x).$$

Para que la señal no crezca sin control al pasar por capas, se escoge $\text{Var}(w)$ proporcional a $1/d$. En redes con ReLU se usa a menudo la inicialización He porque compensa que ReLU anula una parte de las activaciones.

Si los pesos empiezan demasiado grandes, las activaciones pueden saturarse. Si empiezan demasiado pequeños, las señales pueden desaparecer.

Una regla práctica para ReLU es inicialización He:

$$W \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in}}\right).$$

En NumPy:

```
W = rng.normal(0, np.sqrt(2 / fan_in), size=(fan_in, fan_out))
```

De la propagación de varianza a Xavier y He

La regla $1/d$ no se elige solo por costumbre. Si x_j y w_j son independientes, centrados y con igual segundo momento dentro de la capa,

$$\mathbb{E}[z^2] = \mathbb{E}\left[\left(\sum_{j=1}^d w_j x_j\right)^2\right] = d \mathbb{E}[w_j^2] \mathbb{E}[x_j^2],$$

porque los términos cruzados tienen esperanza cero. Para una activación lineal, preservar el segundo momento en la propagación hacia adelante sugiere $\mathbb{E}[w_j^2] = 1/d_{\text{in}}$. Este es el argumento *fan-in* que está en la base de las inicializaciones de tipo Xavier. La variante Glorot/Xavier que equilibra propagación hacia adelante y hacia atrás usa con frecuencia

$$\text{Var}(w_j) = \frac{2}{d_{\text{in}} + d_{\text{out}}}.$$

Cuando $d_{\text{in}} = d_{\text{out}} = d$, ambas escalas coinciden con $1/d$. La distinción importa cuando las capas tienen anchos muy diferentes.

Si z tiene una distribución aproximadamente simétrica alrededor de cero, ReLU conserva el lado positivo y anula el negativo. Bajo esa aproximación,

$$\mathbb{E}[\text{ReLU}(z)^2] \approx \frac{1}{2} \mathbb{E}[z^2].$$

Compensar ese factor $1/2$ lleva a

$$\text{Var}(w_j) \approx \frac{2}{d},$$

que es la escala He. La deducción depende de independencia aproximada, centrado y simetría; durante el entrenamiento esas hipótesis dejan de ser exactas. La inicialización intenta evitar explosión o desaparición al comienzo, no garantiza que todas las capas mantengan la misma varianza para siempre.

Regularización L2 en redes

En este libro y en Lab 3, J es la **pérdida media de datos**. La convención de regularización es:

$$J_\lambda = J + \frac{\lambda}{2} (\|W^{[1]}\|_F^2 + \|W^{[2]}\|_F^2).$$

En los gradientes:

$$dW^{[\ell]} \leftarrow dW^{[\ell]} + \lambda W^{[\ell]}.$$

No penalizamos sesgos por defecto.

En `torch.optim.SGD`, `weight_decay=lambda` añade λW al gradiente de los parámetros incluidos en ese grupo. Para no penalizar sesgos deben ponerse en otro grupo con `weight_decay=0`. `AdamW` usa decaimiento desacoplado y no debe presentarse como la derivada exacta del objetivo anterior.

Ejemplo resuelto: L2 en pérdida y gradiente

Supón una capa con:

$$W = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}, \quad \lambda = 0.1.$$

La norma de Frobenius al cuadrado es:

$$\|W\|_F^2 = 2^2 + (-1)^2 + 0^2 + 3^2 = 14.$$

Si usamos la penalización $\frac{\lambda}{2} \|W\|_F^2$, el aporte a la pérdida es:

$$\frac{0.1}{2} \cdot 14 = 0.7.$$

Si el gradiente de datos fuera:

$$dW_{\text{datos}} = \begin{bmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{bmatrix},$$

el gradiente regularizado sería:

$$dW = dW_{\text{datos}} + \lambda W = \begin{bmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{bmatrix} + 0.1 \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.3 \\ 0.1 & 0.6 \end{bmatrix}.$$

La regularización no “apaga” pesos automáticamente; agrega una fuerza que empuja los pesos hacia valores más pequeños durante la actualización.

Dropout como idea

Dropout apaga unidades al azar durante entrenamiento. En este curso se introduce conceptualmente, no como requisito central del autograder. Si la probabilidad de apagado es p , el dropout invertido usa

$$\tilde{A} = \frac{M}{1-p} A, \quad M \sim \text{Bernoulli}(1-p).$$

Condicionado en A ,

$$\mathbb{E}[\tilde{A} \mid A] = A, \quad \text{Var}(\tilde{A} \mid A) = A^2 \frac{p}{1-p}.$$

La escala $1/(1-p)$ evita tener que reescalar en evaluación, pero muestra también el costo: dropout conserva la activación en esperanza y aumenta su variación durante entrenamiento. En evaluación se desactiva la máscara; dejar dropout activo hace aleatorias las predicciones, salvo que se use deliberadamente un procedimiento como Monte Carlo dropout.

Su intuición:

- evita dependencia excesiva de una neurona;
- funciona como ensamble implícito;
- requiere cuidado al pasar de entrenamiento a evaluación.

Curvas de entrenamiento

Una curva útil muestra:

- pérdida de entrenamiento;
- pérdida de validación;
- métrica principal;
- número de épocas;
- configuración del experimento.

Diagnósticos comunes

Señal	Diagnóstico probable	Acción
Pérdidas train y validación altas	posible underfitting	más capacidad, más épocas, revisar variables
Pérdida train baja y validación alta	posible overfitting	regularización, menos capacidad, más datos
Pérdida explota	learning rate alto o gradiente incorrecto	reducir paso, revisar backward

Señal	Diagnóstico probable	Acción
Pérdida plana	paso bajo, mala inicialización o activaciones saturadas	ajustar inicialización y paso

Ejemplo resuelto: leer una curva

Supón que una red obtiene:

Época	Train loss	Val loss	Train acc	Val acc
5	0.82	0.88	0.74	0.71
20	0.24	0.43	0.94	0.86
50	0.05	0.78	0.99	0.78

Lectura:

1. El entrenamiento sí aprende: **train loss** baja.
2. La validación mejora hasta cierto punto y luego empeora.
3. El diagnóstico probable es overfitting después de la época 20.
4. Acciones razonables: early stopping, más regularización, menor capacidad o más datos.

La conclusión no es “la red es mala”; la conclusión es “esta configuración debe controlarse”.

Checkpoint técnico del proyecto

En semana 9, el proyecto debe mostrar avance real desde el baseline:

1. Métrica de baseline.
2. Modelo mejorado o experimento técnico nuevo.
3. Análisis de errores.
4. Siguiendo experimento y resultado que permitiría evaluarlo.
5. Riesgo técnico o de datos.

Caso longitudinal: curva de entrenamiento en dígitos

Supón que en `load_digits` una red con $h = 64$ obtiene este patrón:

Configuración	Train acc	Dev acc	Lectura
sin regularización	0.99	0.86	posible overfitting
L2 moderado	0.96	0.90	mejor resultado de validación en esta partición

Configuración	Train acc	Dev acc	Lectura
red muy pequeña	0.82	0.80	posible underfitting

Para esta partición y esta arquitectura, L2 moderado reduce la brecha train/dev; la tabla no demuestra que siempre generalice mejor. El registro debe conservar semilla, partición, tamaño de red, learning rate y número de épocas para que la comparación sea reproducible.

Verificaciones seleccionadas

1. Si train accuracy sube y dev accuracy cae después de cierta época, el diagnóstico principal es overfitting, no falta de capacidad.
2. Si train y dev son bajos, aumentar regularización rara vez es la primera acción; primero se revisan capacidad, features, épocas o learning rate.
3. Si cambias arquitectura y regularización al mismo tiempo, no puedes atribuir la mejora a una sola causa sin un experimento adicional.

Ejemplo resuelto: regla de parada

Supón que el dev loss por época es:

Época	Dev loss
10	0.58
20	0.44
30	0.41
40	0.43
50	0.49

Una regla de early stopping con paciencia de 2 revisiones guardaría el modelo de la época 30 y se detendría después de observar que las épocas 40 y 50 no mejoran. La acción correcta no es usar el modelo de la última época por costumbre; es usar el mejor modelo según validación y reportar la regla usada.

Lectura de curvas: tres patrones frecuentes

Una curva se lee comparando entrenamiento y validación, no mirando una sola línea. Hay tres patrones básicos.

Primero, si train loss baja de forma sostenida y dev loss también baja, el entrenamiento todavía mejora la pérdida medida. No conviene detenerlo solo porque ya pasaron muchas épocas. Segundo, si train loss baja pero dev loss sube, la red está memorizando estructura específica del conjunto de entrenamiento. La acción natural es guardar el mejor modelo de validación, revisar regularización,

reducir capacidad o aumentar datos. Tercero, si train y dev loss se mantienen altos, el problema no se resuelve aumentando regularización; probablemente faltan capacidad, variables, épocas, una tasa de aprendizaje adecuada o una revisión de etiquetas.

El reporte debe nombrar el patrón observado. Es distinto decir “el modelo no mejora” que decir “hay overfitting después de la época 30” o “hay underfitting persistente en train y dev”. Esa diferencia reduce ambigüedad para el lector y para quien califica.

Punto de control

Antes de reportar una red como mejora, responde:

- ¿contra qué baseline compite?
- ¿la mejora aparece en validación o solo en entrenamiento?
- ¿qué error específico sigue siendo importante?
- ¿qué hiperparámetro cambiarías primero y por qué?

Para explorar más

- **Extensión matemática:** relaciona la penalización L_2 con el tamaño de los pesos y explica por qué puede reducir varianza.
- **Extensión computacional:** compara curvas train/dev con y sin dropout, manteniendo fija la semilla y el tamaño de red.
- **Conexión con proyecto:** define una regla de parada basada en dev loss y explica qué patrón indicaría sobreentrenamiento.

Revisión del capítulo

Entrenar una red no termina cuando baja la pérdida de entrenamiento. La lectura compara entrenamiento y validación para distinguir aprendizaje, overfitting, underfitting, saturación y problemas de tasa.

La inicialización afecta la escala de activaciones y gradientes. La regularización L2, dropout y early stopping son herramientas para controlar capacidad efectiva, pero ninguna reemplaza una partición de validación ni un análisis de errores.

El producto final del capítulo es una hipótesis concreta. La curva permite proponer una causa probable y elegir una sola modificación verificable: tasa, capacidad, regularización, datos o regla de parada.

Términos clave

- **Activación saturada:** zona donde una activación cambia poco y dificulta el flujo de gradientes.
- **Inicialización:** regla para asignar valores iniciales a pesos antes de entrenar.
- **Regularización L2:** penalización sobre pesos que busca reducir sobreajuste.
- **Dropout:** técnica que desactiva aleatoriamente unidades durante entrenamiento.
- **Curva de entrenamiento:** evolución de pérdida o métrica por época.
- **Early stopping:** detención del entrenamiento cuando validación deja de mejorar.
- **Análisis de error:** revisión de fallas concretas para decidir el siguiente experimento.

Ejercicios

Preguntas conceptuales

1. Explique por qué una red con train accuracy 99 % y validation accuracy 70 % no debe reportarse como exitosa.
2. ¿Por qué la regularización no reemplaza una partición de validación?

Problemas cuantitativos

3. Dada una tabla de pérdidas por época, identifique la primera señal de overfitting y proponga una época candidata para early stopping.
4. Si la penalización L2 es $\frac{\lambda}{2} \sum_j w_j^2$, calcule la penalización para $w = (2, -1, 0.5)$ y $\lambda = 0.1$, y escriba el término que se añade al gradiente.

Práctica computacional

5. Diseñe un experimento para comparar dos valores de `lambda` sin usar el conjunto de test.
6. Grafique pérdida de entrenamiento y validación para al menos dos configuraciones.

Interpretación y pensamiento crítico

7. ¿Qué señal en la curva de pérdida sugeriría revisar el learning rate antes de cambiar la arquitectura?
8. Un modelo mejora la métrica global pero empeora un subconjunto importante. ¿Qué decisión técnica tomaría?

Profundización matemática y algorítmica

9. Deduzca la escala *fan-in* $1/d_{\text{in}}$ a partir de la conservación aproximada del segundo momento en una capa lineal. Explique por qué la variante Glorot/Xavier $2/(d_{\text{in}} + d_{\text{out}})$ coincide con ella cuando ambos anchos son iguales.
10. Justifique el factor 2 de la inicialización He bajo una entrada simétrica y una activación ReLU.
11. Calcule esperanza y varianza condicional de dropout invertido para una activación fija $A = a$ y probabilidad de apagado p .

Proyecto de cierre

12. Redacte un memo técnico de una página sobre una red entrenada: baseline, curva, diagnóstico, cambio propuesto, riesgo y siguiente experimento.

Optimizadores y PyTorch

Pregunta aplicada

Hasta ahora actualizamos parámetros con descenso del gradiente simple:

$$\theta \leftarrow \theta - \alpha \nabla J(\theta).$$

Pero redes neuronales reales suelen entrenarse con variantes que usan memoria del gradiente, escalamiento adaptativo o mini-batches.



Figura 16: Optimizadores: SGD usa el gradiente actual, momentum suaviza direcciones recientes y Adam combina memoria con escala adaptativa por coordenada.

Lectura de la figura. Sin regularización incorporada al optimizador, SGD, momentum y Adam conservan el objetivo y cambian la regla que convierte gradientes en actualizaciones. Por eso el optimizador debe reportarse como parte del experimento, junto con tasa de aprendizaje, batch size, épocas y métrica de validación.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. distinguir batch, mini-batch y época;
2. explicar la intuición de momentum, RMSProp y Adam;
3. ubicar forward, loss, backward y actualización en un loop de PyTorch;
4. comparar una implementación desde cero con PyTorch sin cambiar la pregunta experimental;
5. reconocer que un optimizador no reemplaza validación.
6. escribir las ecuaciones completas de Adam y explicar la corrección de sesgo;
7. comparar costo, memoria y alcance de las garantías de SGD, momentum y Adam.

Gradiente estocástico y filtración

Condicionado en el dataset, el historial de entrenamiento puede verse como un proceso aleatorio generado por mini-batches e inicialización. Si $\mathcal{F}_t$ contiene toda la información disponible antes de escoger el lote B_t , un gradiente de mini-batch uniforme satisface

$$\mathbb{E}[g_t \mid \mathcal{F}_t] = \nabla \hat{R}_m(\theta_t).$$

La diferencia

$$\xi_t = g_t - \nabla \hat{R}_m(\theta_t)$$

es ruido de gradiente con media condicional cero bajo ese esquema de muestreo. Momentum y Adam transforman la secuencia de gradientes; no convierten automáticamente g_t en un mejor estimador del riesgo poblacional.

Comparar optimizadores exige conservar distribución de mini-batches, presupuesto de actualizaciones, inicialización y partición. Cambiar esas fuentes simultáneamente confunde efectos algorítmicos y muestrales.

Mini-batch gradient descent

En lugar de usar todo el dataset en cada paso, usamos lotes pequeños:

```
dataset -> mini-batches -> actualización por batch -> época completa
```

Esto reduce costo por actualización y agrega ruido útil al entrenamiento.

Ese ruido no es una garantía de mejora. El tamaño del lote modifica la varianza del gradiente, el número de actualizaciones por época y el uso de memoria.

Momentum

Momentum acumula una dirección suavizada:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta_t),$$

$$\theta_{t+1} = \theta_t - \alpha v_t.$$

Intuición: evita que cada gradiente aislado cambie la dirección con demasiada fuerza.

Esta es la convención de promedio exponencial usada en el desarrollo del curso. Algunas bibliotecas, incluido `torch.optim.SGD`, parametrizan el buffer sin el factor $(1 - \beta)$; por eso no se comparan tasas de aprendizaje sin revisar la regla exacta del optimizador.

Ejemplo resuelto: dos pasos con momentum

Supón un parámetro escalar $\theta_0 = 5$, tasa $\alpha = 0.1$, momentum $\beta = 0.9$ y velocidad inicial $v_0 = 0$. Los dos primeros gradientes son:

$$g_1 = 4, \quad g_2 = 2.$$

Primer paso:

$$v_1 = 0.9(0) + 0.1(4) = 0.4, \quad \theta_1 = 5 - 0.1(0.4) = 4.96.$$

Segundo paso:

$$v_2 = 0.9(0.4) + 0.1(2) = 0.56, \quad \theta_2 = 4.96 - 0.1(0.56) = 4.904.$$

Aunque el segundo gradiente es menor que el primero, la velocidad conserva parte de la dirección previa. Esa memoria suaviza actualizaciones ruidosas, pero también puede sostener una mala dirección si el learning rate o β no son adecuados.

Derivación guiada: momentum como memoria exponencial

La fórmula

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$

parece local, pero al expandirla se ve su memoria. Como

$$v_{t-1} = \beta v_{t-2} + (1 - \beta)g_{t-1},$$

entonces

$$v_t = (1 - \beta)g_t + \beta(1 - \beta)g_{t-1} + \beta^2 v_{t-2}.$$

Si repetimos la sustitución:

$$v_t = (1 - \beta)g_t + \beta(1 - \beta)g_{t-1} + \beta^2(1 - \beta)g_{t-2} + \dots.$$

Los gradientes recientes pesan más que los antiguos, pero los antiguos no desaparecen de inmediato. Por eso momentum puede avanzar de forma más estable en valles alargados de la pérdida. La contraparte es que una memoria demasiado alta puede retrasar la reacción cuando la dirección útil cambia.

RMSProp y Adam

RMSProp mantiene un promedio exponencial de gradientes al cuadrado y divide cada coordenada por su escala reciente. Adam añade un promedio exponencial del gradiente. Con g_t como gradiente de mini-batch:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t,$$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2)g_t \odot g_t.$$

Como $m_0 = s_0 = 0$, los primeros momentos están sesgados hacia cero. Si, para entender el efecto, imaginamos un gradiente constante g , entonces

$$m_t = (1 - \beta_1^t)g, \quad s_t = (1 - \beta_2^t)(g \odot g).$$

Esto motiva las correcciones

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{s}_t = \frac{s_t}{1 - \beta_2^t}.$$

La actualización es

$$\theta_{t+1} = \theta_t - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{s}_t} + \varepsilon},$$

donde raíz, división y suma se aplican por coordenadas. El término ε evita división por cero y también influye en la escala efectiva cuando $\widehat{s}_t$ es muy pequeño. Usamos s_t para el segundo momento de Adam y reservamos v_t para la velocidad de momentum.

Adam suele ser un punto de partida útil, pero no elimina la necesidad de validar tasa, regularización y arquitectura. Tampoco debe confundirse Adam con AdamW: este último desacopla el decaimiento de pesos de la normalización adaptativa del gradiente.

Ejemplo resuelto: Adam en una coordenada

Supón una coordenada con gradiente inicial $g_1 = 4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $m_0 = 0$ y $s_0 = 0$. El primer momento queda:

$$m_1 = 0.9(0) + 0.1(4) = 0.4.$$

El segundo momento queda:

$$s_1 = 0.999(0) + 0.001(4^2) = 0.016.$$

Como ambos promedios empiezan en cero, Adam usa corrección de sesgo:

$$\widehat{m}_1 = \frac{m_1}{1 - \beta_1} = 4, \quad \widehat{s}_1 = \frac{s_1}{1 - \beta_2} = 16.$$

La dirección adaptativa es aproximadamente:

$$\frac{\widehat{m}_1}{\sqrt{\widehat{s}_1} + \epsilon} \approx 1.$$

La lectura no es que Adam normaliza todo perfectamente. La lectura es que usa información sobre magnitud reciente del gradiente para ajustar el tamaño efectivo del paso por coordenada.

Qué puede afirmarse sobre convergencia

Para SGD sobre una función suave, con gradientes inesgados, varianza controlada y una secuencia de pasos que disminuye apropiadamente, pueden obtenerse resultados de convergencia hacia puntos estacionarios en promedio. Con una tasa constante, el ruido del mini-batch suele impedir convergencia exacta y deja iteraciones en una región cuyo tamaño depende del paso y la varianza.

Las redes hacen el problema no convexo. Llegar a $\|\nabla J\|$ pequeño no prueba que se alcanzó un mínimo global ni que el riesgo de validación sea bajo. Para Adam, la adaptatividad exige hipótesis y variantes técnicas adicionales; no es correcto afirmar que siempre converge mejor que SGD. En este curso, las garantías teóricas orientan el diagnóstico, mientras la comparación empírica se hace con presupuesto y protocolo fijados.

Costo y memoria del optimizador

Sea P el número de parámetros y C_B el costo de forward más backward para un mini-batch. La actualización de SGD cuesta $O(P)$ y almacena parámetros y gradientes. Momentum añade un vector de velocidad de tamaño P . Adam añade dos vectores, m_t y s_t , de modo que su estado auxiliar es aproximadamente $2P$ números.

Método	Estado adicional	Costo de actualización
SGD	ninguno	$O(P)$
momentum	P	$O(P)$
Adam	$2P$	$O(P)$

El costo total por paso sigue dominado normalmente por C_B , pero el estado del optimizador puede ser relevante en modelos grandes. Reportar solo «usamos Adam» omite tasa, betas, epsilon, decaimiento, batch size y número de pasos: todos forman parte del algoritmo ejecutado.

PyTorch: qué automatiza

PyTorch ofrece:

- tensores;
- autograd;
- módulos `nn.Module`;
- pérdidas;
- optimizadores;
- entrenamiento en CPU/GPU.

El ciclo conceptual no cambia:

```
forward -> loss -> backward -> optimizer.step()
```

Para clasificación multiclase, `torch.nn.CrossEntropyLoss` recibe **logits** y etiquetas enteras. Aplicar softmax antes cambia el cálculo y pierde la implementación numéricamente estable que ya combina la función de pérdida.

Loop mínimo

```
model.train()
for X_batch, y_batch in loader:
    optimizer.zero_grad()
    logits = model(X_batch)
    loss = criterion(logits, y_batch)
    loss.backward()
    optimizer.step()
```

La diferencia es que PyTorch construye el grafo de cómputo y calcula los gradientes.

Ejemplo resuelto: lectura de un loop

En el loop mínimo:

```
optimizer.zero_grad()
logits = model(X_batch)
loss = criterion(logits, y_batch)
loss.backward()
optimizer.step()
```

la interpretación es:

Línea	Significado
<code>zero_grad()</code>	borra gradientes acumulados
<code>model(X_batch)</code>	forward propagation
<code>criterion(...)</code>	calcula la pérdida
<code>backward()</code>	backpropagation automático
<code>step()</code>	actualiza parámetros

Si olvidas `zero_grad()`, PyTorch acumula gradientes entre batches. Eso rara vez es lo que quieres en este curso.

Comparación justa

Al comparar NumPy y PyTorch:

- use la misma partición de datos;
- use la misma métrica;
- reporte semilla;
- compare curvas o resultados finales;
- no confunda diferencia de implementación con diferencia de modelo.

Caso longitudinal: experimento reproducible en dígitos

Para `load_digits`, una comparación interpretable entre NumPy y PyTorch debe fijar:

Elemento	Valor que debe registrarse
partición	semilla y proporciones train/dev/test
arquitectura	64 entradas, h ocultas, 10 salidas
pérdida	entropía cruzada multiclase
optimizador	SGD, momentum o Adam
presupuesto	épocas, batch size y learning rate
resultados	curva train/dev y métrica final

Si PyTorch obtiene mejor resultado, hay que preguntar si cambió el modelo, el optimizador, el número de épocas o la inicialización. El nombre de la biblioteca no explica la diferencia; la comparación requiere condiciones controladas.

Punto de control

Una comparación NumPy vs PyTorch solo puede interpretarse si:

1. usa la misma partición;
2. usa la misma métrica;
3. entrena una arquitectura comparable;
4. reporta semilla y número de épocas;
5. interpreta diferencias con cautela.

Verificaciones seleccionadas

1. Con 1024 observaciones y batch size 64 hay $1024/64 = 16$ batches por época.
2. Si se entrenan 20 épocas con 16 batches por época, `optimizer.step()` se ejecuta 320 veces.
3. `zero_grad()` debe ejecutarse antes del siguiente `backward()` para evitar acumular gradientes entre batches, salvo que la acumulación sea intencional.

Ejemplo resuelto: presupuesto de entrenamiento

Comparar dos optimizadores exige presupuesto comparable. Si Adam se entrena 100 épocas y SGD se entrena 20, la diferencia de métrica no aísla el efecto del optimizador. Una comparación mínima fija:

Elemento	Mismo valor
partición	sí
arquitectura	sí
batch size	sí
épocas máximas	sí
criterio de parada	sí
métrica final	sí

Solo después de fijar esos elementos se puede discutir si momentum o Adam mejoraron estabilidad, velocidad o desempeño.

Convergencia no es validación

Un optimizador puede hacer que la pérdida de entrenamiento baje con más rapidez sin producir el mejor modelo para datos nuevos. Por eso el criterio de éxito no es “Adam bajó la pérdida antes”, sino “bajo la misma partición, presupuesto y métrica, el modelo obtiene una mejora de validación”. En particular, una curva suave de train loss puede esconder overfitting si no se acompaña de dev loss o de una métrica externa.

En una entrega, la comparación debe separar tres afirmaciones: velocidad de optimización, estabilidad de la curva y desempeño de validación. La primera se mide por épocas o pasos; la segunda por oscilaciones o sensibilidad a la tasa; la tercera por una métrica en datos no usados para ajustar parámetros.

Para explorar más

- **Extensión matemática:** compara una actualización de SGD con una de Adam en una dimensión y explica qué cambia por los momentos adaptativos.
- **Extensión computacional:** guarda `seed`, arquitectura, optimizador, tasa de aprendizaje y época del mejor modelo en un registro reproducible.
- **Conexión con proyecto:** define qué configuración mínima necesitas reportar para que otra persona pueda repetir tu entrenamiento.

Revisión del capítulo

Los optimizadores modernos modifican la forma de usar gradientes, pero no cambian la lógica experimental. Mini-batch gradient descent estima el gradiente con subconjuntos; momentum acumula dirección; RMSProp y Adam adaptan escalas por parámetro.

PyTorch automatiza autograd y actualización de parámetros, pero el usuario sigue siendo responsable de particiones, métricas, semillas, arquitectura y lectura de curvas. `zero_grad()`, `backward()` y `step()` son operaciones con significado, no rituales de código.

Una comparación justa entre NumPy y PyTorch exige misma pregunta experimental: datos, partición, arquitectura comparable, métrica y presupuesto de entrenamiento. Velocidad de entrenamiento no equivale automáticamente a mejor modelo.

Términos clave

- **Mini-batch:** subconjunto usado para estimar un paso de gradiente.
- **Momentum:** acumulación suavizada de direcciones de actualización pasadas.
- **RMSProp:** adaptación de pasos según magnitudes recientes de gradientes.
- **Adam:** optimizador adaptativo que combina ideas de momentum y RMSProp.
- **Autograd:** mecanismo de diferenciación automática de PyTorch.
- **`zero_grad()`:** instrucción que limpia gradientes acumulados.
- **`backward()`:** cálculo automático de gradientes desde la pérdida.
- **`step()`:** actualización de parámetros del optimizador.

Ejercicios

Preguntas conceptuales

1. Explique por qué Adam no reemplaza la validación.
2. ¿Por qué PyTorch acumula gradientes y por qué normalmente llamamos `zero_grad()`?

Problemas cuantitativos

3. Si un dataset tiene 1024 observaciones y batch size 64, ¿cuántos batches tiene una época?
4. Si se entrenan 20 épocas con 16 batches por época, ¿cuántas veces se ejecuta `optimizer.step()`?

Práctica computacional

5. Identifique en el loop de PyTorch dónde ocurren forward, pérdida, backward y actualización.
6. Implemente un loop mínimo que registre pérdida promedio por época.

Interpretación y pensamiento crítico

7. Proponga una comparación justa entre una red NumPy pequeña y una red PyTorch equivalente.
8. Si PyTorch entrena más rápido que NumPy, ¿qué comparación necesita antes de afirmar que el modelo es mejor?

Profundización matemática y algorítmica

9. Expandir la recurrencia de momentum y determine el peso de g_{t-r} en v_t .
10. Para un gradiente constante, derive los factores $1 - \beta_1^t$ y $1 - \beta_2^t$ que motivan la corrección de sesgo de Adam.
11. Compare la memoria auxiliar de SGD, momentum y Adam para P parámetros.
12. Explique por qué convergencia a un punto estacionario de entrenamiento no implica optimalidad global ni bajo riesgo de validación.

Proyecto de cierre

13. Reproduzca el mismo experimento con NumPy y PyTorch usando la misma partición, semilla y métrica. Entregue una tabla comparativa y una conclusión cautelosa.

Ejercicios integradores: redes neuronales

Estos ejercicios están diseñados para estudiar el módulo como un sistema completo: forward, backward, diagnóstico y entrenamiento con herramientas modernas. La meta no es memorizar fórmulas aisladas, sino verificar que cada objeto matemático tiene forma, propósito e interpretación.

Resultados de práctica

Al terminar este banco deberías poder:

1. seguir las formas de tensores en una red densa;
2. explicar forward propagation como composición de funciones;
3. derivar gradientes de segunda y primera capa;
4. diagnosticar entrenamiento con curvas y métricas;
5. leer un loop PyTorch sin tratarlo como caja negra.

Mapa del banco

Bloque	Qué comprueba
A	compatibilidad de dimensiones y forward pass
B	backpropagation y verificación de gradientes
C	diagnóstico de entrenamiento y generalización
D	lectura de un loop de PyTorch

Cómo usar esta sección

Trabaja primero sin mirar las respuestas. Después revisa:

1. si las formas de matrices son compatibles;
2. si cada gradiente actualiza un parámetro con la misma forma;
3. si tu diagnóstico distingue resultado, causa probable y comprobación;
4. si puedes explicar cada línea crítica de un loop de entrenamiento.

Columna probabilística del módulo

P1. Categórica y softmax. Para tres clases con distribución condicional $\pi = (0.2, 0.5, 0.3)$, calcule la entropía cruzada de reportar $q = (0.1, 0.7, 0.2)$ y compare con reportar $q = \pi$.

P2. Baseline probabilístico. Demuestre que asignar $1/K$ a cada clase produce pérdida $\log K$ para cualquier etiqueta observada. Explique qué diagnóstico permite hacer al inicializar una red.

P3. Gradiente estocástico. Explique respecto de qué aleatoriedad un gradiente de mini-batch es insesgado y por qué eso no implica que sea igual al gradiente poblacional.

P4. Propagación de varianza. Derive $\text{Var}(\sum_j w_j x_j)$ bajo independencia y media cero. Indique qué términos faltan cuando las entradas están correlacionadas.

P5. Dropout. Verifique la esperanza de dropout invertido y calcule su varianza condicional dado $A_j = a$.

P6. Repetición. Entrene la misma red con cinco inicializaciones y reporte media, desviación y valores individuales de la métrica. Explique por qué esto todavía no cuantifica variación por una nueva muestra de la población.

Bloque A: dimensiones y forward

A1. Una red recibe `X.shape == (400, 64)`, tiene 20 unidades ocultas y 10 clases. Escriba las formas de `W1`, `b1`, `Z1`, `A1`, `W2`, `b2`, `Z2` y `P`.

A2. Explique por qué softmax se aplica por fila cuando `Z2` tiene forma `(m, k)`.

A3. Muestre con una frase por qué restar `max(Z, axis=1)` antes de `exp` no cambia softmax.

A4. Para una red $d \rightarrow h \rightarrow k$, derive el número total de parámetros y el costo dominante de un forward con lote de tamaño m . Compare duplicar h con duplicar m .

Bloque B: backpropagation

B1. Partiendo de $dZ2 = (P - Y) / m$, derive las formas de $dW2$ y $db2$.

B2. Describa qué debe guardarse en `cache` durante forward para poder ejecutar backward sin recalcular todo.

B3. Diseñe una prueba de gradientes por diferencias finitas para un solo peso de `W1`.

B4. Derive el Jacobiano de softmax y muestre paso a paso por qué, con entropía cruzada, el gradiente respecto a los logits es $p - y$.

B5. Si una red tiene P parámetros, compare el número de pasadas forward requerido por diferencias centrales para todas las coordenadas con el costo de backpropagation.

Bloque C: diagnóstico

- C1.** Una red tiene pérdida de entrenamiento decreciente y pérdida de validación creciente después de la época 20. ¿Qué diagnóstico haría y qué dos acciones probaría?
- C2.** Si al aumentar el learning rate la pérdida explota, ¿qué resultado reportaría y qué ajuste haría?
- C3.** Explique por qué un checkpoint técnico de proyecto debe incluir análisis de errores, no solo una métrica agregada.
- C4.** Bajo independencia, media cero y entradas simétricas, derive las escalas Xavier y He. Identifique en qué paso se usa la aproximación sobre ReLU.
- C5.** Para dropout invertido con probabilidad de apagado p , calcule esperanza y varianza condicional de una activación fija. Explique la diferencia entre modo entrenamiento y evaluación.

Bloque D: PyTorch

- D1.** En un loop de entrenamiento de PyTorch, explique el propósito de `optimizer.zero_grad()`.
- D2.** ¿Qué significa que `loss.backward()` use diferenciación automática?
- D3.** Proponga dos razones por las que una implementación NumPy y una PyTorch podrían producir métricas distintas aun usando el mismo dataset.
- D4.** Escriba las cuatro ecuaciones de Adam: dos momentos, dos correcciones de sesgo y actualización. Justifique los factores $1 - \beta_1^t$ y $1 - \beta_2^t$.
- D5.** Compare memoria auxiliar y costo por actualización de SGD, momentum y Adam para P parámetros. Separe ese costo del forward y backward.

Respuestas seleccionadas

- A1.** Una convención compatible es: `W1.shape == (64, 20)`, `b1.shape == (20,)`, `Z1.shape == (400, 20)`, `A1.shape == (400, 20)`, `W2.shape == (20, 10)`, `b2.shape == (10,)`, `Z2.shape == (400, 10)` y `P.shape == (400, 10)`.
- B1.** Si `dZ2.shape == (400, 10)` y `A1.shape == (400, 20)`, entonces `dW2 = A1.T @ dZ2` tiene forma `(20, 10)`, igual que `W2`. Además, `db2 = dZ2.sum(axis=0)` tiene forma `(10,)`, igual que `b2`.
- C1.** El patrón es compatible con overfitting después de la época 20. Dos experimentos son early stopping y aumentar regularización L2, uno por vez; también podría reducirse capacidad o ampliar datos si el proyecto lo permite.
- D1.** `optimizer.zero_grad()` borra gradientes acumulados antes del siguiente batch. Si no se llama, PyTorch suma gradientes de batches anteriores y la actualización ya no corresponde al gradiente del batch actual.

Parte IV: Evaluación y estrategia de modelado

Una mejora en la pérdida de entrenamiento no responde por sí sola si un modelo será útil. Para comparar procedimientos es necesario definir qué población y qué errores importan, reservar datos para una evaluación honesta y reconocer que toda métrica calculada con una muestra tiene variación.

Esta parte comienza tratando una métrica como un estadístico. La matriz de confusión deja de ser solo una tabla de conteos y pasa a estimar probabilidades conjuntas y condicionales. La elección de umbral conecta esas probabilidades con los costos del problema. A continuación, la validación cruzada y las curvas de aprendizaje ayudan a separar falta de señal, exceso de complejidad y variación muestral.

El análisis termina localizando los errores en grupos de observaciones y comparando la distribución de los datos disponibles con la distribución de uso. Esta perspectiva permite distinguir una fluctuación pequeña de una falla sistemática y evita que una sola métrica oculte dónde deja de funcionar el modelo.



Figura 17: Selección de métricas: el costo relativo de falsos positivos y falsos negativos orienta si conviene mirar accuracy, precisión, recall o una métrica F beta.

La figura organiza las métricas según el costo relativo de los errores. No ofrece una selección automática: obliga a declarar qué significa la clase positiva, qué decisión seguirá a la predicción y qué denominadores sostienen cada cifra.

Métricas y particiones

Pregunta aplicada

Un modelo puede tener buena accuracy y aun así ser inaceptable si falla en los casos importantes. Antes de entrenar más modelos, debemos decidir:

¿qué error cuesta más y qué comparación será suficiente para elegir?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. elegir una métrica principal alineada con el costo de error;
2. explicar por qué accuracy falla en clases desbalanceadas;
3. construir un baseline útil;
4. separar train, dev y test sin leakage;
5. usar pipelines como defensa contra preprocesamiento incorrecto.
6. interpretar métricas como estimadores de probabilidades poblacionales;
7. deducir un umbral de decisión a partir de costos de error y reconocer cuándo esa deducción no basta;
8. explicar mediante la fórmula de Bayes por qué la prevalencia modifica la precision de un clasificador.

Una métrica es un estadístico

Sea f un procedimiento ya fijado y $L = \ell(f(X), Y)$. El riesgo de uso es

$$R_{P_{\text{uso}}}(f) = \mathbb{E}[L],$$

mientras que una evaluación de n casos produce

$$\widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n L_i.$$

El segundo objeto cambia si cambia la muestra. Bajo casos iid y varianza finita,

$$\text{se}(\widehat{R}_n) = \sqrt{\frac{\text{Var}(L)}{n}}.$$

Para accuracy, $L_i = \mathbf{1}\{f(X_i) \neq Y_i\}$ es Bernoulli con probabilidad de error p_e . Por tanto,

$$\text{Var}(\widehat{\text{accuracy}}) = \frac{a(1-a)}{n},$$

donde $a = 1 - p_e$. En una muestra reemplazamos a por la accuracy observada para obtener un error estándar aproximado. Esta fórmula no corrige selección del modelo, dependencia ni cambio de población.

Cuando la aproximación normal es razonable, un intervalo de referencia para una métrica promedio tiene la forma

$$\widehat{R}_n \pm z_{1-\alpha/2} \widehat{\text{se}}(\widehat{R}_n).$$

Este intervalo cuantifica la variación asociada con la muestra de evaluación para un modelo ya fijado. No incluye la incertidumbre producida por escoger el modelo, las variables o el umbral con esos mismos datos. Para proporciones cercanas a cero o uno, muestras pequeñas o datos dependientes, la aproximación puede ser deficiente y debe reemplazarse por un procedimiento compatible con el diseño de muestreo.

Si dos modelos f_A y f_B se evalúan sobre los mismos casos, la comparación natural es pareada. Definimos

$$D_i = \ell(f_A(X_i), Y_i) - \ell(f_B(X_i), Y_i), \quad \widehat{\Delta} = \frac{1}{n} \sum_{i=1}^n D_i,$$

y estimamos

$$\widehat{\text{se}}(\widehat{\Delta}) = \frac{s_D}{\sqrt{n}}.$$

La diferencia conserva la dificultad particular de cada observación: un caso difícil afecta a ambos modelos antes de calcular D_i . Comparar dos intervalos construidos por separado desperdicia este emparejamiento y puede dar una lectura engañosa. La interpretación sigue condicionada a que el conjunto de evaluación no haya intervenido en la elección de ninguno de los dos modelos.

Matriz de confusión poblacional y empírica

Cada celda de la matriz de confusión estima una probabilidad conjunta, por ejemplo

$$\mathbb{P}(\hat{Y} = 1, Y = 0) \quad \text{mediante} \quad \frac{\#\{i : \hat{y}_i = 1, y_i = 0\}}{n}.$$

Precision y recall son probabilidades condicionales estimadas:

$$\text{precision} = \mathbb{P}(Y = 1 \mid \hat{Y} = 1), \quad \text{recall} = \mathbb{P}(\hat{Y} = 1 \mid Y = 1).$$

El denominador efectivo de cada métrica puede ser mucho menor que n , por lo que su incertidumbre puede ser grande incluso en un test aparentemente amplio.

Las definiciones poblacionales requieren denominadores positivos:

$$\text{precision} = \frac{\mathbb{P}(\hat{Y} = 1, Y = 1)}{\mathbb{P}(\hat{Y} = 1)}, \quad \text{recall} = \frac{\mathbb{P}(\hat{Y} = 1, Y = 1)}{\mathbb{P}(Y = 1)}.$$

Si el clasificador no predice positivos, precision no está definida; asignarle cero es una convención de software, no una identidad matemática. Si el conjunto evaluado no contiene positivos, recall tampoco está definido para esa muestra. Un reporte debe mostrar conteos para que el lector pueda detectar estos casos.

Para recall, el tamaño efectivo es $TP + FN$, es decir, el número de positivos observados. Bajo una aproximación binomial condicional, su error estándar es

$$\widehat{\text{se}}(\text{recall}) \approx \sqrt{\frac{\hat{r}(1 - \hat{r})}{TP + FN}}.$$

La fórmula análoga para precision usa $TP + FP$. Estas aproximaciones requieren casos independientes y un procedimiento fijado; no incluyen la incertidumbre por haber escogido modelo y umbral con los mismos datos.

El condicionamiento inverso y la prevalencia

Recall y precision condicionan en direcciones diferentes. Recall pregunta por la predicción entre los casos realmente positivos; precision pregunta por el estado real entre los casos que el modelo declaró positivos. La fórmula de Bayes conecta ambas direcciones:

$$\mathbb{P}(Y = 1 \mid \hat{Y} = 1) = \frac{\mathbb{P}(\hat{Y} = 1 \mid Y = 1)\mathbb{P}(Y = 1)}{\mathbb{P}(\hat{Y} = 1)}.$$

Si llamamos r al recall, s a la especificidad y $\pi = \mathbb{P}(Y = 1)$ a la prevalencia, entonces

$$\mathbb{P}(\hat{Y} = 1) = r\pi + (1 - s)(1 - \pi),$$

y por tanto

$$\text{precision} = \frac{r\pi}{r\pi + (1 - s)(1 - \pi)}.$$

Esta identidad muestra por qué precision puede cambiar entre poblaciones aunque recall y especificidad permanezcan aproximadamente constantes: depende de la prevalencia. En aplicaciones diagnósticas o de detección, reportar solo una de estas cantidades oculta parte del problema.

Ejemplo: una clase poco frecuente. Supongamos $r = 0.90$, $s = 0.95$ y una prevalencia $\pi = 0.01$. Entonces

$$\text{precision} = \frac{0.90(0.01)}{0.90(0.01) + 0.05(0.99)} \approx 0.154.$$

Aunque el clasificador detecta el 90 % de los positivos y descarta correctamente el 95 % de los negativos, menos de una de cada seis alertas corresponde a un caso positivo. La baja prevalencia no invalida el modelo, pero cambia el uso que puede darse a sus alertas.



Figura 18: Particiones: train ajusta parámetros, dev selecciona modelos y test estima desempeño final; mezclar esos roles produce evaluaciones optimistas.

Lectura de la figura. Cada partición tiene un uso. Train puede aprender parámetros y transformaciones; dev puede guiar elecciones de modelo; test se reserva para estimar el desempeño final. Si test participa en selección, la comparación queda contaminada.

Métrica principal

La **métrica principal** es la regla cuantitativa que se usará para seleccionar modelos durante el desarrollo.

Ejemplos:

Problema	Métrica razonable	Motivo
Diagnóstico médico	recall o F1	falsos negativos costosos
Filtro de alertas	precision	falsos positivos saturan al usuario
Ranking general	ROC-AUC o PR-AUC	importa ordenar riesgos
Regresión de demanda	MAE o RMSE	depende del costo de error

Accuracy no siempre basta

En clases desbalanceadas, un modelo que predice siempre la clase mayoritaria puede tener alta accuracy.

Por eso, en clasificación binaria se debe revisar:

- matriz de confusión;
- precision;
- recall;
- F1;
- ROC-AUC o PR-AUC;
- costo de falsos positivos y falsos negativos.

Ejemplo resuelto: accuracy engañosa

Supón un problema con 1000 observaciones:

- 950 son negativas;
- 50 son positivas;
- un modelo predice todo como negativo.

La accuracy es:

$$\frac{950}{1000} = 0.95.$$

Pero el recall de la clase positiva es:

$$\frac{TP}{TP + FN} = \frac{0}{0 + 50} = 0.$$

Si detectar positivos era el objetivo, este modelo no sirve aunque su accuracy parezca alta.

Ejemplo resuelto: costo de falsos negativos

Supón un problema con 50 positivos y 950 negativos. Comparamos dos umbrales:

Umbral	TP	FP	FN	TN
A	35	40	15	910
B	25	10	25	940

El umbral A tiene:

$$\text{precision}_A = \frac{35}{35 + 40} \approx 0.467, \quad \text{recall}_A = \frac{35}{35 + 15} = 0.70.$$

El umbral B tiene:

$$\text{precision}_B = \frac{25}{25 + 10} \approx 0.714, \quad \text{recall}_B = \frac{25}{25 + 25} = 0.50.$$

Si un falso positivo cuesta 1 unidad y un falso negativo cuesta 10 unidades, el costo total observado de cada umbral es:

$$C_A = 40(1) + 15(10) = 190, \quad C_B = 10(1) + 25(10) = 260.$$

Aunque B tiene mayor precision, A es preferible bajo esta función de costo porque reduce falsos negativos. Si se quisiera un costo promedio por caso, se dividiría cada total por 1000. La métrica correcta depende del uso y de los costos declarados, no de cuál número se ve más alto.

Derivación guiada: F1 como equilibrio operativo

Precision y recall miden errores distintos:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}.$$

F1 es la media armónica de ambas:

$$F1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

La media armónica penaliza valores muy bajos. Por eso un modelo con precision alta pero recall casi cero no obtiene buen F1. Esto es útil cuando ambos tipos de desempeño importan, pero no reemplaza una función de costo si los errores tienen costos muy asimétricos.

De costos a una regla de decisión

Supongamos que el modelo entrega la probabilidad condicional bien calibrada $\eta(x) = \mathbb{P}(Y = 1 \mid X = x)$. Si predecir 1 cuando $Y = 0$ cuesta C_{FP} y predecir 0 cuando $Y = 1$ cuesta C_{FN} , los costos condicionales son

$$\mathcal{C}(1 \mid x) = C_{FP}[1 - \eta(x)], \quad \mathcal{C}(0 \mid x) = C_{FN}\eta(x).$$

Elegimos la clase 1 cuando $\mathcal{C}(1 \mid x) \leq \mathcal{C}(0 \mid x)$. Al reordenar,

$$\eta(x) \geq \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Con costos iguales se obtiene el umbral 1/2. Si los falsos negativos son más costosos, el umbral disminuye. La deducción muestra que el umbral pertenece a la decisión, no al ajuste de la distribución predictiva.

En datos reales usamos $\hat{p}(x)$, no $\eta(x)$ conocida. El umbral teórico solo es aplicable si las probabilidades están suficientemente calibradas, los costos representan el uso y no hay restricciones adicionales de capacidad. Por eso se comprueba y, cuando hace falta, se selecciona con validación; después se fija antes de evaluar test.

Regla de Bayes con dos costos. Bajo probabilidades condicionales conocidas y costos constantes $C_{FP}, C_{FN} > 0$, la regla que minimiza el costo esperado condicional predice 1 exactamente cuando $\eta(x) \geq C_{FP}/(C_{FP} + C_{FN})$.

Baseline

Ningún modelo se reporta sin baseline.

Un baseline puede ser:

- clase mayoritaria;
- promedio de entrenamiento;
- modelo lineal simple;
- regla de negocio;
- modelo anterior del proyecto.

Particiones

La estructura estándar es:

```
train -> ajustar parámetros
dev/validation -> elegir modelo e hiperparámetros
test -> estimar desempeño final
```

Test se toca una vez

Si se usa test para tomar decisiones, deja de ser una estimación honesta del desempeño final.

Advertencia

El conjunto de test no es una herramienta de diseño. Es una auditoría final.

La misma regla se aplica al umbral. Se elige con validación; después se fija y se evalúa una vez en test junto con el modelo final.

Estratificación

En clasificación, la partición debe preservar proporciones de clase cuando sea razonable:

```
train_test_split(X, y, stratify=y, random_state=42)
```

Si no se estratifica un dataset pequeño o desbalanceado, una partición puede cambiar artificialmente la dificultad.

Ejemplo resuelto: conteos en una partición estratificada

Supón un dataset con 2000 observaciones y clase positiva de 8 %. Entonces hay:

$$2000(0.08) = 160$$

observaciones positivas. En una división 60/20/20 estratificada, los tamaños esperados son:

Conjunto	Observaciones	Positivos esperados	Negativos esperados
train	$2000(0.60) = 1200$	$160(0.60) = 96$	1104
dev	$2000(0.20) = 400$	$160(0.20) = 32$	368
test	$2000(0.20) = 400$	$160(0.20) = 32$	368

La tabla no garantiza que cada partición sea perfecta en todos los rasgos del problema. Solo preserva, aproximadamente, la proporción de la clase usada para estratificar. Si hay tiempo, grupos, hospitales, sedes o usuarios repetidos, esas restricciones pueden ser más importantes que una estratificación simple.

Leakage

Leakage ocurre cuando información no disponible en producción entra al entrenamiento o a la evaluación.

Ejemplos:

- escalar usando todo el dataset antes de partir;
- seleccionar variables usando test;
- usar variables creadas después del evento a predecir;
- duplicados entre train y test;
- elegir hiperparámetros mirando test.

Pipeline como defensa

Los pipelines ayudan a aplicar `fit` solo donde corresponde:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

En cross-validation, cada fold ajusta sus transformaciones solo con el train fold.

Caso longitudinal: clasificación con slice reciente

El archivo `clasificacion_binaria_sintetica.csv` acompaña varios capítulos. Tiene variables `x1`, `x2`, una etiqueta binaria `target` y una columna `slice` con valores `base` y `reciente`. El objetivo pedagógico es discutir qué pasa cuando un modelo parece razonable en promedio, pero cambia su comportamiento en una subpoblación o periodo.

Para este caso, una primera partición debe declarar:

Decisión	Criterio
baseline	clase mayoritaria o logística simple
métrica primaria	F1 o recall si la clase positiva importa
partición	train/dev/test con semilla fija
estratificación	preservar proporción de <code>target</code> cuando sea razonable

Decisión	Criterio
auditoría	revisar desempeño por <code>slice</code> antes de recomendar

La columna `slice` no debe usarse automáticamente como variable predictora. A veces se usa para diagnóstico, no para entrenamiento. Esa distinción evita que el modelo aprenda un marcador de origen que no estará disponible o que no se quiere usar en producción.

Punto de control

Antes de comparar modelos, escribe explícitamente:

1. métrica principal;
2. baseline;
3. partición;
4. transformación dentro de pipeline;
5. regla para no tocar test.

Verificaciones seleccionadas

1. Si la clase positiva representa 8% de 2000 observaciones, hay 160 positivos.
2. En una división estratificada 60/20/20, se esperan aproximadamente 96 positivos en train, 32 en dev y 32 en test.
3. Si se ajusta `StandardScaler` antes de partir datos, el promedio de dev/test ya contaminó el preprocesamiento.

Para explorar más

- **Extensión matemática:** calcula precisión, recall, F1 y balanced accuracy para dos matrices de confusión con el mismo accuracy.
- **Extensión computacional:** repite una partición estratificada con varias semillas y mide la variación de la métrica principal.
- **Conexión con proyecto:** escribe qué métrica será primaria, cuál será secundaria y qué baseline debe superar tu modelo.

Revisión del capítulo

Este capítulo separa entrenamiento de evaluación. La métrica principal debe elegirse antes de comparar modelos y debe estar alineada con el costo de error. Accuracy puede ser engañosa cuando las clases están desbalanceadas o cuando un tipo de error pesa más que otro. Precision y recall

condicionan en direcciones diferentes; la fórmula de Bayes muestra que la precisión también depende de la prevalencia de la clase positiva en la población evaluada.

Las particiones `train`, `dev` y `test` cumplen funciones distintas. `Train` ajusta parámetros, `dev` guía decisiones de modelo y `test` estima desempeño final. La estratificación, las particiones temporales o por grupo deben elegirse según el problema, no por costumbre.

El `leakage` suele entrar por preprocesamiento aprendido con demasiados datos. Un **Pipeline** no es solo comodidad de código: es una barrera operativa para que escalamiento, imputación y selección de variables se ajusten donde corresponde. No evita por sí solo variables posteriores al evento, duplicados entre personas o una partición temporal mal construida; esos riesgos pertenecen al diseño del dataset.

Términos clave

- **Métrica principal:** cantidad que guía la selección del modelo.
- **Baseline:** regla simple que fija un punto mínimo de comparación.
- **Prevalencia:** probabilidad poblacional de la clase positiva.
- **Fórmula de Bayes:** identidad que invierte el condicionamiento combinando verosimilitud y prevalencia.
- **Train/dev/test:** separación para entrenar, seleccionar y estimar desempeño final.
- **Estratificación:** partición que preserva proporciones de clase.
- **Leakage:** entrada de información no disponible en producción al entrenamiento o evaluación.
- **Pipeline:** objeto que encadena preprocesamiento y modelo para evitar ajustes fuera de lugar.

Ejercicios

Preguntas conceptuales

1. Para un problema de fraude, explique cuándo preferiría `recall` y cuándo `precision`.
2. ¿Por qué `accuracy` puede ser una mala métrica en clases desbalanceadas?

Problemas cuantitativos

3. Diseñe una partición `train/dev/test` para un dataset de 2000 observaciones con clases desbalanceadas.
4. Si una clase positiva representa 8% de los datos, estime cuántos positivos deberían aparecer en cada partición de una división 60/20/20 estratificada.

Práctica computacional

5. Implemente una partición estratificada con semilla fija.
6. Construya un **Pipeline** que escale variables numéricas y ajuste un modelo.

Interpretación y pensamiento crítico

7. Identifique el leakage: se imputa la media de cada variable usando todo el dataset antes de `train_test_split`.
8. Escriba una regla concreta para decidir cuándo se permite tocar el conjunto de test.

Profundización matemática y estadística

9. Derive el umbral de Bayes para costos C_{FP} y C_{FN} a partir de los dos costos condicionales.
10. Compare el error estándar aproximado de recall 0.80 con 10 y con 500 positivos.
11. Construya un caso donde precision no esté definida y distinga la definición matemática de la convención de una biblioteca.
12. Explique por qué seleccionar el umbral con test invalida la interpretación de su métrica final, aunque el modelo no se vuelva a entrenar.
13. Derive la expresión de precision en términos de recall, especificidad y prevalencia. Evalúela para prevalencias 0.01, 0.10 y 0.50 manteniendo fijos recall y especificidad, e interprete el cambio.

Proyecto de cierre

14. Para su proyecto, escriba una ficha de evaluación con métrica principal, baseline, partición, riesgo de leakage y regla de uso del test.

Validación cruzada y curvas de aprendizaje

Pregunta aplicada

Un solo split puede ser afortunado o desafortunado. Necesitamos estimar variabilidad:

¿el modelo funciona de forma estable o solo tuvo suerte en una partición?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. explicar qué estima cross-validation;
2. reportar media y variabilidad de una métrica;
3. usar grid search sin contaminar test;
4. leer learning curves y validation curves;
5. distinguir problemas de sesgo, varianza y ruido.
6. distinguir validación para selección de validación para estimación;
7. diseñar validación anidada y calcular su costo en número de ajustes.

Qué aleatoriedad intenta representar la validación

Idealmente evaluamos un **procedimiento** A que transforma una muestra de entrenamiento en un modelo $A(D_m)$. Su desempeño esperado para tamaño m es

$$\mathcal{R}_m(A) = \mathbb{E}_{D_m \sim P^m} \mathbb{E}_{Z \sim P} [\ell(A(D_m); Z)].$$

Cross-validation reutiliza una muestra finita para aproximar esta esperanza sobre posibles conjuntos de entrenamiento y observaciones de validación. Los folds no son réplicas independientes: comparten datos y sus modelos están correlacionados. Por eso la desviación entre folds es descriptiva y no debe convertirse automáticamente en un intervalo de confianza.

Independencia de test. Si el procedimiento completo A queda fijado sin usar test y D_{test} es independiente con distribución P , entonces el promedio de pérdidas de test es condicionalmente insesgado para el riesgo de $A(D_m)$:

$$\mathbb{E}[\widehat{R}_{\text{test}}(A(D_m)) \mid D_m] = R_P(A(D_m)).$$

La igualdad deja de describir el protocolo si test modifica hiperparámetros, features, umbral o selección del reporte.

Bootstrap y unidad de remuestreo

El bootstrap empírico genera réplicas $D_n^{*(b)}$ muestreando con reemplazo desde D_n y recalcula un estadístico $T^{*(b)}$. La distribución de las réplicas aproxima la variación muestral de T bajo la distribución empírica.

Antes de remuestrear hay que precisar qué objeto se quiere estudiar. Hay dos preguntas diferentes:

1. **Modelo ajustado fijo.** Se conserva el modelo y se remuestran las unidades del conjunto de evaluación. En cada réplica se vuelve a calcular la métrica. La distribución resultante describe cuánto cambiaría esa métrica al observar otra muestra de evaluación semejante.
2. **Procedimiento de entrenamiento.** Se remuestra el conjunto de entrenamiento y se repite todo el procedimiento: transformaciones, selección de variables, ajuste de hiperparámetros y estimación final. Cada modelo resultante debe evaluarse con un mecanismo que no reutilice los datos empleados para escogerlo. Esta distribución incorpora la variación del ajuste, pero exige mucho más cómputo y un diseño cuidadoso de la evaluación.

Para el primer caso, un esquema básico es:

1. fijar el modelo y las n unidades de evaluación;
2. tomar n unidades con reemplazo;
3. calcular $T^{*(b)}$ sobre la réplica;
4. repetir para $b = 1, \dots, B$ y usar la dispersión o los cuantiles de $\{T^{*(b)}\}_{b=1}^B$.

Un intervalo percentil de nivel aproximado $1 - \alpha$ usa los cuantiles $\alpha/2$ y $1 - \alpha/2$ de las réplicas. Su validez no es automática: depende de que la distribución empírica y el esquema de remuestreo representen la fuente de variación que interesa.

La unidad remuestreada debe corresponder a la unidad aproximadamente independiente. Si hay varias filas por paciente o colegio, se remuestran grupos completos. En series temporales se requieren bloques u otro esquema que conserve la dependencia. Remuestrear filas individuales en estos casos produce réplicas artificialmente homogéneas y subestima la incertidumbre.

Cross-validation

En k -fold cross-validation:

1. Se divide el conjunto de entrenamiento en k folds.
2. Se entrena k veces.
3. Cada fold actúa una vez como validación.
4. Se reporta media y variabilidad.

```
scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring="f1")
```

Qué reportar

Un reporte no debe decir solo:

F1 = 0.84

Dice:

F1 CV = 0.84 ± 0.03

La desviación describe dispersión entre folds. No es, por sí sola, el error estándar de la media ni define un intervalo de confianza: los folds comparten gran parte de sus datos de entrenamiento y sus puntajes no son independientes.

Ejemplo resuelto: elegir entre dos modelos

Supón dos modelos evaluados con 5-fold CV:

Modelo	F1 medio	Desviación
A	0.84	0.02
B	0.86	0.10

El modelo B tiene mayor promedio, pero también mucha mayor variabilidad. Si el contexto exige estabilidad, A puede ser preferible. Si se elige B, debe justificarse y revisarse en slices o con más datos.

La selección no mira solo el máximo promedio.

Derivación guiada: media y variabilidad de CV

Si los puntajes de cinco folds son:

0.80, 0.84, 0.83, 0.86, 0.87,

la media es:

$$\bar{s} = \frac{0.80 + 0.84 + 0.83 + 0.86 + 0.87}{5} = 0.84.$$

La desviación resume qué tanto cambian los resultados entre folds. No convierte la validación cruzada en garantía absoluta, pero sí evita tratar una partición afortunada como regla universal. En reportes del curso, escribe media y variabilidad juntas:


```
F1 CV = 0.84 ± 0.03
```

En Lab 4 usaremos la regla explícita

$$s_{\text{cons}} = \bar{s} - s_{\text{folds}}$$

para ordenar candidatos cuando mayor es mejor. Es un **puntaje conservador heurístico**: penaliza dispersión, pero no tiene cobertura probabilística ni reemplaza la inspección de los folds.

Grid search

Para seleccionar hiperparámetros:

```
GridSearchCV(  
    estimator=pipeline,  
    param_grid=grid,  
    scoring="f1",  
    cv=5  
)
```

El grid search debe ocurrir dentro de entrenamiento/desarrollo, no sobre test.

Selección y validación anidada

Supongamos que comparamos M configuraciones y escogemos

$$\hat{\lambda} = \arg \min_{\lambda \in \Lambda} \hat{R}_{\text{CV}}(A_{\lambda}).$$

El mismo ruido que hace fluctuar los puntajes también influye en qué configuración gana. El mínimo observado tiende a ser optimista porque se escogió después de mirar M estimaciones. Aumentar la grilla puede aumentar este sesgo de selección aunque ninguna configuración nueva sea realmente mejor.

La validación cruzada anidada separa ambos papeles:

```
para cada fold exterior:  
    reservar el fold exterior para evaluación  
    dentro de los datos restantes:  
        comparar hiperparámetros con folds interiores  
        escoger una configuración  
        reajustar con todo el train exterior  
    evaluar una vez en el fold exterior  
agregar los puntajes exteriores
```

Los folds exteriores estiman el desempeño del **procedimiento de selección**, no de una configuración elegida de antemano. Después de cerrar el análisis, se selecciona una configuración con todos los datos de desarrollo, se reajusta y se usa el test externo una sola vez si existe.

Cuándo usarla. En el curso, un train/dev/test bien definido suele ser suficiente para laboratorios y proyecto. La CV anidada resulta apropiada cuando el dataset es pequeño, se comparan muchas configuraciones y se necesita una estimación menos optimista del procedimiento de selección.

Costo computacional de validar

Si una búsqueda evalúa M configuraciones con k folds, requiere Mk ajustes, más el ajuste final. Con validación anidada de r folds exteriores y k interiores, la búsqueda requiere aproximadamente

$$r(Mk + 1)$$

ajustes, antes del modelo final. El costo real depende de que cada fold usa menos datos y de si la biblioteca reutiliza cálculos, pero el orden explica por qué una grilla indiscriminada puede consumir el presupuesto sin aportar comprensión.

La comparación debe fijar también el presupuesto. Si una configuración recibe más épocas, más semillas o una grilla más favorable, el resultado mezcla calidad del modelo con esfuerzo de búsqueda.

Learning curve

Una curva de aprendizaje compara desempeño al aumentar el tamaño del conjunto de entrenamiento.

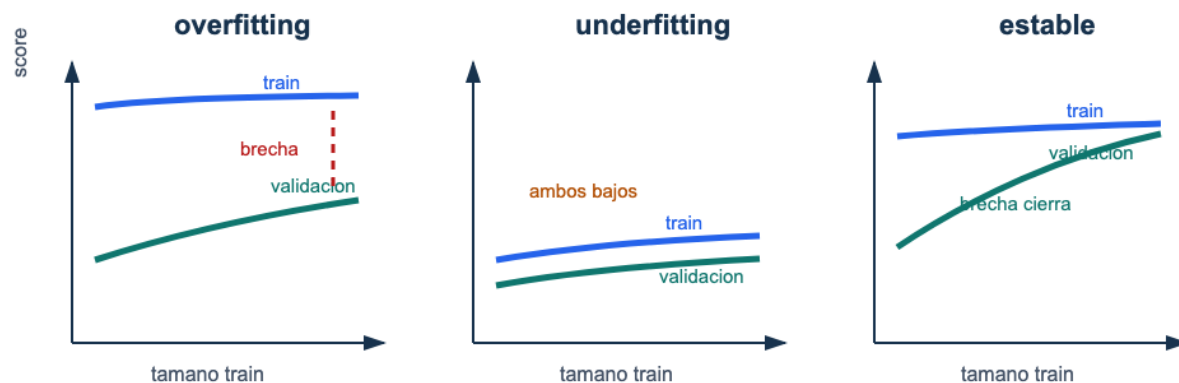


Figura 19: Curvas de aprendizaje: un caso de overfitting mantiene una brecha entre entrenamiento y validación, un caso de underfitting mantiene ambos desempeños bajos, y un caso estable cierra la brecha con más datos.

Lectura de la figura. Una learning curve no se interpreta por el valor final aislado, sino por la relación entre entrenamiento y validación. La brecha sugiere varianza; valores bajos en ambas curvas sugieren sesgo; curvas que se acercan pueden indicar que más datos ayudan.

Señales típicas:

Señal	Diagnóstico	Acción
Train alto y val bajo	overfitting	regularizar, simplificar, más datos
Train bajo y val bajo	underfitting	modelo más expresivo, mejores features
Train y val se acercan	estabilidad	evaluar límite por ruido
Val mejora con más datos	conseguir más datos puede ayudar	

Ejemplo resuelto: leer una learning curve

Supón que una curva de aprendizaje produce:

Tamaño train	F1 train	F1 validación
200	0.98	0.63
800	0.95	0.74
2000	0.92	0.82
5000	0.90	0.86

Lectura:

1. El desempeño de entrenamiento baja al aumentar datos, lo cual es normal.
2. El desempeño de validación sube de forma sostenida.
3. La brecha train-validación se reduce de 0.35 a 0.04.
4. El diagnóstico es alta varianza inicial que mejora con más datos.

La acción razonable no es aumentar complejidad. Primero conviene conseguir más datos, usar regularización moderada o mantener el modelo si el costo de más datos es alto.

Validation curve

Una validation curve varía un hiperparámetro:

```
regularización baja -> modelo flexible  
regularización alta -> modelo simple
```

Permite detectar si el hiperparámetro está empujando al modelo hacia overfitting o underfitting.

Caso longitudinal: estabilidad del modelo de clasificación

En `clasificacion_binaria_sintetica.csv`, una validación de cinco folds puede producir algo como:

Modelo	F1 CV medio	Desviación	Lectura
logística simple	0.71	0.03	baseline estable
logística con features polinomiales	0.76	0.09	mejora media, alta varianza
modelo regularizado	0.74	0.04	balance entre mejora y estabilidad

La selección no es automática. Si el contexto tolera variabilidad y luego se hará error analysis por `slice`, podría explorarse el modelo polinomial. Si se quiere una selección conservadora, el modelo regularizado puede ser preferible. El punto es que media y desviación se leen juntas.

Ejemplo resuelto: decisión con diferencia pequeña

Supón que dos modelos tienen:

Modelo	F1 CV medio	Desviación
A	0.812	0.018
B	0.819	0.061

La diferencia de medias es 0.007, menor que la variabilidad reportada de B. Una lectura cuidadosa no diría que B “gana” de forma clara. Diría que B tiene una mejora media pequeña, pero su inestabilidad exige revisar folds, slices o una partición temporal antes de cambiar la recomendación. En validación, ganar por una milésima no equivale a ganar una decisión.

Sesgo, varianza y ruido

La generalización puede entenderse como una combinación de:

- **sesgo:** error sistemático por modelo demasiado restrictivo;
- **varianza:** sensibilidad a cambios en entrenamiento;
- **ruido:** parte no explicable por las variables disponibles.

No todo error se arregla con un modelo más complejo.

Punto de control

Si una learning curve muestra train alto y validación bajo, no concluyas “necesito una red más grande”. Primero pregunta:

1. ¿hay leakage?
2. ¿la métrica está alineada con la decisión?
3. ¿el modelo ya tiene demasiada varianza?
4. ¿más datos podría cerrar la brecha?

Verificaciones seleccionadas

1. Evaluar 4 valores de `C`, 3 valores de `penalty` y 5 folds entrena $4 \cdot 3 \cdot 5 = 60$ ajustes.
2. Si la media CV mejora poco pero la desviación se triplica, la mejora puede no justificar el cambio de modelo.
3. `GridSearchCV` debe recibir el pipeline completo cuando hay escalamiento, imputación o selección de variables.

Ejemplo resuelto: cuándo no hacer grid más grande

Si una grilla inicial muestra que todos los valores de regularización fuerte tienen train y dev bajos, aumentar la grilla alrededor de regularización más fuerte probablemente no ayudará. La acción más razonable es revisar features, capacidad del modelo o métrica. Grid search no reemplaza diagnóstico: solo prueba combinaciones dentro del espacio que tú definiste.

Para explorar más

- **Extensión matemática:** interpreta una curva con alto sesgo y otra con alta varianza usando brechas train/dev.
- **Extensión computacional:** genera curvas de aprendizaje para dos tamaños de entrenamiento y reporta si más datos parecen útiles.
- **Conexión con proyecto:** decide qué experimento harías primero: más datos, más regularización, más features o modelo más flexible.

Criterio práctico de decisión

Cuando dos modelos tienen métricas cercanas, la elección no debe depender solo del tercer decimal. Un criterio razonable compara desempeño medio, variabilidad, complejidad, costo de explicación y estabilidad por slice. Si el modelo más complejo mejora 0.003 en F1, pero tiene mayor varianza entre folds y errores más difíciles de explicar, puede conservarse el modelo simple como baseline operativo.

La validación no declara un campeón universal. Compara candidatos bajo una partición, una métrica y un presupuesto. Elegir bien también significa registrar cuándo la diferencia observada no basta para preferir una alternativa más complicada.

Revisión del capítulo

La validación cruzada estima desempeño con variabilidad. Reportar solo una media oculta si el modelo es estable o si depende demasiado de una partición particular. La desviación entre folds puede cambiar una decisión técnica.

Grid search debe evaluar pipelines completos para no contaminar cada fold. Las learning curves y validation curves ayudan a distinguir sesgo, varianza, falta de datos, hiperparámetros mal calibrados y ruido irreducible.

La pregunta no es únicamente qué modelo gana. También importa si los resultados justifican cambiar complejidad, recolectar datos, ajustar regularización o aceptar un límite. La curva debe terminar en un experimento, no solo en una imagen.

Términos clave

- **Cross-validation:** estimación de desempeño usando varias particiones train/validation.
- **Grid search:** búsqueda sistemática sobre combinaciones de hiperparámetros.
- **Learning curve:** curva que compara desempeño al variar tamaño de entrenamiento.
- **Validation curve:** curva que compara desempeño al variar un hiperparámetro.
- **Sesgo:** error sistemático por modelo demasiado restrictivo.
- **Varianza:** sensibilidad del modelo a cambios en datos de entrenamiento.
- **Ruido:** variación no explicable con las variables disponibles.

Ejercicios

Preguntas conceptuales

1. Una learning curve muestra train F1 0.99 y validation F1 0.70. Diagnostique y proponga dos acciones.
2. ¿Por qué reportar desviación estándar de cross-validation puede cambiar una decisión de modelo?

Problemas cuantitativos

3. Compare dos modelos con medias CV similares pero desviaciones estándar diferentes. ¿Cuál elegiría si la estabilidad es prioritaria?
4. Si se evalúan 4 valores de C , 3 valores de **penalty** y 5 folds, ¿cuántos ajustes se entrenan?

Práctica computacional

5. Explique por qué `GridSearchCV` debe recibir un `Pipeline` completo si hay escalamiento.
6. Genere una tabla con media y desviación estándar de CV para al menos tres configuraciones.

Interpretación y pensamiento crítico

7. Una validation curve mejora hasta cierto punto y luego cae. Explique qué sugiere sobre el hiperparámetro.
8. ¿Cuándo buscar más datos es más razonable que cambiar de algoritmo?

Profundización matemática y estadística

9. Explique por qué el mínimo entre muchas estimaciones ruidosas de CV tiende a ser optimista para la configuración seleccionada.
10. Diseñe una validación anidada con 5 folds exteriores, 4 interiores y 12 configuraciones. Calcule el número aproximado de ajustes.
11. Distinga el riesgo de una configuración fija del riesgo del procedimiento que escoge una configuración con los datos.
12. Explique por qué dividir la desviación entre folds por $\sqrt{k}$ no produce automáticamente un error estándar válido.

Proyecto de cierre

13. Construya una learning curve o validation curve para un modelo de su proyecto y escriba un diagnóstico de sesgo/varianza/ruido con una acción siguiente.

Error analysis y data mismatch

Pregunta aplicada

Dos modelos pueden tener la misma métrica agregada y fallar en casos completamente distintos.

El análisis de errores pregunta:

¿dónde falla el modelo, para quién falla y qué experimento debe seguir?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. separar errores por tipo: falsos positivos, falsos negativos y casos ambiguos;
2. definir slices relevantes para un proyecto;
3. detectar señales de data mismatch;
4. escribir una recomendación con resultado, diagnóstico y acción;
5. reconocer cuándo una métrica agregada oculta daño o riesgo.
6. cuantificar la variación de una métrica por slice usando su denominador efectivo;
7. separar slices predefinidos de hallazgos exploratorios seleccionados después de mirar los errores.

Riesgo condicional por slice

Para un slice $S \subseteq \mathcal{X} \times \mathcal{Y}$ con $\mathbb{P}_P(S) > 0$, definimos

$$R_P(f \mid S) = \mathbb{E}_P[\ell(f(X), Y) \mid (X, Y) \in S].$$

El promedio global satisface una descomposición por una partición $S_1, \dots, S_J$:

$$R_P(f) = \sum_{j=1}^J \mathbb{P}_P(S_j) R_P(f \mid S_j).$$

Un slice pequeño puede tener riesgo alto y aportar poco al promedio global. Por eso el análisis debe reportar simultáneamente tamaño, prevalencia, conteos de error y métrica condicional.

Cambio de distribución como cambio de medida

Train y uso corresponden a distribuciones P_{train} y P_{uso} sobre el mismo espacio de observaciones. Si P_{uso} es absolutamente continua respecto de P_{train} , el riesgo de uso puede escribirse formalmente como

$$R_{\text{uso}}(f) = \mathbb{E}_{P_{\text{train}}} [w(X, Y) \ell(f(X), Y)], \quad w = \frac{dP_{\text{uso}}}{dP_{\text{train}}}.$$

Esta identidad no resuelve el problema: las razones pueden ser desconocidas o muy inestables, y puede haber regiones de uso sin soporte en entrenamiento. Sirve para precisar por qué comparar histogramas no basta para garantizar transportabilidad.

La expresión general reúne situaciones distintas, pero distinguirlas orienta el diagnóstico:

- En un **cambio de covariables**, cambia $P(X)$ mientras $P(Y | X)$ permanece aproximadamente estable. Reponderar observaciones puede ser útil solo si las regiones relevantes de uso tienen soporte en entrenamiento.
- En un **cambio de prevalencia**, cambia $P(Y)$ y se supone aproximadamente estable $P(X | Y)$. Las probabilidades posteriores, la precisión y el umbral operativo pueden cambiar aunque las distribuciones dentro de cada clase sean semejantes.
- En un **cambio de concepto**, cambia $P(Y | X)$. Un patrón que antes estaba asociado con la respuesta deja de tener el mismo significado; reponderar X no corrige por sí solo este problema.
- En un **cambio de medición o de soporte**, se modifica el instrumento, la definición de una variable, la población observable o aparecen regiones que no existían en los datos de entrenamiento. Sin observaciones comparables, el riesgo de uso no puede recuperarse solo a partir del conjunto original.

Por eso no existe una prueba única de *data mismatch*. Comparar marginales de variables puede revelar cambio de covariables, pero no demuestra estabilidad de $P(Y | X)$. Comparar prevalencias detecta otra clase de cambio, y auditar el proceso de medición resulta indispensable cuando las columnas conservan el mismo nombre pero ya no representan el mismo objeto.

Error analysis

Un análisis mínimo incluye:

1. Casos correctamente clasificados.
2. Falsos positivos.
3. Falsos negativos.
4. Subgrupos con peor desempeño.
5. Variables o condiciones asociadas al error.

Error analysis: el promedio no cuenta toda la historia

Un modelo con métrica global aceptable puede fallar en un slice que importa para el uso previsto.

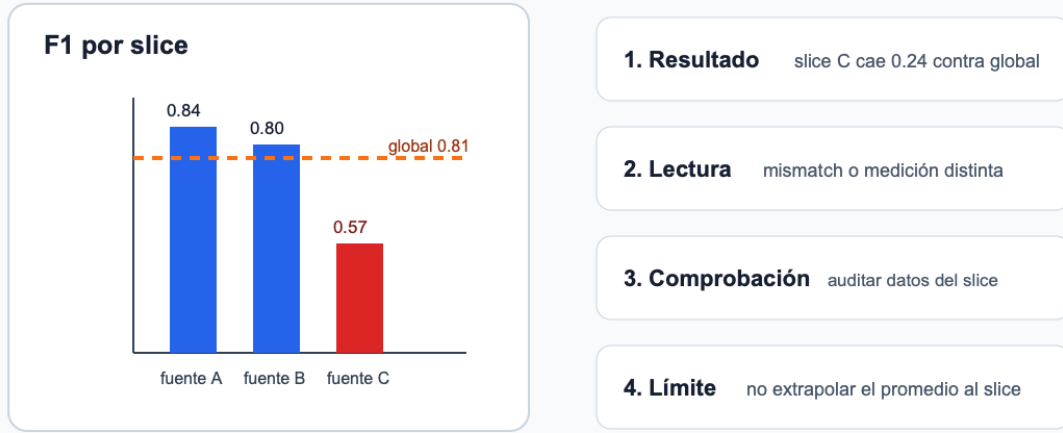


Figura 20: Error analysis por slices: el promedio global puede ocultar un slice crítico con desempeño bajo y exige una recomendación con resultado, diagnóstico, acción y límite.

Lectura de la figura. El promedio global puede ser suficiente para comparar dos prototipos, pero no para recomendar una decisión si un slice importante falla. La figura propone la secuencia mínima: resultado, diagnóstico, acción y límite.

Slices

Un **slice** es un subconjunto interpretable de datos:

- pacientes mayores de cierta edad;
- observaciones con medición faltante;
- imágenes de baja calidad;
- transacciones de monto alto;
- ejemplos de una fuente específica.

El objetivo es detectar fallas ocultas por el promedio.

Incertidumbre y búsqueda entre slices

Sea n_s el número de observaciones de un slice. Para una pérdida promedio,

$$\widehat{R}_S = \frac{1}{n_S} \sum_{i: Z_i \in S} L_i, \quad \widehat{\text{se}}(\widehat{R}_S) = \frac{s_S}{\sqrt{n_S}}.$$

Si L_i es el indicador de error, una aproximación binomial usa

$$\sqrt{\frac{\widehat{e}_S(1 - \widehat{e}_S)}{n_S}}.$$

Para recall, el denominador no es n_S , sino el número de positivos del slice; para precision es el número de predicciones positivas. Una métrica extrema con denominador 6 no tiene la misma estabilidad que la misma métrica con denominador 600.

Hay además un problema de selección. Si se inspeccionan cien slices y se reporta solo el peor, parte de la caída puede deberse a variación aleatoria. Los slices ligados a riesgos conocidos deben definirse antes de mirar resultados. Los hallazgos posteriores son hipótesis exploratorias y necesitan confirmación en otro periodo o muestra.

No imponemos una prueba de hipótesis automática para cada slice. Exigimos algo más básico y útil para este nivel: tamaño, denominadores de la métrica, intervalo o error estándar cuando sea razonable, comparación contra baseline y etiqueta clara de análisis confirmatorio o exploratorio.

Data mismatch

Data mismatch ocurre cuando train, dev o test no vienen de la misma distribución relevante.

Ejemplos:

- entrenar con datos de una ciudad y probar en otra;
- entrenar con datos históricos y usar en una temporada distinta;
- usar datos limpios de laboratorio y desplegar en datos ruidosos;
- combinar fuentes sin controlar origen.

Diagnóstico de mismatch

Compare:

- distribución de variables clave;
- proporción de clases;
- desempeño por fuente;
- error por periodo;
- desempeño en dev vs test.

Si validación y test difieren mucho, no basta con cambiar hiperparámetros. Puede existir mismatch.

Ejemplo resuelto: mismatch por proporción de clase

Supón la siguiente auditoría:

Conjunto	Observaciones	Positivos	Porcentaje positivo	Recall
train	5000	1000	20 %	0.84
dev	1000	210	21 %	0.82
test	1000	80	8 %	0.55

Train y dev se parecen en proporción de clase y desempeño. Test, en cambio, tiene una clase positiva mucho menos frecuente y recall mucho menor. El cambio de prevalencia puede alterar precisión, pero no explica por sí solo la caída de recall. Esa caída obliga a comparar también las distribuciones condicionadas a la clase y el proceso de medición.

La primera hipótesis no debería ser “el modelo necesita más hiperparámetros”. Una explicación más plausible es que test proviene de otra población, fuente o periodo. La acción técnica sería:

1. revisar cómo se construyó test;
2. comparar distribuciones de variables clave;
3. reportar desempeño por fuente o periodo;
4. decidir si se necesita un dev set más parecido a producción.

Si test representa producción real, el resultado indica un riesgo de despliegue, no solo una oportunidad de tuning.

Derivación guiada: qué métricas pueden promediarse por slices

Una pérdida promedio o la exactitud puede descomponerse por observación. Si un conjunto tiene dos slices con tamaños n_A, n_B y promedios L_A, L_B , entonces:

$$L_{\text{global}} = \frac{n_A L_A + n_B L_B}{n_A + n_B}.$$

Esto **no** vale en general para F1, precisión o recall. Para obtener su valor global se suman primero los conteos TP , FP , FN y TN de los slices y luego se aplica la fórmula de la métrica. Un promedio ponderado de F1 por tamaño puede dar otro número. En ambos casos, un slice grande puede ocultar la caída de uno pequeño; por eso se reportan el tamaño y los conteos junto a la métrica.

Recomendación técnica

Una recomendación debe seguir la forma:

Resultado -> diagnóstico -> acción

Ejemplo:

El recall cae de 0.84 a 0.62 en el grupo de mayor riesgo.

Diagnóstico: el modelo no generaliza bien a ese slice.

Acción: revisar representación de variables, ampliar datos del slice y ajustar métrica de selección.

Ejemplo resuelto: dos modelos con el mismo F1

Supón dos modelos con F1 global 0.82.

Slice	Modelo A	Modelo B
Datos recientes	0.80	0.83
Datos antiguos	0.84	0.81
Grupo de alto riesgo	0.55	0.76

Aunque el F1 global sea igual, el Modelo B puede ser preferible si el grupo de alto riesgo es central para el uso propuesto. Antes de escogerlo hay que revisar el tamaño del slice y sus conteos de error.

Ejemplo resuelto: brecha contra el promedio

Supón que un modelo tiene F1 global de 0.81, pero el análisis por slice muestra:

Slice	Observaciones	F1
fuelle A	1400	0.84
fuelle B	900	0.80
fuelle C	220	0.57

La brecha del slice C contra el promedio global es:

$$0.81 - 0.57 = 0.24.$$

Esa diferencia es grande y afecta una fuente específica. La recomendación no debería ser simplemente “el modelo tiene F1 0.81”. Una recomendación concreta sería:

Resultado: el F1 global es 0.81, pero en fuente C cae a 0.57 con 220 casos.

Diagnóstico: el modelo generaliza mal a la fuente C o esa fuente tiene un proceso de medición distinto.

Acción: auditar variables, faltantes y distribución de fuente C; entrenar/dev con datos representativos de esa fuente antes de recomendar despliegue.

Límite: el desempeño global no debe extrapolarse a fuente C hasta cerrar la brecha.

La magnitud de la brecha no prueba por sí sola la causa. Sí prueba que el promedio global es insuficiente para tomar la decisión.

Caso longitudinal: slice reciente

En `clasificacion_binaria_sintetica.csv`, la columna `slice` permite construir una tabla mínima:

Slice	Observaciones	F1	Recall	Lectura
base	180	0.82	0.79	desempeño esperado
reciente	60	0.61	0.48	posible mismatch

Aunque el número exacto dependa de la partición y del modelo, la pregunta permanece: si el uso real se parece al slice **reciente**, el promedio global no justifica despliegue. La acción razonable es construir un dev set más parecido a producción, revisar variables que cambian en el tiempo y reportar la limitación.

Punto de control

Una recomendación técnica es incompleta si no responde:

1. ¿qué resultado observaste?;
2. ¿qué diagnóstico es compatible con ese resultado?;
3. ¿qué acción concreta sigue?;
4. ¿qué límite o riesgo debe comunicarse?

Verificaciones seleccionadas

1. Si el F1 global es 0.81 y el F1 de un slice es 0.57, la brecha es 0.24.
2. Una brecha por slice no prueba la causa; prueba que el promedio global es insuficiente para la decisión.
3. Si dev y test tienen proporciones de clase muy distintas, primero se audita la construcción de particiones antes de hacer más tuning.

Procedimiento de auditoría por slice

Una auditoría mínima por slice sigue cuatro pasos. Primero, predefine los slices que sostendrán conclusiones confirmatorias: periodo, fuente, región, rango de edad, instrumento de medición o categoría operativa relevante. Segundo, calcula para cada slice tamaño, tasa de clase positiva, métrica principal y tipo de error dominante. Tercero, compara contra el desempeño global y contra el baseline; un slice puede tener bajo desempeño absoluto pero seguir mejorando el baseline, o puede

empeorar aunque el promedio global suba. Cuarto, escribe una acción: recoger más datos, cambiar partición, ajustar umbral, crear feature, separar modelo o limitar el uso.

Los slices encontrados después de inspeccionar errores son útiles para generar hipótesis, pero deben marcarse como exploratorios y confirmarse con otra muestra o periodo. De lo contrario se seleccionan, sin decirlo, los grupos con la caída más llamativa.

La auditoría también debe proteger contra conclusiones exageradas. Un slice con 12 observaciones no sostiene una afirmación fuerte, pero sí puede activar una revisión. Un slice grande con caída persistente sí cambia la recomendación. En el reporte, el tamaño del slice y la métrica deben aparecer juntos.

Para explorar más

- **Extensión matemática:** calcula tasas de error por slice y estima la brecha respecto al promedio global.
- **Extensión computacional:** crea una tabla de errores con columnas para predicción, etiqueta, slice, score y tipo de fallo.
- **Conexión con proyecto:** define dos slices que podrían revelar data mismatch y explica qué acción tomarías si fallan.

Revisión del capítulo

El análisis de errores convierte una métrica agregada en diagnóstico. Separar falsos positivos, falsos negativos, casos ambiguos y slices críticos revela fallas que un promedio puede ocultar.

Data mismatch aparece cuando train, dev o test no representan la misma población, periodo o proceso de medición. Antes de ajustar hiperparámetros, hay que revisar si la caída de desempeño se explica por cambio de distribución.

Una recomendación técnica debe tener estructura: resultado observado, diagnóstico probable, acción concreta y límite. Sin esa estructura, el reporte queda como una opinión difícil de auditar.

Términos clave

- **Error analysis:** revisión sistemática de casos donde el modelo falla.
- **Slice:** subconjunto de datos definido por una característica relevante.
- **Data mismatch:** diferencia entre distribuciones o condiciones de dev, test o producción.
- **Diagnóstico:** explicación técnica probable de un patrón de error.
- **Recomendación técnica:** acción proporcional al resultado, diagnóstico y límites.
- **Límite:** condición bajo la cual la conclusión no debe extrapolarse.

Ejercicios

Preguntas conceptuales

1. Proponga tres slices relevantes para un proyecto de predicción de abandono de clientes.
2. ¿Por qué un F1 global puede ocultar una falla grave?

Problemas cuantitativos

3. Dada una tabla de métricas por slice, identifique el slice crítico y calcule la brecha contra el desempeño global.
4. Si dev tiene 30 % de clase positiva y test 12 %, ¿qué sospecha antes de ajustar hiperparámetros?

Práctica computacional

5. Escriba una función que calcule una métrica por grupo.
6. Compare distribución de una variable clave entre dev y test con una tabla o gráfico.

Interpretación y pensamiento crítico

7. ¿Qué resultados sugerirían data mismatch entre dev y test?
8. Escriba una recomendación técnica con la estructura resultado-diagnóstico-acción.

Profundización matemática y estadística

9. Para un slice con 20 casos y tasa de error 0.30, calcule un error estándar binomial aproximado. Repita con 500 casos.
10. Explique por qué el denominador efectivo de recall por slice es el número de positivos y no el tamaño total del slice.
11. Se inspeccionaron 80 slices y se publicó solo el peor. Describa el sesgo de selección y diseñe una comprobación confirmatoria.
12. Indique qué falla en la ponderación por densidades cuando producción contiene una región sin soporte en entrenamiento.

Proyecto de cierre

13. Para su proyecto, entregue una tabla de error analysis con al menos tres slices, diagnóstico por slice y una acción concreta.

Ejercicios integradores: estrategia de machine learning

Esta sección entrena la habilidad central del módulo: convertir métricas y particiones en comparaciones verificables. Una buena respuesta nombra el resultado, explica por qué es relevante y reconoce qué todavía no se sabe.

Resultados de práctica

Al terminar este banco deberías poder:

1. elegir una métrica según costo de error;
2. diseñar particiones que respeten el problema;
3. detectar leakage antes de confiar en una métrica;
4. interpretar curvas de aprendizaje;
5. convertir error analysis en una recomendación técnica.

Mapa del banco

Bloque	Qué comprueba
A	métrica principal, métrica secundaria y baseline
B	particiones, validación y leakage
C	curvas, variabilidad y validación cruzada
D	análisis de errores por slices

Criterio de respuesta

En problemas aplicados, una respuesta completa suele tener esta forma:

`métrica elegida -> baseline -> partición -> riesgo de leakage -> decisión o siguiente experimento`

Si falta una de esas piezas, la comparación queda débil.

Columna probabilística del módulo

- P1. Accuracy aleatoria.** Un modelo fijo acierta 180 de 200 casos iid. Calcule accuracy y un error estándar aproximado. Explique qué supuesto permite usar la fórmula Bernoulli.
- P2. Denominadores.** Dos grupos tienen recall 0.80; uno contiene 10 positivos y otro 500. Compare la información que aportan ambas cifras sin usar solo el valor puntual.
- P3. Procedimiento y modelo.** Distinga el riesgo de un modelo ya fijado del riesgo esperado de un procedimiento que selecciona hiperparámetros a partir de una muestra.
- P4. Dependencia de folds.** Explique por qué los puntajes de 5-fold CV no son independientes y por qué $\text{media} \pm \text{desviación}$ no constituye automáticamente un intervalo de confianza.
- P5. Bootstrap por grupos.** Diseñe un remuestreo para datos con diez mediciones por paciente. Compare con remuestrear filas individualmente.
- P6. Riesgo por slices.** Construya dos slices con pesos poblacionales y riesgos condicionales dados; verifique la descomposición del riesgo global y muestre cómo un slice crítico puede quedar oculto.
- P7. Soporte.** Dé un ejemplo de observaciones posibles en producción con probabilidad positiva bajo P_{uso} y probabilidad cero bajo P_{train} . Explique por qué ponderar entrenamiento no basta.

Bloque A: métricas

- A1.** En un problema con falsos negativos costosos, explique por qué accuracy puede ser una mala métrica principal.
- A2.** Dada una matriz de confusión, calcule precision, recall y F1.
- A3.** Proponga una métrica principal y una métrica secundaria para un proyecto de detección temprana de riesgo.
- A4.** Derive el umbral de Bayes para costos C_{FP} y C_{FN} . Indique qué supuestos sobre calibración y costos necesita para usarlo con un score estimado.
- A5.** Dos clasificadores tienen recall 0.80, uno con 12 positivos y otro con 480. Calcule errores estándar aproximados y discuta qué comparación puede sostenerse.

Bloque B: particiones y leakage

- B1.** Explique por qué escalar antes de partir datos puede contaminar validación.
- B2.** Diseñe una partición para datos temporales. ¿Por qué `shuffle=True` puede ser inadecuado?
- B3.** Identifique dos formas de leakage en proyectos estudiantiles.
- B4.** Explique por qué elegir un umbral o una figura después de mirar test es selección, aunque los parámetros del modelo permanezcan fijos.

Bloque C: validación y curvas

- C1.** Interprete una curva con train alto y validation bajo.
- C2.** Explique cuándo más datos probablemente ayudarían.
- C3.** ¿Qué información agrega cross-validation frente a un único split?
- C4.** Diseñe una validación cruzada anidada para 20 configuraciones, 5 folds interiores y 4 exteriores. Calcule el número aproximado de ajustes.
- C5.** Explique por qué la mejor de muchas estimaciones de CV suele ser optimista para el candidato seleccionado.

Bloque D: error analysis

- D1.** Proponga slices para analizar errores en clasificación de dígitos escritos a mano.
- D2.** Escriba una recomendación técnica basada en un slice donde el F1 cae de 0.80 a 0.55.
- D3.** Explique por qué una métrica agregada no reemplaza inspección de errores.
- D4.** Se exploraron cincuenta slices y se reportó el de peor resultado. Diseñe un protocolo para distinguir hallazgo exploratorio de confirmación y especifique qué denominadores publicaría.

Respuestas seleccionadas

- A1.** Accuracy puede ser alta aunque el modelo ignore la clase importante. Si los falsos negativos son costosos, recall, F1 o una métrica ponderada por costo pueden reflejar mejor la decisión.
- B1.** Escalar antes de partir datos usa media y desviación calculadas con información de validación. Eso contamina la evaluación porque el preprocesamiento ya vio datos que debían permanecer independientes.
- C3.** Cross-validation muestra la dispersión de resultados entre folds. Un único split puede favorecer o perjudicar accidentalmente a un modelo. La desviación entre folds no es automáticamente un intervalo de confianza.
- D2.** Una recomendación mínima sería: “El F1 global es 0.80 y el del slice crítico es 0.55. El patrón es compatible con una falla de representación o mismatch. Antes de recomendar uso, se deben revisar tamaño y conteos de error del slice y confirmar la caída en otra muestra.”

Parte V: Aprendizaje no supervisado

Hasta ahora una respuesta observada ha guiado el ajuste y la evaluación. En el aprendizaje no supervisado no hay una variable objetivo que cumpla ese papel. Los algoritmos buscan estructura mediante varianza, error de reconstrucción, distancias o patrones de interacción, y la interpretación depende con mayor fuerza de las decisiones de representación.

PCA introduce primero una pregunta poblacional: ¿qué direcciones concentran mayor variación en un vector aleatorio? La matriz de covarianza muestral y la SVD proporcionan una respuesta empírica y computable. K-Means sustituye la varianza proyectada por distancias a centroides y muestra por qué el escalamiento y la inicialización pueden cambiar la solución. La recomendación, finalmente, utiliza matrices dispersas para estudiar similitud y estructura latente.

En los tres casos, descubrir una regularidad matemática no basta para asignarle un significado de dominio. La estabilidad frente a muestras, inicializaciones y transformaciones forma parte del resultado, al igual que la descripción de las unidades y variables que originaron la estructura.

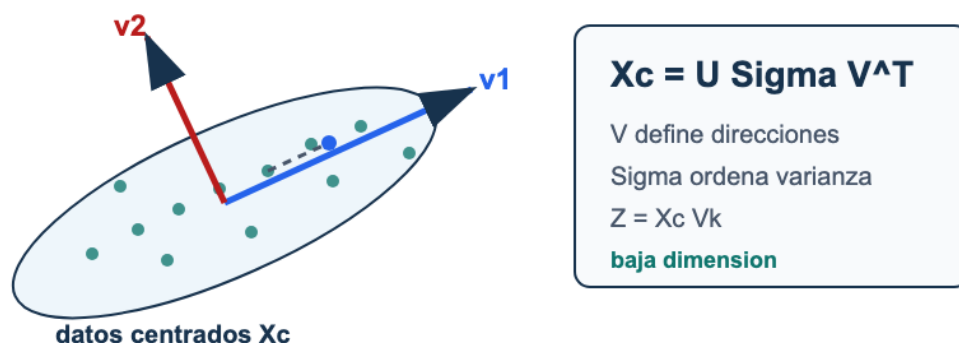


Figura 21: PCA como proyección: datos centrados se factorizan con SVD; los primeros vectores de V definen ejes principales y la matriz Z contiene coordenadas de baja dimensión.

La figura presenta PCA como una proyección calculada a partir de la SVD de los datos centrados. Las coordenadas reducidas conservan una parte de la variación, pero la elección del número de componentes y su interpretación requieren volver a las variables originales y al propósito del análisis.

SVD y PCA

Pregunta aplicada

Un dataset puede tener decenas o miles de variables. ¿Podemos representarlo en menos dimensiones sin perder la estructura principal?

PCA responde:

buscar direcciones ortogonales que capturan la mayor varianza posible.

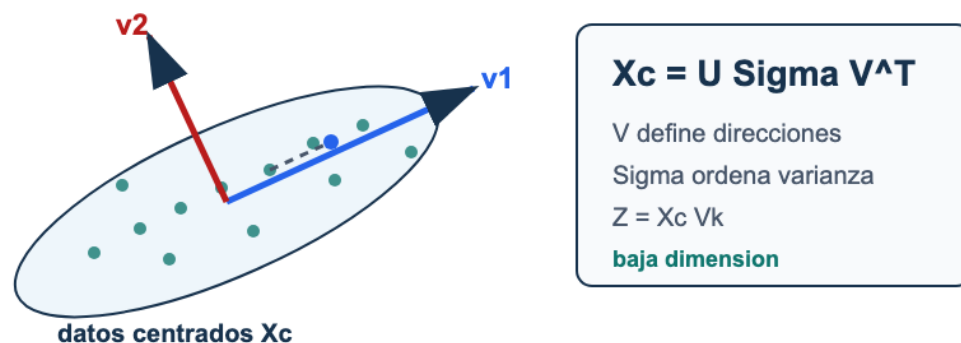


Figura 22: PCA como proyección: datos centrados se factorizan con SVD; los primeros vectores de V definen ejes principales y la matriz Z contiene coordenadas de baja dimensión.

Lectura de la figura. PCA empieza con datos centrados, usa la SVD para encontrar direcciones de alta varianza y proyecta sobre V_k . La proyección puede servir para visualizar, comprimir o alimentar otro modelo, pero no prueba causalidad.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. escribir la SVD de una matriz;
2. explicar por qué PCA requiere centrar datos;
3. calcular la forma de una proyección $Z = X_c V_k$;
4. interpretar varianza explicada y error de reconstrucción;
5. distinguir reducción de dimensión de interpretación causal.

6. derivar las direcciones principales mediante un problema restringido;
7. comparar la descomposición por covarianza con una SVD directa en costo y estabilidad.

PCA poblacional y PCA empírico

Sea $X \in \mathbb{R}^p$ un vector aleatorio con media μ y matriz de covarianza

$$\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^\top].$$

La primera dirección principal poblacional resuelve

$$v_1^* = \arg \max_{\|v\|_2=1} \text{Var}(v^\top X) = \arg \max_{\|v\|_2=1} v^\top \Sigma v.$$

Por el cociente de Rayleigh, v_1^* es un autovector asociado al mayor autovalor de Σ . No conocemos Σ ; con una muestra usamos

$$\widehat{\Sigma}_m = \frac{1}{m} X_c^\top X_c.$$

Si $X_c = U \Sigma_s V^\top$ es la SVD, entonces

$$\widehat{\Sigma}_m = V \frac{\Sigma_s^2}{m} V^\top.$$

Así, los vectores singulares derechos son direcciones principales **empíricas** y σ_j^2/m son autovalores de la covarianza empírica. Cambiar la muestra puede cambiar direcciones, especialmente cuando autovalores consecutivos son cercanos.

Máxima varianza empírica. Para X_c centrada,

$$\max_{\|v\|=1} \frac{1}{m} \|X_c v\|_2^2 = \frac{\sigma_1^2}{m},$$

y cualquier primer vector singular derecho alcanza el máximo.

La afirmación se obtiene con multiplicadores de Lagrange. Para una matriz de covarianza simétrica Σ , maximizamos $v^\top \Sigma v$ sujeto a $v^\top v = 1$. La función de Lagrange es

$$\mathcal{L}(v, \lambda) = v^\top \Sigma v - \lambda(v^\top v - 1).$$

Su condición estacionaria respecto a v es

$$2\Sigma v - 2\lambda v = 0,$$

de modo que una dirección candidata debe satisfacer $\Sigma v = \lambda v$. Además, en un autovector unitario,

$$v^\top \Sigma v = \lambda.$$

Por tanto, el máximo corresponde al mayor autovalor. Para k direcciones ortonormales, el problema se escribe

$$\max_{V_k^\top V_k = I_k} \text{tr}(V_k^\top \Sigma V_k),$$

y el máximo es la suma de los k mayores autovalores. Si hay autovalores repetidos, el subespacio principal está determinado, pero una base particular dentro de él no es única.

La indeterminación de signo de un componente no es incertidumbre muestral: v y $-v$ describen el mismo eje. La inestabilidad entre subespacios al cambiar la muestra sí es una cuestión estadística y debe estudiarse por remuestreo o particiones.

SVD

Para una matriz $X \in \mathbb{R}^{m \times p}$, su SVD reducida puede escribirse:

$$X = U_r \Sigma_r V_r^\top,$$

donde $r = \text{rank}(X)$, $U_r \in \mathbb{R}^{m \times r}$, $\Sigma_r \in \mathbb{R}^{r \times r}$ y $V_r \in \mathbb{R}^{p \times r}$.

Donde:

- U contiene direcciones en el espacio de observaciones;
- Σ contiene valores singulares no negativos;
- V contiene direcciones en el espacio de variables.

PCA como SVD del dato centrado

Primero centramos cada columna con el vector de medias $\mu \in \mathbb{R}^p$:

$$X_c = X - \mathbf{1}_m \mu^\top.$$

Luego:

$$X_c = U \Sigma V^\top.$$

Los componentes principales son las primeras filas de $V^\top$, o columnas de V , asociadas a los valores singulares más grandes.

Centrar no es una formalidad. Sin centrado se maximiza $m^{-1} \|Xv\|^2$, que mezcla variación con la distancia del promedio al origen. La primera dirección puede entonces apuntar hacia la media de las observaciones, no hacia la dirección en que fluctúan alrededor de ella. Si además se estandarizan

columnas, PCA analiza la matriz de correlaciones y responde una pregunta distinta: cada variable aporta en unidades de desviación estándar en lugar de conservar su escala original.

Proyección

Si tomamos k componentes:

$$Z = X_c V_k.$$

Aquí $Z \in \mathbb{R}^{m \times k}$ es la representación reducida.

Varianza explicada

La varianza explicada por el componente j es proporcional a:

$$\sigma_j^2.$$

La razón de varianza explicada:

$$\frac{\sigma_j^2}{\sum_{\ell} \sigma_{\ell}^2}.$$

Con la convención de covarianza empírica $1/m$, el autovalor es $\hat{\lambda}_j = \sigma_j^2/m$; el factor $1/m$ se cancela en la razón. Por eso la fórmula es exacta para la muestra observada, pero no afirma qué proporción de varianza explicaría otra muestra de la población.

Ejemplo resuelto: error desde valores singulares

Supón que una matriz centrada tiene valores singulares:

$$\sigma_1 = 9, \quad \sigma_2 = 4, \quad \sigma_3 = 1.$$

La energía total bajo la norma de Frobenius es:

$$9^2 + 4^2 + 1^2 = 98.$$

Si usamos solo $k = 1$, descartamos σ_2 y σ_3 . El error de reconstrucción cuadrático es:

$$4^2 + 1^2 = 17.$$

La varianza explicada acumulada por el primer componente es:

$$\frac{9^2}{98} \approx 0.827.$$

La lectura tiene dos partes: un componente conserva cerca de 82.7% de la variabilidad medida por PCA, pero todavía descarta una parte que puede ser importante para ciertos grupos, variables o decisiones posteriores.

Derivación guiada: reconstrucción y pérdida de información

Si usamos k componentes, retenemos:

$$X_c \approx U_k \Sigma_k V_k^\top.$$

La parte descartada corresponde a componentes $k+1, \dots, r$. Bajo la norma de Frobenius, el error de reconstrucción queda ligado a los valores singulares descartados:

$$\|X_c - \hat{X}_c\|_F^2 = \sum_{j=k+1}^r \sigma_j^2.$$

El teorema de Eckart-Young establece que esta reconstrucción truncada minimiza el error de Frobenius entre todas las matrices de rango a lo sumo k . Esta fórmula explica por qué la varianza explicada acumulada y el error de reconstrucción son dos caras del mismo cálculo. Retener pocos componentes puede ser útil para visualizar, pero siempre descarta variación.

Una forma de ver el resultado usa proyecciones ortogonales. Sea B una matriz de rango a lo sumo k y sea S su espacio columna. Entre todas las matrices cuyas columnas pertenecen a S , la más cercana a X_c es $P_S X_c$, donde P_S es la proyección ortogonal sobre S . Por Pitágoras,

$$\|X_c - P_S X_c\|_F^2 = \|X_c\|_F^2 - \|P_S X_c\|_F^2.$$

Minimizar el error equivale entonces a escoger un subespacio de dimensión k que capture la mayor energía $\|P_S X_c\|_F^2$. La caracterización variacional aplicada a $X_c X_c^\top$ selecciona $S = \text{span}(u_1, \dots, u_k)$, las primeras direcciones singulares izquierdas. Si $X_c = U \Sigma V^\top$,

$$P_S X_c = U_k U_k^\top U \Sigma V^\top = U_k \Sigma_k V_k^\top.$$

El error restante es $\sum_{j>k} \sigma_j^2$. Esta es una justificación de optimalidad respecto a error cuadrático de reconstrucción, no respecto a clasificación, interpretabilidad o utilidad de una variable minoritaria.

Costo y elección del método numérico

Hay dos rutas densas comunes. Formar la covarianza $X_c^\top X_c/m$ cuesta $O(mp^2)$ y diagonalizarla cuesta $O(p^3)$. Una SVD directa evita formar la covarianza y no eleva al cuadrado el número de condición. Si $m \geq p$, su costo denso también es del orden de $O(mp^2)$; si $p \gg m$, puede ser preferible trabajar con $X_c X_c^\top$ o con una SVD truncada.

Cuando solo se necesitan $k \ll \min(m, p)$ componentes, algoritmos iterativos o aleatorizados aproximan el subespacio dominante sin calcular toda la SVD. Deben verificarse con error de reconstrucción y estabilidad, porque ahora aparece una aproximación algorítmica adicional a la variación muestral.

Elección	Ventaja	Riesgo principal
autodescomposición de covarianza	conecta directamente con varianza	forma $X_c^\top X_c$ y empeora condicionamiento
SVD directa de X_c	mayor estabilidad numérica	puede ser costosa si solo se requieren pocos componentes
SVD truncada	aprovecha k pequeño	agrega tolerancia y posible error aproximado

El ajuste del centrado, el escalamiento y los componentes debe ocurrir dentro de cada conjunto de entrenamiento. Proyectar validación o test usa las medias, escalas y direcciones ya aprendidas; recalcularlas sería leakage.

Ejemplo resuelto: elegir k

Supón que los primeros componentes explican:

Componente	Varianza explicada	Acumulada
1	0.42	0.42
2	0.23	0.65
3	0.12	0.77
4	0.06	0.83

Si la meta es visualizar, $k = 2$ puede ser suficiente. Si la meta es reconstruir con poca pérdida, $k = 4$ puede ser más razonable. La elección depende del uso posterior y de una tolerancia declarada, no de un porcentaje universal.

Reconstrucción

Con k componentes:

$$\hat{X}_c = ZV_k^\top.$$

La reconstrucción en escala original:

$$\hat{X} = \hat{X}_c + \mathbf{1}_m \mu^\top.$$

El error de reconstrucción ayuda a medir pérdida de información.

Ejemplo resuelto: proyección y reconstrucción

Considera dos observaciones con dos variables:

$$X = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

Primero centramos por columnas. La media es $\bar{X} = (1, 1)$, entonces:

$$X_c = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Una dirección principal para estos datos es

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

La proyección con $k = 1$ es:

$$Z = X_c v_1 = \begin{bmatrix} \sqrt{2} \\ -\sqrt{2} \end{bmatrix}.$$

La reconstrucción centrada queda:

$$\hat{X}_c = Z v_1^\top = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Al sumar la media:

$$\hat{X} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

En este ejemplo la reconstrucción con un componente es exacta porque los datos centrados tienen rango 1. En un dataset real, $k = 1$ normalmente conserva solo una parte de la variación.

Caso longitudinal: PCA sintético

El archivo `pca_sintetico.csv` contiene cuatro variables generadas desde una estructura latente de baja dimensión. El ejercicio consiste en justificar qué se conserva y qué se pierde.

Resultado	Pregunta
media de entrenamiento	¿con qué parámetros se centró?
varianza explicada acumulada	¿cuántos componentes se retienen?
error de reconstrucción	¿qué se pierde al reducir dimensión?
cargas principales	¿qué variables dominan cada componente?
advertencia	¿qué interpretación causal no se puede afirmar?

Si los dos primeros componentes explican mucha varianza, puedes usarlos para visualizar o comprimir. Eso no significa que los componentes sean causas, grupos naturales o factores reales del dominio. Esa lectura requiere información de dominio adicional.

PCA no es interpretabilidad automática

Advertencia

Un componente principal es una combinación lineal de variables. Puede ser útil, pero no siempre tiene una interpretación humana directa.

Interpretar PCA requiere:

- revisar cargas de variables;
- medir varianza explicada;
- visualizar proyecciones;
- conectar componentes con el dominio.

Además, el signo de cada componente es arbitrario: si v_j es una dirección principal, $-v_j$ representa el mismo eje y produce scores con signo opuesto. Dos implementaciones pueden diferir en esos signos y tener idéntica reconstrucción. Las pruebas deben comparar el subespacio o la reconstrucción, no forzar una orientación.

Punto de control

Antes de usar PCA en un proyecto, responde:

1. ¿centraste con parámetros aprendidos en entrenamiento?
2. ¿cuánta varianza explican los componentes usados?
3. ¿la proyección se usa para visualizar, modelar o comprimir?
4. ¿qué interpretación no puedes afirmar?

Verificaciones seleccionadas

1. Si X tiene forma 500×64 y se usan $k = 2$ componentes, la proyección $Z = X_c V_k$ tiene forma 500×2 .
2. Si los primeros 10 componentes explican 85 % de la varianza, todavía se pierde 15 % de la variabilidad bajo esa medida.
3. Un scatterplot PCA 2D puede sugerir separación visual, pero no prueba causalidad ni garantiza buen desempeño predictivo.

Ejemplo resuelto: PCA dentro de un pipeline

Si PCA se usa antes de un clasificador, el centrado y los componentes deben aprenderse solo con entrenamiento. En scikit-learn, la forma segura es:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=10)),
    ("model", LogisticRegression())
])
```

En validación cruzada, cada fold aprende su propio `scaler` y su propio PCA con el train fold. Hacer PCA con todo el dataset antes de partir datos introduce leakage porque la proyección ya vio validación o test.

`StandardScaler` también cambia la geometría: PCA sobre variables estandarizadas equivale a trabajar sobre correlaciones, mientras PCA solo centrado conserva las unidades y puede quedar dominado por variables de gran varianza. La elección debe declararse.

Para explorar más

- **Extensión matemática:** reconstruye una matriz pequeña con rango 1 y compara el error de reconstrucción con el de rango 2.
- **Extensión computacional:** grafica varianza explicada acumulada y marca el punto donde agregar componentes deja de cambiar la interpretación.
- **Conexión con proyecto:** decide si PCA se usará para visualización, compresión, diagnóstico o como paso previo de modelado.

Revisión del capítulo

SVD factoriza una matriz en direcciones y magnitudes de variación. PCA usa esa estructura sobre datos centrados para construir proyecciones de baja dimensión que conservan tanta varianza como sea posible bajo el número de componentes elegido.

La forma de los objetos importa: V_k define direcciones, $Z = X_c V_k$ contiene scores y la reconstrucción aproxima los datos originales. La varianza explicada ayuda a elegir k , pero no convierte automáticamente los componentes en causas o conceptos de dominio.

PCA es una herramienta de compresión, visualización o preprocesamiento. Su uso exige declarar qué conserva, qué pierde y qué interpretación no está justificada por la proyección.

Términos clave

- **SVD:** factorización $X = U\Sigma V^\top$ que expone direcciones principales.
- **PCA:** proyección lineal que conserva la mayor varianza posible en pocas dimensiones.
- **Dato centrado:** matriz después de restar la media de cada variable.
- **Componente principal:** dirección lineal de alta varianza.
- **Varianza explicada:** proporción de variabilidad capturada por componentes.
- **Reconstrucción:** aproximación del dato original desde una proyección de baja dimensión.

Ejercicios

Preguntas conceptuales

1. Explique por qué PCA debe centrar los datos antes de calcular componentes.
2. ¿Por qué un componente principal no es automáticamente interpretable?

Problemas cuantitativos

3. Si X tiene forma (500, 64) y usamos $k=2$, ¿qué forma tiene la proyección PCA?
4. ¿Qué significa que los primeros 10 componentes expliquen 85 % de la varianza?

Práctica computacional

5. Implemente una proyección PCA usando SVD sobre datos centrados.
6. Calcule y grafique varianza explicada acumulada.

Interpretación y pensamiento crítico

7. Si dos componentes explican poca varianza pero separan visualmente grupos, ¿qué conclusión puede y no puede afirmar?

Profundización matemática y algorítmica

8. Use multiplicadores de Lagrange para derivar el problema de máxima varianza de la primera dirección principal.
9. Demuestre que el problema de k direcciones maximiza la traza $\text{tr}(V_k^\top \Sigma V_k)$.
10. Explique el argumento central de Eckart-Young y distinga optimalidad de reconstrucción de utilidad predictiva.
11. Compare costo y estabilidad de formar la covarianza frente a calcular la SVD directa de la matriz centrada.

Proyecto de cierre

12. Aplique PCA a un dataset de su proyecto o a uno pequeño generado para práctica, justifique k , muestre una proyección y escriba una advertencia de interpretación.

K-Means y clustering

Pregunta aplicada

Si no tenemos etiquetas, aún podemos preguntar:

¿hay grupos de observaciones con patrones similares?

K-Means busca particionar datos en k grupos representados por centroides.

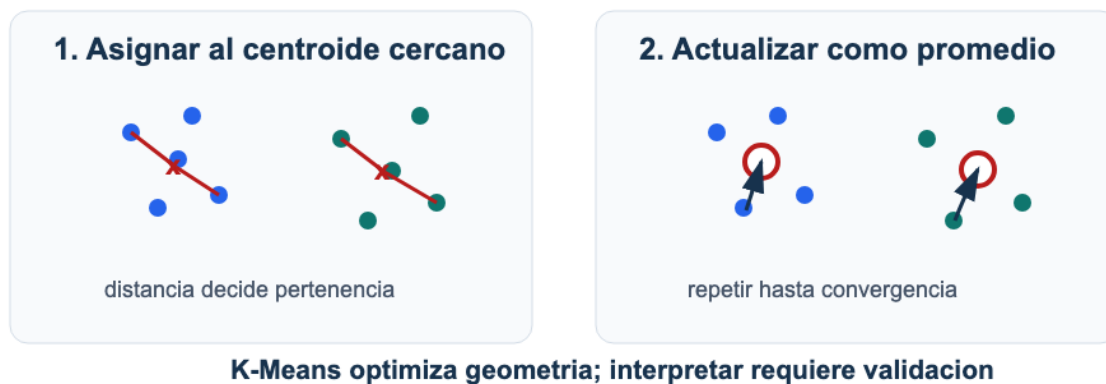


Figura 23: K-Means alterna dos pasos: asignar cada punto al centroide más cercano y actualizar centroides como promedio de los puntos asignados.

Lectura de la figura. K-Means alterna entre asignar observaciones al centroide más cercano y mover cada centroide al promedio de su grupo. La interpretación aplicada viene después de validar que esa geometría tenga sentido.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. escribir la función objetivo de K-Means;
2. explicar los pasos de asignación y actualización;
3. interpretar inercia y silhouette sin absolutizarlos;
4. justificar el escalamiento antes de usar distancias euclídeas;
5. reconocer cuándo un cluster no debe traducirse directamente en una decisión.
6. demostrar por qué cada paso no aumenta la función objetivo;

7. distinguir convergencia a un punto fijo de optimalidad global y estimar el costo del algoritmo.

Distorsión poblacional y objetivo empírico

Para centros $C = (\mu_1, \dots, \mu_k)$, la distorsión poblacional es

$$R_P(C) = \mathbb{E}_{X \sim P} \left[\min_{1 \leq j \leq k} \|X - \mu_j\|_2^2 \right].$$

K-Means minimiza su versión empírica

$$\widehat{R}_m(C) = \frac{1}{m} \sum_{i=1}^m \min_{1 \leq j \leq k} \|x_i - \mu_j\|_2^2.$$

`inertia_` es $m\widehat{R}_m(C)$ en la escala ajustada. Un valor bajo demuestra buena compactación de la muestra para ese k ; no prueba que existan clases naturales en la población.

Hay dos fuentes de variación diferentes:

- **inicialización algorítmica:** distintos centros iniciales pueden llegar a mínimos locales diferentes para el mismo dataset;
- **variación muestral:** otro conjunto de observaciones puede producir otra partición aunque el algoritmo encuentre su mejor mínimo empírico.

Una evaluación de estabilidad debe repetir semillas y remuestrear unidades independientes. Repetir solo `n_init` estudia optimización; no estudia estabilidad poblacional.

Función objetivo

Con indicadores $r_{ij} \in \{0, 1\}$ y $\sum_j r_{ij} = 1$, K-Means minimiza

$$J(R, C) = \sum_{i=1}^m \sum_{j=1}^k r_{ij} \|x_i - \mu_j\|_2^2.$$

La representación separa dos bloques de variables:

- $R = (r_{ij})$ contiene asignaciones discretas;
- $C = (\mu_1, \dots, \mu_k)$ contiene centroides continuos.

Algoritmo

1. Inicializar centroides.
2. Asignar cada punto al centroide más cercano.
3. Recalcular cada centroide como promedio del grupo.
4. Repetir hasta convergencia.

La frase «hasta convergencia» necesita una regla: ausencia de cambios en las asignaciones, desplazamiento máximo de centroides menor que una tolerancia, reducción de inercia menor que una tolerancia o número máximo de iteraciones. También debe definirse qué ocurre con empates y clusters vacíos.

Por qué la inercia no aumenta

Con centroides fijos, cada fila r_i aparece solo en

$$\sum_{j=1}^k r_{ij} \|x_i - \mu_j\|^2.$$

Asignar x_i al centroide más cercano minimiza ese término entre todas las asignaciones posibles. Por tanto, el paso de asignación no aumenta J .

Con asignaciones fijas, cada centroide resuelve un problema independiente:

$$\min_{\mu_j} \sum_{i:r_{ij}=1} \|x_i - \mu_j\|^2.$$

Como se deriva más adelante, su minimizador es el promedio del cluster. El paso de actualización tampoco aumenta J . La secuencia de inercias es no creciente y está acotada inferiormente por cero.

La monotonía implica que la secuencia de valores de J converge. Para concluir terminación finita de las asignaciones hace falta algo más: que todo cambio de asignación reduzca J estrictamente. Esto ocurre, por ejemplo, cuando no hay empates de distancia y ningún cluster queda vacío. Como solo hay un número finito de asignaciones, bajo esas condiciones el algoritmo alcanza una que ya no cambia. Con empates o degeneraciones puede haber pasos de costo idéntico; una implementación debe fijar desempate, tolerancia y número máximo de iteraciones.

Descenso por bloques de Lloyd. Cada iteración completa de asignación y actualización no aumenta la función objetivo de K-Means. Si todo cambio de asignación produce descenso estricto y no aparecen clusters vacíos, Lloyd termina en un punto fijo después de un número finito de iteraciones. Ese punto no tiene por qué ser un óptimo global.

K-Means++ escoge centros iniciales favoreciendo puntos alejados de los ya seleccionados. Reduce la probabilidad de una inicialización muy mala, pero no elimina mínimos locales. `n_init` ejecuta varias inicializaciones y conserva la de menor inercia.

Inertia

inertia es la suma de distancias cuadradas dentro de clusters. La mejor inertia alcanzable no aumenta al incrementar k : una solución con $k + 1$ centros puede reproducir una de k .

Por eso menor inertia no basta para elegir una segmentación:

- favorece valores grandes de k ;
- no está normalizada;
- puede favorecer clusters poco interpretables.

Silhouette score

Para una observación:

$$s = \frac{b - a}{\max(a, b)}.$$

Donde:

- a : distancia media al propio cluster;
- b : distancia media al cluster vecino más cercano.

Valores cercanos a 1 sugieren separación; valores cercanos a 0 sugieren solapamiento y valores negativos indican que otra asignación puede quedar más cerca. Silhouette requiere al menos dos clusters y menos clusters que observaciones.

Ejemplo resuelto: por qué escalar cambia clusters

Supón dos variables: ingreso mensual en pesos y número de visitas a una página. Dos observaciones son:

$$x_1 = (5,000,000, 10), \quad x_2 = (5,100,000, 80).$$

La diferencia en ingreso es 100,000, mientras la diferencia en visitas es 70. En distancia euclídea sin escalar, el ingreso domina casi todo:

$$\sqrt{100000^2 + 70^2} \approx 100000.$$

K-Means tenderá a agrupar por ingreso aunque visitas sea más relevante para el uso posterior. Después de estandarizar, cada variable se mide en desviaciones estándar. Esto elimina la dominancia causada únicamente por unidades, pero no demuestra que todas las variables deban tener el mismo peso.

Ejemplo resuelto: asignación a centroides

Supón dos centroides en una dimensión:

$$\mu_1 = 2, \quad \mu_2 = 8.$$

Para una observación $x = 5$:

$$|5 - 2| = 3, \quad |5 - 8| = 3.$$

La observación está empatada. En implementación debe haber una regla de desempate. Este ejemplo muestra que K-Means es geométrico: asigna por distancia, no por significado de negocio.

Ejemplo resuelto: una iteración completa

Supón cuatro puntos en una dimensión:

$$x = (1, 2, 8, 10)$$

y dos centroides iniciales:

$$\mu_1 = 1, \quad \mu_2 = 10.$$

Paso 1: asignar puntos.

Punto	Distancia a μ_1	Distancia a μ_2	Cluster
1	0	9	1
2	1	8	1
8	7	2	2
10	9	0	2

La inercia inicial con estas asignaciones es:

$$(1 - 1)^2 + (2 - 1)^2 + (8 - 10)^2 + (10 - 10)^2 = 5.$$

Paso 2: actualizar centroides.

$$\mu_1^{nuevo} = \frac{1 + 2}{2} = 1.5, \quad \mu_2^{nuevo} = \frac{8 + 10}{2} = 9.$$

Con los nuevos centroides, la inercia para las mismas asignaciones es:

$$(1 - 1.5)^2 + (2 - 1.5)^2 + (8 - 9)^2 + (10 - 9)^2 = 2.5.$$

La inercia bajó, pero eso no prueba que $k = 2$ sea la mejor decisión aplicada. Solo muestra que la actualización mejoró el objetivo geométrico de K-Means.

Derivación guiada: por qué el centroide es el promedio

Para un cluster con puntos $x_1, \dots, x_m$, K-Means escoge un centroide μ que minimiza:

$$\sum_{i=1}^m \|x_i - \mu\|_2^2.$$

En una dimensión, derivar respecto a μ da:

$$\frac{d}{d\mu} \sum_{i=1}^m (x_i - \mu)^2 = -2 \sum_{i=1}^m (x_i - \mu).$$

Igualando a cero:

$$\sum_{i=1}^m x_i - m\mu = 0 \quad \Rightarrow \quad \mu = \frac{1}{m} \sum_{i=1}^m x_i.$$

En varias dimensiones ocurre componente a componente. Por eso la actualización del centroide es el promedio de los puntos asignados.

Complejidad y casos degenerados

Calcular las distancias de m puntos en $\mathbb{R}^p$ a k centroides cuesta $O(mkp)$ por iteración. Actualizar centroides cuesta $O(mp)$. Con T iteraciones y s inicializaciones, el costo dominante es

$$O(sTmkp).$$

Una implementación vectorizada puede almacenar una matriz de distancias $m \times k$, con memoria $O(mk)$; también puede procesar bloques para reducirla.

Un cluster vacío no tiene promedio. Las implementaciones deben conservar el centro anterior, reubicarlo o reinicializarlo según una regla explícita. Los empates también necesitan desempate determinista si se quiere reproducibilidad. Estas decisiones rara vez aparecen en la fórmula del objetivo, pero forman parte del algoritmo ejecutado.

Escalamiento

K-Means usa distancia euclídea. Por tanto, las escalas importan.

```
Pipeline([
    ("scaler", StandardScaler()),
    ("kmeans", KMeans(n_clusters=3))
])
```

Limitaciones

K-Means funciona mejor cuando los clusters son:

- aproximadamente esféricos;
- de tamaños comparables;
- separables por distancia euclídea.

Puede fallar con formas no convexas, densidades distintas o variables mal escaladas.

Caso longitudinal: clusters sintéticos

El archivo `clusters_sinteticos.csv` fue generado con tres centros compactos. Eso lo hace útil para aprender K-Means, pero también peligroso si se generaliza demasiado: favorece justo la geometría que K-Means prefiere.

Resultado	Uso
scatterplot escalado	inspección geométrica inicial
inertia por k	reducción de compactación
silhouette	separación relativa
tamaños de clusters	detectar grupos diminutos
estabilidad por semilla	revisar sensibilidad

Si $k = 3$ gana en este dataset, no significa que “siempre hay tres segmentos”. Significa que, bajo esta geometría sintética y esta escala, tres centroides resumen bien la nube de puntos.

Estabilidad por inicialización

K-Means puede converger a soluciones distintas si cambian los centroides iniciales. Por eso un reporte debe fijar `random_state` y usar varias inicializaciones (`n_init`). Si dos corridas con el mismo k producen centroides muy distintos o clusters con tamaños incompatibles, la segmentación no es estable.

La estabilidad puede medirse comparando asignaciones con una cantidad invariante a permutaciones de etiquetas, como adjusted Rand index, o emparejando centroides. No convierte los clusters en categorías reales; solo indica que el algoritmo encuentra una partición parecida bajo variaciones de inicialización.

Punto de control

Antes de reportar clusters, verifica:

1. ¿las variables están en escalas comparables?
2. ¿probaste más de un k ?
3. ¿los clusters son interpretables con variables originales?
4. ¿hay un cluster demasiado grande o demasiado pequeño?
5. ¿qué decisión concreta usaría esta segmentación?

Verificaciones seleccionadas

1. El óptimo global de inercia es no creciente con k ; por eso no basta para elegir el número de clusters. Una corrida local mal inicializada puede no mostrar la monotonía esperada.
2. Si dos variables tienen escalas muy distintas, K-Means puede agrupar por la variable de mayor escala aunque no sea la más relevante.
3. Un cluster no es una categoría real hasta que se interpreta con variables originales y contexto de decisión.

Ejemplo resuelto: elegir k con información incompleta

Supón:

k	Inertia	Silhouette
2	420	0.48
3	260	0.55
4	210	0.51

La inercia baja de 3 a 4, pero silhouette cae. Puede elegirse $k = 3$ como representación candidata, siempre que los clusters tengan tamaño razonable y puedan describirse con variables originales. La tabla no prueba que existan tres grupos naturales; solo apoya que $k = 3$ es una representación útil bajo esta geometría.

Para explorar más

- **Extensión matemática:** ejecuta una iteración de K-Means a mano con cuatro puntos y dos centroides iniciales distintos.
- **Extensión computacional:** compara inercia y silueta para varios valores de k , usando la misma escala de variables.
- **Conexión con proyecto:** describe qué significaría cada cluster para una decisión real y qué variables dominan esa interpretación.

Revisión del capítulo

K-Means busca centroides que reduzcan la suma de distancias cuadráticas dentro de clusters. El algoritmo alterna asignación de puntos al centroide más cercano y actualización de centroides como promedios.

La inercia óptima no aumenta cuando crece k , por lo que no puede usarse sola como prueba de calidad. Silhouette aporta otra señal, pero también depende de geometría, escala y estructura de los datos.

Una segmentación requiere una escala justificada, varias inicializaciones, descripción con variables originales y cautela aplicada. Un cluster matemático no es automáticamente un grupo útil para una acción ni una categoría real de dominio.

Términos clave

- **Clustering:** agrupación no supervisada de observaciones.
- **Centroide:** punto representativo de un cluster.
- **Inercia:** suma de distancias cuadradas a centroides.
- **Silhouette:** métrica que compara cercanía al cluster propio frente a otros.
- **Escalamiento:** transformación necesaria cuando las variables tienen unidades distintas.
- **Segmentación:** interpretación aplicada de grupos, que requiere validación de negocio o dominio.

Ejercicios

Preguntas conceptuales

1. Explique por qué la inercia óptima no aumenta cuando crece k .
2. ¿Por qué K-Means debe usarse con escalamiento en variables de unidades distintas?

Problemas cuantitativos

3. Asigne tres puntos en una dimensión a dos centroides dados y calcule inercia.
4. Si un cluster contiene 90 % de las observaciones, ¿qué revisaría antes de reportarlo?

Práctica computacional

5. Ajuste K-Means para varios valores de k y reporte inercia y silhouette.
6. Construya un pipeline con escalamiento y K-Means.

Interpretación y pensamiento crítico

7. Proponga una situación donde clusters matemáticamente separados no deberían convertirse directamente en una decisión aplicada.
8. ¿Qué resultados usaría para nombrar un cluster sin caer en sobreinterpretación?

Profundización matemática y algorítmica

9. Demuestre que el paso de asignación y el paso de actualización no aumentan la función objetivo.
10. Explique por qué monotonía de la inercia no implica alcanzar el mínimo global.
11. Calcule el costo asintótico para T iteraciones, s inicializaciones, m observaciones, k clusters y p variables.
12. Proponga reglas reproducibles para un empate y para un cluster vacío.

Proyecto de cierre

13. Genere una segmentación con K-Means, describa cada cluster usando variables originales y escriba una recomendación cautelosa de uso.

Recomendación básica y estructura latente

Pregunta aplicada

Si sabemos qué ítems han gustado o sido usados por ciertos usuarios, podemos preguntar:

¿qué ítems son similares y qué recomendaríamos a un usuario?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. construir una matriz usuario-ítem simple;
2. calcular e interpretar similitud coseno;
3. describir una recomendación item-item;
4. explicar la idea de factores latentes por SVD;
5. reconocer problemas de popularidad, cold start y sesgo.
6. formular una factorización regularizada sobre entradas observadas y derivar sus gradientes;
7. estimar costo, memoria y límites de identificación de los factores.

El mecanismo de observación no es neutral

Sea R_{ui} la preferencia potencial de un usuario u por un ítem i y $O_{ui} \in \{0, 1\}$ el indicador de que esa interacción fue observada. El dataset contiene principalmente valores con $O_{ui} = 1$. En plataformas reales,

$$\mathbb{P}(O_{ui} = 1 \mid R_{ui}, u, i)$$

no suele ser constante: exposición previa, posición, popularidad y políticas del sistema afectan qué llega a registrarse. Por tanto, la distribución observada

$$P(R_{ui} \mid O_{ui} = 1)$$

puede diferir de la distribución de preferencias sobre todos los pares usuario-ítem. Tratar faltantes como desagrado supone un mecanismo que rara vez está justificado.

Riesgo offline y utilidad de uso

Una métrica offline estima desempeño bajo el protocolo que generó sus pares de evaluación. La utilidad relevante sería una esperanza bajo exposición futura,

$$\mathbb{E}_{(u,i) \sim P_{\text{uso}}} [U(u, i, \text{recomendar}(u))],$$

que no se identifica automáticamente con precisión@k o recall@k histórico. Esta brecha explica por qué una comparación offline no demuestra efecto causal sobre satisfacción, diversidad o descubrimiento.

Matriz usuario-ítem

Una matriz R puede representar:

- filas: usuarios;
- columnas: ítems;
- entradas: interacción, rating, compra, vista o preferencia.

Ejemplo:

$$R_{ui} = 1$$

si el usuario u interactuó con el ítem i .

Similitud coseno

Para dos ítems representados por vectores x y y :

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}.$$

La similitud coseno compara dirección más que magnitud.

Ejemplo resuelto: similitud coseno

Supón dos ítems con vectores de interacción:

$$x = (1, 1, 0, 0), \quad y = (1, 0, 1, 0).$$

Entonces:

$$x^\top y = 1, \quad \|x\| = \sqrt{2}, \quad \|y\| = \sqrt{2}.$$

Por tanto:

$$\cos(x, y) = \frac{1}{2} = 0.5.$$

La interpretación es moderada similitud: comparten un usuario, pero cada uno tiene otro usuario distinto.

Derivación guiada: qué ignora la similitud coseno

La similitud coseno normaliza por las normas:

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}.$$

Eso significa que dos ítems pueden tener similitud alta si apuntan en dirección parecida, aunque uno tenga muchas más interacciones que el otro. Esta propiedad puede ser útil para reducir el sesgo por popularidad, pero no lo elimina. Si un ítem aparece en casi todos los perfiles, puede seguir dominando rankings por cobertura y disponibilidad de datos.

La lectura de una similitud debe incluir tres preguntas: qué usuarios comparten los ítems, cuántas interacciones sustentan el cálculo y qué ítems quedan invisibles por falta de datos.

Recomendación item-item

Flujo básico:

1. Construir matriz usuario-ítem.
2. Calcular similitud entre ítems.
3. Para un ítem semilla, buscar ítems similares.
4. Excluir ítems ya vistos si se recomienda a un usuario.

Ejemplo resuelto: recomendación item-item

Supón la siguiente matriz binaria de interacción:

Usuario	A	B	C	D
$u1$	1	1	0	0
$u2$	1	1	1	0
$u3$	0	0	1	1
$u4$	0	1	0	1

Los vectores de ítems son:

$$A = (1, 1, 0, 0), \quad B = (1, 1, 0, 1), \quad C = (0, 1, 1, 0), \quad D = (0, 0, 1, 1).$$

Para recomendar a partir del ítem A , calculamos similitudes:

$$\cos(A, B) = \frac{2}{\sqrt{2}\sqrt{3}} \approx 0.816,$$

$$\cos(A, C) = \frac{1}{\sqrt{2}\sqrt{2}} = 0.5, \quad \cos(A, D) = 0.$$

Si el usuario semilla interactuó con A , el ítem más similar es B . Si ese usuario ya vio B , el siguiente candidato por similitud sería C . La recomendación debe excluir ítems ya vistos y declarar que esta regla no resuelve cold start ni sesgo de popularidad.

Evaluación offline: Precision@k

Una evaluación mínima de recomendación separa algunas interacciones conocidas como si fueran futuras. Luego pregunta si el recomendador las recupera entre sus primeras sugerencias.

Si el orden temporal está disponible, se ocultan interacciones posteriores y el modelo se ajusta solo con las anteriores. Un split aleatorio puede usar el futuro para explicar el pasado. Los usuarios evaluados deben conservar al menos una interacción de entrenamiento; los casos de cold start se reportan por separado.

Si para un usuario ocultamos los ítems relevantes $\{C, D\}$ y el sistema recomienda los tres primeros ítems:

$$[B, C, E],$$

entonces solo uno de los tres recomendados es relevante. La precisión en 3 es:

$$\text{Precision@3} = \frac{1}{3}.$$

Si además queremos saber cuántos relevantes recuperó:

$$\text{Recall}@3 = \frac{1}{2}.$$

Estas métricas no prueban satisfacción real. Solo verifican si el ranking recupera interacciones ocultas bajo una simulación offline. Un reporte debe decir qué se ocultó, cómo se partió el dato y qué usuarios o ítems quedaron sin recomendación.

Baseline de popularidad

Antes de celebrar un recomendador personalizado, compáralo contra una regla simple: recomendar los ítems más populares que el usuario no ha visto. La popularidad se calcula solo con entrenamiento. Este baseline suele ser fuerte porque muchos datasets tienen concentración de interacciones en pocos ítems.

Si el modelo latente no supera popularidad en Precision@k o cobertura, todavía no hay una mejora offline de personalización. Si lo supera, el reporte debe mostrar cuánto mejora y para qué grupos de usuarios. La personalización no se presume: se compara.

SVD para factores latentes

SVD puede aproximar:

$$R \approx U_k \Sigma_k V_k^T.$$

Recomendación: completar una matriz con estructura latente



Lectura: factores latentes explican patrones; la validacion decide si recomiendan bien.

Figura 24: Sistemas de recomendación: una matriz usuario-item incompleta se aproxima mediante factores latentes de usuarios e ítems; una calificación faltante se predice con un producto punto.

Lectura de la figura. La factorización representa usuarios e ítems en una baja dimensión compartida. Una recomendación es una predicción incompleta de la matriz, pero también una intervención que puede reforzar sesgos o popularidad.

Esto produce representaciones latentes de usuarios e ítems.

Esta igualdad requiere una advertencia. Una SVD directa trata cada entrada de la matriz numérica como observada; si se rellenan ratings faltantes con cero, se impone el supuesto de que “no observado” equivale a cero. Los métodos de factorización para recomendación suelen optimizar solo sobre entradas observadas o usar objetivos propios para retroalimentación implícita. Aquí la SVD introduce la geometría latente; no resuelve por sí sola el manejo de faltantes.

Factorización sobre entradas observadas

Sea Ω el conjunto de pares (u, i) para los que existe una interacción o rating. Asignamos a cada usuario un vector $p_u \in \mathbb{R}^k$ y a cada ítem un vector $q_i \in \mathbb{R}^k$. Un objetivo básico para ratings explícitos es

$$J(P, Q) = \frac{1}{2} \sum_{(u, i) \in \Omega} (R_{ui} - p_u^\top q_i)^2 + \frac{\lambda}{2} \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right).$$

La suma recorre solo entradas observadas; no convierte faltantes en ceros. Si $e_{ui} = R_{ui} - p_u^\top q_i$, los gradientes son

$$\begin{aligned} \nabla_{p_u} J &= - \sum_{i: (u, i) \in \Omega} e_{ui} q_i + \lambda p_u, \\ \nabla_{q_i} J &= - \sum_{u: (u, i) \in \Omega} e_{ui} p_u + \lambda q_i. \end{aligned}$$

Para obtener actualizaciones por interacción consistentes con este mismo objetivo, definimos los grados

$$d_u = |\{i : (u, i) \in \Omega\}|,$$

$$d_i = |\{u : (u, i) \in \Omega\}|.$$

La penalización puede distribuirse exactamente entre las interacciones:

$$J(P, Q) = \frac{1}{2} \sum_{(u,i) \in \Omega} \left[e_{ui}^2 + \frac{\lambda}{d_u} \|p_u\|^2 + \frac{\lambda}{d_i} \|q_i\|^2 \right].$$

Cada vector aparece en tantas sumas como indica su grado, de modo que esta expresión coincide con el objetivo global anterior. Una actualización SGD sobre un par observado usa entonces

$$\begin{aligned} p_u &\leftarrow p_u + \alpha \left(e_{ui} q_i - \frac{\lambda}{d_u} p_u \right), \\ q_i &\leftarrow q_i + \alpha \left(e_{ui} p_u^{\text{anterior}} - \frac{\lambda}{d_i} q_i \right). \end{aligned}$$

Se conserva p_u^{anterior} en la segunda expresión para que ambas actualizaciones correspondan al mismo gradiente. Usar λp_u y λq_i en cada interacción también es posible, pero corresponde a una penalización ponderada por los grados, no al objetivo global escrito arriba. También pueden añadirse un promedio global y sesgos de usuario e ítem; sin ellos, los factores deben explicar diferencias de nivel que no son necesariamente interacciones latentes.

Alternancia, costo e identificación

El objetivo no es conjuntamente convexo en (P, Q) , pero sí es cuadrático en un bloque cuando el otro queda fijo. Alternating least squares fija Q y resuelve un problema Ridge por usuario; luego fija P y resuelve uno por ítem. Cada paso no aumenta el objetivo si los subproblemas se resuelven exactamente, pero el resultado puede depender de la inicialización.

Una época SGD que recorre $|\Omega|$ interacciones cuesta $O(|\Omega|k)$ y almacena $O((n_{\text{usuarios}} + n_{\text{ítems}})k)$ parámetros. Esta estructura aprovecha dispersión: el costo depende de interacciones observadas, no del producto completo de usuarios por ítems.

Los factores no son únicos. Si A es invertible,

$$p_u^\top q_i = (A^\top p_u)^\top (A^{-1} q_i).$$

Por tanto, una coordenada latente aislada no tiene significado intrínseco. La regularización reduce parte de la indeterminación, pero las interpretaciones de «factor de acción», «factor de drama» o similares requieren evidencia externa. Lo identificable para el objetivo son principalmente los productos y rankings, no los nombres de los ejes.

Para retroalimentación implícita, una ausencia puede significar falta de exposición. Se usan objetivos ponderados, muestreo de negativos o pérdidas de ranking; el objetivo cuadrático anterior no debe trasladarse sin declarar ese cambio de modelo.

Caso longitudinal: interacciones de recomendación

El archivo `recomendacion_interacciones.csv` contiene usuarios, ítems y ratings ficticios. Su función es mostrar la lógica de una matriz usuario-item pequeña:

Resultado	Pregunta
matriz usuario-item	¿qué entradas se observaron y cuáles faltan?
promedio por ítem	¿hay sesgo de popularidad?
similitud item-item	¿qué ítems parecen cercanos?
cobertura	¿a cuántos usuarios/ítems puede recomendar?
evaluación offline	¿qué ratings ocultos se predicen?

Un recomendador puede producir rankings plausibles y aun así amplificar popularidad, ignorar ítems nuevos o recomendar sin diversidad. Por eso la pregunta aplicada no es solo “qué ítem tiene mayor puntaje”, sino “qué se omite y a quién afecta”.

Riesgos de recomendación

Los sistemas de recomendación pueden:

- reforzar popularidad;
- encerrar usuarios en burbujas;
- amplificar sesgos;
- optimizar interacción sin optimizar bienestar;
- fallar con usuarios o ítems nuevos.

Punto de control

Antes de recomendar, responde:

1. ¿la similitud refleja preferencia o solo popularidad?
2. ¿se excluyen ítems ya vistos?
3. ¿qué ocurre con usuarios nuevos?
4. ¿qué sesgo puede amplificarse?

Verificaciones seleccionadas

1. Una entrada faltante en la matriz usuario-item no significa rating cero; significa no observado.
2. La similitud coseno alta entre dos ítems no prueba que sean equivalentes; solo indica patrones de interacción parecidos bajo los datos disponibles.
3. Un ranking sin cobertura puede verse preciso para usuarios frecuentes y fallar para usuarios o ítems con pocas interacciones.

Ejemplo resuelto: cobertura mínima

Supón un recomendador que produce al menos una recomendación para 70 de 100 usuarios y cubre 12 de 50 ítems disponibles. Entonces:

$$\text{cobertura de usuarios} = 70/100 = 70\%, \quad \text{cobertura de ítems} = 12/50 = 24\%.$$

Aunque las recomendaciones visibles parezcan razonables, el sistema deja sin recomendación a 30 % de usuarios y concentra exposición en pocos ítems. Esos conteos deben aparecer antes de afirmar que el recomendador funciona bien.

Para explorar más

- **Extensión matemática:** calcula una similitud coseno item-item y compara la recomendación con una basada en producto punto.
- **Extensión computacional:** oculta algunas interacciones conocidas y mide si el sistema las recupera entre las mejores recomendaciones.
- **Conexión con proyecto:** explica qué sesgo de popularidad podría aparecer y cómo lo reportarías sin exagerar la calidad del sistema.

Revisión del capítulo

Un sistema de recomendación básico parte de una matriz usuario-ítem y mide similitudes o factores latentes para sugerir elementos no observados. La similitud coseno compara dirección de interacción y puede mitigar parcialmente diferencias de escala.

Las recomendaciones item-item son útiles porque se pueden explicar a partir de ítems similares, pero siguen dependiendo del historial observado. La idea de factores latentes resume usuarios e ítems en representaciones de baja dimensión.

El límite central es que el comportamiento pasado no define por completo la preferencia futura. Cold start, popularidad y sesgo deben declararse antes de presentar una recomendación como útil o justa.

Términos clave

- **Matriz usuario-ítem:** tabla que representa interacciones entre usuarios e ítems.
- **Similitud coseno:** medida angular entre vectores de interacción.
- **Recomendación item-item:** recomendación basada en ítems similares a uno observado.
- **Factor latente:** representación de baja dimensión aprendida de patrones de interacción.
- **Cold start:** dificultad para recomendar a usuarios o ítems sin historial.
- **Bucle de retroalimentación:** refuerzo de patrones pasados por recomendaciones futuras.

Ejercicios

Preguntas conceptuales

1. Explique por qué similitud coseno puede ser útil cuando algunos ítems son mucho más populares que otros.
2. ¿Qué problema aparece para recomendar a un usuario nuevo sin historial?

Problemas cuantitativos

3. Calcule similitud coseno entre dos vectores binarios de interacción.
4. Dada una matriz usuario-ítem pequeña, identifique los dos ítems más similares.

Práctica computacional

5. Implemente una función de similitud coseno para matrices.
6. Construya una recomendación item-item simple excluyendo ítems ya vistos.

Interpretación y pensamiento crítico

7. Proponga una limitación ética de un sistema de recomendación basado solo en comportamiento pasado.
8. ¿Cómo detectaría si el recomendador solo amplifica popularidad?

Profundización matemática y algorítmica

9. Formule el objetivo regularizado de factorización sobre Ω y derive los gradientes respecto a un vector de usuario y uno de ítem.
10. Escriba una actualización SGD para un rating observado, distribuya la regularización mediante d_u y d_i , y explique por qué debe conservarse el valor anterior de uno de los factores.
11. Compare el costo de una época dispersa $O(|\Omega|k)$ con operar sobre toda la matriz usuario-ítem.
12. Demuestre la indeterminación de los factores bajo un cambio invertible de base y explique qué limita su interpretación.

Proyecto de cierre

13. Diseñe un recomendador item-item mínimo, reporte tres recomendaciones y escriba una advertencia sobre cold start, popularidad o sesgo.

Ejercicios integradores: aprendizaje no supervisado

En aprendizaje no supervisado no hay una etiqueta que valide automáticamente el significado. Por eso estos ejercicios combinan cálculo, visualización, estabilidad e interpretación prudente.

Resultados de práctica

Al terminar este banco deberías poder:

1. calcular formas de SVD y proyecciones PCA;
2. explicar qué sí y qué no justifica una reducción de dimensión;
3. usar inercia y silhouette como resúmenes geométricos limitados;
4. construir una recomendación item-item mínima;
5. separar estructura matemática de interpretación aplicada.

Mapa del banco

Bloque	Qué comprueba
A	PCA, SVD, proyección e interpretación cautelosa
B	K-Means, selección de k y diagnóstico geométrico
C	similitud, recomendación y sesgos de interacción

Antes de responder

Para cada ejercicio, identifica:

1. qué estructura matemática estás usando;
2. qué cantidad se puede calcular;
3. qué conclusión sí está permitida;
4. qué conclusión sería una sobreinterpretación.

Columna probabilística del módulo

P1. Covarianza y SVD. Demuestre que si $X_c = U\Sigma_s V^\top$, entonces $X_c^\top X_c/m = V\Sigma_s^2 V^\top/m$. Identifique autovalores y autovectores de la covarianza empírica.

P2. Dirección poblacional. Explique la diferencia entre maximizar $\text{Var}(v^\top X)$ y maximizar $\|X_c v\|^2/m$ para una muestra observada.

P3. Estabilidad PCA. Proponga un bootstrap para comparar subespacios de dos componentes. Explique por qué comparar signos de vectores no es suficiente.

P4. Distorsión. Distinga $R_P(C)$, $\widehat{R}_m(C)$ e `inertia_`. Indique cuál calcula `KMeans` y cuál interesa para nuevas observaciones.

P5. Dos estabilidades. Diseñe un experimento que separe variación por inicialización de K-Means y variación por remuestreo del dataset.

P6. Exposición. Construya un ejemplo donde un ítem popular tenga más interacciones observadas aunque no tenga mayor preferencia promedio. Identifique la variable O_{ui} y el sesgo inducido.

P7. Evaluación offline. Explique qué distribución representa un test creado ocultando aleatoriamente interacciones históricas y qué escenarios de uso no representa.

Bloque A: PCA

A1. Derive las formas de U , Σ y $V^\top$ si X_c tiene forma $(100, 20)$.

A2. Implemente mentalmente los pasos de PCA desde cero: centrar, SVD, seleccionar componentes, proyectar.

A3. Explique por qué un componente con alta varianza no necesariamente es causal ni justo para tomar decisiones.

A4. Use multiplicadores de Lagrange para obtener la ecuación de autovectores del problema de máxima varianza.

A5. Explique por qué la SVD truncada es óptima para reconstrucción en norma de Frobenius y por qué eso no la hace óptima para una tarea supervisada posterior.

A6. Compare costo y condicionamiento de formar $X_c^\top X_c$ frente a calcular una SVD directa o truncada.

Bloque B: K-Means

B1. Escriba la función objetivo de K-Means y explique cada término.

B2. Compare `inertia` y `silhouette` como criterios geométricos para elegir k .

B3. Proponga un diagnóstico si K-Means produce un cluster con 95 % de los datos.

B4. Demuestre que los dos pasos de Lloyd no aumentan la inercia y explique por qué el resultado sigue dependiendo de la inicialización.

B5. Derive el costo $O(sTmkp)$ y proponga una regla para clusters vacíos.

Bloque C: recomendación

C1. Calcule similitud coseno entre dos vectores binarios simples.

C2. Explique cómo excluir ítems ya vistos por el usuario.

C3. ¿Qué diferencia hay entre recomendar por similitud de contenido y por comportamiento colaborativo?

C4. Formule una factorización regularizada sobre el conjunto observado Ω y derive los gradientes de usuario e ítem.

C5. Compare una época SGD de costo $O(|\Omega|k)$ con una SVD sobre una matriz densa rellena con ceros. Identifique el supuesto estadístico que cambia.

C6. Demuestre que una transformación invertible de la base puede conservar los productos usuario-ítem y discuta por qué los factores no tienen nombres intrínsecos.

Respuestas seleccionadas

A1. En SVD reducida, una forma usual es $U \in \mathbb{R}^{100 \times 20}$, $\Sigma \in \mathbb{R}^{20 \times 20}$ y $V^\top \in \mathbb{R}^{20 \times 20}$. En SVD completa, U podría tener forma 100×100 .

A3. Alta varianza significa que una dirección explica dispersión de los datos, no que represente una causa, una variable ética ni una regla justa de decisión.

B3. Un cluster con 95 % de los datos puede indicar k inadecuado, variables mal escaladas, ausencia de estructura separable o outliers que deforman centroides. Antes de interpretar, conviene revisar escalamiento, probar otros k y visualizar variables originales.

C3. La similitud de contenido compara atributos de los ítems. La similitud colaborativa compara patrones de interacción de usuarios. La segunda puede capturar comportamiento colectivo, pero también amplificar popularidad y sesgos históricos.

Parte VI: Reproducibilidad y comunicación

Un análisis no termina cuando aparece una métrica. Para que el resultado pueda examinarse, otra persona debe conocer los datos utilizados, las transformaciones, el código, el entorno y los parámetros que produjeron cada cifra. La reproducibilidad convierte esos elementos en parte del argumento, no en anexos opcionales.

Esta parte reúne primero las conexiones matemáticas entre los modelos estudiados: qué distribución o estructura intentan aproximar, qué objetivo empírico calculan y qué fuentes de variación permanecen. Después presenta un paquete reproducible mínimo y discute cómo registrar artefactos, versiones y resultados sin confundir una semilla fija con replicación estadística.

Los capítulos finales se ocupan de la escritura y la exposición oral. Una conclusión técnica debe distinguir resultados observados, interpretación y alcance; una figura debe permitir reconstruir la comparación que presenta; y una defensa debe hacer visibles tanto las decisiones tomadas como las afirmaciones que los datos no permiten sostener.

Reproducibilidad: que otra persona pueda reconstruir el resultado

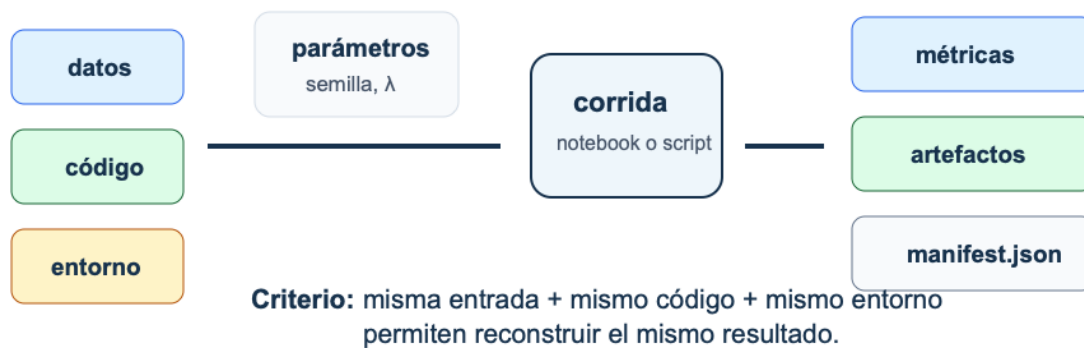


Figura 25: Reproducibilidad: datos, código, entorno y parámetros alimentan una corrida que produce métricas, artefactos y un manifiesto verificable.

La figura muestra esa trazabilidad como un flujo: datos, código, entorno y parámetros producen una corrida; la corrida genera métricas y artefactos; un manifiesto registra cómo volver a encontrarlos y verificarlos.

Síntesis conceptual: redes, estrategia y no supervisado

Pregunta aplicada

Una pregunta del parcial puede comenzar con logits, pasar por una tabla de validación y terminar con un slice que falla. ¿Cómo se organiza una respuesta sin mezclar esos objetos?

Alcance

Este capítulo prepara el Parcial 2 e integra tres bloques:

- redes neuronales desde primeros principios;
- estrategia ML y diagnóstico;
- aprendizaje no supervisado.

El examen evalúa cálculos, implementación y lectura de resultados. Recordar el nombre de una técnica no reemplaza ninguno de esos tres componentes.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. conectar forward/backward de redes con diagnóstico de entrenamiento;
2. explicar cómo métrica, baseline y partición sostienen una comparación;
3. interpretar PCA, K-Means o recomendación como descripciones estructurales;
4. responder preguntas integradoras sin depender de memoria aislada;
5. escribir una recomendación con resultado, lectura, siguiente comprobación y límite.

Mapa probabilístico de síntesis

Todo método del curso puede ubicarse en cuatro niveles:

Nivel	Pregunta	Ejemplo
población	¿qué cantidad define P ?	$\mathbb{E}[Y \mid X = x]$, $P(Y = k \mid X = x)$, Σ

Nivel	Pregunta	Ejemplo
muestra	¿qué observaciones se realizaron?	$D_m \sim P^m$ o muestra dependiente
objetivo empírico	¿qué promedio o geometría se calcula?	MSE, CE, distorsión, covarianza empírica
algoritmo	¿cómo se obtiene una solución?	<code>lstsq</code> , <code>backprop</code> , Adam, SVD, K-Means

Confundir niveles produce errores típicos: tratar una probabilidad estimada como verdadera, una métrica como riesgo conocido, una componente empírica como dirección poblacional fija o una semilla reproducible como estabilidad estadística.

Principio de riesgo empírico. Regresión, clasificación y redes minimizan promedios de pérdidas sobre una muestra. Esa operación es una aproximación al riesgo poblacional solo bajo supuestos sobre muestreo, dependencia y comparabilidad con el uso. La corrección del algoritmo no demuestra esos supuestos.

Mapa de contenidos

Bloque	Pregunta central	Qué debe aparecer
Redes	¿Cómo se calculan predicciones y gradientes?	dimensiones, forward, pérdida y actualización
Estrategia ML	¿Cómo comparo modelos sin usar test para escoger?	baseline, partición, métrica, validación y análisis de errores
No supervisado	¿Qué estructura encontré sin etiqueta?	varianza, reconstrucción, separación, estabilidad e interpretación

Redes neuronales

Forward propagation. Para una capa densa:

$$Z^{[\ell]} = A^{[\ell-1]}W^{[\ell]} + b^{[\ell]}, \quad A^{[\ell]} = g(Z^{[\ell]}).$$

Las dimensiones son parte del razonamiento. Si $A^{[\ell-1]} \in \mathbb{R}^{m \times d}$ y $W^{[\ell]} \in \mathbb{R}^{d \times h}$, entonces $Z^{[\ell]} \in \mathbb{R}^{m \times h}$.

Softmax y entropía cruzada

Para clasificación multiclase, softmax convierte logits en probabilidades:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}.$$

La implementación estable resta el máximo por fila antes de exponenciar. Esto no cambia las probabilidades, pero evita overflow.

Diagnóstico de modelos

Una comparación evaluable requiere:

1. baseline explícito;
2. métrica alineada con el costo de error;
3. particiones sin leakage;
4. validación o estimación de variabilidad;
5. análisis de errores;
6. conclusión con el alcance de los resultados disponibles.

Un puntaje alto no basta si no sabemos contra qué baseline compite, qué variabilidad tiene y dónde falla.

No supervisado

En PCA se decide cuántos componentes retener usando varias señales:

- varianza explicada acumulada;
- error de reconstrucción;
- interpretabilidad;
- utilidad para la tarea posterior.

En K-Means se evita elegir k solo por inercia porque inercia disminuye al aumentar k . Silhouette ayuda, pero también tiene sesgos geométricos: favorece grupos compactos y separados.

Forma del Parcial 2

Parte	Tipo	Propósito
Notebook autograded, 70 %	implementación y resultados numéricos	verificar funciones y cálculos
Preguntas estructuradas, 20 %	fórmulas, selección, respuestas numéricas	comprobar conceptos con respuesta cerrada

Parte	Tipo	Propósito
Explicación breve, 10 %	rúbrica 0/1/2 escalada a 10	leer un resultado y acotar su uso

Regla de preparación

Una respuesta breve puede seguir esta estructura:

resultado -> lectura -> siguiente comprobación -> límite

Cómo leer una pregunta integradora

Una pregunta integradora suele mezclar cálculo, implementación y decisión. Antes de responder, separa cuatro capas:

Capa	Pregunta que debes responder
Objeto matemático	¿qué función, matriz, métrica o pérdida aparece?
Procedimiento	¿qué algoritmo o transformación se aplica?
Resultado	¿qué número, curva, tabla o slice sostiene la afirmación?
Alcance	¿qué comprobación sigue y qué no puede afirmarse todavía?

Por ejemplo, si una pregunta combina PCA y clasificación, no basta con decir que PCA “mejora” el modelo. Debes decir si PCA se ajustó dentro del pipeline, cuánta varianza conserva, cómo cambia la métrica de validación y qué interpretación no puede hacerse de los componentes. Esta lectura evita respuestas largas pero desordenadas.

Ejemplo resuelto: softmax estable en síntesis

Supón que una red produce logits:

$$z = (1000, 1001, 1002).$$

Calcular e^{1002} directamente puede producir overflow. Restamos el máximo:

$$z - \max(z) = (-2, -1, 0).$$

Entonces:

$$\text{softmax}(z) = \frac{(e^{-2}, e^{-1}, 1)}{e^{-2} + e^{-1} + 1}.$$

Como $e^{-2} \approx 0.135$ y $e^{-1} \approx 0.368$, la suma es aproximadamente 1.503. Por tanto:

$$\text{softmax}(z) \approx (0.090, 0.245, 0.665).$$

La respuesta integradora debe decir dos cosas: la resta del máximo no cambia las probabilidades y evita un problema numérico. Si la clase correcta es la tercera, la pérdida de esa observación sería $-\log(0.665) \approx 0.408$.

Ejemplo resuelto: estructura no supervisada dentro de una respuesta

Supón que PCA muestra separación visual entre dos grupos y K-Means produce dos clusters con silhouette razonable. Son dos resúmenes de la misma geometría, no dos pruebas independientes. Pueden sugerir estructura, pero no prueban que los grupos correspondan a clases útiles para una decisión supervisada.

Una respuesta integradora adecuada sería:

Resultado: PCA y K-Means resumen la nube en dos grupos geométricos.
Lectura: puede haber estructura de baja dimensión, pero no hay etiqueta ni costo de error.
Comprobación: tratar los clusters como hipótesis exploratoria y compararlos con variables de d
Límite: no afirmar que los clusters son segmentos reales ni que predicen una decisión.

La síntesis del curso exige separar un resultado exploratorio de una evaluación predictiva. Un método no supervisado puede abrir una pregunta; no siempre puede cerrarla.

Ejemplo resuelto: respuesta integradora

Caso: una red mejora accuracy global, pero el análisis por slice muestra caída fuerte en datos recientes.

Respuesta posible:

Resultado: la accuracy global mejora, pero el slice reciente cae.
Lectura: el patrón es compatible con data mismatch temporal.
Comprobación: validar con partición temporal, revisar variables y medir el cambio por periodo.
Límite: no recomendar despliegue si el uso real se parece al slice reciente.

La clave es no quedarse en “mejoró la métrica”.

Caso longitudinal: respuesta integradora

Retoma `clasificacion_binaria_sintetica.csv`. Una respuesta integradora no debe decir solo “el modelo regularizado ganó”. Debe conectar varias capas del análisis:

Pieza	Resultado o registro esperado
modelo	logística regularizada o baseline comparable
validación	media y variabilidad de F1/recall
error analysis	desempeño por <code>slice</code>
reproducibilidad	semilla, partición, pipeline y manifest
comunicación	recomendación y límite

Una respuesta de parcial aceptable sería:

```
El modelo regularizado mejora el baseline en F1 medio, pero el recall cae en el slice reciente. El diagnóstico probable es mismatch temporal o cambio de distribución. Recomendaría usarlo solo como baseline interno y construir una validación temporal antes de despliegue.
```

Esta respuesta integra métrica, validación, diagnóstico y alcance.

Verificaciones seleccionadas

1. Una mejora global no basta si el slice relevante para producción empeora.
2. PCA o K-Means pueden apoyar exploración, pero no sustituyen una métrica supervisada cuando la decisión depende de etiquetas.
3. La secuencia **resultado** -> **lectura** -> **comprobación** -> **límite** obliga a nombrar qué se observó y cómo se podría refutar la explicación propuesta.

Ejemplo resuelto: matriz de decisión integradora

Supón dos modelos:

Resultado	Modelo A	Modelo B
F1 global	0.83	0.85
F1 slice reciente	0.62	0.51
desviación CV	0.03	0.09
manifest completo	sí	no

Aunque B tiene mayor F1 global, A es el candidato más prudente si el slice reciente representa el uso futuro: su caída es menor y su validación es más estable. El manifest faltante de B impide auditar esa corrida, pero no demuestra que B prediga peor; es un problema de trazabilidad que debe corregirse antes de comparar de nuevo.

Mini-rúbrica para una respuesta integradora

Una respuesta de parcial puede calificarse de forma consistente con cuatro criterios 0/1/2.

Criterio	0	1	2
resultado	no cita resultados	cita un resultado aislado	conecta métrica global, slice y variabilidad
diagnóstico	no explica causa probable	nombra una causa sin sostenerla	relaciona patrón observado con causa plausible
acción	propone algo genérico	propone una acción parcial	propone acción proporcional y verificable
límite	no declara límite	límite vago	límite específico sobre uso, datos o despliegue

Esta rúbrica también guía el estudio. Antes del parcial, el estudiante debe practicar respuestas que no sean listas de conceptos, sino argumentos breves con resultados identificables. Para el profesor, la ventaja es que la parte manual se concentra en juicio técnico observable, mientras los cálculos, formas y métricas pueden quedar autocalificados.

Un error común es responder solo con el nombre del método: “usaría regularización” o “miraría PCA”. Eso rara vez basta. La respuesta debe decir qué resultado activa esa propuesta y qué esperaría observar después. Si la acción propuesta no produce una verificación posterior, todavía no es una decisión técnica cerrada. El objetivo es mostrar criterio técnico, no solo vocabulario aislado.

Para explorar más

- **Extensión matemática:** construye un mapa de dependencias entre pérdida, métrica, regularización, validación y decisión.
- **Extensión computacional:** rehace un experimento anterior desde cero y verifica si produce las mismas métricas dentro de tolerancias razonables.
- **Conexión con proyecto:** escribe qué comprobación mínima usarías para estudiar si tu resultado depende de una partición afortunada.

Revisión del capítulo

Este capítulo reúne los conceptos necesarios para responder preguntas integradoras. Una buena respuesta no enumera fórmulas: conecta modelo, métrica, validación, resultados y alcance.

El Parcial 2 exige reconocer cuándo una mejora global es insuficiente, cuándo un slice crítico cambia la recomendación y cuándo una técnica no supervisada solo permite una conclusión exploratoria. También exige cálculos básicos estables, como softmax con resta del máximo.

La estructura de respuesta esperada es resultado -> lectura -> comprobación -> límite. Permite calificar con claridad y evita que una explicación plausible se presente como hecho demostrado.

Términos clave

- **Respuesta integradora:** explicación que conecta modelo, métrica, resultado y límite.
- **Data mismatch temporal:** diferencia entre datos antiguos y recientes que afecta generalización.
- **Síntesis conceptual:** capacidad de relacionar contenidos de varios módulos en un argumento.
- **Alcance proporcional:** conclusión ajustada a la partición, métrica y datos disponibles.

Ejercicios

Preguntas conceptuales

1. Explique por qué una mejora global no basta si un slice crítico empeora.
2. ¿Qué diferencia hay entre diagnosticar overfitting y diagnosticar data mismatch?

Problemas cuantitativos

3. Compare dos modelos con métricas globales similares y desempeño distinto por slice; elija uno y justifique.
4. Calcule softmax estable para logits (1000, 1001, 1002) usando resta del máximo.

Práctica computacional

5. Escriba una función que evalúe una métrica global y la misma métrica por slice.
6. Cree una tabla resumen para preparar una respuesta de parcial: baseline, métrica, error principal y acción.

Interpretación y pensamiento crítico

7. Escriba una respuesta de 120 palabras con estructura resultado -> lectura -> comprobación -> límite.

Proyecto de cierre

8. Tome un resultado real o simulado de su proyecto y conviértalo en una respuesta oral de dos minutos para Parcial 2.

Reproducibilidad y tracking

Pregunta aplicada

Si otra persona recibe nuestros resultados, ¿puede reconstruir qué datos, código, parámetros y decisiones produjeron la conclusión?

Qué significa reproducir

Reproducibilidad: capacidad de volver a ejecutar el análisis con los mismos datos, código, entorno y parámetros para obtener resultados iguales o numéricamente equivalentes bajo una tolerancia declarada.

Esta propiedad permite comparar corridas y depurar errores. No demuestra que el modelo sea correcto ni que el resultado se replique con otra muestra.

Reproducibilidad: que otra persona pueda reconstruir el resultado

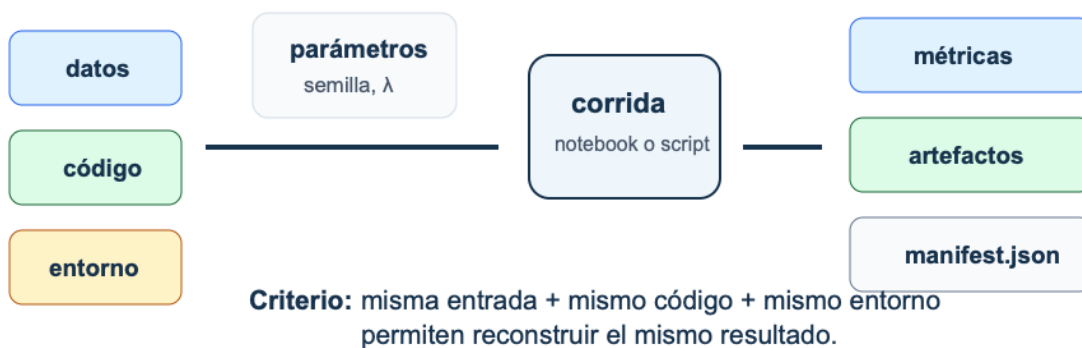


Figura 26: Reproducibilidad: datos, código, entorno y parámetros alimentan una corrida que produce métricas, artefactos y un manifiesto verificable.

Lectura de la figura. Una corrida reproducible no se identifica solo por el resultado final. Necesita registrar datos, código, entorno, parámetros, métricas y artefactos para reconstruir la corrida.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. listar los componentes mínimos de un proyecto reproducible;
2. explicar qué controla una semilla y qué no controla;
3. construir un `manifest.json` útil;
4. registrar parámetros, métricas y artefactos;
5. usar hashes para detectar cambios en archivos.

Reproducibilidad computacional y replicación estadística

Conviene separar dos preguntas:

1. **Reproducibilidad computacional:** con los mismos bytes, código y entorno, ¿se reconstruye el mismo resultado?
2. **Replicación estadística:** con otra muestra del proceso de interés, ¿la conclusión se mantiene dentro de una variación razonable?

La primera condiciona en el dataset observado. La segunda considera $D'_m \sim P^m$ y estudia la distribución de $T(D'_m)$, donde T puede ser un coeficiente, una métrica o una recomendación.

Una semilla fija selecciona una realización de generadores pseudoaleatorios. Debe registrarse, pero no reduce $\text{Var}_{D_m}(T(D_m))$ ni corrige cambio de distribución.

Un manifest completo registra la semilla y también el protocolo estadístico: unidad de muestreo, partición, repeticiones, intervalos o dispersión y distribución de uso pretendida.

El paquete mínimo reproducible

Elemento	Pregunta que responde
README.md	¿Qué hace el proyecto y cómo se corre?
requirements.txt o environment.yml	¿Qué entorno necesita?
src/ o funciones claras	¿Dónde vive la lógica reutilizable?
notebooks/	¿Dónde está la exploración documentada?
data/README.md	¿Qué datos se usaron y qué restricciones tienen?
reports/	¿Cuál es la conclusión final?
artifacts/	¿Qué modelos, métricas o figuras se generaron?
manifest.json	¿Qué entorno, parámetros, métricas y hashes identifican la corrida?

Semillas

La semilla controla generadores pseudoaleatorios:

```
RANDOM_STATE = 42
```

Pero una semilla no garantiza reproducibilidad total si cambian dependencias, hardware, versiones o datos. Algunas operaciones paralelas también pueden ser no deterministas. Por eso se registra junto con el entorno y los artefactos.

Manifest de experimento

Un manifest es un resumen estructurado:

```
{
  "schema_version": 1,
  "run_id": "logreg-c01-s2105",
  "code": {"commit": "4d92a1f"},
  "environment": {
    "python": "3.13.5",
    "packages": {"numpy": "2.3.1", "scikit-learn": "1.7.1"}
  },
  "inputs": {"datos.csv": "6f7d...a12c"},
  "params": {
    "model": "LogisticRegression",
    "C": 1.0,
    "seed": 2105,
    "split": {"train": 0.6, "dev": 0.2, "test": 0.2}
  },
  "metrics": {"dev_f1": 0.81},
  "artifacts": {"metricas.csv": "c2a8...90bf"}
}
```

Los hashes se muestran abreviados solo para lectura; el archivo real conserva los 64 caracteres hexadecimales de SHA-256. La pregunta que responde el manifest es: “¿qué corrida produjo este número?”.

Ejemplo resuelto: manifest mínimo

Un manifest suficientemente útil para un baseline podría ser:

```
{
  "schema_version": 1,
  "run_id": "baseline-logreg-s2105",
  "environment": {
    "python": "3.13.5",
    "packages": {"scikit-learn": "1.7.1"}
  },
  "params": {
    "model": "LogisticRegression",
    "C": 1.0,
    "seed": 2105,
    "split": "60/20/20 estratificado"
  },
  "metrics": {"dev_f1": 0.81},
  "artifacts": {"metricas.csv": "c2a8...90bf"}
}
```

No es necesario que sea largo. Debe distinguir la partición de cada métrica y permitir localizar sus salidas. `test_f1` se agrega cuando el protocolo queda cerrado y se ejecuta la evaluación final.

Ejemplo resuelto: resultado no reproducible

Supón que un equipo reporta `test_f1 = 0.84`, pero al ejecutar el notebook desde cero aparece `test_f1 = 0.77`. La primera hipótesis no debe ser mala fe; debe ser falta de trazabilidad. Revisa en orden:

1. si la partición usa la misma semilla;
2. si el pipeline aprende transformaciones solo en entrenamiento;
3. si el dataset tiene el mismo hash;
4. si las versiones de librerías cambiaron;
5. si el reporte copió una métrica de una corrida anterior.

Un resultado no reproducible no se corrige con memoria. Se reconstruye la secuencia de entradas y transformaciones hasta localizar por qué cambió la métrica.

Ejemplo resuelto: auditar un manifest

Supón que el manifest registra:

```
{
  "run_id": "ridge_v3",
  "environment": {"python": "3.13.5", "packages": {"numpy": "2.3.1"}},
  "params": {"model": "Ridge", "alpha": 10.0, "seed": 2105},
  "metrics": {"dev_rmse": 12.4},
```

```
"artifacts": {"predictions.csv": "abc123..."}
}
```

Dos semanas después, el archivo `predictions.csv` existe, pero su hash calculado es `sha256:def987`.

La conclusión correcta no es “el modelo está mal”. La conclusión correcta es:

1. el artefacto actual no coincide con el artefacto registrado;
2. la métrica `dev_rmse=12.4` ya no queda respaldada por ese archivo;
3. hay que volver a ejecutar la evaluación o recuperar el artefacto original;
4. cualquier reporte que use esa corrida debe marcarse como no verificado hasta resolver la discrepancia.

Un hash no evalúa calidad estadística. Evalúa identidad del archivo.

Tracking de experimentos

Herramientas como MLflow organizan experimentos en corridas con parámetros, métricas y artefactos. En este curso no se exige un servidor MLflow; el patrón conceptual sí se exige:

Concepto	En el curso
experimento	carpeta o manifest
run	una corrida con semilla y parámetros
params	hiperparámetros, semilla y protocolo de partición
metrics	resultados numéricos
artifacts	modelo, figuras, reporte, predicciones

Persistencia de modelos

Para proyectos con `scikit-learn`, una práctica común es guardar pipelines completos:

```
from joblib import dump

dump(pipeline, "artifacts/model.joblib")
```

Nunca cargues archivos `pickle` o `joblib` de fuentes no confiables. La persistencia basada en `pickle` puede ejecutar código durante la carga. Para producción o intercambio abierto, considera formatos más seguros o flujos de confianza explícita.

Hash de artefactos

Un hash SHA-256 permite detectar si un archivo cambió:

```
import hashlib

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()
```

Checklist de reproducibilidad

1. ¿Está la semilla definida en un lugar visible?
2. ¿La partición de datos es reproducible?
3. ¿El preprocesamiento está dentro de un pipeline?
4. ¿El entorno está documentado?
5. ¿Las métricas finales se pueden recalcular?
6. ¿Los artefactos tienen nombre, versión o hash?
7. ¿El reporte distingue resultados, interpretación y recomendación?

Punto de control

Si alguien abre tu proyecto dos semanas después, debería poder responder:

1. ¿qué datos se usaron?;
2. ¿con qué entorno se ejecutó?;
3. ¿qué modelo produjo la métrica reportada?;
4. ¿qué archivos sostienen los resultados y cuáles son auxiliares?;
5. ¿qué decisión se tomó a partir de la corrida?

Caso longitudinal: manifest del clasificador

Para `clasificacion_binaria_sintetica.csv`, un manifest mínimo debería registrar:

Campo	Ejemplo
dataset	<code>clasificacion_binaria_sintetica.csv</code>
hash o versión	SHA-256 del archivo publicado
target	<code>target</code>
slices auditados	<code>base, reciente</code>
partición	semilla y proporciones

Campo	Ejemplo
pipeline	escalamiento dentro del pipeline y modelo
métrica primaria	F1 o recall
artefactos	tabla de métricas, matriz de confusión, error por slice

El manifest no garantiza que el modelo sea bueno. Permite reconstruir qué corrida produjo los archivos registrados. Esa diferencia es central para una defensa técnica.

Verificaciones seleccionadas

1. Un notebook ejecutable sin semilla puede ser reproducible en estructura, pero no necesariamente en resultados numéricos.
2. Si el dataset cambia y no cambia el manifest, ya no sabes qué archivo sostiene la conclusión.
3. Una tabla de métricas sin partición asociada no permite reconstruir la evaluación.

Ejemplo resuelto: manifest mínimo en JSON

Un manifest pequeño puede verse así:

```
{
  "schema_version": 1,
  "environment": {
    "python": "3.13.5",
    "packages": {"pandas": "2.3.1", "scikit-learn": "1.7.1"}
  },
  "params": {
    "dataset": "clasificacion_binaria_sintetica.csv",
    "target": "target",
    "seed": 2105,
    "split": "60/20/20 estratificado",
    "metric_primary": "f1"
  },
  "metrics": {"dev_f1": 0.81},
  "artifacts": {
    "metrics.csv": "c2a8...90bf",
    "slice_report.csv": "151f...2d8e"
  }
}
```

Este archivo no sustituye el reporte, pero permite auditar qué corrida produjo qué resultado. Si cambian datos, métrica o partición, el manifest debe cambiar.

Qué debe poder volver a ejecutarse

Un paquete reproducible no exige que todo el curso se ejecute en segundos, pero sí que exista una ruta verificable. Como mínimo, otra persona debe poder ejecutar un script o notebook que cargue datos permitidos, reconstruya particiones, entrene o cargue el modelo declarado, calcule métricas principales y regenere la tabla final. Si el entrenamiento completo es costoso, se debe incluir una versión reducida o un modo de evaluación que cargue artefactos ya generados y verifique sus métricas.

La trazabilidad se rompe cuando una figura se pega manualmente, una métrica se copia sin archivo de origen o una tabla final no se puede regenerar. En ese caso, la discusión técnica puede ser buena, pero el resultado no es auditable. El manifest debe apuntar a los archivos que sostienen cada afirmación importante.

Para explorar más

- **Extensión matemática:** identifica qué cantidades numéricas deben quedar fijas para comparar dos corridas de entrenamiento.
- **Extensión computacional:** crea un manifest mínimo con datos, commit, semilla, entorno, métricas y hash de salida.
- **Conexión con proyecto:** define qué archivos entregados permitirían auditar tu resultado sin abrir tu notebook manualmente.

Revisión del capítulo

Reproducibilidad significa que otra persona puede reconstruir el resultado desde datos, código, entorno, parámetros y artefactos. Una semilla ayuda, pero no basta si cambian datos, librerías, hardware o pasos manuales.

El manifest registra qué corrida produjo qué métricas y qué artefactos. Los hashes detectan cambios en archivos, pero no prueban que el contenido sea correcto. El tracking de experimentos ordena comparaciones y evita depender de memoria o nombres informales.

Un paquete reproducible debe permitir auditar el resultado final. Eso implica separar archivos de entrada, scripts, outputs, métricas, modelos y reporte, y declarar qué archivo sostiene cada afirmación cuantitativa.

Términos clave

- **Reproducibilidad:** capacidad de reconstruir resultados desde datos, código, entorno y parámetros.
- **Manifest:** archivo que registra configuración, rutas, semillas, métricas y artefactos.
- **Run:** corrida individual de un experimento.
- **Artefacto:** salida guardada, como modelo, figura, reporte, predicciones o tabla.
- **Hash:** huella de un archivo usada para detectar cambios.

- **Persistencia:** guardado de un modelo o pipeline para reutilizarlo.

Ejercicios

Preguntas conceptuales

1. ¿Por qué una semilla fija no garantiza reproducibilidad completa?
2. ¿Qué diferencia hay entre código que corre y un resultado reproducible?

Problemas cuantitativos

3. Diseñe una tabla mínima con tres runs, dos hiperparámetros y dos métricas.
4. Explique qué detecta y qué no detecta un hash SHA-256.

Práctica computacional

5. Cree un `manifest.json` mínimo para un experimento.
6. Calcule el hash SHA-256 de un archivo de resultados.

Interpretación y pensamiento crítico

7. ¿Qué riesgos tiene cargar modelos `pickle` o `joblib` de fuentes no confiables?
8. Si no puede reproducir una métrica final, ¿qué partes del proyecto revisaría primero?

Proyecto de cierre

9. Prepare un paquete mínimo reproducible para su proyecto: manifest, entorno, script de evaluación, tabla de métricas y lista de artefactos.

Comunicación técnica

Pregunta aplicada

¿Cómo convertimos experimentos, métricas y errores en una recomendación técnica que una audiencia pueda evaluar y cuestionar?

Tesis de un reporte

Un reporte final no debe ser una bitácora de todo lo que se intentó. Debe defender una conclusión.



Figura 27: Comunicación técnica: un reporte organiza tesis, resultados, diagnóstico, recomendación y límites para que una audiencia pueda auditarla.

Lectura de la figura. La tesis técnica aparece primero, pero no vive sola: debe estar sostenida por resultados, una lectura y límites. Si una afirmación no puede conectarse con una tabla, figura, cálculo o archivo, debe suavizarse o eliminarse.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. escribir una tesis técnica breve;
2. seleccionar figuras que respondan preguntas específicas;
3. distinguir resultado, interpretación y recomendación;
4. evitar lenguaje desproporcionado;
5. preparar una respuesta oral precisa.

Comunicar una cantidad aleatoria

Una métrica, coeficiente o componente estimada depende de la muestra. Un reporte matemáticamente proporcionado identifica:

estimando + estimador + estimación + incertidumbre + alcance.

Ejemplo:

En el test temporal de 2025, el MAE fue 4.8 unidades. Un bootstrap por mes dio percentiles 2.5 y 97.5 de 4.1 y 5.7. El intervalo describe remuestreo de meses observados; no cubre un cambio de política posterior.

No toda cifra necesita un intervalo formal, pero toda cifra usada para recomendar debe declarar tamaño efectivo, unidad de muestreo y fuente principal de incertidumbre. **media ± desviación entre folds** no se llamará intervalo de confianza.

Tesis técnica: frase que resume la recomendación, el resultado principal y la limitación más importante.

Ejemplo:

El modelo regularizado mejora el baseline en F1 y mantiene estabilidad en validación cruzada, pero requiere revisar falsos negativos en el subgrupo de mayor riesgo antes de una recomendación operativa.

Estructura recomendada

Sección	Pregunta
Resumen ejecutivo	¿Qué se recomienda y con qué cautela?
Problema y datos	¿Qué pregunta se modeló y qué datos la soportan?
Método	¿Qué pipeline se usó y por qué?
Validación	¿Qué comparación numérica respalda la recomendación?
Error analysis	¿Dónde falla y qué patrón importa?
Reproducibilidad	¿Cómo se vuelve a ejecutar o audita?

Sección	Pregunta
Limitaciones	¿Qué no puede concluirse?
Siguiente paso	¿Qué experimento o acción sigue?

Figuras útiles

Una figura debe cumplir una función. No se incluye porque “se ve bien”.

Figura	Uso
curva de aprendizaje	diagnosticar underfitting/overfitting
matriz de confusión	identificar tipos de error
tabla de métricas con baseline	comparar modelos
slices de error	encontrar fallas por subgrupo
PCA 2D	explorar estructura, no probar causalidad
perfil de clusters	interpretar segmentos

Lenguaje de alcance

Evita:

- “el modelo es bueno”;
- “la gráfica demuestra”;
- “el accuracy es alto”;
- “se recomienda implementar”.

Prefiere:

- “mejora el baseline por...”;
- “en validación se observa...”;
- “el resultado es estable/inestable porque...”;
- “se recomienda el siguiente experimento antes de...”.

Ejemplo resuelto: de bitácora a argumento

Una bitácora dice:

Probamos regresión logística, random forest y varias variables. La mejor métrica fue la del modelo regularizado. También hicimos gráficas y revisamos errores.

Un argumento técnico dice:

La regresión logística regularizada se elige porque mejora el baseline en F1 medio y mantiene menor variabilidad que el modelo más complejo. El análisis por slice muestra caída en observaciones recientes; por eso la recomendación se limita a usarlo como baseline interno y priorizar validación temporal.

La diferencia no es estilo literario. La bitácora enumera acciones; el argumento conecta criterio, resultado, diagnóstico y límite. Un lector puede auditar el segundo texto porque sabe qué tabla buscar y qué afirmación verificar.

Ejemplo resuelto: mejorar una conclusión

Conclusión débil:

El modelo es bueno porque tiene accuracy alta.

Conclusión revisada:

El modelo mejora el baseline de F1 en validación cruzada y mantiene desempeño similar en test. Sin embargo, el recall cae en el slice de observaciones recientes; por eso se recomienda usarlo solo como baseline interno mientras se revisa data mismatch temporal.

La segunda versión nombra comparación, partición, error y límite de uso.

Ejemplo resuelto: mejora absoluta y relativa

Supón que el baseline tiene F1 de 0.60 y el modelo final tiene F1 de 0.72.

La mejora absoluta es:

$$0.72 - 0.60 = 0.12.$$

La mejora relativa respecto al baseline es:

$$\frac{0.72 - 0.60}{0.60} = 0.20 = 20\%.$$

Una frase precisa sería:

El modelo mejora el baseline en 0.12 puntos de F1; al dividir esa diferencia por el F1 del baseline se obtiene 20 %. Ambos valores provienen de la misma partición de validación y deben leerse junto con los conteos de error.

La mejora relativa puede sonar más grande que la absoluta. Además, un aumento relativo de 20 % en F1 no significa que el costo de errores se haya reducido 20 %. El reporte muestra ambas cantidades solo si ayudan a interpretar el caso.

Respuesta oral de 45 segundos

Una respuesta oral de defensa debe caber en 45 segundos:

```
Elegimos [método] porque [criterio].  
El resultado principal fue [métrica + partición].  
El mayor error aparece en [slice o caso].  
La limitación más importante es [límite].  
Por eso recomendamos [acción proporcional].
```

Caso longitudinal: conclusión del clasificador

Con `clasificacion_binaria_sintetica.csv`, una conclusión débil sería:

El modelo final tiene buen F1 y se recomienda usarlo.

Una conclusión revisada sería:

El modelo regularizado mejora el baseline en F1 medio de validación, pero el recall del slice **reciente** es menor que el del slice **base**. Por tanto, el modelo puede usarse como baseline interno para priorizar revisión, pero no como sistema automático final hasta construir una validación temporal y auditar variables que cambian entre slices.

La segunda versión tiene tesis, resultado, diagnóstico, recomendación y límite. También deja claro qué experimento falta.

Verificaciones seleccionadas

1. Una mejora relativa de 20 % puede sonar grande; siempre debe leerse junto con la mejora absoluta y el baseline.
2. Una figura funcional debe responder una pregunta: comparación, error, curva, slice o trazabilidad.
3. Si una conclusión no declara límite, probablemente exagera el alcance del resultado.

Ejemplo resuelto: tabla de soporte de afirmaciones

Antes de entregar un reporte, cruza cada afirmación fuerte contra un soporte identificable:

Afirmación	Soporte necesario	Estado
supera el baseline	tabla baseline vs modelo	verificable
mantiene desempeño en la muestra de test	métrica, tamaño y protocolo de test	incompleto si no hay test
funciona para casos recientes	métrica por <code>slice</code>	falso si el slice cae

Afirmación	Soporte necesario	Estado
la corrida es reproducible	manifest, entorno y hashes	verificable

Una afirmación sin soporte debe cambiar de tono. Por ejemplo, “generaliza bien” puede convertirse en “la media de F1 fue estable entre los cinco folds; falta la evaluación final en test”. Incluso un test final estima desempeño sobre una muestra concreta, no garantiza todos los datos futuros.

Plantilla de resultados

Una sección de resultados puede tener esta forma:

```
Baseline. [regla simple] obtiene [métrica] en [partición].
Modelo. [modelo elegido] obtiene [métrica media ± desviación] bajo [validación].
Error principal. El peor patrón aparece en [slice/tipo de error].
Recomendación. Con estos resultados se propone [acción proporcional].
Límite. No se puede afirmar [conclusión excesiva] porque [razón].
```

Esta plantilla evita dos extremos: una bitácora larga sin tesis y una conclusión fuerte sin soporte. Si una línea no puede llenarse con una tabla, figura, cálculo o archivo, el reporte todavía no está listo.

De resultado a recomendación

La recomendación final debe tener un verbo de acción. “El modelo alcanza 0.84 de F1” es un resultado; “usar el modelo como filtro preliminar con revisión humana en casos de baja confianza” es una recomendación. Entre ambos debe aparecer el razonamiento: baseline superado, error principal, costo de fallo y límite de uso. En ciencia de datos aplicada, una métrica sin acción suele quedarse corta.

“Baja confianza” solo tiene sentido si el puntaje tiene una interpretación adecuada y se revisó calibración. Una probabilidad logística o softmax no queda calibrada por el solo hecho de estar entre cero y uno.

También importa el tono. Si el resultado viene de validación interna, la recomendación debe decirlo. Si el test final es pequeño, debe aparecer el tamaño. Si un slice falla, no se debe esconder detrás del promedio global. La escritura calidad de la escritura no consiste en sonar segura; consiste en precisar qué permiten sostener los datos disponibles.

Una buena prueba de lectura es subrayar cada afirmación del reporte y preguntar: ¿qué tabla, figura, cálculo o archivo la sostiene? Si no hay respuesta, la frase debe debilitarse, eliminarse o convertirse en trabajo futuro. Esta disciplina hace que el texto sea más corto, pero más fuerte y auditable. Además, ayuda a distinguir entre una conclusión lista para entrega y una idea que aún necesita validación adicional.

Para explorar más

- **Extensión matemática:** convierte una mejora de métrica en mejora absoluta, relativa y límite de interpretación.
- **Extensión computacional:** genera una tabla final reproducible que combine baseline, modelo elegido, métrica principal, variabilidad y nota de validación.
- **Conexión con proyecto:** prepara una conclusión de 45 segundos que incluya recomendación, resultado principal, error, limitación y siguiente paso.

Revisión del capítulo

La comunicación técnica transforma resultados en una conclusión verificable. Una tesis de reporte debe decir qué problema se abordó, qué se obtuvo, qué se recomienda y bajo qué límites.

Las figuras funcionales responden preguntas concretas: desempeño contra baseline, errores por slice, curva de validación o estructura de datos. Una figura decorativa puede distraer; una figura funcional reduce ambigüedad.

El lenguaje debe ser proporcional. En lugar de afirmar que un modelo “es bueno”, el reporte debe explicar contra qué baseline mejora, con qué métrica, en qué datos, dónde falla y qué acción se recomienda.

Términos clave

- **Tesis del reporte:** afirmación central vinculada a resultados identificables.
- **Resumen ejecutivo:** síntesis de recomendación, resultado y límite para lector no técnico.
- **Figura funcional:** visualización incluida porque responde una pregunta técnica.
- **Limitación:** frontera explícita de lo que no se puede concluir.
- **Acción proporcional:** recomendación ajustada al alcance de los resultados.

Ejercicios

Preguntas conceptuales

1. ¿Por qué “el modelo es bueno” no es una conclusión técnica suficiente?
2. ¿Qué diferencia hay entre una figura decorativa y una figura funcional?

Problemas cuantitativos

3. Reduzca una tabla con cinco métricas a las dos más relevantes para una decisión de alto costo.
4. Compare un modelo contra baseline y escriba la mejora absoluta y relativa de la métrica principal.

Práctica computacional

5. Cree una tabla final de resultados con baseline, modelo elegido, métrica, media y desviación entre folds, y nota de validación. No llame intervalo a **media $\pm$ desviación**.
6. Genere una figura de error analysis con título, eje claro y caption interpretativo.

Interpretación y pensamiento crítico

7. Reescriba una conclusión exagerada para que incluya resultado, límite y siguiente paso.

Proyecto de cierre

8. Escriba el resumen ejecutivo de su proyecto en 180 palabras: problema, método, resultado principal, error, limitación y recomendación.

Entrega final y defensa

Pregunta aplicada

¿Qué debe entregar un equipo para que su proyecto pueda leerse, ejecutarse, auditarse y defenderse sin depender de una explicación informal?

Paquete final

El paquete final debe permitir tres acciones:

1. leer la conclusión;
2. ejecutar o auditar el flujo principal;
3. evaluar la contribución individual.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. organizar un repositorio de proyecto con resultados auditables;
2. preparar una entrega que se pueda leer y ejecutar;
3. explicar tu contribución individual;
4. responder preguntas sobre métrica, validación, leakage y limitaciones;
5. distinguir presentación grupal de defensa individual.

Defensa del fundamento probabilístico

La defensa individual debe poder responder, con notación y datos del proyecto:

1. ¿cuál es la población o distribución de uso P_{uso} ?
2. ¿qué mecanismo produjo la muestra y qué dependencias existen?
3. ¿qué objeto poblacional aproxima el modelo?
4. ¿qué riesgo o métrica empírica se calculó y sobre qué partición?
5. ¿qué variación tiene la cifra principal y cómo se estimó?
6. ¿qué cambio de distribución invalidaría la recomendación?

Una respuesta que solo describe una función de biblioteca no demuestra comprensión matemática del proyecto.

Estructura del repositorio de proyecto

```
proyecto-final/  
  README.md  
  requirements.txt  
  data/  
    README.md  
  notebooks/  
    01_exploracion.ipynb  
    02_modelado.ipynb  
    03_reporte_final.ipynb  
  src/  
    features.py  
    evaluation.py  
  reports/  
    reporte_final.pdf  
  artifacts/  
    metrics.json  
    manifest.json
```

No todos los proyectos tendrán exactamente la misma estructura, pero sí deben separar datos, código, resultados y reporte.

Criterio de paquete ejecutable

Un proyecto no está listo porque “corre en mi computador”. Está listo cuando otra persona puede reconstruir el resultado principal con instrucciones mínimas. El README debe responder:

Pregunta	Archivo o respuesta esperada
¿qué problema estudia?	pregunta aplicada y alcance
¿qué datos usa?	fuentes, licencia, columnas y restricciones
¿cómo se ejecuta?	comandos o notebooks en orden
¿qué resultado debe aparecer?	métrica, tabla o figura esperada
¿qué no se publica?	datos privados, credenciales o archivos sensibles

Si el notebook final depende de rutas locales, celdas ejecutadas fuera de orden o archivos no incluidos, la defensa se vuelve una explicación informal. La entrega debe eliminar esa fragilidad antes de la entrega.

Ejemplo resuelto: contribución individual

Respuesta vaga:

Ayudé con el modelo.

Respuesta verificable:

Implementé la partición train/dev/test, construí el baseline logístico y comparé F1/recall contra el modelo regularizado. También preparé la tabla de errores por slice de edad y documenté el riesgo de falsos negativos.

La segunda respuesta permite verificar autoría y comprensión.

Peso del proyecto en el curso

El proyecto completo representa 40 % de la nota:

Componente	Peso del curso	Momento
Hitos y checkpoints	15 %	durante el semestre
Reporte y repositorio final	15 %	entrega de Semana 16
Defensa individual	10 %	jornadas de defensa

Los hitos usan formularios y plantillas breves. La tabla siguiente califica el reporte y el repositorio sobre 100 puntos y luego se multiplica por 0.15.

Rúbrica del reporte y repositorio

Criterio	Peso sugerido
Pregunta, datos y alcance	15 %
Fundamento matemático y algoritmo	20 %
Validación, métricas y error analysis	25 %
Reproducibilidad y organización	15 %
Comunicación visual y escrita	15 %
Limitaciones, ética y decisión	10 %

Ejemplo resuelto: cálculo de nota ponderada

Supón que un proyecto recibe:

Criterio	Puntaje	Peso
Pregunta, datos y alcance	12/15	15 %
Fundamento matemático y algoritmo	16/20	20 %
Validación, métricas y error analysis	20/25	25 %
Reproducibilidad y organización	13/15	15 %

Criterio	Puntaje	Peso
Comunicación visual y escrita	12/15	15 %
Limitaciones, ética y decisión	8/10	10 %

Como los pesos suman 100 puntos y los denominadores coinciden con esos pesos, la nota total es:

$$12 + 16 + 20 + 13 + 12 + 8 = 81.$$

El porcentaje del componente es $81/100 = 81\%$, que aporta

$$0.81(15) = 12.15$$

puntos a la nota del curso. La tabla muestra en qué criterio se perdieron los 2.85 puntos restantes del componente.

Defensa oral individual

La defensa no repite la presentación. Verifica autoría, comprensión y criterio.



Figura 28: Defensa oral: la respuesta individual conecta pregunta, respuesta breve, archivo verificable y límite específico con una rúbrica 0/1/2.

Lectura de la figura. Una defensa fuerte no empieza con “yo hice varias cosas”; empieza con una respuesta breve, la conecta con un archivo o resultado y declara un límite. Esa estructura permite evaluar rápido sin convertir la defensa en una conversación improvisada.

Preguntas típicas:

1. ¿Cuál fue tu contribución técnica concreta?
2. ¿Por qué esa métrica es la correcta?
3. ¿Dónde falla el modelo?
4. ¿Qué parte del pipeline podría producir leakage?
5. ¿Qué cambiaría si tuvieras dos semanas más?
6. ¿Qué conclusión no puedes afirmar con estos datos?

Rúbrica de defensa 0/1/2

Criterio	0	1	2
Autoría	no identifica contribución	contribución vaga	contribución concreta y verificable
Matemática	errores conceptuales	idea parcial	explicación correcta y conectada
Validación	no justifica métrica	justifica parcialmente	conecta métrica, baseline y error
Reproducibilidad	no localiza la corrida	menciona archivos aislados	explica entorno, semilla, manifest y artefacto
Limitaciones	no reconoce límites	límite genérico	límite específico y relevante

Cada criterio aporta 0, 1 o 2 puntos. Los cinco criterios suman directamente los 10 puntos porcentuales de la defensa en la nota del curso.

Caso longitudinal: defensa del clasificador

Para defender el caso `clasificacion_binaria_sintetica.csv`, prepara una tabla de archivos antes de hablar:

Pregunta posible	Archivo o resultado que debes abrir
¿por qué esa métrica?	costo de falsos positivos/falsos negativos y baseline
¿dónde falla?	matriz de confusión y tabla por <code>slice</code>
¿hay leakage?	pipeline y partición
¿qué hiciste tú?	commit, notebook, función o tabla verificable
¿qué no puedes afirmar?	límite sobre el <code>slice reciente</code>

Una defensa de 45 segundos:

Mi contribución fue construir el pipeline de evaluación y la tabla por slice. El modelo regularizado mejora el baseline en F1, pero el recall cae en el slice reciente. Por eso no recomiendo despliegue; recomiendo usarlo como baseline y crear una validación temporal antes de tomar decisiones.

Verificaciones seleccionadas

1. “Ayudé con el modelo” no prueba autoría; “implementé la partición y la tabla por slice en el archivo X” sí es verificable.
2. Una defensa fuerte puede reconocer que el resultado no basta para despliegue.
3. La rúbrica 0/1/2 funciona cuando la respuesta tiene un soporte observable y no depende de impresiones.

Ejemplo resuelto: pregunta difícil

Pregunta:

Si tu modelo tiene mejor F1, ¿por qué no recomiendas desplugarlo?

Respuesta precisa:

Porque la mejora global no se mantiene en el slice reciente. La tabla por slice muestra una caída de recall, y ese slice se parece más al uso esperado. Recomendando mantener el modelo como baseline interno y construir una validación temporal antes de despliegue.

La respuesta obtiene puntaje alto porque no niega la mejora; la ubica dentro de su límite y propone una acción técnica.

Checklist individual antes de entrar a defensa

Cada estudiante debe poder abrir un soporte concreto para estas preguntas:

Pregunta	Soporte mínimo
¿qué hiciste tú?	archivo, función, notebook, tabla o commit
¿qué fórmula o métrica usaste?	definición y razón aplicada
¿qué baseline superaste?	tabla comparativa
¿dónde falla el modelo?	matriz de confusión, residuos o slice
¿cómo evitas leakage?	partición y pipeline
¿qué no puedes concluir?	limitación específica

Si una respuesta depende de “eso lo hizo mi compañero”, la defensa individual no está lista. La colaboración es válida, pero cada persona debe defender una parte técnica concreta del proyecto.

Carga docente mínima

Para grupos grandes:

- revisar el reporte con rúbrica estructurada;
- usar checklist de reproducibilidad antes de leer detalles;
- evaluar defensa individual con 5 criterios 0/1/2;
- pedir soporte de contribución en una tabla;
- muestrear notebooks solo cuando haya inconsistencia.

Contribución individual y trazabilidad

La defensa individual debe apoyarse en archivos concretos. Una respuesta fuerte no dice solamente “yo hice el modelo”; señala el archivo, la función, el commit, la tabla o la figura que prueba la contribución. Esta trazabilidad evita dos problemas frecuentes: estudiantes que entienden el proyecto pero no pueden mostrar su aporte, y estudiantes que presentan artefactos sin poder defender las decisiones técnicas.

Una forma simple de prepararse es construir una tabla con tres columnas: pregunta posible, archivo que abriría y respuesta de 30 segundos. Por ejemplo, si la pregunta es “¿cómo evitaron leakage?”, el soporte puede ser el pipeline de preprocesamiento y la partición; la respuesta debe explicar qué pasos se ajustaron solo con entrenamiento y cuáles se aplicaron después a validación o test. Esta preparación hace la defensa más justa y reduce tiempo de evaluación.

Para explorar más

- **Extensión matemática:** anticipa una pregunta sobre pérdida, métrica o validación y prepara una respuesta con una fórmula o cálculo simple.
- **Extensión computacional:** ejecuta el proyecto en un entorno limpio y registra cualquier dependencia faltante antes de la defensa.
- **Conexión con proyecto:** arma una tabla de preguntas difíciles con archivo fuente y respuesta breve.

Revisión del capítulo

La entrega final debe ser auditable. El repositorio, el reporte, los artefactos y la defensa deben permitir reconstruir qué se hizo, quién lo hizo, qué resultado se obtuvo y qué conclusión se puede sostener.

La defensa individual no repite la presentación grupal. Su función es verificar criterio técnico: contribución concreta, elección de métrica, prevención de leakage, lectura de errores, limitaciones y siguiente experimento razonable.

La carga docente se reduce cuando los soportes están estructurados. Una rúbrica 0/1/2 funciona si cada criterio se puede observar rápidamente en archivos, tablas, respuestas orales o artefactos reproducibles.

Términos clave

- **Paquete final:** conjunto de reporte, código, datos permitidos y artefactos.
- **Defensa oral:** instancia para verificar autoría, comprensión y criterio.
- **Contribución individual:** trabajo técnico concreto que puede ser explicado y verificado.
- **Rúbrica:** criterio explícito para evaluar con consistencia.
- **Carga docente mínima:** diseño de evaluación que concentra revisión manual donde aporta juicio.

Ejercicios

Preguntas conceptuales

1. ¿Por qué una defensa oral no debe repetir simplemente la presentación?
2. ¿Qué archivo o resultado distingue una contribución concreta de una afirmación vaga?

Problemas cuantitativos

3. Con la rúbrica propuesta, calcule la nota de un proyecto que obtiene 12/15, 16/20, 20/25, 13/15, 12/15 y 8/10.
4. Si una defensa usa cinco criterios 0/1/2, diseñe una conversión transparente a porcentaje.

Práctica computacional

5. Cree una estructura de carpetas final para su proyecto y liste qué archivo prueba cada resultado importante.
6. Prepare una tabla de contribuciones individuales con tarea, soporte y responsable.

Interpretación y pensamiento crítico

7. Escriba una respuesta precisa a: “¿Qué cambiaría si tuviera dos semanas más?”

Proyecto de cierre

8. Arme un checklist final de entrega con 12 ítems verificables antes de subir el proyecto.

Ejercicios integradores: reproducibilidad y defensa

Esta sección sirve como ensayo final del curso. Cada ejercicio conecta matemática, implementación y comunicación. La respuesta no termina en el cálculo: debe decir qué permite concluir y qué falta comprobar.

Resultados de práctica

Al terminar este banco deberías poder:

1. estabilizar cálculos numéricos básicos;
2. transformar métricas en diagnóstico y acción;
3. usar PCA y reconstrucción como herramientas verificables;
4. diseñar un manifest reproducible;
5. defender límites antes de recomendar uso real.

Mapa del banco

Ejercicio	Qué comprueba
1	estabilidad numérica en softmax
2	diagnóstico con baseline, CV, test y slice crítico
3	PCA, scores y reconstrucción
4	manifest y trazabilidad de experimento
5	defensa técnica con límites explícitos

Criterio de calidad

Una respuesta final de curso debe:

1. usar una fórmula o procedimiento correcto;
2. señalar el resultado observado;
3. traducirlo en una lectura técnica;
4. proponer una acción proporcional;
5. declarar un límite.

Columna probabilística del cierre

P1. Cuatro niveles. Para una red de clasificación, identifique distribución poblacional, muestra, riesgo empírico y algoritmo. Repita para PCA y K-Means.

P2. Dos reproducibilidades. Dé un ejemplo donde una corrida sea perfectamente reproducible, pero la conclusión cambie mucho con otra muestra.

P3. Semilla. Explique qué variables aleatorias controla `random_state=2105` en su pipeline y cuáles fuentes de incertidumbre permanecen.

P4. Manifest estadístico. Diseñe campos JSON para registrar población de uso, unidad independiente, partición, repeticiones y resumen de incertidumbre.

P5. Comunicación. Reescriba “el modelo tiene F1 de 0.87” como una afirmación que identifique partición, tamaño efectivo, procedimiento fijado, incertidumbre y límite de transporte.

P6. Defensa. Prepare una respuesta de 90 segundos que conecte un resultado de su proyecto con P_{uso} , $\hat{R}$, fuente de variación y siguiente comprobación.

Ejercicio 1: Softmax estable

Sea

$$Z = \begin{bmatrix} 1000 & 1001 & 1002 \end{bmatrix}.$$

1. Explica por qué una implementación directa puede fallar.
2. Calcula softmax después de restar el máximo.
3. Verifica que las probabilidades suman 1.

Ejercicio 2: Diagnóstico

Un modelo tiene:

- baseline F1 = 0.62;
- modelo F1 CV = 0.78, desviación entre folds 0.09;
- test F1 = 0.70;
- slice crítico de test F1 = 0.48 en 40 casos.

Escribe una recomendación en cuatro partes:

```
resultado -> lectura -> comprobación -> límite
```

Ejercicio 3: PCA y reconstrucción

Tienes una matriz centrada X_c y su SVD $X_c = U\Sigma V^\top$.

1. Escribe la fórmula de los scores con k componentes.
2. Escribe la reconstrucción aproximada.
3. Explica por qué el error de reconstrucción no aumenta al aumentar k .

Ejercicio 4: Reproducibilidad

Diseña un `manifest.json` mínimo para un experimento de clasificación. Debe incluir:

- semilla;
- entorno y versiones;
- protocolo de partición;
- modelo;
- hiperparámetros;
- métrica principal con nombre de partición;
- hash de artefactos;
- commit o versión del código.

Ejercicio 5: Defensa

Responde en 120 palabras:

¿Qué resultados y comprobaciones necesitarías antes de recomendar el uso de un modelo?

Respuestas seleccionadas

Ejercicio 1. La implementación directa puede fallar porque e^{1002} excede el rango numérico usual. Al restar el máximo, se calcula softmax sobre $(-2, -1, 0)$. Las probabilidades son proporcionales a $(e^{-2}, e^{-1}, 1)$ y se normalizan dividiendo por $e^{-2} + e^{-1} + 1$. La suma final es 1 por construcción.

Ejercicio 2. Una respuesta posible sería: “Resultado: el modelo supera el baseline, pero varía entre folds, baja en test y obtiene F1 0.48 en 40 casos del slice crítico. Lectura: el resultado es inestable y puede haber una falla en ese subgrupo. Comprobación: revisar sus conteos de error y repetir la medición en otro periodo; no volver a escoger el modelo con test. Límite: no recomendar uso si el slice representa casos de alto costo.”

Ejercicio 3. Los scores con k componentes son $Z = X_c V_k$. La reconstrucción aproximada es $\hat{X}_k = Z V_k^\top$. Al aumentar k , el subespacio disponible contiene al anterior, así que la mejor reconstrucción no puede empeorar.

Ejercicio 4. El manifest debe registrar `environment`, `params`, `metrics` y `artifacts`. En `params` se incluyen semilla y partición; cada métrica nombra su conjunto, y cada artefacto conserva el SHA-256 completo. Un hash identifica los bytes del archivo, no certifica que el análisis sea correcto.

Ejercicio 5. La revisión mínima incluye baseline, partición independiente, métrica alineada con el costo, análisis por slices con tamaños y conteos, matriz de errores, revisión de leakage, estabilidad ante cambios de datos y límites. Una métrica global aislada no justifica una recomendación de uso.

Apéndices de consulta

Prerrequisitos matemáticos mínimos

Este apéndice resume el lenguaje matemático que el libro usa de forma recurrente. Presupone los cursos indicados de álgebra lineal y cálculo, pero no un curso formal previo de probabilidad. Su propósito es ofrecer una referencia rápida para leer capítulos, notebooks y ejercicios sin perder la forma de los objetos; la teoría probabilística se desarrolla dentro del texto y se reúne en los capítulos de herramientas.

Cómo usar este apéndice

Úsalo cuando aparezca una duda de notación antes de resolver un ejercicio:

1. identifica el objeto: escalar, vector, matriz, función o variable aleatoria;
2. revisa la forma esperada;
3. conecta la fórmula con la operación de NumPy o PyTorch;
4. vuelve al capítulo y verifica si la interpretación tiene sentido.

La pregunta central no es “¿recuerdo la fórmula?”, sino “¿sé qué objeto calcula, con qué forma, sobre qué datos y para qué decisión?”.

Álgebra lineal mínima

Objeto	Notación	Forma típica	Lectura
vector de variables	x	p o $p \times 1$	una observación con p entradas
matriz de diseño	X	$n \times p$	n observaciones y p variables
parámetros	θ o w	p o $p \times 1$	pesos que convierten entradas en predicción
predicción	$\hat{y} = X\theta$	n	una salida por observación
residual	$r = y - \hat{y}$	n	error observado
matriz de pesos	$W^{[\ell]}$	$d_{\ell-1} \times d_{\ell}$	conecta capas de una red

Producto matricial:

$$(X\theta)_i = \sum_{j=1}^p X_{ij}\theta_j.$$

En NumPy:

```
y_hat = X @ theta
```

Chequeo básico:

Operación	Requisito
<code>X @ theta</code>	columnas de <code>X</code> = largo de <code>theta</code>
<code>X.T @ residual</code>	largo de <code>residual</code> = filas de <code>X</code>
<code>A @ W + b</code>	columnas de <code>A</code> = filas de <code>W</code> ; <code>b</code> broadcast por columnas

Normas, distancias y pérdida cuadrática

La norma euclidiana de un vector es:

$$\|r\|_2 = \sqrt{\sum_i r_i^2}.$$

El MSE usa errores cuadrados:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

En regresión, minimizar MSE equivale a buscar predicciones cercanas a y en promedio cuadrático. En optimización usamos a menudo:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

porque el factor $1/2$ simplifica el gradiente.

Cálculo y gradientes mínimos

Una derivada mide sensibilidad local. Un gradiente reúne derivadas parciales:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}.$$

Regla práctica del curso:

Si quieres	Usas
reducir una pérdida	moverte en dirección $-\nabla J$
detectar convergencia	norma del gradiente, cambio de pérdida o cambio de parámetros
entrenar una red	regla de la cadena organizada como backpropagation

Para MSE:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2, \quad \nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Notación probabilística de primera aparición

Antes de observar un caso, Y representa una cantidad incierta. Después de observarlo, y_i es una realización.

Idea	Lectura operativa
distribución empírica	lo que se observa en el dataset disponible
valor esperado	promedio ideal bajo un proceso de generación
ruido	variación que el modelo no explica con las variables disponibles
probabilidad condicional	$P(Y = 1 \mid X = x)$: probabilidad dada una entrada
pérdida empírica	error promedio observado en un conjunto finito
riesgo esperado	error promedio ideal en casos futuros

Estas nociones aparecen primero dentro de modelos concretos. Los capítulos de herramientas probabilísticas precisan después sus definiciones, propiedades y condiciones de uso.

Notación NumPy y PyTorch

Matemática	NumPy	PyTorch	Cuidado
$X\theta$	<code>X @ theta</code>	<code>X @ theta</code> o <code>model(X)</code>	revisar formas
media	<code>np.mean(x)</code>	<code>torch.mean(x)</code>	eje correcto
exponencial	<code>np.exp(z)</code>	<code>torch.exp(z)</code>	overflow
logaritmo	<code>np.log(p)</code>	<code>torch.log(p)</code>	evitar <code>log(0)</code>
gradiente	cálculo manual	<code>loss.backward()</code>	limpiar gradientes
actualización	<code>theta -= alpha * grad</code>	<code>optimizer.step()</code>	no olvidar <code>zero_grad()</code>

PyTorch acumula gradientes por defecto. Por eso el loop mínimo contiene:

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Checklist de formas

Antes de ejecutar un notebook, verifica:

- ¿cada fila representa una observación?
- ¿la variable objetivo tiene una entrada por observación?
- ¿la matriz de diseño incluye o no intercepto?
- ¿los pesos tienen una dimensión compatible con las entradas?
- ¿la pérdida produce un escalar?
- ¿la métrica se calcula sobre la partición correcta?
- ¿las transformaciones se ajustan solo con entrenamiento?

Ejemplo resuelto: gradiente de MSE en una línea

Sea:

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \theta = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

El residual es:

$$X\theta - y = \begin{bmatrix} -1 \\ -3 \end{bmatrix}.$$

Como $n = 2$,

$$\nabla J(\theta) = \frac{1}{2} X^\top (X\theta - y) = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -3 \end{bmatrix} = \begin{bmatrix} -2 \\ -1.5 \end{bmatrix}.$$

La actualización con $\alpha = 0.1$ sería:

$$\theta_{\text{nuevo}} = \theta - \alpha \nabla J(\theta) = \begin{bmatrix} 0.2 \\ 0.15 \end{bmatrix}.$$

Respuestas rápidas de autoverificación

1. Si X tiene forma 100×8 y θ tiene largo 8, entonces $X\theta$ tiene largo 100.
2. Si una red produce logits de forma 64×10 , softmax se aplica por fila para obtener una distribución por observación.
3. Si `loss.backward()` se llama dos veces sin `zero_grad()`, los gradientes se acumulan.
4. Si `test` se usa para escoger hiperparámetros, `test` deja de ser estimación final.

Glosario y notación

Este apéndice reúne términos y símbolos que se usan repetidamente en el curso. No reemplaza las definiciones de cada capítulo; sirve como referencia rápida.

Términos clave

Término	Significado operativo en el curso
Unidad de observación	Entidad, evento o intervalo representado por un registro.
Unidad de análisis	Entidad sobre la que se formula una conclusión; puede diferir de la unidad de observación.
Variable	Característica con un dominio de valores y una regla de medición o construcción.
Población objetivo	Conjunto de unidades, lugares y periodos sobre el que se desea concluir.
Marco de observación	Conjunto de unidades que el procedimiento de recolección podía identificar o incluir.
Variable aleatoria	Función medible que asigna un valor a cada resultado posible de un experimento aleatorio.
Realización	Valor concreto observado de una variable aleatoria.
Muestra aleatoria	Colección de variables aleatorias antes de observar sus realizaciones.
Distribución empírica	Distribución que asigna el mismo peso a cada observación de una muestra.
Valor esperado	Integral o promedio de una variable respecto a su distribución.
Estimando	Cantidad poblacional que se desea conocer.
Estimador	Función de una muestra aleatoria diseñada para aproximar un estimando.
Estimación	Valor obtenido al aplicar un estimador a la muestra observada.
Distribución muestral	Distribución de un estadístico bajo repetición del mecanismo de muestreo.
Error estándar	Desviación estándar estimada de la distribución muestral de un estimador.
Ruido	Componente aleatoria no explicada dentro de un modelo probabilístico declarado.

Término	Significado operativo en el curso
Matriz de diseño	Matriz X que contiene las variables usadas por el modelo.
Parámetro	Cantidad que identifica un elemento de una familia de modelos o distribuciones.
Predicción	Salida producida por el modelo para una observación o lote de observaciones.
Residual	Diferencia entre valor observado y predicción en regresión.
Pérdida	Costo numérico asignado a una acción o predicción y su resultado.
Pérdida empírica	Promedio de pérdidas calculado sobre datos observados.
Riesgo esperado	Esperanza de una pérdida bajo una distribución especificada.
Métrica	Cantidad usada para evaluar desempeño y tomar decisiones.
Baseline	Modelo o regla simple que sirve como punto mínimo de comparación.
Gradiente	Vector de derivadas parciales que indica dirección local de mayor aumento.
Tasa de aprendizaje	Escala del paso en descenso del gradiente.
Regularización	Penalización o restricción que busca mejorar generalización.
Validación	Evaluación en datos no usados para ajustar parámetros.
Leakage	Uso accidental de información que no estaría disponible al predecir en producción.
Regresión logística	Modelo probabilístico para clasificación binaria.
Probabilidad condicional	Probabilidad calculada dado que se conoce cierta información, como $P(Y = 1 \mid X = x)$.
Umbral	Regla que convierte probabilidad o score en decisión discreta.
Matriz de confusión	Tabla de verdaderos/falsos positivos y negativos.
Precision	Proporción de predicciones positivas que fueron correctas.
Recall	Proporción de positivos reales detectados por el modelo.
F1	Media armónica entre precision y recall.
Forward propagation	Cálculo de predicciones capa por capa en una red neuronal.
Backpropagation	Cálculo eficiente de gradientes en una red usando regla de la cadena.
Cross-validation	Evaluación repetida con particiones de entrenamiento y validación.
Bootstrap	Aproximación de una distribución muestral mediante remuestreo de los datos observados.

Término	Significado operativo en el curso
Cambio de distribución	Diferencia relevante entre la distribución que produjo los datos y la distribución de uso.
PCA	Proyección lineal que conserva la mayor varianza posible en pocas dimensiones.
SVD	Factorización matricial que sustenta PCA y otras aproximaciones de bajo rango.
K-Means	Algoritmo de clustering basado en centroides.
Reproducibilidad	Capacidad de reconstruir resultados a partir de datos, código, entorno y decisiones documentadas.

Notación frecuente

Símbolo	Uso
m, n	Números de observaciones; el símbolo se declara en cada contexto.
p	Número de columnas usadas por el modelo después de codificación.
d	Número de variables originales o dimensión de entrada, según contexto.
X	Vector aleatorio de entradas o matriz de diseño, según se declare en el contexto.
x_i	Vector de variables de la observación i .
y	Vector de valores observados.
y_i	Valor observado de la observación i .
Y	Variable aleatoria objetivo antes de observar un caso.
$D_m = (Z_1, \dots, Z_m)$	Muestra aleatoria de tamaño m .
$d_m = (z_1, \dots, z_m)$	Realización observada de una muestra.
P	Distribución poblacional o de uso especificada.
$\hat{P}_m$	Distribución empírica de una muestra observada.
$\hat{y}$	Vector de predicciones.
$\mathbb{E}[Y]$	Valor esperado de Y .
$P(Y = 1 \mid X = x)$	Probabilidad condicional de clase positiva dado x .
ε	Ruido o componente no explicado en un modelo como $Y = f(X) + \varepsilon$.
θ	Vector de parámetros en modelos lineales.
w, b	Pesos y sesgo de una neurona o capa.
$J(\theta)$	Función de pérdida como función de parámetros.
∇J	Gradiente de la función de pérdida.
λ	Intensidad de regularización.

Símbolo	Uso
α	Tasa de aprendizaje o hiperparámetro, según contexto.
Z	Pre-activación o representación proyectada.
A	Activación de una capa.
$U, \Sigma, V^\top$	Factores de la descomposición SVD.

Convención de formas

Cuando se use código Python, el libro favorece esta convención:

Objeto	Forma típica
X	<code>(n, p)</code>
y	<code>(n,)</code> o <code>(n, 1)</code>
θ	<code>(p,)</code> o <code>(p, 1)</code>
$X @ \theta$	<code>(n,)</code> o <code>(n, 1)</code>
W en una capa densa	<code>(d_in, d_out)</code>
b en una capa densa	<code>(d_out,)</code>

Las formas pueden cambiar por biblioteca o por convención local. Lo importante es declarar la convención antes de derivar o implementar.

Índice temático

Este índice ayuda a encontrar conceptos por problema de estudio, no por el orden lineal de los capítulos. Puede utilizarse para recuperar una definición, ubicar una derivación o seguir una conexión entre modelos.

Cómo usar este índice

- Si estás empezando un tema, lee primero el capítulo indicado como entrada.
- Al resolver un problema, revisa también el apéndice o banco integrador asociado.
- La última columna indica una conexión que conviene reconstruir junto con la entrada principal.

Modelación y representación

Tema	Entrada recomendada	Conexión principal
Observaciones, variables y matriz de datos	Observaciones, variables y preguntas	definir qué representa una fila y qué conclusión se quiere formular
Variable aleatoria y realización	Capítulo 1; referencia formal	distinguir proceso aleatorio, objetivo futuro y valores observados
Distribución y distribución empírica	Capítulo 1; referencia formal	describir variación poblacional y muestral sin confundirlas
Condicionamiento, ruido y residual	Capítulo 1; referencia formal	formular el objetivo condicional y distinguir fuentes de error
Pérdida empírica y riesgo esperado	Capítulo 1; referencia formal	explicar por qué la evaluación muestral tiene alcance limitado
Matriz de diseño e intercepto	Capítulo 1	documentar columnas, transformaciones y forma final de $\mathbf{X}$
Parámetros, predicciones y residuales	Capítulo 1	interpretar qué aprende el modelo y qué errores quedan
Features e interacciones	Capítulo 3	justificar transformaciones sin introducir leakage
Baselines	Capítulo 1 y Capítulo 11	comparar contra una regla simple en la misma muestra de evaluación

Tema	Entrada recomendada	Conexión principal
Casos recurrentes	Cómo leer este libro	seguir los casos sintéticos y el caso de Bogotá a través de formulación, cálculo y diagnóstico

Álgebra lineal y optimización

Tema	Entrada recomendada	Conexión principal
Prerrequisitos matemáticos mínimos	Apéndice de prerrequisitos	repasar formas, gradientes, probabilidad mínima y notación antes de resolver ejercicios
Producto matricial y formas	Capítulo 1	detectar errores antes de entrenar
Norma cuadrática y MSE	Capítulo 1	explicar qué error numérico se minimiza
Ecuación normal	Capítulo 1	comparar solución cerrada contra implementación de biblioteca
Gradiente de MSE	Capítulo 2	justificar la actualización iterativa
Escalamiento y condicionamiento	Capítulo 2	decidir cuándo una variable necesita transformación
SVD y bajo rango	Capítulo 15	explicar qué estructura se conserva al reducir dimensión

Regresión, regularización y clasificación

Tema	Entrada recomendada	Conexión principal
Regresión lineal múltiple	Capítulo 1	construir un baseline de regresión verificable
Descenso del gradiente	Capítulo 2	diagnosticar convergencia y tasa de aprendizaje
Overfitting y complejidad	Capítulo 3	elegir complejidad usando validación, no test
Ridge y Lasso	Capítulo 3	controlar varianza y seleccionar hiperparámetros
Regresión logística	Capítulo 4	convertir scores en probabilidades interpretables

Tema	Entrada recomendada	Conexión principal
Probabilidad condicional	Capítulo 4 y referencia formal	interpretar $P(Y = 1 \mid X = x)$ antes de escoger umbral
Umbral de decisión	Capítulo 4	ajustar decisión al costo relativo de errores
Ejercicios integradores de regresión	Capítulo 5	practicar derivación, código e interpretación conjunta

Validación, métricas y diagnóstico

Tema	Entrada recomendada	Conexión principal
Train, dev, validation y test	Capítulo 3 y Capítulo 11	separar ajuste, selección y estimación final
Leakage	Capítulo 11	impedir que el pipeline vea información de validación o test
Matriz de confusión	Capítulo 4	explicar tipos de error
Precision, recall y F1	Capítulo 4 y Capítulo 11	escoger métrica primaria según costo
Curvas de aprendizaje	Capítulo 12	distinguir sesgo, varianza y límite por datos
Error analysis por slice	Capítulo 13	encontrar fallas que el promedio oculta
Data mismatch	Capítulo 13	decidir si un modelo puede salir del contexto de entrenamiento

Redes neuronales

Tema	Entrada recomendada	Conexión principal
Perceptrón y capas densas	Capítulo 6	decidir si hace falta no linealidad
Forward propagation	Capítulo 6	revisar formas de entradas, pesos, activaciones y logits
Backpropagation	Capítulo 7	verificar gradientes y cache
Diagnóstico de entrenamiento	Capítulo 8	interpretar curvas train/dev
Regularización en redes	Capítulo 8	decidir entre menor capacidad, L2, dropout o más datos
Optimizadores y PyTorch	Capítulo 9	reportar configuración reproducible de entrenamiento

Aprendizaje no supervisado y recomendación

Tema	Entrada recomendada	Conexión principal
PCA	Capítulo 15	reducir dimensión sin afirmar causalidad
Varianza explicada	Capítulo 15	justificar cuántos componentes conservar
K-Means	Capítulo 16	describir clusters antes de convertirlos en decisiones
Inercia y silueta	Capítulo 16	evaluar estructura sin absolutizar una métrica
Recomendación item-item	Capítulo 17	explicar similitud, cobertura y sesgo de popularidad
Ejercicios integradores no supervisados	Capítulo 18	practicar PCA, clustering y recomendación con límites

Reproducibilidad y comunicación

Tema	Entrada recomendada	Conexión principal
Síntesis conceptual	Capítulo 19	conectar pérdidas, métricas, validación y decisión
Manifest de experimento	Capítulo 20	registrar datos, código, entorno, parámetros y artefactos
Trazabilidad de datos	Capítulo 20	permitir auditoría de resultados
Reporte técnico	Capítulo 21	separar evidencia, interpretación y recomendación
Defensa oral	Capítulo 22	responder preguntas difíciles con evidencia concreta
Paquete final	Capítulo 22 y Capítulo 23	entregar reporte, código, datos permitidos y evidencia reproducible

Recursos de consulta rápida

Necesidad	Recurso
Definiciones y símbolos	Glosario y notación
Prerrequisitos de lectura	Prerrequisitos matemáticos mínimos
Algoritmos, datasets y funciones	Índice de algoritmos, datasets y funciones
Respuestas seleccionadas	Formularios y respuestas seleccionadas
Datos descargables	Manifiesto de datos
Autoría, accesibilidad y errata	Sobre esta edición

Necesidad	Recurso
Fuentes matemáticas y técnicas	Referencias y atribuciones

El índice no reemplaza el desarrollo de los capítulos. Su propósito es reunir rutas de regreso cuando una idea reaparece con una función distinta.

Índice de algoritmos, datasets y funciones

Este apéndice funciona como una tabla de referencia rápida. No enseña los conceptos desde cero; indica dónde aparecen, qué función o clase suele usarse y qué debe revisar el estudiante antes de confiar en un resultado.

Cómo usar este apéndice

Usa este índice cuando encuentres una función, clase o algoritmo en un notebook y necesites ubicar su primer uso conceptual. La columna de revisión necesaria indica qué debe comprobarse antes de convertir un output en evidencia.

El primer uso fuerte no siempre coincide con la primera mención. Se registra el punto donde el libro espera que puedas explicar el objeto matemático, ejecutar o leer una implementación y conectar el resultado con una decisión.

Mapa por primer uso

Capítulo	Objetos técnicos introducidos	Uso posterior
Observaciones, variables y preguntas	unidad de observación, diccionario de datos, matriz de diseño, familia y algoritmo	definición y representación matemática del problema
Capítulo 1	par aleatorio, media condicional, riesgo, matriz de diseño, MSE, ecuación normal, <code>np.linalg.lstsq</code>	regresión, comparación con bibliotecas y baselines
Capítulo 2	descenso del gradiente, <code>np.linalg.cond</code> , historial de pérdida	optimización, escala y diagnóstico de convergencia
Capítulo 3	features polinomiales, Ridge, Lasso, validación	selección de modelo, regularización y prevención de leakage
Capítulo 4	sigmoide estable, log-loss, matriz de confusión, umbral	clasificación binaria y métricas de decisión
Capítulo 6	capas densas, activaciones, softmax estable	redes neuronales y clasificación multiclase
Capítulo 7	regla de la cadena, cache, gradientes vectorizados	entrenamiento de redes y verificación numérica

Capítulo	Objetos técnicos introducidos	Uso posterior
Capítulo 9	<code>nn.Module</code> , <code>loss.backward</code> , <code>optimizer.step</code>	loops de entrenamiento reproducibles
Capítulo 11	particiones, <code>Pipeline</code> , métricas y prevención de leakage	evaluación y selección de modelos
Capítulo 12	curvas de aprendizaje, curvas de validación, <code>GridSearchCV</code>	diagnóstico de sesgo/varianza e hiperparámetros
Capítulo 15	centrado, SVD, PCA, varianza explicada	reducción de dimensión e interpretación cautelosa
Referencia: variables, distribuciones y muestras	espacio de probabilidad, distribuciones, momentos, muestra, <code>np.mean</code> , <code>np.var</code> , <code>np.random.default_rng</code>	consulta y profundización probabilística
Referencia: condicionamiento, estimación y riesgo	Bayes, estimadores, bootstrap y cambio de distribución	consulta y profundización estadística
Capítulo 16	<code>KMeans</code> , inercia, silhouette, escalamiento	clustering, segmentación y estabilidad
Capítulo 17	similitud coseno, ranking y cobertura	recomendación y evaluación offline
Capítulo 20	manifest de experimento, semillas, entorno	trazabilidad y reconstrucción de resultados

Referencia de API por capítulo

Primer uso	API o patrón	Qué aprende o calcula	Riesgo principal
Estimación y riesgo	<code>train_test_split</code>	separa observaciones	mezclar selección y test
Estimación y riesgo	<code>DummyClassifier</code>	baseline de clasificación	compararlo con otra partición
Estimación y riesgo	<code>make_pipeline</code>	encadena transformación y modelo	olvidar qué paso aprende parámetros
Distribuciones y muestras	<code>np.mean</code> , <code>np.var</code>	resumen de distribución empírica	confundir resumen muestral con garantía futura
Distribuciones y muestras	<code>np.random.default_rng</code>	simulación reproducible	interpretar simulación como dato real
Cap. 1	<code>np.column_stack</code> , <code>np.hstack</code>	construye matriz de diseño	duplicar intercepto
Cap. 1	<code>np.linalg.solve</code>	resuelve sistema lineal	usar matriz mal condicionada
Cap. 2	historial de pérdida	registra convergencia	no revisar escala de gradiente

Primer uso	API o patrón	Qué aprende o calcula	Riesgo principal
Cap. 3	<code>Ridge</code> , <code>Lasso</code>	ajusta regularización	escoger hiperparámetro con test
Cap. 4	<code>np.clip</code> , <code>np.log</code>	estabiliza log-loss	ocultar probabilidades extremas sin revisar
Cap. 6	<code>softmax</code> estable	convierte logits en probabilidades	overflow y ejes incorrectos
Cap. 9	<code>loss.backward()</code>	calcula gradientes	acumular gradientes sin limpiar
Cap. 11	<code>Pipeline</code>	evita leakage en transformaciones	ajustar pasos fuera del pipeline
Cap. 12	<code>GridSearchCV</code>	selecciona hiperparámetros	buscar sobre pipeline incompleto
Cap. 15	<code>np.linalg.svd</code>	factoriza matriz centrada	olvidar centrar antes
Cap. 16	<code>KMeans</code>	ajusta centroides	no escalar o no fijar semilla
Cap. 17	<code>cosine_similarity</code>	mide similitud angular	ignorar datos faltantes y popularidad

Algoritmos principales

Algoritmo o método	Primer uso fuerte	Implementación o referencia	Revisión necesaria
Regresión lineal	Capítulo 1	ecuación normal, <code>np.linalg.solve</code>	dimensiones, intercepto, MSE y residual
Descenso del gradiente	Capítulo 2	actualización vectorizada en NumPy	escala de variables, pérdida y convergencia
Regresión polinomial	Capítulo 3	matriz de features y validación	overfitting, grado y partición
Ridge	Capítulo 3	ecuación normal regularizada, <code>Ridge</code>	no penalizar intercepto y validar <code>lambda</code>
Lasso	Capítulo 3	<code>Lasso</code>	<code>sparsity</code> , escalamiento y validación
Regresión logística	Capítulo 4	sigmoide, log-loss, <code>LogisticRegression</code>	umbral, calibración y matriz de confusión
Perceptrón/logística multicapa	Capítulo 6	forward propagation	formas de matrices y activaciones
Backpropagation	Capítulo 7	regla de la cadena vectorizada	gradientes, cache y prueba numérica

Algoritmo o método	Primer uso fuerte	Implementación o referencia	Revisión necesaria
Momentum/Adam	Capítulo 9	loop NumPy o PyTorch	learning rate, estabilidad y validación
PCA	Capítulo 15	centrado y <code>np.linalg.svd</code>	varianza explicada e interpretación
K-Means	Capítulo 16	KMeans	escalamiento, inercia, silhouette y estabilidad
Recomendación item-item	Capítulo 17	similitud coseno	sesgo de popularidad y cobertura

Datasets y generadores

Recurso	Uso en el curso	Advertencia
<code>regresion_lineal_sintetica.csv</code>	caso longitudinal de regresión, gradiente, regularización y comunicación	sintético; no representa una población real
<code>clasificacion_binaria_sintetica.csv</code>	caso longitudinal de clasificación, métricas, validación, error analysis y defensa	el slice reciente ilustra mismatch de forma controlada
<code>pca_sintetico.csv</code>	PCA/SVD, varianza explicada y reconstrucción	estructura latente generada; no afirmar causalidad
<code>clusters_sinteticos.csv</code>	K-Means, inercia, silhouette y estabilidad	clusters compactos por construcción
<code>recomendacion_interacciones.csv</code>	recomendación item-item y cobertura	datos ficticios; no inferir preferencias reales
<code>load_wine</code>	arranque, clasificación multiclase y pipeline básico	dataset limpio; no prueba despliegue real
<code>load_diabetes</code>	regresión, Ridge/Lasso y validación	objetivo clínico simplificado; requiere cautela aplicada
<code>load_breast_cancer</code>	clasificación binaria, métricas y umbrales	no usar como recomendación clínica real
<code>load_digits</code>	redes, PCA y clasificación multiclase	imágenes pequeñas 8x8; no representa visión moderna
<code>make_regression</code>	regresión generada y gradiente	útil para revisar código, no para concluir aplicación
<code>make_blobs</code>	clustering sintético	favorece clusters compactos
<code>make_moons</code>	fronteras no lineales	muestra límites geométricos de modelos lineales

Funciones NumPy frecuentes

Función	Uso típico	Riesgo frecuente
<code>np.asarray</code>	convertir entradas a arreglo numérico	ocultar copias o formas inesperadas
<code>np.ones</code> , <code>np.hstack</code> , <code>np.column_stack</code>	construir matriz de diseño	duplicar intercepto
<code>@</code>	producto matricial	confundir producto elemento a elemento
<code>np.mean</code> , <code>np.sum</code> <code>np.linalg.solve</code>	pérdidas, métricas y gradientes resolver sistemas lineales	olvidar eje o factor $1/m$ usar matrices mal condicionadas
<code>np.linalg.cond</code>	diagnosticar condicionamiento	calcularlo después de leakage o escalamiento incorrecto
<code>np.linalg.svd</code> <code>np.exp</code> , <code>np.log</code> , <code>np.clip</code> <code>np.random.default_rng</code>	PCA y bajo rango sigmoide y log-loss reproducibilidad	olvidar centrar datos overflow o logaritmo de cero no fijar semilla o mezclar generadores

Herramientas scikit-learn frecuentes

Objeto	Uso típico	Revisión necesaria
<code>train_test_split</code>	separar entrenamiento, validación o test	estratificación y semilla
<code>StandardScaler</code>	escalar variables numéricas	ajustar solo con entrenamiento
<code>Pipeline</code> o <code>make_pipeline</code>	evitar leakage en transformaciones	ordenar pasos y revisar qué se aprende
<code>DummyClassifier</code>	baseline de clasificación	comparar antes de celebrar métricas
<code>LinearRegression</code>	referencia de regresión	misma matriz y mismo intercepto
<code>Ridge</code> , <code>Lasso</code>	regularización	escala, alpha y selección por validación
<code>LogisticRegression</code>	clasificación lineal	umbral, regularización y clases
<code>KMeans</code>	clustering por centroides	n_init , semilla y escalamiento
<code>silhouette_score</code>	evaluar separación de clusters	no absolutizar en geometrías no convexas
<code>cosine_similarity</code>	recomendación y similitud	normalización y datos faltantes

Métricas y diagnósticos

Métrica o diagnóstico	Dónde aparece	Pregunta que responde
MSE/MAE	regresión	¿qué tan grande es el error numérico?
matriz de confusión	clasificación	¿qué tipos de error se cometen?
precision	clasificación	¿cuántas predicciones positivas son correctas?
recall	clasificación	¿cuántos positivos reales se detectan?
F1	clasificación	¿cómo balancear precision y recall?
ROC-AUC	clasificación	¿cómo ordena scores el modelo?
learning curve	estrategia ML	¿hay sesgo, varianza o límite por datos?
validation curve	estrategia ML	¿cómo cambia el desempeño con un hiperparámetro?
inertia	K-Means	¿qué tan compactos son los clusters?
silhouette	clustering	¿qué tan separados están los clusters?
varianza explicada	PCA	¿cuánta estructura conserva la proyección?

PyTorch mínimo del curso

Elemento	Uso	Error frecuente
<code>torch.Tensor</code>	datos y parámetros en tensores	mezclar tipos o dispositivos sin control
<code>nn.Module</code>	definir modelo entrenable	esconder formas de entrada/salida
<code>loss_fn</code>	calcular pérdida	usar pérdida incompatible con logits o probabilidades
<code>loss.backward()</code>	calcular gradientes	olvidar limpiar gradientes antes
<code>optimizer.zero_grad()</code>	reiniciar acumulación de gradientes	acumular gradientes sin querer
<code>optimizer.step()</code>	actualizar parámetros	actualizar sin revisar escala de pérdida

Regla de lectura rápida

Cuando encuentres una función o clase en un notebook, pregunta:

1. ¿qué objeto matemático representa?
2. ¿qué aprende con `fit` o durante entrenamiento?
3. ¿qué datos puede ver sin producir leakage?
4. ¿qué métrica verifica que funciona?
5. ¿qué decisión se justifica y cuál no?

Una biblioteca estable reduce trabajo mecánico, no reduce responsabilidad técnica. El estudiante sigue siendo responsable de la pregunta, la partición, la métrica, el diagnóstico y la interpretación.

Formularios y respuestas seleccionadas

Los apéndices reúnen fórmulas, criterios operativos, patrones de código y respuestas seleccionadas de cada bloque. No reemplazan el estudio de los capítulos: sirven como referencia rápida y como verificación después de resolver ejercicios.

Las respuestas seleccionadas son deliberadamente parciales. El objetivo es orientar, no convertir el libro en un solucionario completo. Las soluciones oficiales de laboratorios, parciales y pruebas ocultas pertenecen al paquete restringido de instructor.

Cómo usar esta parte

1. Intenta resolver primero el ejercicio sin mirar la respuesta.
2. Revisa la fórmula o checklist correspondiente.
3. Compara tu resultado con la respuesta seleccionada cuando exista.
4. Si tu respuesta difiere, identifica si el error es de forma, fórmula, cálculo, código o interpretación.
5. Vuelve al capítulo si no puedes explicar por qué la respuesta es correcta.

Mapa de apéndices

Apéndice	Uso principal
Módulo 1	Regresión, gradiente, regularización, logística y Parcial 1.
Módulo 2	Redes densas, backpropagation, diagnóstico y lectura de PyTorch.
Módulo 3	Métricas, particiones, validación, leakage y análisis de errores.
Módulo 4	PCA/SVD, K-Means, recomendación y límites de interpretación.
Módulo 5	Parcial 2, reproducibilidad, reporte técnico y defensa final.

Política de respuestas

Las respuestas públicas cubren una muestra representativa: algunas verifican cálculo, otras verifican razonamiento y otras muestran el nivel esperado de una explicación técnica breve. Los proyectos de cierre no tienen solución única; por eso se evalúan con criterios, evidencia y límites explícitos.

La política completa de separación entre respuestas públicas, manual de estudiante y recursos restringidos está en [Respuestas, solucionarios y recursos restringidos](#).

Apéndice Módulo 1: formulario y respuestas seleccionadas

Resumen de fórmulas, chequeos y verificación

Cómo usar este apéndice

Este apéndice resume las fórmulas y chequeos del primer bloque matemático del curso: regresión, optimización, regularización y clasificación binaria. Úsalo para verificar derivaciones, notebooks y preparación del Parcial 1.

No memorices fórmulas aisladas. Para cada objeto, verifica forma, significado, hipótesis y el lugar que ocupa en el pipeline de modelación.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de diseño
$y \in \mathbb{R}^n$	variable respuesta o etiqueta binaria
$\theta \in \mathbb{R}^p$	vector de parámetros
$\hat{y}$	predicción de regresión
$\hat{p}$	probabilidad predicha en clasificación binaria
λ	intensidad de regularización
τ	umbral de clasificación
TP, TN, FP, FN	conteos de matriz de confusión

Formulario mínimo

Regresión lineal

Predicción:

$$\hat{y} = X\theta.$$

Residual:

$$r = y - X\theta.$$

MSE:

$$\text{MSE}(\theta) = \frac{1}{n} \|X\theta - y\|_2^2.$$

Pérdida escalada:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2.$$

Gradiente:

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Ecuación normal:

$$X^\top X\theta = X^\top y.$$

Solución si $X^\top X$ es invertible:

$$\theta^* = (X^\top X)^{-1} X^\top y.$$

Descenso del gradiente

Actualización:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}).$$

Para MSE:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{1}{n} X^\top (X\theta^{(t)} - y).$$

Estandarización:

$$z_j = \frac{x_j - \mu_j}{s_j}.$$

Ridge

Objetivo:

$$J_{\text{ridge}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \frac{\lambda}{2} \|\theta_{1:p}\|_2^2.$$

Gradiente:

$$\nabla J_{\text{ridge}}(\theta) = \frac{1}{n} X^\top (X\theta - y) + \lambda \tilde{\theta}.$$

Solución sin penalizar intercepto:

$$\theta^* = (X^\top X + n\lambda D)^{-1} X^\top y,$$

donde $D_{00}=0$ y $D_{jj}=1$ para $j>0$.

Lasso

Objetivo:

$$J_{\text{lasso}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \lambda \|\theta_{1:p}\|_1.$$

Regresión logística

Sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Probabilidad:

$$\hat{p} = \sigma(X\theta).$$

Log-loss:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

Gradiente:

$$\nabla J(\theta) = \frac{1}{n} X^\top (\hat{p} - y).$$

Clasificación por umbral:

$$\hat{y}_i = 1\{\hat{p}_i \geq \tau\}.$$

Métricas

Accuracy:

$$\frac{TP + TN}{TP + TN + FP + FN}.$$

Precision:

$$\frac{TP}{TP + FP}.$$

Recall:

$$\frac{TP}{TP + FN}.$$

F1:

$$2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Patrones de código

```
def add_intercept(X):  
    X = np.asarray(X, dtype=float)  
    return np.hstack([np.ones((X.shape[0], 1)), X])
```

```
def mse_loss(X, y, theta):  
    residual = X @ theta - y  
    return np.mean(residual ** 2)
```

```
def mse_gradient(X, y, theta):  
    return (X.T @ (X @ theta - y)) / X.shape[0]
```

```
def ridge_closed_form(X, y, lam):  
    p = X.shape[1]  
    D = np.eye(p)  
    D[0, 0] = 0  
    return np.linalg.solve(X.T @ X + X.shape[0] * lam * D, X.T @ y)
```

```
def sigmoid(z):
    z = np.asarray(z, dtype=float)
    out = np.empty_like(z)
    pos = z >= 0
    out[pos] = 1 / (1 + np.exp(-z[pos]))
    exp_z = np.exp(z[~pos])
    out[~pos] = exp_z / (1 + exp_z)
    return out
```

Checklist antes de entregar

- El notebook corre desde cero.
- Las funciones no dependen de variables globales ocultas.
- Las formas de matrices y vectores son consistentes.
- El preprocesamiento se ajusta solo con entrenamiento.
- Hay baseline.
- Hay métrica adecuada.
- Hay interpretación breve.
- La conclusión menciona al menos una limitación.

Respuestas seleccionadas

Estas respuestas no reemplazan el desarrollo. Sirven para verificar dirección.

Capítulo 1

Ejercicio 1.1. Si X tiene forma $(120, 4)$, entonces `theta` debe tener forma $(4,)$ o $(4, 1)$ para que `X @ theta` produzca 120 predicciones.

Ejercicio 1.2. La columna de unos convierte el intercepto en un coeficiente más: `theta_0 * 1`.

Ejercicio 1.6. Para

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, y = \begin{bmatrix} 2 \\ 2 \\ 5 \end{bmatrix},$$

se obtiene:

$$X^T X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 9 \\ 12 \end{bmatrix}.$$

Capítulo 2

Ejercicio 2.2. Usamos `theta - alpha * grad` porque el gradiente apunta hacia mayor aumento de la pérdida.

Ejercicio 2.4. Escalar ayuda porque reduce diferencias extremas de curvatura entre direcciones. El descenso hace pasos más directos y estables.

Ejercicio 2.8. Una curva que sube y baja violentamente sugiere una tasa de aprendizaje demasiado grande o un problema de escala.

Capítulo 3

Ejercicio 3.2. No se escoge `lambda` con prueba porque entonces prueba deja de ser una evaluación final independiente.

Ejercicio 3.8. Para `theta=(3,4,-1)`, sin intercepto:

$$\|\theta_{1:p}\|_2^2 = 4^2 + (-1)^2 = 17, \quad \|\theta_{1:p}\|_1 = 4 + 1 = 5.$$

Ejercicio 3.16. Preferiría `lambda=0.1` si la meta es generalizar: aumenta el error de entrenamiento, pero reduce mucho el error de validación.

Capítulo 4

Ejercicio 4.6.

$$\sigma(0) = 0.5, \quad \sigma(2) \approx 0.881, \quad \sigma(-2) \approx 0.119.$$

Ejercicio 4.7. Si TP=40, FP=10, TN=80, FN=20:

$$\text{accuracy} = 0.80, \quad \text{precision} = 0.80, \quad \text{recall} = 0.667.$$

F1 es aproximadamente 0.727.

Ejercicio 4.16. Accuracy alta y recall bajo puede indicar clases desbalanceadas: el modelo acierta muchos negativos, pero falla positivos.

Tema	Ejercicios recomendados
------	-------------------------

Guía de estudio para el Parcial 1

Prioriza estos ejercicios si tienes poco tiempo:

Tema	Ejercicios recomendados
Regresión lineal	Cap. 1: 6, 7, 10, 14
Gradiente	Cap. 2: 6, 9, 13, 14
Validación	Cap. 3: 2, 6, 10, 16
Regularización	Cap. 3: 7, 8, 11, 13
Logística	Cap. 4: 6, 7, 11, 12
Umbrales	Cap. 4: 14, 15, 16

Para cerrar, escribe una página de resumen con:

1. una fórmula que todavía te cuesta;
2. un error común que ya sabes evitar;
3. una métrica que usarías en tu proyecto;
4. una pregunta concreta para clase o monitoría.

Apéndice Módulo 2: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice es una referencia para el Módulo 2. Úsalo después de intentar los ejercicios del capítulo o antes de revisar un notebook de redes. La meta no es memorizar símbolos, sino poder auditar cada objeto: forma, significado, operación que lo produjo y riesgo de implementación.

En redes neuronales, una respuesta puede tener código correcto y aun así estar mal argumentada si no explica formas, pérdida, diagnóstico o decisión de entrenamiento.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de entrada: n observaciones, p variables
$Y \in \mathbb{R}^{n \times k}$	etiquetas one-hot para k clases
$W^{[\ell]}, b^{[\ell]}$	pesos y sesgos de la capa ℓ
$Z^{[\ell]}$	preactivación de la capa ℓ
$A^{[\ell]}$	activación de la capa ℓ
$\hat{P}$	probabilidades predichas por softmax
J	pérdida promedio

Fórmulas clave de forward

Forward

$$Z^{[1]} = XW^{[1]} + b^{[1]}$$

$$A^{[1]} = \text{ReLU}(Z^{[1]})$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$$

$$\hat{P} = \text{softmax}(Z^{[2]})$$

Si $X \in \mathbb{R}^{n \times p}$, una capa oculta con h neuronas y k clases usa las formas:

Objeto	Forma
$W^{[1]}$	$p \times h$
$b^{[1]}$	$h \text{ o } 1 \times h$
$Z^{[1]}, A^{[1]}$	$n \times h$
$W^{[2]}$	$h \times k$
$b^{[2]}$	$k \text{ o } 1 \times k$
$Z^{[2]}, \hat{P}, Y$	$n \times k$

Softmax

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{\ell=1}^k e^{z_\ell}}$$

Versión estable:

$$\text{softmax}(z)_j = \frac{e^{z_j - \max(z)}}{\sum_{\ell=1}^k e^{z_\ell - \max(z)}}.$$

Cross-entropy

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k Y_{ij} \log(\hat{P}_{ij})$$

Fórmulas clave de backward

$$dZ^{[2]} = \frac{1}{n}(\hat{P} - Y)$$

$$dW^{[2]} = (A^{[1]})^\top dZ^{[2]}, \quad db^{[2]} = \sum_i dZ_i^{[2]}$$

$$dA^{[1]} = dZ^{[2]}(W^{[2]})^\top$$

$$dZ^{[1]} = dA^{[1]} \odot \mathbf{1}\{Z^{[1]} > 0\}$$

$$dW^{[1]} = X^\top dZ^{[1]}, \quad db^{[1]} = \sum_i dZ_i^{[1]}$$

Diagnóstico de entrenamiento

Evidencia	Diagnóstico probable	Acción inicial
Train mejora y validation empeora	overfitting	early stopping, L2, dropout, menor capacidad
Train y validation son malos	underfitting o mala señal	revisar features, arquitectura, learning rate
La pérdida oscila o explota	learning rate alto o escala inestable	bajar learning rate, revisar inicialización
Probabilidades casi uniformes	entrenamiento insuficiente o logits pequeños	revisar inicialización, gradientes y épocas
Gradientes con forma correcta pero norma cero	saturación o camino bloqueado	revisar activación, inicialización y datos

Patrones de código

```
def softmax_stable(Z):
    Z = np.asarray(Z, dtype=float)
    shifted = Z - np.max(Z, axis=1, keepdims=True)
    exp_scores = np.exp(shifted)
    return exp_scores / exp_scores.sum(axis=1, keepdims=True)
```

```
def cross_entropy(P, y):
    eps = 1e-12
    P = np.clip(P, eps, 1 - eps)
    return -np.mean(np.log(P[np.arange(len(y)), y]))
```

```
def relu_backward(dA, Z):
    return dA * (Z > 0)
```

Checklist antes de entregar

- Cada matriz tiene la forma esperada.
- Softmax se aplica por fila.
- La pérdida usa probabilidades recortadas o softmax estable.
- dZ_2 tiene la misma forma que Y .
- Cada gradiente tiene la misma forma que el parámetro que actualiza.
- Los gradientes se dividen por n exactamente una vez.
- El diagnóstico usa curva de entrenamiento y validación.
- La comparación con PyTorch usa misma partición, semilla y métrica.

Respuestas seleccionadas

Estas respuestas verifican dirección. No sustituyen el desarrollo completo ni las pruebas del laboratorio.

Verificaciones de ejemplos resueltos

Softmax estable. Para $z = (2, 1, 0)$, restar el máximo produce $(0, -1, -2)$. La distribución aproximada es $(0.665, 0.245, 0.090)$. La pérdida de una observación se calcula con la probabilidad de la clase correcta, no con la probabilidad más alta en abstracto.

Diferencia finita. Para $J(w) = (w - 3)^2$, $w = 2$ y $\varepsilon = 0.01$, la diferencia central da -2 , igual a la derivada exacta $2(w - 3)$.

Regularización L2. Si $\|W\|_F^2 = 14$ y $\lambda = 0.1$, la penalización $\frac{\lambda}{2}\|W\|_F^2$ vale 0.7. El gradiente suma λW al gradiente de datos.

Momentum. Con $\theta_0 = 5$, $\alpha = 0.1$, $\beta = 0.9$, $v_0 = 0$, $g_1 = 4$ y $g_2 = 2$, se obtiene $v_1 = 0.4$, $\theta_1 = 4.96$, $v_2 = 0.56$ y $\theta_2 = 4.904$.

Capítulo 1

Ejercicio 1.1. Sin activaciones no lineales, una composición de transformaciones lineales sigue siendo una transformación lineal. Por ejemplo:

$$(XW_1)W_2 = X(W_1W_2).$$

La red puede reescribirse como un solo modelo lineal.

Ejercicio 1.3. Si X tiene forma $(250, 20)$ y la capa oculta tiene 12 neuronas, entonces $W1$ debe tener forma $(20, 12)$, $b1$ forma $(12,)$ o $(1, 12)$, y $Z1$ forma $(250, 12)$.

Ejercicio 1.4. Para 5 clases y batch de 32, logits, probabilidades softmax y matriz one-hot tienen forma $(32, 5)$.

Capítulo 2

Ejercicio 2.1. Backpropagation necesita valores guardados del forward porque las derivadas locales dependen de objetos intermedios: por ejemplo, ReLU necesita conocer dónde $Z^{[1]} > 0$, y $dW^{[2]}$ necesita $A^{[1]}$.

Ejercicio 2.2. ReLU vale z cuando $z > 0$ y vale 0 cuando $z \leq 0$. Por tanto, su derivada es 1 en la región positiva y 0 en la región no positiva. En código, $(Z1 > 0)$ actúa como máscara.

Ejercicio 2.4. Si $A^{[1]} \in \mathbb{R}^{80 \times 16}$ y $dZ^{[2]} \in \mathbb{R}^{80 \times 4}$, entonces

$$dW^{[2]} = (A^{[1]})^\top dZ^{[2]} \in \mathbb{R}^{16 \times 4}, \quad db^{[2]} \in \mathbb{R}^4.$$

Capítulo 3

Ejercicio 3.1. Train accuracy 99 % y validation accuracy 70 % sugiere overfitting. El modelo aprendió patrones específicos del entrenamiento que no generalizan. Antes de reportarlo como mejora habría que revisar regularización, capacidad, partición de datos y análisis de errores.

Ejercicio 3.4. Para $w = (2, -1, 0.5)$ y $\lambda = 0.1$,

$$\lambda \sum_j w_j^2 = 0.1(4 + 1 + 0.25) = 0.525.$$

Ejercicio 3.8. Si una métrica global mejora pero un subconjunto crítico empeora, no basta con reportar la mejora. El siguiente paso es aislar el slice, verificar tamaño y costo del error, y proponer una acción antes de usar el modelo.

Capítulo 4

Ejercicio 4.2. PyTorch acumula gradientes porque permite sumar contribuciones de varias pérdidas o batches antes de actualizar. En entrenamiento estándar se llama `zero_grad()` para que el batch actual no se mezcle con gradientes del batch anterior.

Ejercicio 4.3. Con 1024 observaciones y batch size 64, una época tiene $1024/64 = 16$ batches.

Ejercicio 4.4. Si se entrenan 20 épocas con 16 batches por época, `optimizer.step()` se ejecuta $20 \times 16 = 320$ veces.

Ejercicios integradores

Bloque A. Para auditar dimensiones, no empieces por el código. Escribe primero las formas esperadas de X , $W^{[1]}$, $b^{[1]}$, $Z^{[1]}$, $A^{[1]}$, $W^{[2]}$, $b^{[2]}$ y $\hat{P}$. Luego compara cada objeto del notebook contra esa tabla.

Bloque C. Una buena respuesta de diagnóstico tiene tres partes: resultado observado, causa probable y comprobación. Ejemplo: “validation loss sube después de la época 15 mientras train loss sigue bajando; esto sugiere overfitting; probaría early stopping y mayor regularización”.

Apéndice Módulo 3: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice resume las reglas que evitan conclusiones engañosas en machine learning aplicado: métrica mal escogida, particiones contaminadas, validación mal interpretada y análisis de error insuficiente. Úsalo como checklist antes de comparar modelos o escribir una recomendación técnica.

Una métrica no es una conclusión. La conclusión aparece cuando conectas métrica, baseline, partición, error principal, costo de error y límite de uso.

Notación mínima

Símbolo	Significado
TP, TN, FP, FN	conteos de la matriz de confusión
s_k	métrica obtenida en el fold k
$\bar{s}$	media de validación cruzada
σ_s	variabilidad entre folds
dev	conjunto de validación para elegir modelos
test	conjunto final para estimar desempeño una vez
slice	subconjunto relevante para análisis de error

Métricas de clasificación binaria

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Validación cruzada

$$\bar{s} = \frac{1}{K} \sum_{k=1}^K s_k, \quad \sigma_s = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (s_k - \bar{s})^2}.$$

Reporta media y variabilidad. Si dos modelos tienen medias similares, la variabilidad puede cambiar la decisión.

Reglas operativas

1. El test se toca una vez.
2. La métrica principal se decide antes de comparar modelos.
3. Las transformaciones se ajustan dentro de cada fold o partición de entrenamiento.
4. Todo modelo se compara contra baseline.
5. Un promedio no reemplaza análisis de slices.

Checklist de evaluación

- La métrica principal está alineada con el costo de error.
- Hay baseline trivial o operativo.
- La partición respeta tiempo, grupos o estratificación cuando aplica.
- El preprocesamiento está dentro de un **Pipeline**.
- La validación no usa test para elegir hiperparámetros.
- El reporte incluye variabilidad o intervalo cuando sea razonable.
- Hay análisis de al menos un slice crítico.
- La recomendación incluye un límite explícito.

Patrones de decisión

Situación	Señal	Decisión prudente
Clase positiva rara	accuracy alta con recall bajo	usar recall, F1, PR-AUC o costo explícito
Métricas train muy superiores a dev	brecha alta	diagnosticar overfitting o leakage
Dev estable y test cae	fuentes/población distinta	investigar data mismatch
Mejor media pero alta variabilidad	folds inconsistentes	preferir modelo más estable si el riesgo importa
Slice crítico empeora	promedio oculta daño	no recomendar despliegue sin mitigación

Patrones de código

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

```
def metric_by_group(df, group_col, y_col, yhat_col, metric_fn):
    rows = []
    for group, part in df.groupby(group_col):
        rows.append({
            "group": group,
            "n": len(part),
            "score": metric_fn(part[y_col], part[yhat_col])
        })
    return pd.DataFrame(rows)
```

Respuestas seleccionadas

Verificaciones de ejemplos resueltos

Costo de errores. Con umbral A, $FP = 40$, $FN = 15$, costo de FP igual a 1 y costo de FN igual a 10, el costo es $40 + 150 = 190$. Con umbral B, el costo es $10 + 250 = 260$. Si el falso negativo es mucho más costoso, A puede ser preferible aunque tenga menor precisión.

Partición estratificada. Si hay 2000 observaciones y 8% de positivos, hay 160 positivos. En una división 60/20/20 se esperan 96 positivos en train, 32 en dev y 32 en test. Esta cuenta solo verifica proporción de clase; no valida tiempo, grupos ni duplicados.

Learning curve. Si F1 de validación sube de 0.63 a 0.86 al aumentar datos y la brecha train-validación baja de 0.35 a 0.04, el patrón sugiere que más datos podría ayudar a reducir varianza. No es una señal para aumentar complejidad como primera acción.

Data mismatch. Si train y dev tienen cerca de 20% de positivos, pero test tiene 8% y recall cae de 0.82 a 0.55, antes de ajustar hiperparámetros hay que auditar fuente, periodo, población y construcción del split.

Brecha por slice. Si el F1 global es 0.81 y en fuente C es 0.57, la brecha es $0.81 - 0.57 = 0.24$. La recomendación debe explicar resultado, lectura, comprobación y límite; no basta reportar el promedio global.

Capítulo 1

Ejercicio 1.1. En fraude, preferir recall es razonable cuando perder un fraude es mucho más costoso que revisar falsos positivos. Preferir precision es razonable cuando cada alerta tiene alto costo operativo o afecta injustamente a usuarios legítimos.

Ejercicio 1.4. Con 2000 observaciones y clase positiva de 8 %, hay 160 positivos. En una partición 60/20/20 estratificada se esperan aproximadamente 96 positivos en train, 32 en dev y 32 en test.

Ejercicio 1.7. Imputar la media usando todo el dataset antes de partir es leakage porque la media incorpora información de validación/test. La media debe aprenderse solo con entrenamiento, idealmente dentro de un **Pipeline**.

Capítulo 2

Ejercicio 2.1. Train F1 0.99 y validation F1 0.70 sugiere overfitting. Acciones razonables: aumentar regularización, simplificar el modelo, revisar leakage, recolectar más datos o cambiar features.

Ejercicio 2.2. La desviación estándar de CV importa porque una media alta con mucha variabilidad puede indicar que el modelo depende demasiado de la partición. Si la aplicación exige estabilidad, un modelo con media apenas menor pero menor variabilidad puede ser preferible.

Ejercicio 2.4. Si se evalúan 4 valores de **C**, 3 valores de **penalty** y 5 folds, se entrenan $4 \times 3 \times 5 = 60$ ajustes.

Capítulo 3

Ejercicio 3.2. Un F1 global puede ocultar una falla grave si el modelo funciona bien en grupos numerosos y mal en un grupo pequeño pero importante. Por eso se reportan métricas por slice.

Ejercicio 3.3. Si una tabla por slice muestra F1 global 0.81 y un slice con F1 0.57, la brecha es 0.24. Si ese slice corresponde a una fuente, grupo o periodo relevante para producción, debe tratarse como riesgo técnico aunque el promedio global parezca aceptable.

Ejercicio 3.4. Si dev tiene 30 % de clase positiva y test 12 %, antes de ajustar hiperparámetros sospecharía data mismatch, cambio temporal, sesgo de muestreo o partición no representativa.

Ejercicio 3.7. Data mismatch puede sospecharse si el desempeño en dev es estable pero cae mucho en test, especialmente si test proviene de otra fuente, periodo o población.

Ejercicios integradores

Bloque A. Una respuesta completa sobre métricas debe nombrar el costo del error. Por ejemplo, no basta decir “F1 es mejor”; hay que explicar si el problema castiga más falsos positivos, falsos negativos o ambos.

Bloque D. La estructura mínima de recomendación técnica es: resultado observado, lectura probable, comprobación concreta y límite. Si falta una de esas cuatro partes, la recomendación es difícil de auditar.

Apéndice Módulo 4: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice reúne fórmulas y criterios para aprendizaje no supervisado. En este módulo, el cálculo correcto no basta: también debes decir qué conclusión permite el método y qué conclusión sería una sobreinterpretación.

En aprendizaje no supervisado no hay una etiqueta que confirme automáticamente que la estructura encontrada es útil. La respuesta debe combinar cálculo, visualización, estabilidad, interpretación y límite aplicado.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de datos
X_c	matriz centrada por columnas
$U\Sigma V^\top$	descomposición SVD
V_k	primeros k vectores derechos
$Z = X_c V_k$	scores o coordenadas PCA
μ_j	centroide del cluster j
c_i	asignación de cluster de la observación i
R	matriz usuario-ítem

SVD

$$X = U\Sigma V^\top$$

Si $X \in \mathbb{R}^{n \times p}$ y $r = \text{rank}(X)$, entonces:

Objeto	Forma
U reducida	$n \times r$
Σ reducida	$r \times r$
V reducida	$p \times r$
V_k	$p \times k$
$Z = X_c V_k$	$n \times k$

PCA

$$X_c = X - \bar{X}$$

$$Z = X_c V_k$$

$$\hat{X} = Z V_k^\top + \bar{X}$$

Varianza explicada por el componente j :

$$\frac{\sigma_j^2}{\sum_{\ell} \sigma_{\ell}^2}.$$

Varianza explicada acumulada con k componentes:

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_{\ell} \sigma_{\ell}^2}.$$

K-Means

$$\min_{\mu, c} \sum_{i=1}^n \|x_i - \mu_{c_i}\|_2^2$$

Silhouette

$$s = \frac{b - a}{\max(a, b)}$$

Cosine similarity

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}$$

Checklist de interpretación

- Los datos se centraron o escalaron según el método.
- La forma de la proyección PCA es consistente.
- La elección de k tiene un criterio: varianza, reconstrucción, curva o uso.
- K-Means se ejecutó con variables comparables en escala.
- Inertia no se interpreta como métrica absoluta de calidad.
- Silhouette se usa como señal, no como verdad final.
- Los clusters se describen con variables originales.
- El recomendador declara cold start, popularidad o sesgo cuando aplica.

Patrones de código

```
def pca_scores_from_svd(X, k):  
    X = np.asarray(X, dtype=float)  
    mean = X.mean(axis=0, keepdims=True)  
    Xc = X - mean  
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)  
    Z = Xc @ Vt[:k].T  
    return Z, Vt[:k], mean, S
```

```
def cosine_similarity_matrix(A):  
    A = np.asarray(A, dtype=float)  
    norms = np.linalg.norm(A, axis=1, keepdims=True)  
    norms = np.maximum(norms, 1e-12)  
    A_norm = A / norms  
    return A_norm @ A_norm.T
```

Respuestas seleccionadas

Capítulo 1

Ejercicio 1.1. PCA centra porque busca direcciones de variación alrededor de la media. Sin centrar, el primer componente puede capturar desplazamiento de la nube respecto al origen, no estructura de variabilidad.

Ejercicio 1.3. Si X tiene forma (500, 64) y usamos $k=2$, la proyección PCA tiene forma (500, 2).

Ejercicio 1.4. Que los primeros 10 componentes expliquen 85 % de la varianza significa que, en el subespacio de 10 dimensiones, se conserva 85 % de la variabilidad cuadrática medida por PCA. No significa que esos componentes expliquen causalmente el fenómeno.

Verificación de proyección. Para

$$X = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \quad \bar{X} = (1, 1),$$

se obtiene:

$$X_c = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Con $v_1 = (1, -1)^\top / \sqrt{2}$, los scores son $(\sqrt{2}, -\sqrt{2})^\top$. La reconstrucción con ese único componente recupera exactamente X_c , porque la matriz centrada tiene rango 1.

Capítulo 2

Ejercicio 2.1. Inertia baja al aumentar k porque cada punto puede quedar más cerca de algún centroide. En el extremo $k = n$, cada punto puede ser su propio centroide y la inertia llega a cero.

Ejercicio 2.2. K-Means debe usar escalamiento cuando las variables tienen unidades distintas porque la distancia euclídea queda dominada por variables con mayor escala numérica.

Ejercicio 2.4. Si un cluster contiene 90 % de las observaciones, revisaría escalamiento, elección de k , inicializaciones, variables dominantes y si la estructura real quizá no es aproximadamente esférica.

Verificación de K-Means. Para puntos 1, 2, 8, 10 y centroides iniciales $\mu_1 = 1$, $\mu_2 = 10$, las asignaciones son $\{1, 2\}$ y $\{8, 10\}$. La inertia inicial es

$$0^2 + 1^2 + 2^2 + 0^2 = 5.$$

Los centroides actualizados son 1.5 y 9. Con ellos, la inertia baja a

$$0.5^2 + 0.5^2 + 1^2 + 1^2 = 2.5.$$

Esto verifica el objetivo geométrico, no la calidad aplicada de la segmentación.

Capítulo 3

Ejercicio 3.1. La similitud coseno puede ser útil cuando algunos ítems son más populares porque compara dirección de interacción, no solo magnitud. Aun así, no elimina por completo el sesgo de popularidad.

Ejercicio 3.2. El problema de usuario nuevo se conoce como cold start: sin historial, no hay vector de preferencias suficiente para comparar o inferir gustos.

Ejercicio 3.8. Para detectar si un recomendador amplifica popularidad, compararía la popularidad media de los ítems recomendados contra la popularidad del catálogo y contra un baseline simple de popularidad.

Verificación item-item. Si

$$A = (1, 1, 0, 0), \quad B = (1, 1, 0, 1), \quad C = (0, 1, 1, 0), \quad D = (0, 0, 1, 1),$$

entonces:

$$\cos(A, B) = \frac{2}{\sqrt{2}\sqrt{3}} \approx 0.816, \quad \cos(A, C) = 0.5, \quad \cos(A, D) = 0.$$

Por similitud con A , el primer candidato es B . Si B ya fue visto por el usuario, el siguiente candidato entre estos ítems sería C .

Ejercicios integradores

Bloque A. Si dos componentes explican poca varianza pero separan grupos en un gráfico, puedes afirmar que la proyección muestra una separación visual en esos datos. No puedes afirmar causalidad ni que la separación aparezca en otra muestra sin una comprobación adicional.

Bloque B. Una buena descripción de cluster debe volver a las variables originales. Nombrar clusters solo por la posición en el gráfico suele producir etiquetas frágiles.

Bloque C. Una recomendación item-item mínima debe excluir ítems ya vistos y declarar qué ocurre con usuarios o ítems sin historial.

Apéndice Módulo 5: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice sirve para cerrar el curso: preparar el Parcial 2, revisar reproducibilidad, escribir el reporte final y defender una decisión técnica. La pregunta central ya no es solo “¿cuál fórmula uso?”, sino “¿qué resultado puedo explicar y qué límite debo declarar?”.

Una defensa precisa no exagera. Presenta un resultado identificable, una lectura proporcional, una comprobación concreta y límites claros.

Notación mínima

Símbolo	Significado
$\hat{p}_{i,y_i}$	probabilidad asignada a la clase correcta
$\bar{s}, \sigma_s$	media y variabilidad de validación
Z_k	representación PCA con k componentes
$\hat{X}$	reconstrucción aproximada
a_i, b_i	distancia intra-cluster y vecino más cercano en silhouette
manifest	archivo que registra datos, parámetros, métricas y artefactos

Fórmulas clave

Softmax

$$\text{softmax}(z)_j = \frac{e^{z_j - \max(z)}}{\sum_{k=1}^K e^{z_k - \max(z)}}.$$

Entropía cruzada multiclase

$$L = -\frac{1}{n} \sum_{i=1}^n \log(\hat{p}_{i,y_i}).$$

Score CV

$$\bar{s} = \frac{1}{K} \sum_{k=1}^K s_k, \quad \sigma_s = \sqrt{\frac{1}{K} \sum_{k=1}^K (s_k - \bar{s})^2}.$$

PCA desde SVD

$$X_c = U \Sigma V^\top, \quad Z_k = X_c V_k, \quad \hat{X} = Z_k V_k^\top + \mu.$$

Silhouette

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}.$$

Hash SHA-256

Un hash no prueba que un resultado sea correcto. Prueba que el archivo no cambió respecto a la versión registrada.

Checklist de paquete reproducible

- Hay una instrucción clara para ejecutar el flujo principal.
- Datos, scripts y resultados tienen rutas documentadas.
- La semilla y las particiones están registradas.
- El entorno está especificado con **requirements**, **environment** o equivalente.
- El preprocesamiento final está dentro del pipeline o descrito paso a paso.
- Las métricas finales se pueden recalcular.
- Los artefactos importantes tienen versión, nombre estable o hash.
- La conclusión distingue resultado, interpretación y recomendación.

Estructura mínima de manifest

```
{
  "schema_version": 1,
  "run_id": "baseline-logreg-s2105",
  "environment": {
    "python": "3.13.5",
    "packages": {"scikit-learn": "1.7.1"}
  },
}
```

```
"params": {
  "model": "logistic_regression",
  "C": 1.0,
  "seed": 2105,
  "split": "60/20/20 estratificado"
},
"metrics": {"dev_f1": 0.72, "dev_recall": 0.68},
"artifacts": {"metrics.csv": "c2a8...90bf"}
}
```

Lenguaje de defensa

Frase débil	Versión precisa
“El modelo es bueno.”	“El modelo supera el baseline en F1, pero falla en un slice crítico.”
“PCA demuestra grupos.”	“La proyección PCA sugiere separación visual; falta validación adicional.”
“El experimento es reproducible.”	“El experimento tiene semilla, entorno, manifest y script de evaluación.”
“Recomendamos usarlo.”	“Recomendamos una prueba limitada bajo estas restricciones.”

Respuestas seleccionadas

Verificaciones de ejemplos resueltos

- Softmax estable.** Para logits (1000, 1001, 1002), restar el máximo produce (−2, −1, 0). La distribución queda aproximadamente (0.090, 0.245, 0.665). La resta evita overflow y no cambia softmax.
- Manifest.** Si el manifest registra sha256:abc123 y el archivo actual tiene sha256:def987, el artefacto no coincide con la corrida registrada. La métrica asociada debe considerarse no verificada hasta volver a ejecutar o recuperar el archivo correcto.
- Mejora relativa.** Si F1 pasa de 0.60 a 0.72, la mejora absoluta es 0.12 y la relativa es $0.12/0.60 = 20\%$.
- Rúbrica ponderada.** Con puntajes 12, 16, 20, 13, 12, 8 sobre pesos que suman 100, la nota final es 81 %.

Síntesis para Parcial 2

Capítulo 1, ejercicio 1. Una mejora global no basta si un slice crítico empeora porque la métrica agregada puede ocultar daño concentrado. La respuesta debe revisar tamaño del slice, conteos, costo del error, estabilidad de la medición y acción de mitigación.

Capítulo 1, ejercicio 4. Para logits (1000, 1001, 1002), resta el máximo: $(-2, -1, 0)$. Softmax queda proporcional a $(e^{-2}, e^{-1}, 1)$, es decir aproximadamente (0.090, 0.245, 0.665).

Reproducibilidad

Capítulo 2, ejercicio 1. Una semilla fija no garantiza reproducibilidad completa porque pueden cambiar librerías, hardware, orden de datos, implementaciones paralelas o archivos de entrada.

Capítulo 2, ejercicio 4. Un hash SHA-256 detecta si un archivo cambió respecto a la versión registrada. No dice si el contenido es correcto, ético o suficiente para la decisión.

Capítulo 2, ejercicio 8. Si no se puede reproducir una métrica final, revisa primero partición, versión de datos, pipeline de preprocesamiento, semilla, parámetros del modelo y script de evaluación.

Comunicación técnica

Capítulo 3, ejercicio 1. “El modelo es bueno” no es suficiente porque no dice contra qué baseline, con qué métrica, en qué partición, bajo qué costo de error ni con qué limitaciones.

Capítulo 3, ejercicio 4. Si el baseline tiene F1 0.60 y el modelo final F1 0.72, la mejora absoluta es 0.12 puntos de F1 y la mejora relativa es $(0.72 - 0.60)/0.60 = 20\%$.

Capítulo 3, ejercicio 7. Una conclusión exagerada debe reescribirse con cuatro partes: resultado, límite, acción y condición de uso. Por ejemplo: “El modelo mejora F1 frente al baseline, pero su recall cae en clientes nuevos; recomendamos analizar ese slice antes de una prueba controlada”.

Entrega y defensa

Capítulo 4, ejercicio 3. Con puntajes 12/15, 16/20, 20/25, 13/15, 12/15 y 8/10, el total es 81/100.

Capítulo 4, ejercicio 4. Si una defensa usa cinco criterios 0/1/2, el puntaje máximo es 10. Una conversión transparente a porcentaje es $\text{porcentaje} = 10 \times \text{puntaje}$.

Capítulo 4, ejercicio 7. Una respuesta precisa a “¿qué cambiaría con dos semanas más?” debe nombrar una acción priorizada y una razón: “ampliaría análisis del slice con peor recall porque ahí está el riesgo operativo principal”.

Ejercicios integradores

Ejercicio 1. Restar el máximo evita overflow porque reemplaza exponenciales enormes por valores en una escala estable. La operación no cambia softmax porque multiplica numerador y denominador por la misma constante.

Ejercicio 2. Una respuesta sólida: el modelo mejora el baseline, pero tiene variabilidad alta, cae en test y falla en un slice crítico. La acción prudente es diagnosticar ese slice y ajustar datos, features o umbral antes de recomendar uso operativo.

Ejercicio 3. Con más componentes, el subespacio de reconstrucción contiene al anterior. Por eso la mejor aproximación con $k + 1$ componentes no puede tener mayor error que con k .

Ejercicio 4. Un manifest válido incluye parámetros, métricas, rutas de artefactos, hashes y una decisión. Su valor principal es auditar qué corrida produjo una conclusión.

Ejercicio 5. Una defensa final fuerte debe declarar al menos un límite: datos insuficientes, slice inestable, métrica parcial, supuestos de despliegue o riesgo de cambio de distribución.

Referencias y atribuciones

Esta página reúne las fuentes técnicas, las referencias generales y las atribuciones del libro. Las fuentes específicas de un resultado o de un conjunto de datos aparecen, además, en el lugar donde se utilizan.

Referencias de exposición y edición

La organización narrativa del texto se contrastó con dos obras de naturaleza distinta:

- Christensen, R. (2016). *Analysis of Variance, Design, and Regression: Linear Modeling for Unbalanced Data* (2nd ed.). CRC Press.
- OpenStax. *Principles of Data Science*. Página del libro: <https://openstax.org/details/books/principles-data-science>.
- OpenStax. Prefacio de *Principles of Data Science*: <https://openstax.org/books/principles-data-science/pages/preface>.

De Christensen se retoma como criterio general la progresión desde un problema concreto hacia su formulación matemática y el regreso posterior a la interpretación. OpenStax sirve como referencia para accesibilidad, atribución, erratas y publicación abierta. Los ejemplos, las derivaciones y los problemas de este libro son propios, salvo cuando se indica expresamente otra fuente.

Referencias matemáticas y algorítmicas

Estas referencias son útiles para revisión técnica y profundización:

- Blitzstein, J. K., & Hwang, J. (2019). *Introduction to Probability*. Chapman and Hall/CRC.
- Casella, G., & Berger, R. L. (2002). *Statistical Inference*. Duxbury.
- Wasserman, L. (2004). *All of Statistics*. Springer.
- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman and Hall/CRC.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Trefethen, L. N., & Bau, D. (1997). *Numerical Linear Algebra*. SIAM.
- Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press.

Documentación técnica

El curso usa documentación oficial como referencia para implementación:

- NumPy documentation: <https://numpy.org/doc/stable/>.
- pandas documentation: <https://pandas.pydata.org/docs/>.
- scikit-learn user guide and API: <https://scikit-learn.org/stable/>.
- scikit-learn toy datasets: https://scikit-learn.org/stable/datasets/toy_dataset.html.
- Matplotlib documentation: <https://matplotlib.org/stable/index.html>.
- seaborn documentation: <https://seaborn.pydata.org/>.
- PyTorch documentation: <https://docs.pytorch.org/docs/stable/index.html>.
- Project Jupyter documentation: <https://docs.jupyter.org/>.
- Quarto books documentation: <https://quarto.org/docs/books/>.

Datasets y generadores

El libro incluye un paquete público de CSV de práctica generados por `scripts/generar_datasets_libro.py` y documentados en [el manifiesto de datos](#). Estos archivos son originales del curso y se usan para práctica reproducible.

Además, algunos ejemplos y laboratorios pueden usar datasets de `scikit-learn` o generadores de datos controlados con semilla fija:

- `load_wine`: clasificación multiclase en el módulo de arranque.
- `load_diabetes`: regresión y regularización.
- `load_breast_cancer`: clasificación binaria, métricas y umbrales.
- `load_digits`: redes neuronales, PCA y clasificación multiclase.
- `make_regression`: regresión generada y descenso del gradiente.
- `make_blobs`: clustering y geometría de grupos.
- `make_moons`: fronteras no lineales y límites de modelos lineales.

El caso de vivienda nueva por subzona adapta el conjunto [Caracterización precios vivienda nueva](#), publicado por la Secretaría Distrital del Hábitat de Bogotá con licencia CC BY 4.0. La adaptación y sus limitaciones están registradas en [el manifiesto de datos](#). Los criterios de autoría, licencia y reproducción se resumen en [Sobre esta edición](#).

Figuras

Las figuras del atlas visual y de las aperturas de parte son originales del curso MATE-2105. Fueron creadas para explicar objetos matemáticos y decisiones técnicas del libro. El inventario auditable está publicado en [figuras/manifest_figuras.json](#) e incluye, para cada figura, archivo SVG, archivo PNG, caption, texto alternativo, autoría, estado de derechos, primer uso significativo y propósito pedagógico.

Cada figura debe conservar:

- título o caption;

- texto alternativo o descripción equivalente;
- archivo fuente o versión editable cuando exista;
- indicación de autoría del curso si se reutiliza fuera del contexto original.

Si una edición futura incorpora imágenes, tablas, datasets o textos de terceros, debe registrar fuente, autor, licencia, enlace original y modificaciones realizadas.

Cómo citar este libro

Galindo, C. (2026). *Matemáticas y Algoritmos en Ciencia de Datos Aplicada: Libro del curso MATE-2105*. Universidad de los Andes.

<https://cesarneyit-code.github.io/mate2105/libro/>

El repositorio incluye `CITATION.cff` para herramientas compatibles con GitHub y gestores bibliográficos.

Sobre esta edición

Esta es la edición de trabajo 2026-20 de *Matemáticas y Algoritmos en Ciencia de Datos Aplicada*, escrita por César Galindo para el curso MATE-2105 de la Universidad de los Andes. La versión HTML es la edición principal de lectura; el PDF se ofrece como copia estable para consulta sin conexión.

Autoría y cita

La responsabilidad matemática, pedagógica y editorial corresponde a César Galindo. La cita sugerida es:

Galindo, C. (2026). *Matemáticas y Algoritmos en Ciencia de Datos Aplicada: libro del curso MATE-2105*. Universidad de los Andes.

La URL permanente del libro es <https://cesarneyit-code.github.io/mate2105/libro/>.

Accesibilidad

El HTML utiliza estructura semántica, navegación por teclado, texto alternativo para figuras, encabezados en tablas y MathJax para la notación. El PDF es un respaldo visual y no sustituye la versión HTML como formato accesible. Las barreras encontradas deben reportarse por los canales del curso indicando página, dispositivo, navegador y tecnología de apoyo utilizada.

Datos, código y atribuciones

Los datasets originales del curso se publican junto con un manifiesto que registra procedencia, uso previsto, limitaciones y SHA-256. Las referencias matemáticas, técnicas y de datos se encuentran en [Referencias y atribuciones](#). El caso de vivienda nueva por subzona adapta un recurso de la Secretaría Distrital del Hábitat de Bogotá publicado con licencia CC BY 4.0.

Licencia y materiales reservados

El repositorio fuente contiene un archivo LICENSE de uso docente restringido. La lectura y el uso dentro del curso están permitidos; la redistribución, adaptación pública o explotación comercial requieren autorización expresa del autor. El libro público no incluye soluciones oficiales, pruebas ocultas, autograders ni guías docentes privadas.

Errata y actualización

Una corrección se incorpora cuando puede localizarse, reproducirse y verificarse en las fuentes. Para reportar una posible errata debe indicarse el capítulo, el fragmento afectado, la corrección propuesta y, cuando corresponda, un cálculo o una referencia que permita comprobarla. La fecha de esta edición aparece en el prefacio y en los metadatos bibliográficos del libro.